

CATEDRA DE ARQUITECTURA DE COMPUTADORAS

REPRESENTACION DE DATOS

Introducción

- En **INFORMATICA (Información Automática)**, se trabaja con un conjunto de datos para elaborarlo, procesarlo y obtener un resultado en un proceso de:

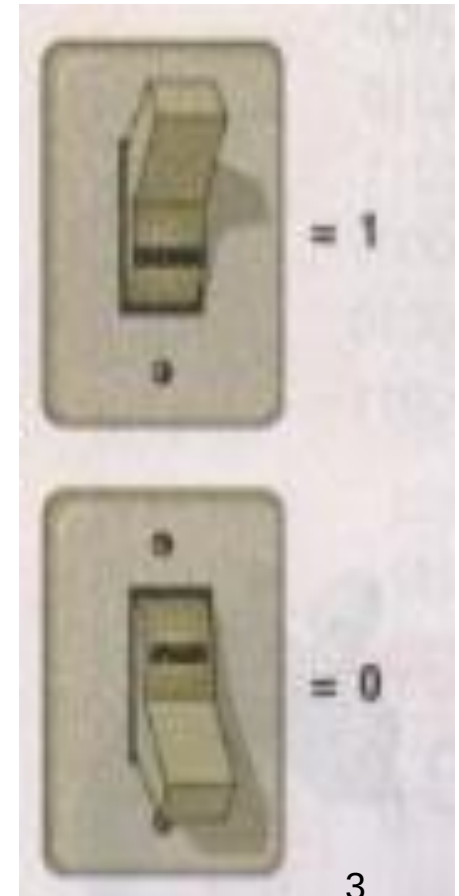
ENTRADA ➡ **MEMORIA** ➡ **PROCESO** ➡ **SALIDA**

Elaboración: Se produce en el interior de la máquina, y esta funciona con energía eléctrica.

Necesitamos analizar como se adapta la matemática y la aritmética para que pueda funcionar con energía eléctrica.

Sistemas de Representación

- Las computadoras representan todos sus datos en **sistema de numeración binario**.
- Los datos viajan, se procesan y se almacenan en las computadoras a través de **impulsos eléctricos**. Estos impulsos se representan por dos estados:
 - ✓ Encendido o apagado = 1 ó 0

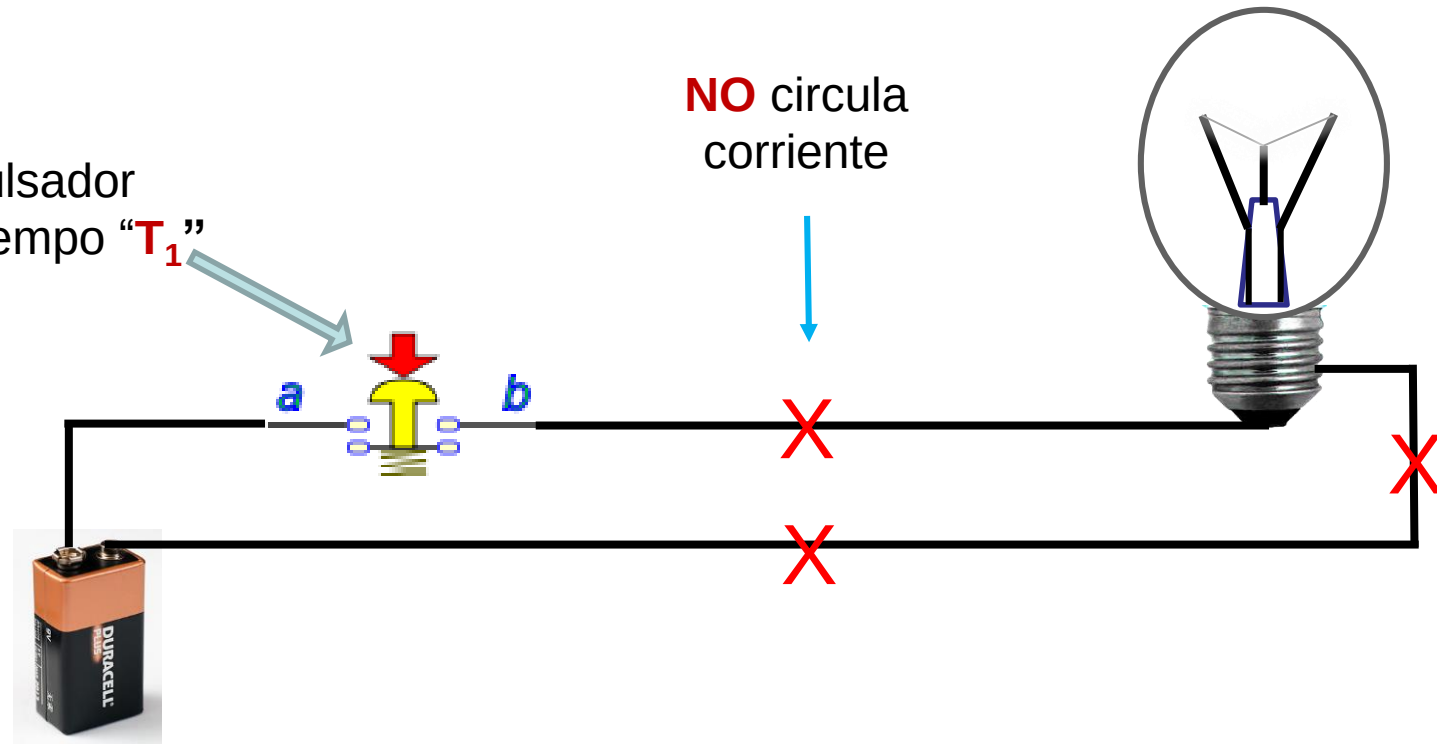


Sistemas de Representación

❑ EJEMPLO 1:

- ✓ SI NO PRESIONAMOS EL PULSADOR DURANTE UN TIEMPO T_1 . NO CIRCULARÁ CORRIENTE Y LA LÁMPARA PERMANECERÁ APAGADA DURANTE ESE TIEMPO T_1 .

NO presionar el pulsador
Durante el intervalo tiempo " T_1 "

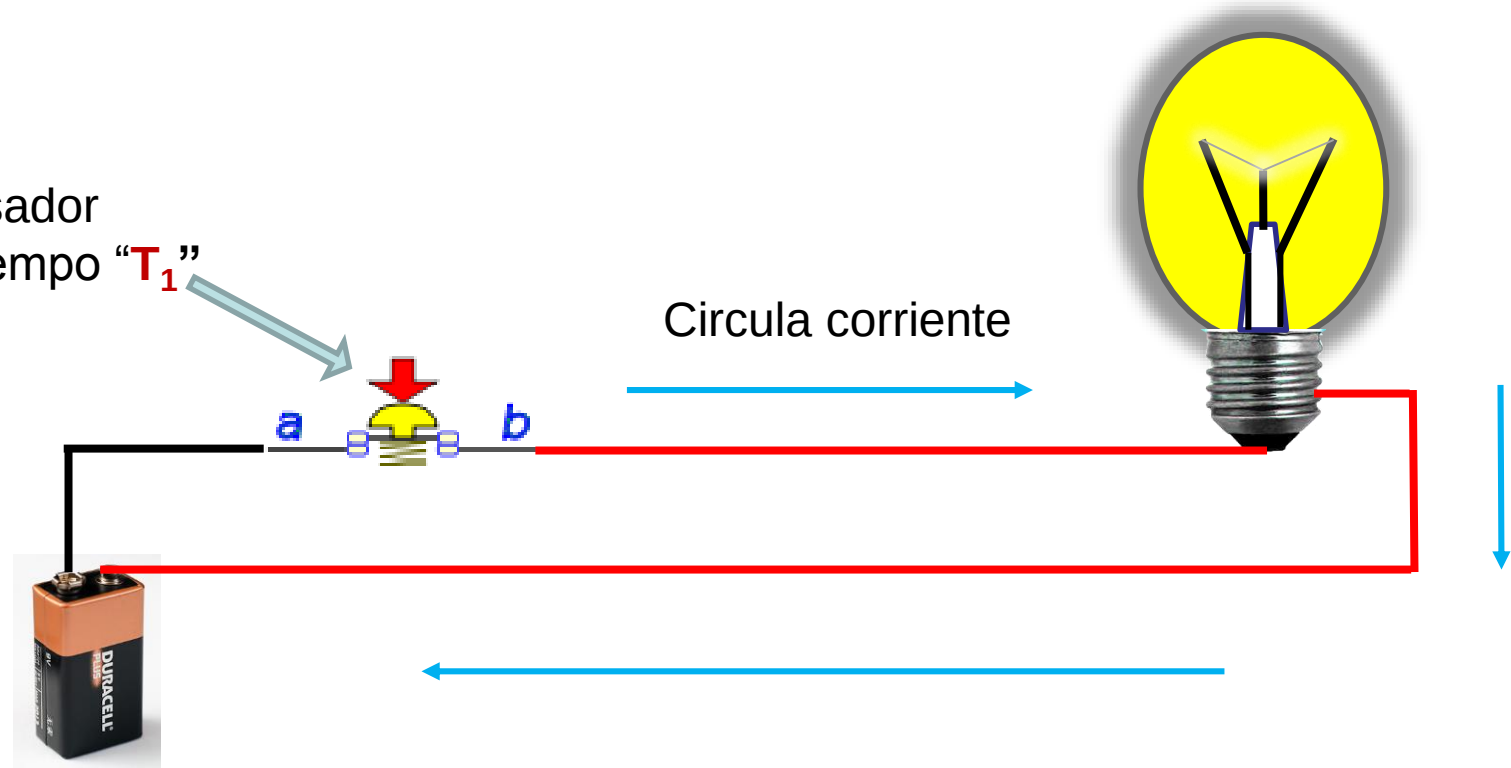


Sistemas de Representación

❑ EJEMPLO 2:

- ✓ **SI** PRESIONAMOS EL PULSADOR DURANTE UN TIEMPO T_1 . CIRCULARÁ CORRIENTE Y LA LÁMPARA PERMANECERÁ **ENCENDIDA** DURANTE ESE TIEMPO T_1 .

SI Presionar pulsador durante el intervalo tiempo " T_1 "



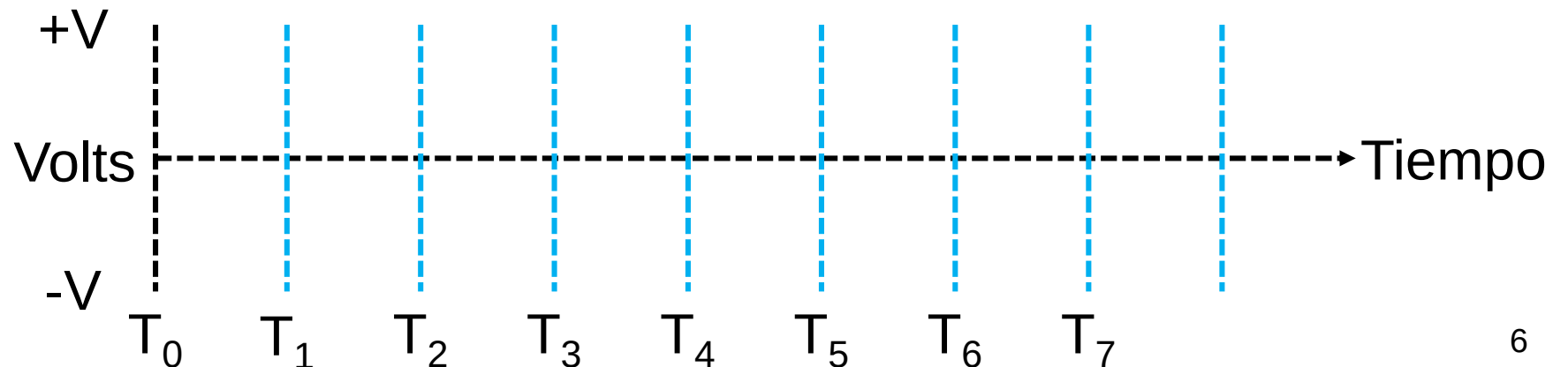
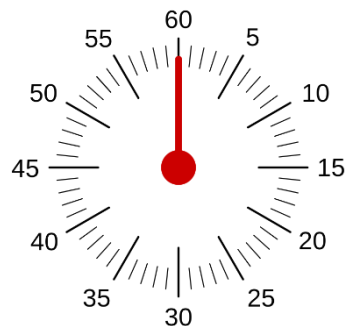
Sistemas de Representación

❑ EJEMPLO 3: SINCRONIZAR LA OPERATORIA:

- ✓ LA ACCIÓN QUE DECIDAMOS REALIZAR, YA SEA PRESIONAR EL PULSADOR O DEJARLO SIN PRESIONAR, DEBERÁ MANTENERSE CONSTANTE DURANTE UN TIEMPO T_i . AL FINALIZAR, ARBITRAMOS LA SIGUIENTE ACCIÓN Y DEBEREMOS MANTENERLA CONSTANTE DURANTE EL TIEMPO T_{i+1} Y ASÍ SUCESIVAMENTE.
- ✓ LOS INTERVALOS DE TIEMPO SON FIJOS, O SEA $T_1 = T_2 = \dots T_N$.

SUPONGAMOS:

$$0 - 60 = T_i$$

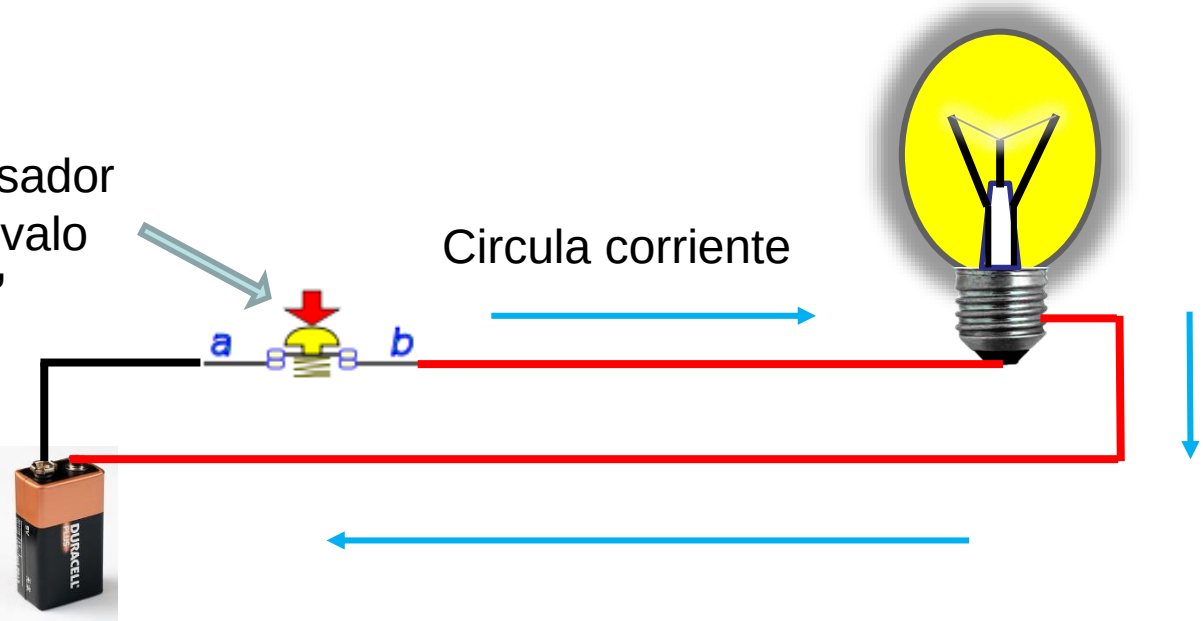


Sistemas de Representación

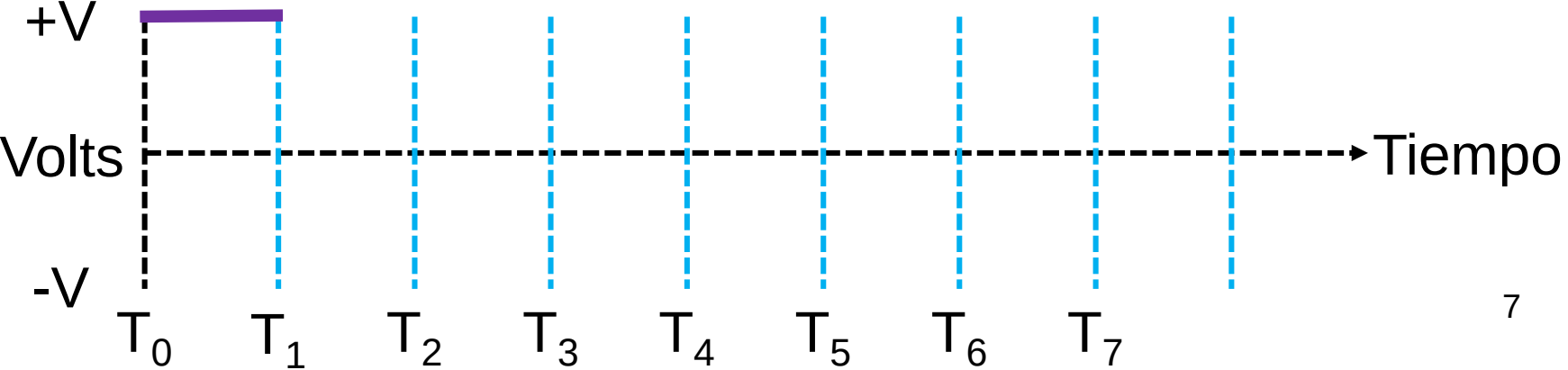
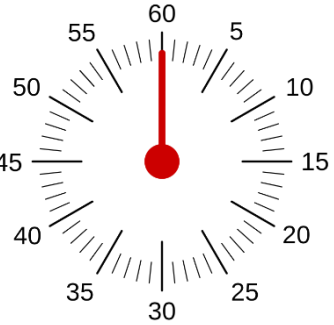
☐ ✓ ACCIÓN 1:

PULSAR SI / NO	PASO DE CORRIENTE
SI	✓

SI Presionar pulsador
durante el intervalo
tiempo " T_1 "



SUPONGAMOS:
 $0 - 60 = T_i$

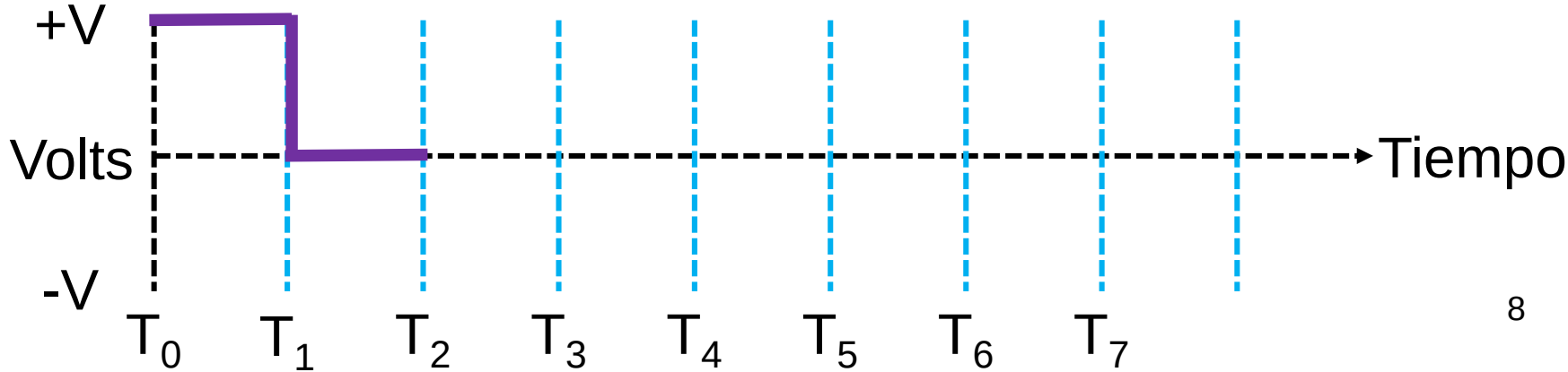
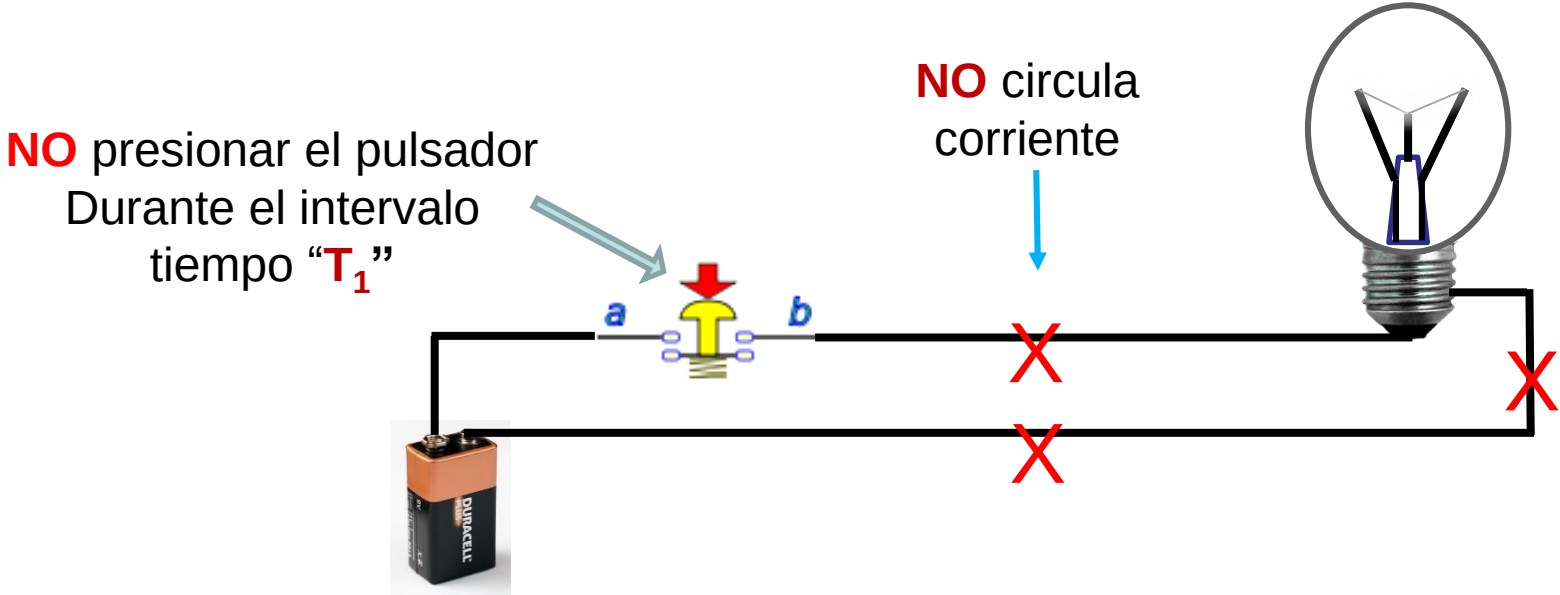
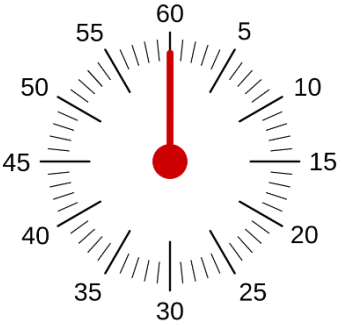


Sistemas de Representación

☐ ✓ ACCIÓN 2:

PULSAR SI / NO	PASO DE CORRIENTE
SI	✓
NO	X

SUPONGAMOS:
 $0 - 60 = T_i$

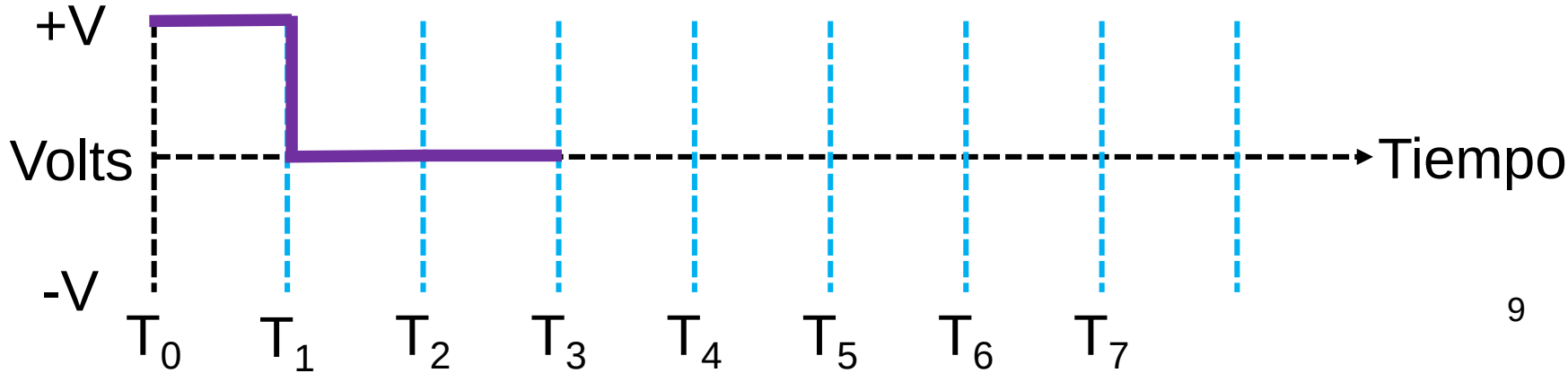
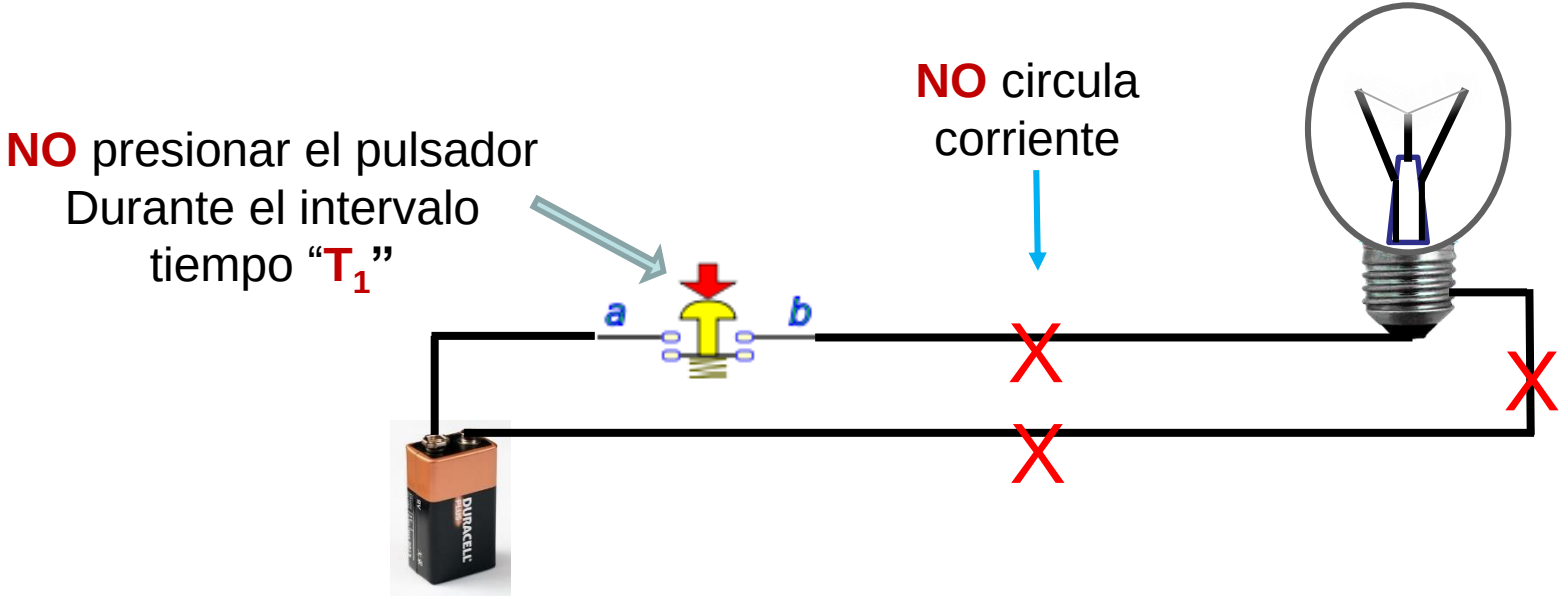
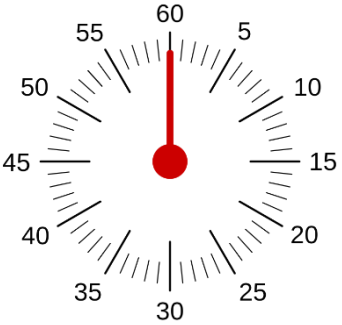


Sistemas de Representación

☐ ✓ ACCIÓN 3:

PULSAR SI / NO	PASO DE CORRIENTE
SI	✓
NO	X
NO	X

SUPONGAMOS:
0 – 60 = T_i

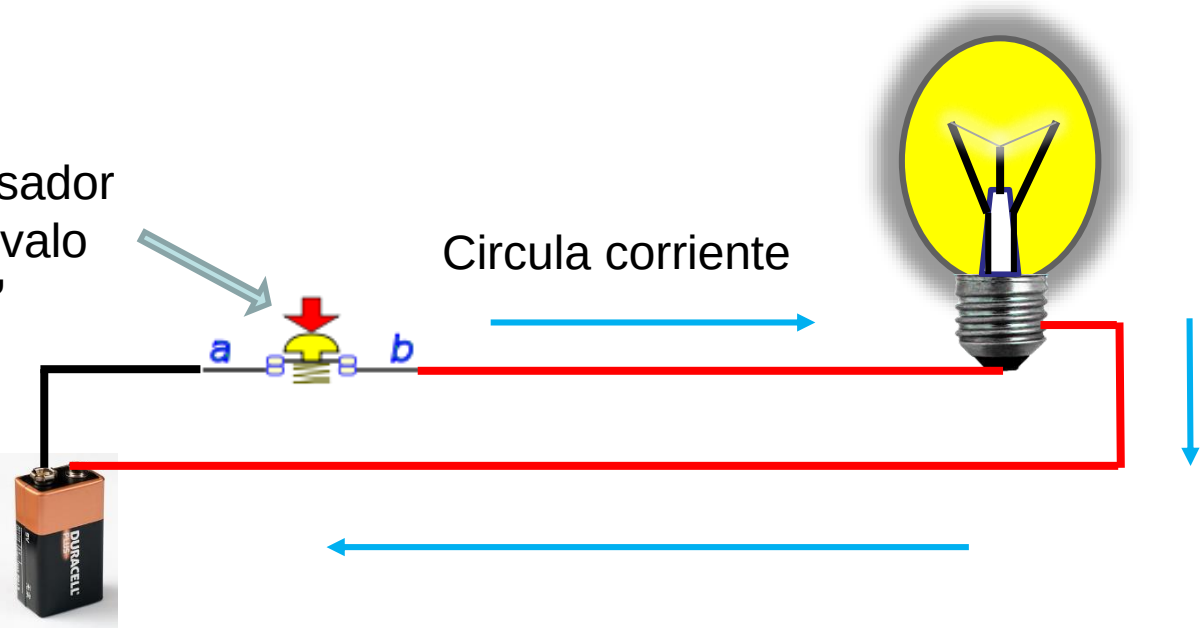


Sistemas de Representación

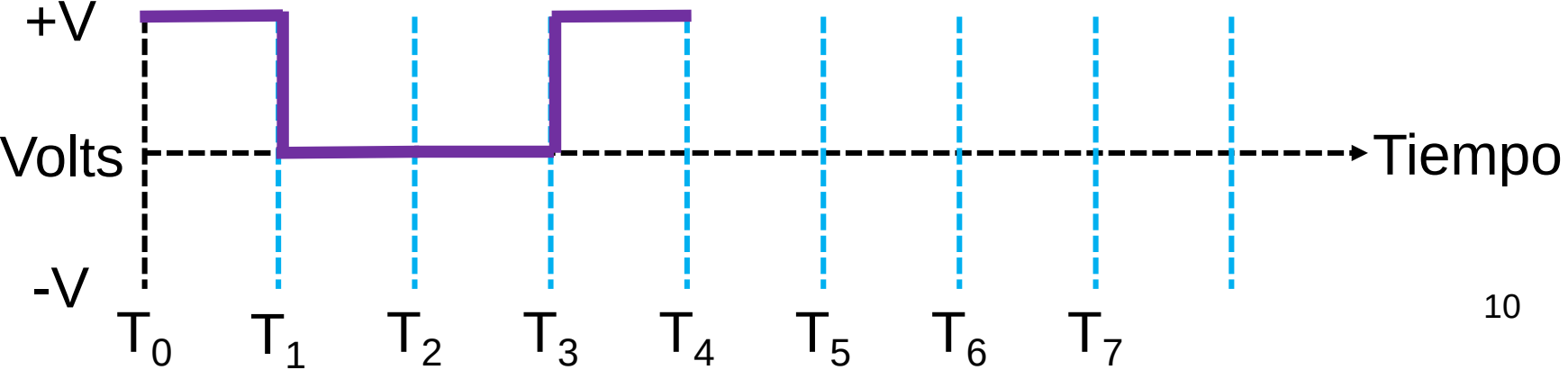
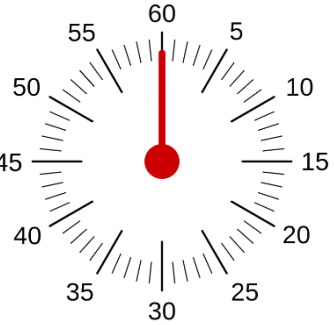
☐ ✓ ACCIÓN 4:

PULSAR SI / NO	PASO DE CORRIENTE
SI	✓
NO	X
NO	X
SI	✓

SI Presionar pulsador
durante el intervalo
tiempo " T_1 "



SUPONGAMOS:
 $0 - 60 = T_i$

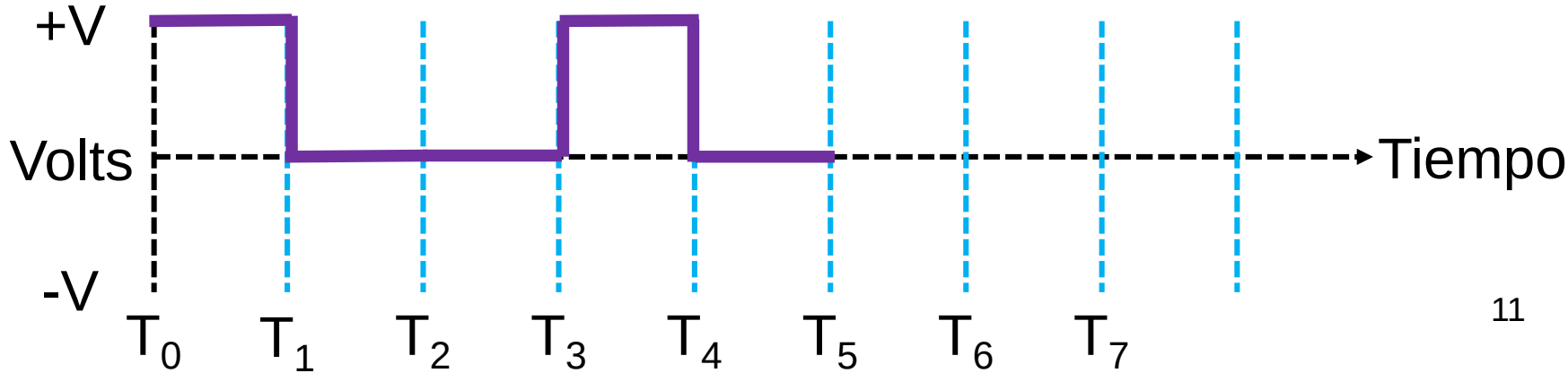
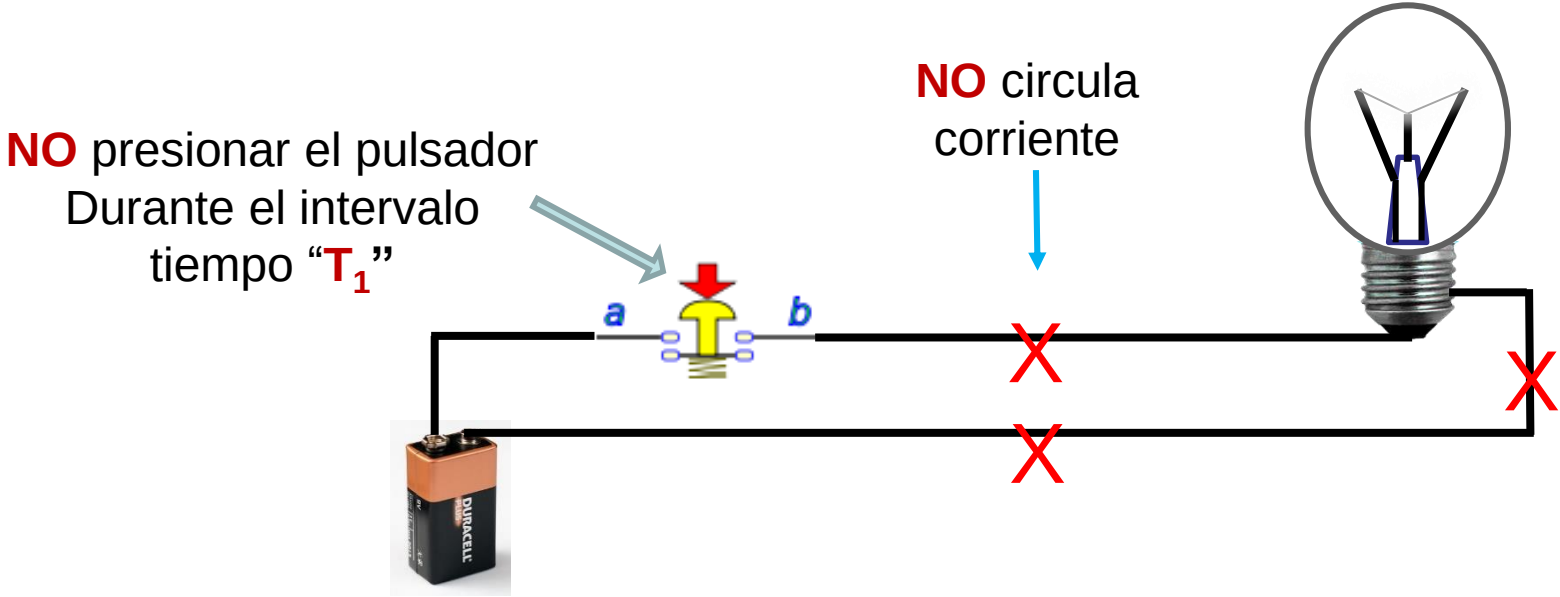
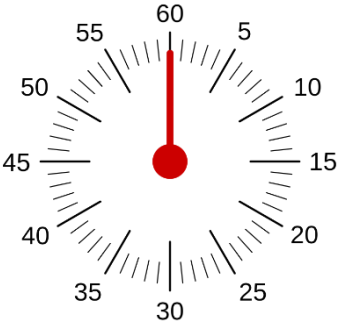


Sistemas de Representación

☐ ✓ ACCIÓN 5:

PULSAR SI / NO	PASO DE CORRIENTE
SI	✓
NO	X
NO	X
SI	✓
NO	X

SUPONGAMOS:
0 – 60 = T_i

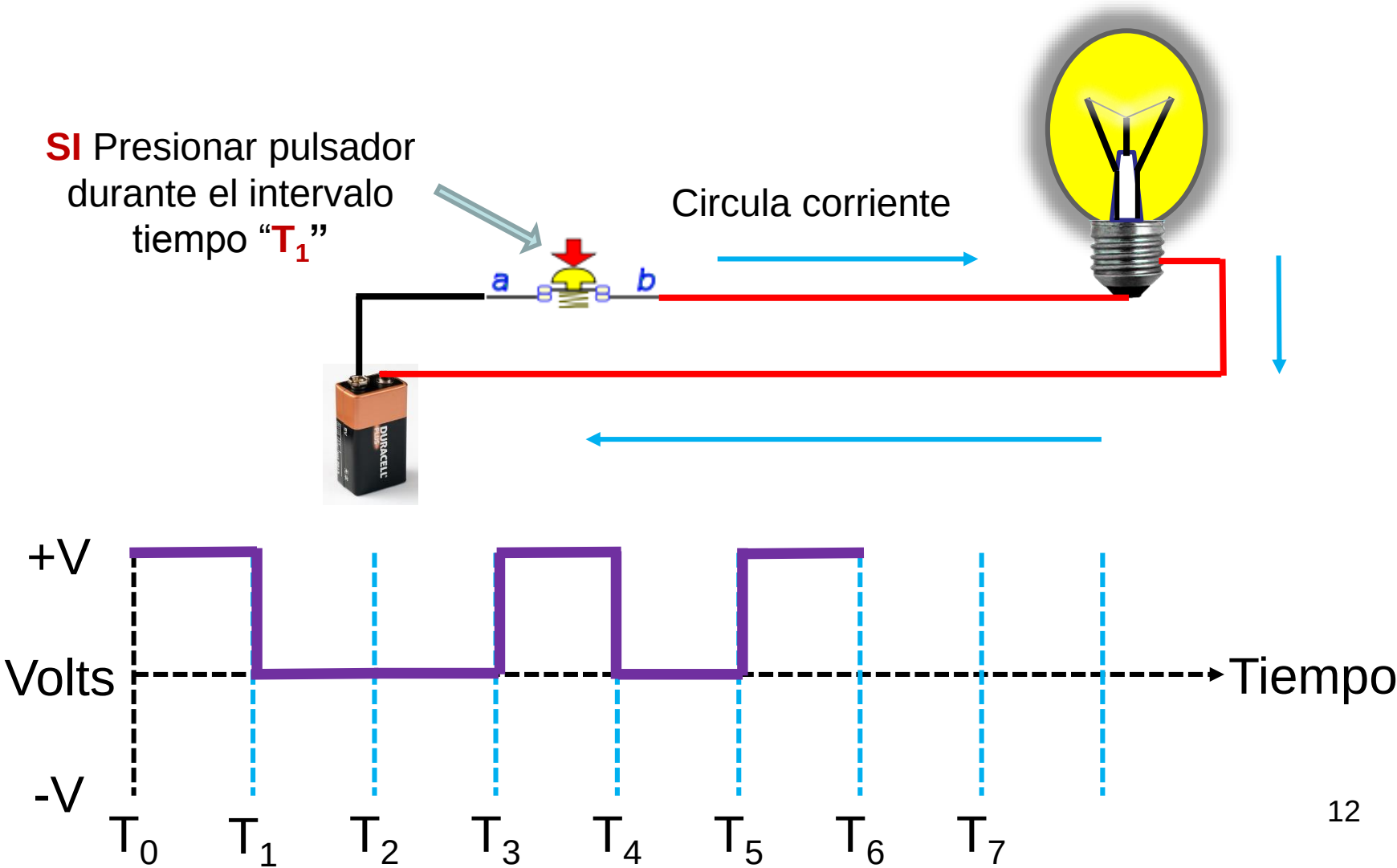
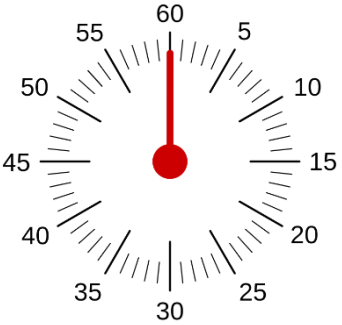


Sistemas de Representación

☐ ✓ ACCIÓN 6:

PULSAR SI / NO	PASO DE CORRIENTE
SI	✓
NO	X
NO	X
SI	✓
NO	X
SI	✓

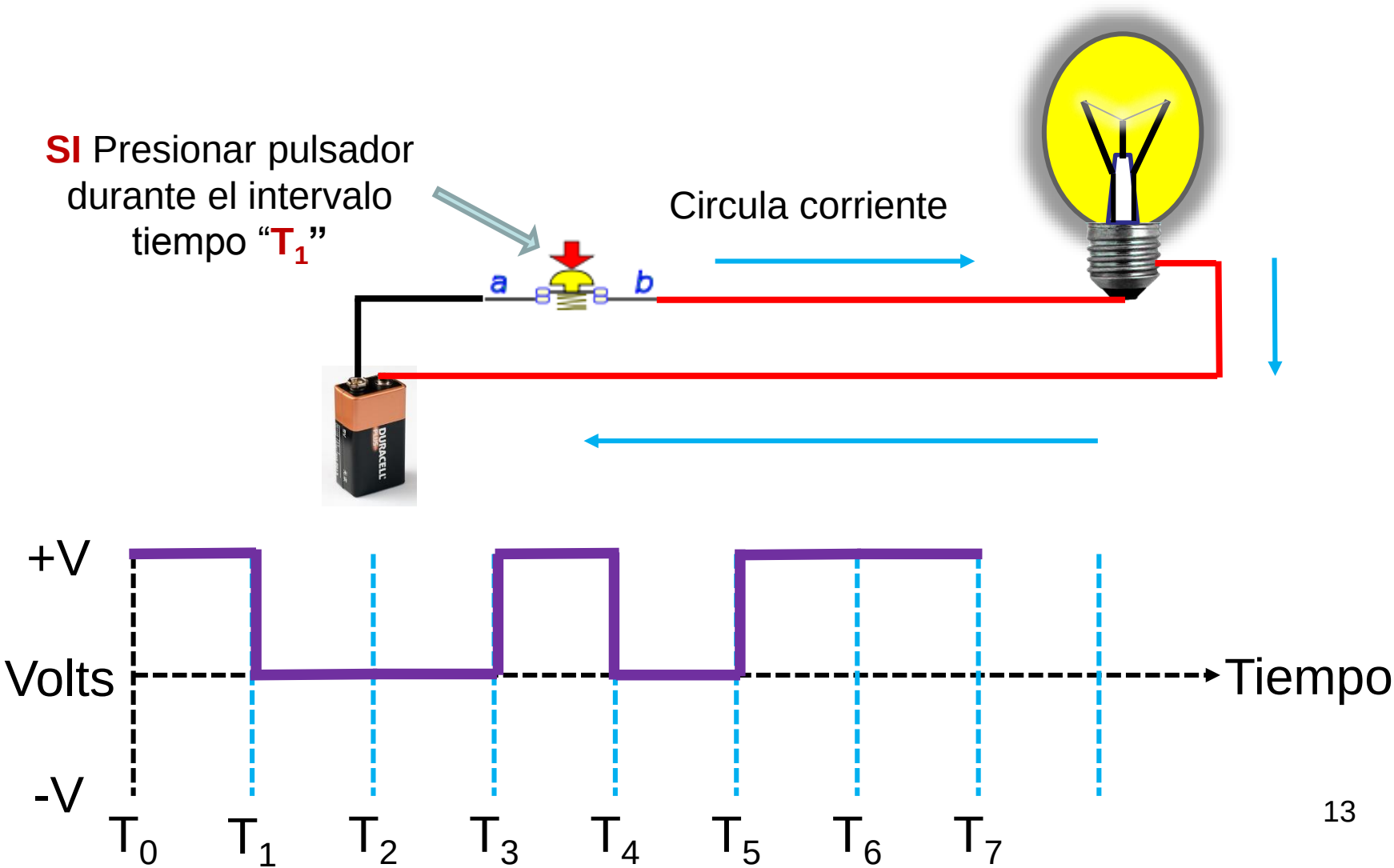
SUPONGAMOS:
 $0 - 60 = T_i$



Sistemas de Representación

☐ ✓ ACCIÓN 7:

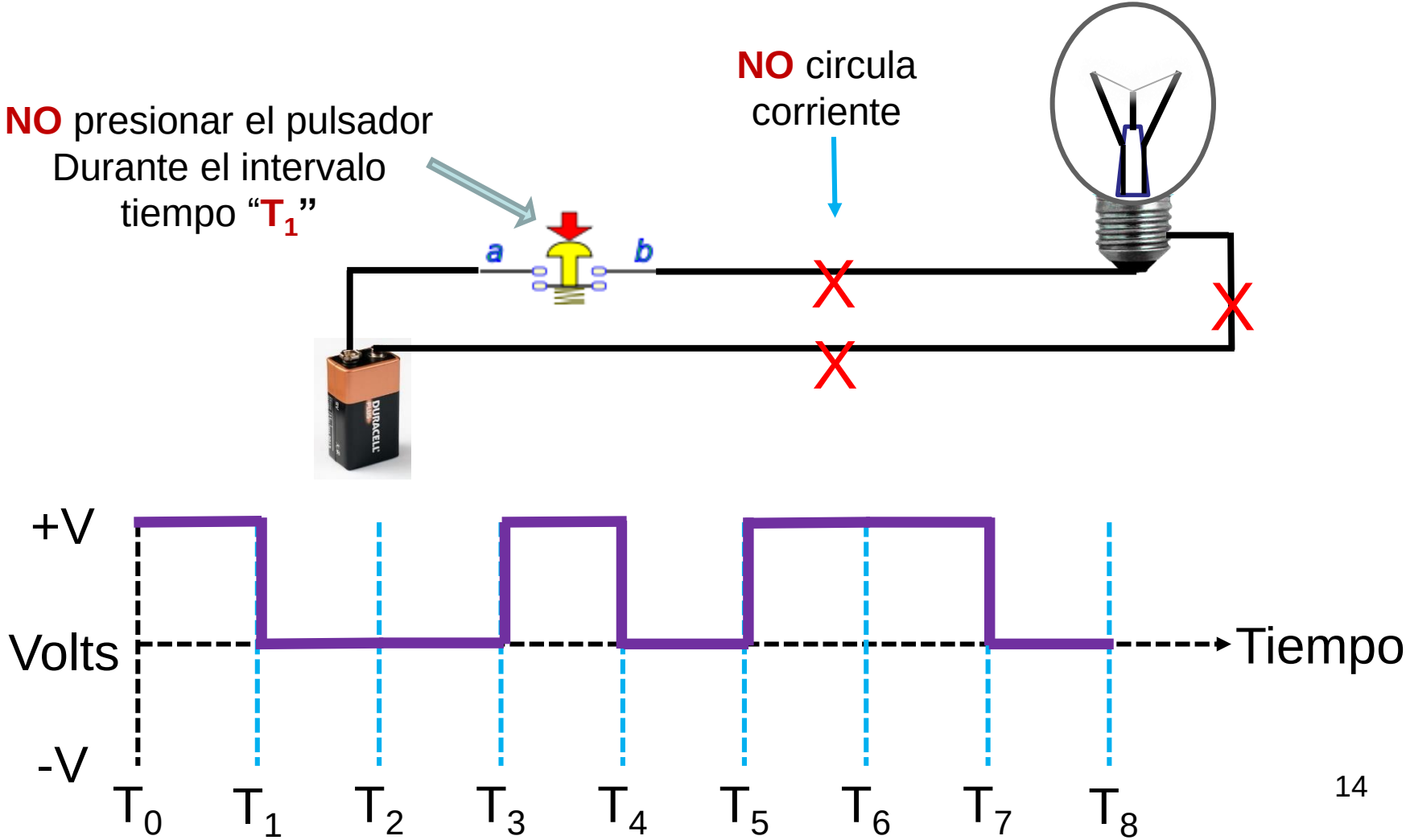
PULSAR SI / NO	PASO DE CORRIENTE
SI	✓
NO	X
NO	X
SI	✓
NO	X
SI	✓
SI	✓



Sistemas de Representación

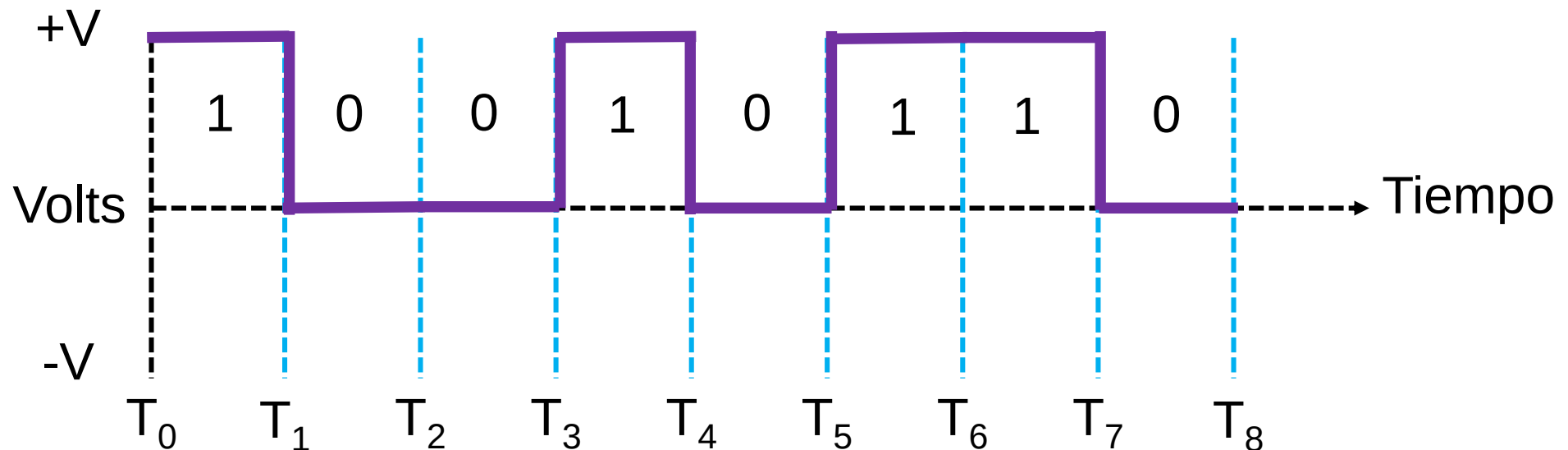
☐ ✓ ACCIÓN 8:

PULSAR SI / NO	PASO DE CORRIENTE
SI	✓
NO	X
NO	X
SI	✓
NO	X
SI	✓
SI	✓
NO	X



Sistemas de Representación

- ❑ ✓ EL MÉTODO DE TRANSMISIÓN, EN ESTE CASO, ES A TRAVÉS DE PULSOS ELÉCTRICOS QUE VARÍAN ENTRE DOS NIVELES DISTINTOS DE VOLTAJE:
- ❑ ✓ COMO EN ESTE CASO SE UTILIZAN DOS ESTADOS, APLICAMOS PARA SU REPRESENTACIÓN EL SISTEMA BINARIO, O SEA EL SISTEMA NUMÉRICO DE DOS DIGITOS :
 - **DIGITO “0” PARA INDICAR AUSENCIA DE LA CORRIENTE DURANTE UN INSTANTE T_i .**
 - **DIGITO “1” PARA INDICAR PRESENCIA DE LA CORRIENTE DURANTE UN INSTANTE T_i .**



Sistemas de Representación

- Las computadoras almacenan datos e instrucciones en memoria.
Para ello utilizan el **sistema binario**.
- **Razones:**
 - ✓ El dispositivo se encuentra en uno de dos estados posibles (0 o 1).
 - ✓ Identificar el estado es más fácil si sólo hay dos.
- **Ejemplo:**
 - ✓ Lámpara encendida o apagada.
 - ✓ Lámpara encendida con 10 intensidades distintas,
- Para saber si está encendida es más fácil en el primer caso que determinar alguna de las 10 intensidades distintas.

Unidades de medida para almacenamiento de datos

- Los computadores comprenden el lenguaje de los números.
- La organización de un computador depende entre otros factores del sistema de representación numérica adoptado.
- Se trabaja con el sistema binario, de donde proviene el término: **Bit**: Acrónimo de **Binary Digit** (**Dígito Binario**), es la mínima unidad de información que viaja y se almacena en un computador. Cada bit representa un impulso eléctrico (**1** o **0**).
- A un grupo de **8 Bits** se lo denomina “**BYTE**” (y es el **número de Bits necesarios para representar a un carácter**) “**a**” = **01100001**

Sistemas de Numeración – Introducción

- Los números naturales aparecen al contar los objetos de un conjunto.
- En la primera infancia se empieza por aprender a contar, luego más tarde se coordinan conjuntos prescindiendo del orden.
- Con posterioridad a la invención del número natural surge la necesidad de ampliar el concepto de número.
- Surgen así el **número entero**, el **número real**, etc.

Podemos definir un **Sistema de Numeración** como:

“El conjunto de reglas y convenciones que permiten la representación de todos los números mediante varios signos, o varias palabras”

Existen Sistemas de Numeración muy conocidos como el **Romano** que descompone el número en suma o diferencia de otros varios, cada uno de los cuales está representado por un símbolo especial:

I, V, X, L, C, D, M

Otro sistema es el **Decimal** que en vez de introducir nuevos símbolos para estos diversos sumandos, **utiliza el principio del valor relativo (posicional)**, es decir **una misma cifra representa valores distintos según el lugar que ocupa.**

Sistemas de Representación

- **Sistema Decimal:** Es el sistema de numeración utilizado en la vida cotidiana, cuya base es diez, utilizando los símbolos **0, 1, 2, 3, 4, 5, 6, 7, 8 y 9**
- **Sistema Binario:** los **dos** símbolos utilizados son el **0** y el **1**, los que reciben el nombre de **bit** (**Binary Digit**)
- **Sistema Octal:** de base **8**, los símbolos utilizados son **0, 1, 2, 3, 4, 5, 6, 7**
- **Sistema Hexadecimal:** de base 16, los símbolos utilizados son
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

Sistema Decimal: Inventado en la India en el siglo IV antes de Cristo y llevado a Europa por los árabes en la Edad Media.

Está fundado en el número fijo que llamamos **diez** (habiendo elegido este y no otro porque tenemos diez dedos en las manos).

De aquí surgen los **SISTEMAS NUMERICOS DE NOTACION POSICIONAL** donde:

NUMERO:

Representación de una cantidad, se compone de **1 o más dígitos**.

DÍGITO:

Elemento componente de un **NUMERO** para representar una cantidad.

BASE:

Cantidad de símbolos diferentes que pueden representar **UN DIGITO**

Ecuación General: $N = \sum_{i=-k}^{i=n} d_i * 10^i$ Donde 10 es la “Base”

Ej: $\begin{array}{|c|c|c|} \hline 2 & 1 & 2 \\ \hline \end{array})_d = 2*10*10 + 1*10 + 2*1$

$$= 2*10^2 + 1*10^1 + 2*10^0$$

$$= 200 + 10 + 2$$

Sistema de Numeración Binario

En el **Sistema Binario** los símbolos utilizados son el **0** y el **1**, llamados dígitos binario (bits).

Es posible, aplicando el algoritmo de numeración, obtener los números binarios correspondientes a las primeras **16₁₀** (el subíndice **10** indica que el número esta en el Sistema Decimal) cantidades.

Igualmente podríamos obtener los números en el Sistema Octal y en el Sistema Hexadecimal.

DECIMAL b = 10 ₁₀	BINARIO b = 2 ₁₀	OCTAL b = 8 ₁₀	HEXADECIMAL b = 16 ₁₀
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Observemos que en el **Sistema Hexadecimal** se han agregado las letras **A, B, C, D, E** y **F** para obtener los **16 símbolos distintos necesarios**.

La razón de tener en cuenta este Sistema es que **$2^4 = 16$** , por lo tanto a cada **4** cifras **Binarias** se corresponderá **1** cifra **Hexadecimal**.

También en el **Sistema Octal** se da algo parecido ya que **$2^3 = 8$** , por lo tanto cada tres cifras **binarias** se corresponderá una **Octal**.

Esto último nos permite encontrar rápidamente las equivalencias entre **Binario, Octal** y **Hexadecimal**, por ejemplo:

$$\underbrace{0010}_2 \underbrace{1001}_9 \underbrace{1110}_E \underbrace{0101}_5)_2 = 29E5)_{16}$$

$$\underbrace{110}_6 \underbrace{111}_7 \underbrace{111}_7 \underbrace{001}_1 \underbrace{000}_0 \underbrace{110}_6)_2 = 677106)_8$$

Conversión entre números de distintas bases

Se subdivide en dos etapas:

a-) Por un lado la **parte entera**

b-) Por otro lado la **parte decimal**

a-) Se divide el número por la nueva Base. El **cociente** se escribe debajo del número original y repetimos el procedimiento hasta que el **residuo** sea menor que **divisor**.

El nuevo número se forma de derecha a izquierda a partir del último cociente tomando todos los residuos de las divisiones anteriores.

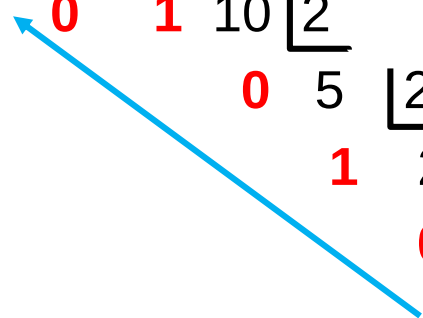
b-) La **parte fraccionaria** se convierte de la siguiente manera: Multiplicamos el número por la nueva **Base**, tomamos la parte entera del resultado y la parte fraccionaria la volvemos a multiplicar sucesivamente hasta llegar a cero.

Ej: Pasar el número $42,375)_{10}$ a “**Binario**”:

a) **Parte Entera:**

$$\begin{array}{r} 42 \div 2 \\ 21 \div 2 \\ 10 \div 2 \\ 5 \div 2 \\ 2 \div 2 \\ 1 \end{array}$$

0 1 0 1 0 1



Parte Entera: 101010

b) **Parte Fraccionaria:**

$$\begin{array}{l} 0,375 * 2 = 0,75 \\ 0,75 * 2 = 1,50 \\ 0,50 * 2 = 1,00 \end{array}$$

0 1 1



Parte Fraccionaria: 011

Por lo tanto: $42,375)_{10} = 101010,011)_2$

Conversión a Base Decimal

Tomamos los dígitos y los multiplicamos por la base elevada a la posición que ocupa cada dígito:

Ej: Pasar el número $204)_8$ a “**Decimal**”:

$$\begin{aligned} 204)_8 &= 2 * 8^2 + 0 * 8^1 + 4 * 8^0 = \\ &= 2 * 64 + 0 + 4 * 1 = \\ &= 128 + 0 + 4 = 132)_{10} \end{aligned}$$

Ejercicio:

$$10101)_2 = ???$$

Pasaje de una Base a otra en forma directa

El pasaje de **2** a **8** o **16**, o el pasaje en sentido contrario, se puede realizar en forma mecánica, sin necesidad de realizar los pasos anteriores.

Cuando una base es potencia de otra, se reemplaza cada dígito por su equivalente numérico en la segunda base.

Sistemas Potencia de “2”: 2^n

$n = 1 \rightarrow 2^1 = 2 \rightarrow$ (BINARIO)

$n = 3 \rightarrow 2^3 = 8 \rightarrow$ (OCTAL)

$n = 4 \rightarrow 2^4 = 16 \rightarrow$ (HEXADECIMAL)

cada dígito en la base se
representa con “**n**” dígitos
binarios

Conversión de Binario a Octal por Agrupación

El pasaje de **2** a **8** o **16**, o el pasaje en sentido contrario, se puede realizar en forma mecánica, sin necesidad de realizar ningún cálculo.

Cuando una base es potencia de otra (ya sea en forma positiva o en forma negativa) como por ejemplo lo son las bases **8** y **2** (**$8 = 2^3$**) *se reemplaza cada dígito por su equivalente numérico en la segunda base.*

Demostración para el pasaje de base 2 a 8

$$N)_b = 110\ 101, 110)_2 =$$

$$= 1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 \quad + \quad 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \quad + \quad 1 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3}$$

$$= [1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0] \quad + \quad [1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0] \quad + \quad [1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0]$$

$$= [4 \quad + \quad 2 \quad + \quad 0] \quad + \quad [4 \quad + \quad 0 \quad + \quad 1] \quad + \quad [4 \quad + \quad 2 \quad + \quad 0]$$

$$= \quad 6 \quad \times \quad 8^1 \quad + \quad 5 \times 8^0 \quad + \quad 6 \times 8^{-1}$$

$$= \quad 6 \quad 5 \quad , \quad 6)_8$$

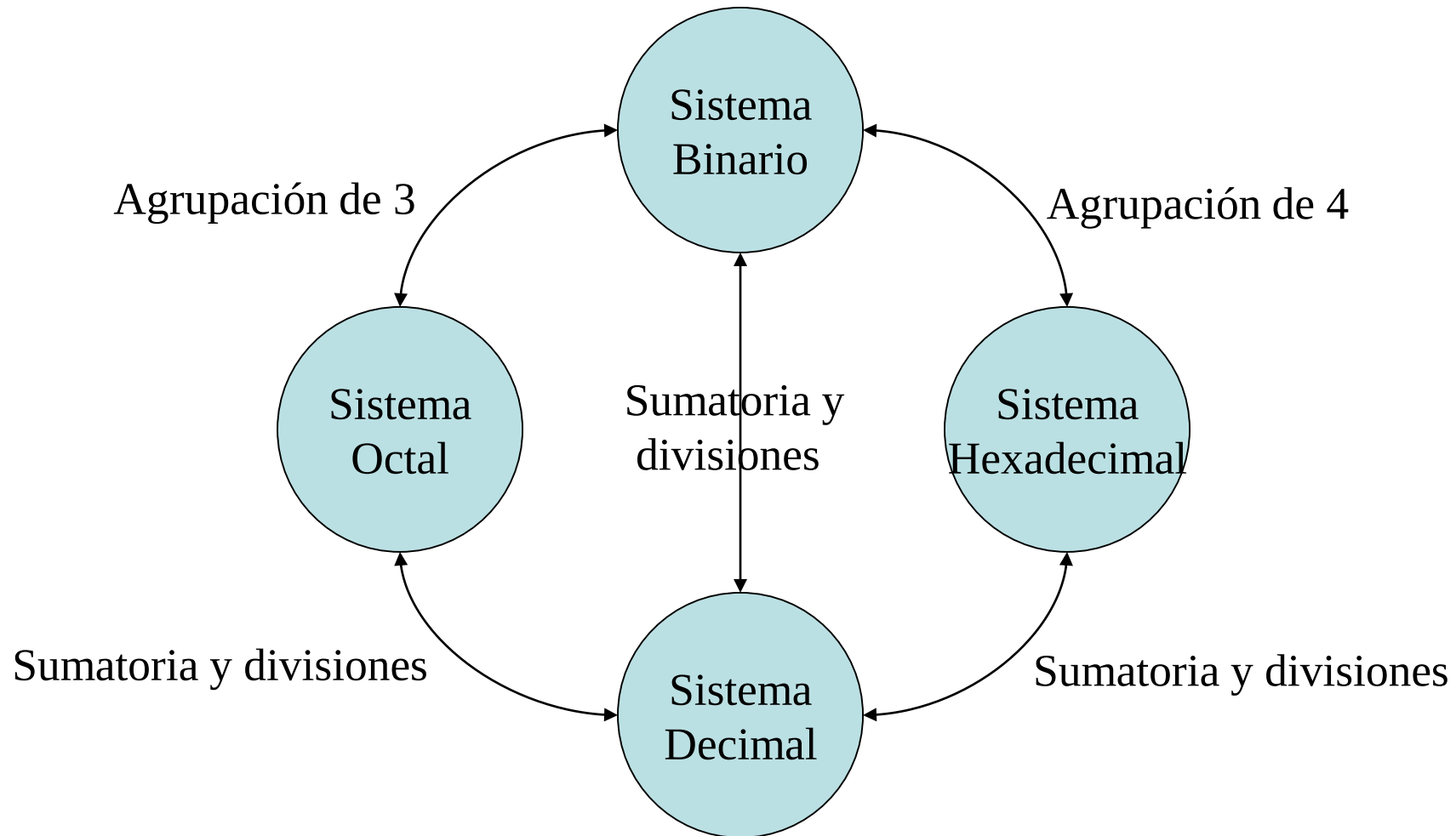
Conversión de Binario a Hexadecimal Por agrupación

$0110\ 1101\ 0010\ ,\ 0101\ 1010\ 0010\]_2$
 $\underbrace{\hspace{1.5em}}\underbrace{\hspace{1.5em}}\underbrace{\hspace{1.5em}}\underbrace{\hspace{1.5em}}\underbrace{\hspace{1.5em}}\underbrace{\hspace{1.5em}}_6\ D\ 2\ ,\ 5\ A\ 2\]_{16}$

Conversión de Hexadecimal a Binario Por agrupación

$A\ 2\ ,\ 1\ 7\]_{16}$
 $\underbrace{\hspace{1.5em}}\underbrace{\hspace{1.5em}}\underbrace{\hspace{1.5em}}\underbrace{\hspace{1.5em}}_{1010\ 0010\ ,\ 0001\ 0111\]_2$

Conversión de bases



Ejercicios

- Pasar a **Binario**

$$204)_8 = ???$$

$$A3E5)_{16} = ???$$

- Pasar a **Octal**

$$1F4)_{16} = ???$$

$$101010)_2 = ???$$

- Pasar a **Hexadecimal**

$$4275)_8 = ???$$

$$11110010)_2 = ???$$

Aritmética binaria

- Una computadora digital consume la mayor parte de su vida útil en la realización de operaciones de aritmética binaria.
- Las reglas de la aritmética binaria son similares a las de la aritmética decimal. Los conceptos de acarreo y préstamo, por ejemplo, se aplican también a la aritmética binaria.
- Todas las operaciones aritméticas estudiadas en el álgebra aplicadas a números al **Sistema Decimal**, son aplicables a los **Números Binarios** dado que ambos Sistemas responden al mismo algoritmo de numeración.

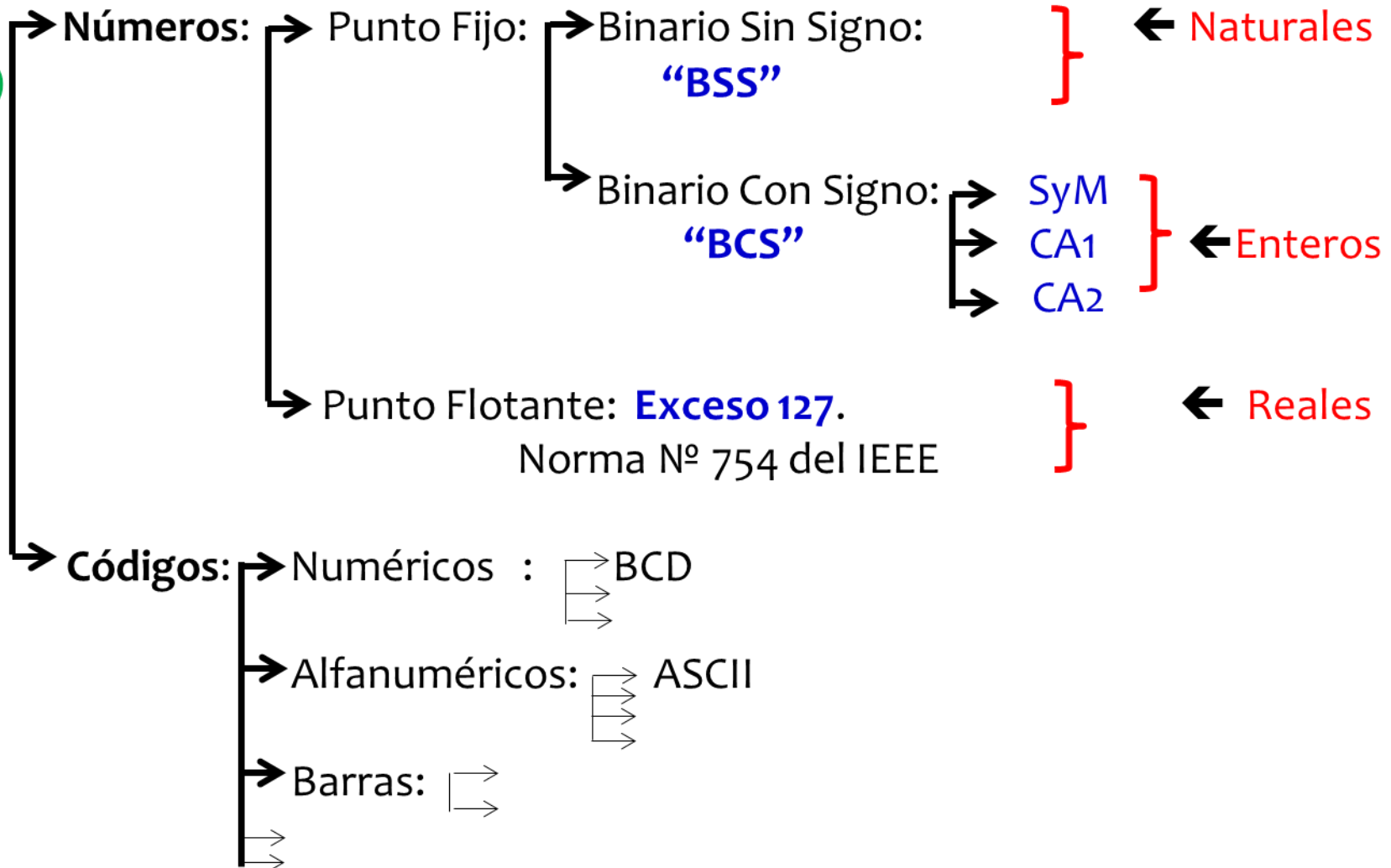
Operaciones de aritmética binaria

Todas las operaciones aritméticas estudiadas en el álgebra aplicadas a números al **Sistema Decimal**, son aplicables a los **Números Binarios** dado que ambos Sistemas responden al mismo algoritmo de numeración.

Dígitos		Suma		Resta		Multiplicación
a_1	a_2	$a_1 + a_2$		$a_1 - a_2$		$a_1 \cdot a_2$
		Suma	Acarreo	Diferencia	Préstamo	Producto
0	0	0	0	0	0	0
0	1	1	0	1	1	0
1	0	1	0	1	0	0
1	1	0	1	0	0	1

Representación de

Datos:
("0 y 1")



Números sin Signo (BSS)

En binario para “n” cantidad de bits podemos expresar esto:

Número mínimo = 0

Número máximo = $2^n - 1$

Rango = 2^n

En Binario y Sin Signo (BSS) según la cantidad de bits tenemos

Con 4 bits puedo representar 16 números y el 15 es el mayor.

Con 8 bits puedo representar 256 números y el 255 es el mayor.

Con 10 bits puedo representar 1.024 números y el 1.023 es el mayor.

Con 12 bits puedo representar 4.096 números y el 4.095 es el mayor.

Con 16 bits puedo representar 65.536 números y el 65.535 es el mayor.

Con 32 bits puedo representar 4.294.967.296 números y el 4.294.967.295 es el mayor.

Con 64 bits puedo representar 18.446.744.073.709.551.616 números

Representación de Números con Signo (BCS)

- Con n bits, **1 bit** representa al **signo** y **$n-1$ bits** a la **magnitud**

b_{n-1}	b_{n-2}	b_0
Signo	Magnitud	

- El bit **$n-1$ (extremo izquierdo)** representa sólo al **signo**
- Los bits **0 a $n-2$** la **magnitud**
- ✓ Un **0** en el bit de signo indica que el número es positivo
- ✓ Un **1** en el bit de signo indica que el número es negativo
- ✓ Los bits **0 a $n-2$** representan el valor absoluto en binario
- ✓ El rango: depende de la convención (**SyM, Ca2 o Ca1**)

Representación de Números con Signo (BCS)

En matemáticas, los números negativos en cualquier base se representan del modo habitual (**precediéndolos con el símbolo “-”**)

Sin embargo, en una computadora existen varias formas de representar el signo de un número:

- **Signo y Módulo**
- **Complemento a la Base**
- **Complemento a la Base-1**
- **Exceso**

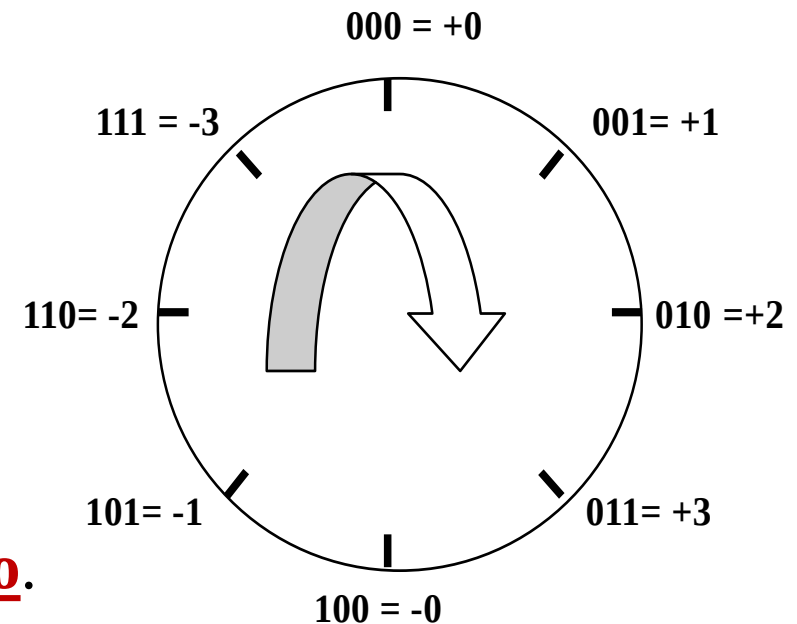
Representación en Signo y Módulo

Para representar un número signado de “**n bits**” se realiza lo siguiente:

1- Asignar un bit para representar el signo (**el MSB - Most Significant Bit o Bit Más Significativo**) y por convención:

- Si es igual a “**0**”: indica un número **positivo**.
- Si es igual a “**1**”: indica un número **negativo**.

2- Los “**n-1 bits**” restantes se utilizan para representar el significado, que es la **magnitud del número en valor absoluto**.



Este enfoque es directamente comparable a la forma habitual de mostrar el signo (colocando “**+**” o “**-**” al lado de la magnitud del número).

Ejemplo de Signo y Módulo

Sea una representación que nos permite codificar un número entero en binario con **8 bits (un byte)**

Esto nos otorga **1 bit para el signo** y **7 bits para la magnitud**.

8 bits permiten representar en teoría al menos **$2^8 = 256$ números**.

128 números positivos (Bit de Signo en “0”)

128 números negativos (Bit de Signo en “1”)

Convención Signo y Modulo

➤ Ejemplo:

✓ $+5)_{10} = 0000\ 0101)_2$

64 32 16 8 4 2
1 ↑ ↑ ↑ ↑ ↑ ↑

5

+

✓ $+25)_{10} = 0001\ 1001)_2$

64 32 16 8 4 2 1

16 + 8 + 1 = 25

$-5)_{10} = 1000\ 0101)_2$

5

-

$-25)_{10} = 1001\ 1001)_2$

Tabla para n=8 (8 bits) en Signo y Módulo

Valores de 8 Bits	Interpretado en Signo y Módulo en Decimal	Interpretado como entero sin Signo y Módulo
00000000	0	0
00000001	1	1
00000010	2	2
00000011	3	3
.....		
01111110	126	126
01111111	127	127
10000000	-0	128
10000001	-1	129
10000010	-2	130
10000011	-3	131
.....		
11111101	-125	253
11111110	-126	254
11111111	-127	255

Ejemplo: Representar el número -97_{10}

Pasos:

1- Como el número -97 es **negativo** $\Rightarrow$ deberá llevar como Bit de Signo “1”

2- Realizar la conversión en Valor Absoluto de $-97_{10} \Rightarrow |-97_{10}| = 97_{10}$
Que en Binario corresponde al número 1100001_2

3- Armar el número, el número -97_{10} en binario con formato de **Signo y Módulo** es :

$11100001_2 \Rightarrow$ Donde el **1** en el **MSB** indica un número **negativo**, y

1100001_2 es el significado en **Valor Absoluto**.

Desventajas de la Representación en Signo y Módulo

(Siguiendo el ejemplo de un número de 8 bits)

- * **Es más complejo operar aritméticamente.**

Para realizar una suma por ejemplo: Primero hay que determinar si los dos números poseen el mismo signo, y en caso de ser así, realizar la suma de la parte significativa, pero en caso contrario, restar el mayor del menor y asignar el signo del mayor.

- * **Posee la doble representación del Cero.**

Aparece el cero signado:

00000000₂ (+0₁₀)

10000000₂ (-0₁₀)

Ventajas de la Representación en Signo y Módulo

(Siguiendo el ejemplo de un número de 8 bits)

* **Posee un rango simétrico.**

Los número van desde el $+127_{10} = 01111111_2$, Pasando por el $+0_{10} = 00000000_2$ y el $-0_{10} = 10000000_2$, hasta el $-127_{10} = 11111111_2$

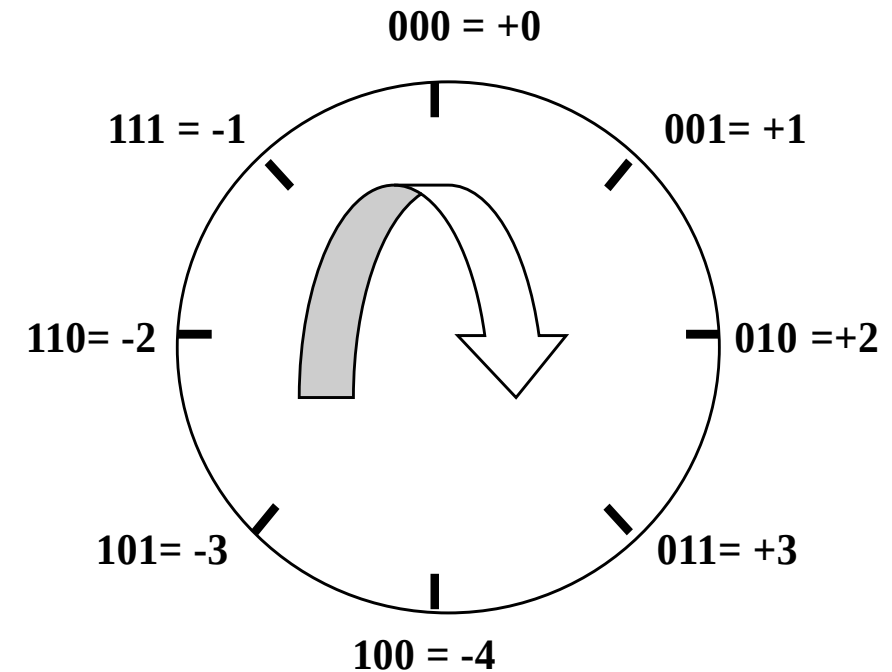
En forma general:

Para “**n bits**”, el rango (en decimal) para **Signo y Módulo** es:

$$(-2^{n-1} - 1; 2^{n-1} - 1) \text{ o bien } +/- 2^{n-1} - 1$$

Complemento a la Base (C_{a2})

- El complemento a la base de un número N , es lo que lo falta a N para llegar al modulo 10^n .
- Los números positivos comienzan con **cero** (0)
- Los negativos comienzan con **uno** (1)
- El rango es asimétrico y va desde $-(2^{n-1})$ a $+(2^{n-1}-1)$
- Existe una sola representación del **cero** (0)



Expresión General: $N + C_B = 10^n \rightarrow C_B = 10^n - N$

Complemento a la Base (C_{a2})

$$N + C_B = 10^n \Rightarrow C_B = 10^n - N$$

$$C_{a2} = 2^n - N$$

► Ejemplo:

mplo:

$4 + 1 = 5$

64 32 16 8 4 2 1

↑ ↑ ↑ ↑ ↑ ↑ ↑

✓ + 5)₁₀ = **0**000 0101)₂ -5)₁₀ = **1**111 1011)₂

Regla práctica:

Tomamos el numero positivo y de derecha a izquierda se copia hasta el primer “1” y luego invertimos todos los demás, 0 por 1 y 1 por 0.

Esto se puede realizar para pasar de Nro. positivo al negativo y también para pasar de negativo al positivo

✓ $+25)_{10} = 0001\ 1001)_2$ $-25)_{10} = 1110\ 0111)_2$

64 32 16 8 4 2 1

16 + 8 + 1 = 25

Ejemplo (en Binario):

Dado $+2_{10}$ expresar su C_2 con 3 bits:

$$+2_{10} = 010_2$$

$$2^n = 2^3 = 1000_2$$

$$C_n = 10^n - N$$

$$C_2 = 2^3 - N = 2^3 - 2 = 1000_2 - 010_2$$



$$\begin{array}{r} 1000_2 \\ - 010_2 \\ \hline 110_2 \end{array}$$

Regla de Nemotécnica:

$$C_b = C_{b-1} + 1$$

Operaciones en Complemento a 2

Suma en Complemento a 2

- Los dos números en una suma se denominan **sumandos**.
- El resultado es la suma de ambos.
- Cuando se suman dos números binarios con signo pueden producirse cuatro casos:
 1. **Ambos números son positivos.**
 2. **El número positivo es mayor que el negativo en valor absoluto.**
 3. **El número negativo es mayor que el positivo en valor absoluto.**
 4. **Ambos números son negativos.**

Operaciones en Complemento a 2

Suma en Complemento a 2

- Ambos números son positivos:

$$\begin{array}{r} 00000111 \\ + 00000100 \\ \hline 00001011 \end{array} \quad \begin{array}{r} 7 \\ + 4 \\ \hline 11 \end{array}$$

- La suma es positiva y, por tanto, es un número binario real (no complementado).

Operaciones en Complemento a 2

Suma en Complemento a 2

- El número positivo es mayor que el número negativo en valor absoluto:

$$\begin{array}{r} 00001111 \quad 15 \\ + 11111010 \quad -6 \\ \hline 1\,00001001 \quad 9 \end{array}$$

Acarreo que se descarta →

- El bit de acarreo final no se tiene en cuenta. La suma es positiva y, por tanto es un número binario real (no complementado).

Operaciones en Complemento a 2

Suma en Complemento a 2

- El número negativo es mayor que el número positivo en valor absoluto:

$$\begin{array}{r} 00010000 \\ + 11101000 \\ \hline 11111000 \end{array} \quad \begin{array}{r} 16 \\ + -24 \\ \hline -8 \end{array}$$

- La suma es negativa y, por tanto, está en complemento a 2.

Operaciones en Complemento a 2

Suma en Complemento a 2

- Ambos números son negativos:

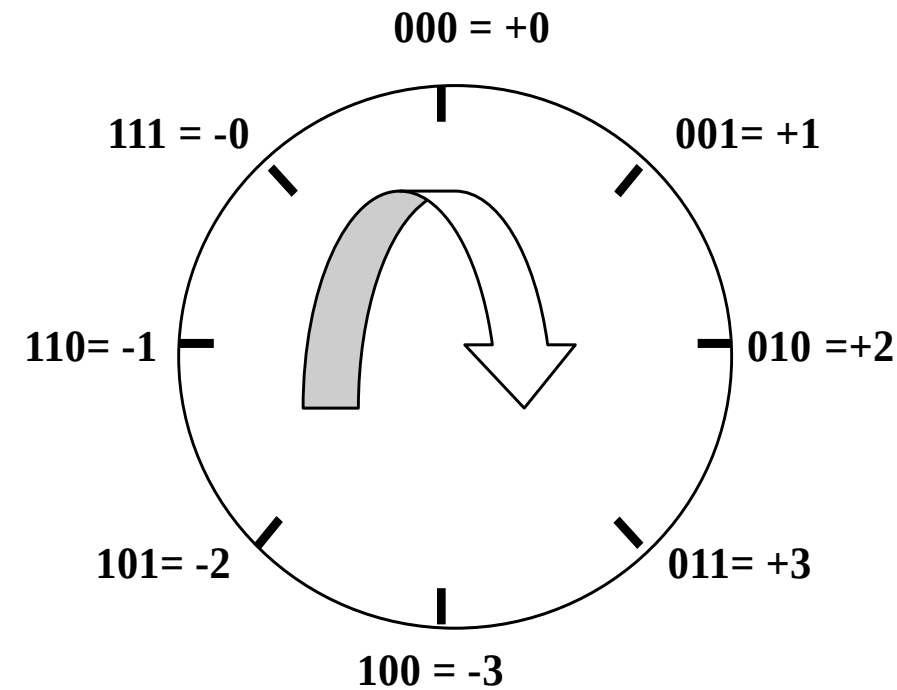
$$\begin{array}{r} 11111011 \quad -5 \\ + 11110111 \quad + -9 \\ \hline 11110010 \quad -14 \end{array}$$

Acarreo que se descarta → 1

- El bit de acarreo final no se tiene en cuenta. La suma es negativa y, por tanto, está en complemento a 2.

Complemento a la Base-1 (C_{a1})

- El **complemento a la base menos uno** de un número N , es lo que lo falta a N para llegar a el modulo 10^{n-1}
 - En **Base 2** se da una regla práctica en donde el C_{a1} de un número se obtiene invirtiendo todos los bits
 - Si el número es positivo, los n bits poseen la representación binaria del número (como siempre)
 - Si el número es negativo, los n bits poseen el C_{a1} del valor deseado.
- ✓ Los n bits representan al número.
 - ✓ Los positivos empiezan con **cero** (0)
 - ✓ Los negativos empiezan con **uno** (1)
 - ✓ El rango: **$-(2n-1-1)$ a $+(2n-1-1)$ con doble representación del cero (+0 y -0)**



Expresión General:

$$N + C_{B-1} = 10^n - 1 \rightarrow$$

"n= Cantidad de Bits"

$$C_{B-1} = (10^n - 1) - N$$

Complemento a la Base-1 (C_{a1})

$$C_{B-1} = (10^n - 1) - N_{10}$$

$$C_{1a} = (2^n - 1) - N_2$$

➤ Ejemplo:

$4 + 1 = 5$
 64 32 16 8 4 2 1
 ↑ ↑ ↑ ↑ ↑ ↑ ↑
 ✓ $+ 5)_{10} = 0000\ 0101)_2$ $-5)_{10} = 1111\ 1010)_2$

Regla práctica:

Se invierten todos los bits (0 por 1 y 1 por 0).

Esto se puede realizar para pasar de Nro. positivo al negativo y también para pasar de negativo al positivo

✓ $+25)_{10} = 0001\ 1001)_2$ $-25)_{10} = 1110\ 0110)_2$
 ↓ ↓ ↓ ↓ ↓ ↓ ↓
 64 32 16 8 4 2 1
 16 + 8 + 1 = 25

Ejemplo (en Binario):

(C_{a1})

Dado $+2_{10}$ expresar su **C_{a1}** con 3 bits:

$$+2_{10} = 010_2 \quad (\text{con 3 bits})$$

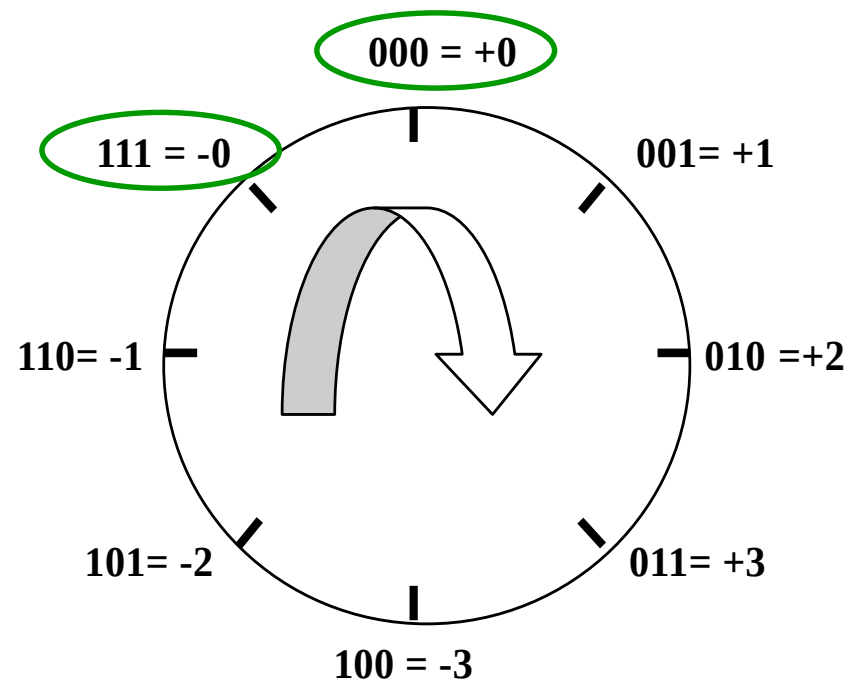
$$2^n = 2^3 = 1000_2$$

$$\boxed{C_{a1} = (10^n - 1) - N}$$

$$2^n - 1 = 2^3 - 1 = 1000_2 - 1_2 = 111_2$$

$$C_1 = 2^3 - 1 - 2 = 111_2 - 010_2$$

$$\begin{array}{r} 111_2 \\ - 010_2 \\ \hline \boxed{101_2} \end{array} \Rightarrow C_1 \text{ de } +2_{10}$$



- Se puede observar la “**Doble Representación del Cero**” (**000** y **111**), además de una propiedad particular por la disposición de los números que es la Siguiente:
- Dado un número **POSITIVO** o **NEGATIVO** de “n” bits, su complemento tiene los “n” bits todos invertidos. Ej.: $+1_{10} = 001_2$ y $-1_{10} = 110_2$
- **Regla de Conversión:**

**DADO EL NÚMERO “N”, SU COMPLEMENTO A LA BASE MENOS 1
POSEE TODOS SUS BITS INVERTIDOS.**

Complemento a la Base-1 (C_{a1})

$$C_{B-1} = (10^n - 1) - N$$

➤ Ejemplo:

Dada una cadena de bits ¿qué número decimal representa si lo interpretamos en Ca_1 ?

✓ Cuando es positivo:

$$0000\ 0101 = 1 \times 2^2 + 1 \times 2^0 = 4 + 1 = +5$$

✓ Cuando es negativo, saco el Ca_1 del número y **obtengo el positivo** y luego **cambio el signo**

$$1111\ 1010 \rightarrow 0000\ 0101 = +5_{10}$$

$$1111\ 1010 = -5_{10}$$

Operaciones en Complemento a 1

Resta o Sustracción en Complemento a 1

- La sustracción es un caso especial de la suma.
- Por ejemplo, restar +6 (el **sustraendo**) de +9 (el **minuendo**) es equivalente a sumar -6 a +9.
- Básicamente la operación de la sustracción cambia el signo del sustraendo y le suma al minuendo.
- El resultado de una sustracción se denomina **diferencia**.
 - El signo de un número binario positivo o negativo se cambia calculando su complemento a 1.
 - Para restar dos números con signo se calcula el complemento a 1 del sustraendo y se suman. Cualquier bit de acarreo final se suma al LSB (de más a la derecha).

Operaciones en Complemento a 1

Resta o Sustracción en Complemento a 1

- Ejemplo:

Forma normal

00111111
- 00011100

Forma en complemento a 1

00111111 63₁₀
+ 11100011 - 28₁₀

100100010

Acarreo final → 1

00100011 35₁₀

- Si no hubiera un bit de acarreo final, entonces el resultado es un número negativo representado en la forma de complemento a 1.

Operaciones en Complemento a 1

Resta o Sustracción en Complemento a 1

- **Ejemplo:** Restar 63_{10} de 28_{10} .

Forma normal		Forma en complemento a 1
00011100		00011100 28_{10}
- 00111111	+	<u>11000000</u> <u>- 63_{10}</u>
No hay acarreo final → 11011100 - 35_{10}		

- No se tiene un bit de acarreo final, por tanto el resultado es un número negativo en complemento a 1.
- Se debe determinar su complemento a 1 para obtener su magnitud; en este caso es: 00100011 o 35_{10} .
- Como su signo es negativo, el resultado real es: -35_{10} .

Representación de números sin/con signo, para 3 bits

BSS			BCS					
			S y M			Ca1		
						Ca2		
7	111	2^{n-1}						
6	110					$C_{a1} = 10^n - 1 - N_o$		
5	101					Se invierten todos los bits		
4	100					De derecha a Izq. se copia hasta el 1ºbits luego se invierten todos		
3	011		+3	011	$+ (2^{n-1} - 1)$	+3	011	$+ (2^{n-1} - 1)$
2	010		+2	010		+2	010	
1	001		+1	001		+1	001	
0	000	0	0	000	0	0	000	0
			-1	101		-1	111	
			-2	110		-2	110	
			-3	111	$- (2^{n-1} - 1)$	-3	101	
						-4	100	$- (2^{n-1})$

Representación en Punto Flotante

- Es una convención utilizada para trabajar con números Reales de alto rango, o sea, para poder trabajar con números de mayor cantidad de bits de los que posee la PALABRA de la máquina.
- Los números Naturales y Enteros que vimos hasta ahora se denominan “**Punto Fijo**” o “**Coma Fija**”, y si por ejemplo nuestra computadora puede operar solamente con 3 dígitos, podríamos trabajar con 1000 combinaciones.
- Sería como una representación exponencial de los números decimales Convencional, semejante a la notación científica.

Ej. : $12,3 \times 10^{+6}]_d$ o $3,25 \times 10^{-8}]_d$ $\Rightarrow$

$$\mathbf{Nd = \pm m * 10^{\pm P}}$$

Punto Flotante

Exceso 127 IEEE Normalizado

- Utiliza para la escritura Base 2 para las fracciones y notación en exceso para los exponentes.
- La norma define tres formatos: Precisión Sencilla (32 bits), Precisión Doble (64 bits) y Precisión Extendida (80 bits).
- Utilizaremos el formato de **Precisión Sencilla (32 bits)**:
 - ✓ Se inicia con **1 bit de signo**, (**0** = Positivos y **1** = Negativos)
 - ✓ **8 bits para el exponente** (desplazado en 127)
 - ✓ **23 bits para la mantisa**

Posee el siguiente formato:

- Partiendo del $N^o = +/- m * b^c$
- Donde “**m**”: Mantisa y “**b**”: La base en la cual se encuentra expresado el nro.
- La característica $C = 127 +/- p$
- La normalización se realiza de la siguiente forma: $m = 1, F = 1, X_1 X_2 X_3 \dots X_n$

Reglas básicas para la norma:

- ✓ La representación se realiza en binario y de la forma: $+/- 1, F * 10_2^{+/-p}$ donde $F = X_1 X_2 X_3 \dots X_n$
- ✓ En precisión simple se utilizan **32 bits (4bytes)**
- ✓ La parte fraccionaria se representa en los **últimos 23 bits**, sobreentendiéndose que posee un “**1**” en la parte entera, **que no se representa en su normalización.**
- ✓ **El primer bit del primer byte dará el signo de la mantisa**, tratándose de representar la fracción como un número del tipo **Signo y Modulo.**
- ✓ Al **exponente** se lo desplaza **127**, dando la característica “**+/- p + 127**” y se anotan en los **8 bits** siguientes al signo.
- ✓ El número “**0**” se representa como “**+0**”.

Formato:

S	CARACTERISTICA	MANTISA
S	E E E E E E E E	X ₁ X ₂ X ₃ X _n
1 bit	8 bits	23 bits

Total= 1 + 8 + 23 = 32 bits

Ej:

$5_{10} = 101_2 = +1,01 * 10_2^{10} \Rightarrow$ Se desplazó la coma 2 lugares a la izquierda para normalizar

$C = 127 +/- p = 127 + 2 = 129_{10} = 10000001_2$

S	CARACTERISTICA	MANTISA
0	1 0 0 0 0 0 0 1	0 1 0

Signo

Exceso 127 IEEE Normalizado

$$N_o = \pm m 10^{\pm e}$$

Signo de mantisa → +/- = 0/1

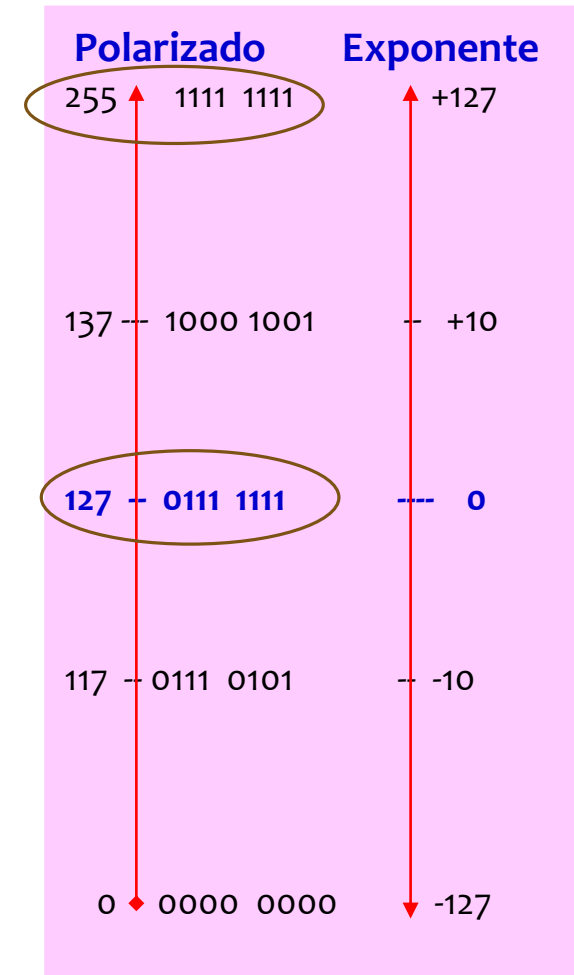
Polarizado

de exponente

Normalizado de mantisa

$$C = 127 \pm e ; 1 \leq m < 2)_d \rightarrow m = 1, b_1 b_2 b_3 \dots b_n$$

Precisión SIMPLE (32 blts) - EXCESO 127				
Signo	Exponente		Mantisa	
0/1	Polarizado ("C")		Normalizada	
S	C C C C	C C C	C b ₁ b ₂ b ₃ b ₄ b ₅ b ₆ b ₇	B ₈ b ₉ b ₁₀ b ₁₁ b ₁₂ b ₁₃ b ₁₄ b ₁₅ b ₁₆ b ₂₃
1 byte		1 byte	1 byte	1 byte



Punto Flotante - Exceso 127

Los valores extremos que se utilizan para representar :

- *Nos permite manejar números desde ± 38 dígitos y lograr una exactitud entre 6 y 7 dígitos:*
- con **C = 1111 1111** y **m=0**, se representa el $\pm$ infinito según el bit de signo
- con **C= 1111 1111** y **m <> 0**, se indica operaciones no válidas, ej. 0 * infinito
- con **C=0000 0000**, **m=0** y el bits de signo de valor 1 se representa el cero.
- con **C=0000 0000** y **m<>0**, el N^o esta desnormalizado.

BANDERAS (FLAGS)

SIGNO: (S)

S=0 Resultado positivo.

S=1 Resultado negativo.

CERO: (Z)

Z=0 Resultado $\neq 0$

Z=1 Resultado $= 0$

S	Z	C	V
---	---	---	---

CARRY: (C)

C=0 No existe acarreo.

C=1 Existe acarreo.

OVERFLOW: (V)

V=0 No existe overflow. (si sumando 2 nros. de igual signo y el resultado no cambia de signo, o si se suman 2 nros. de signos distintos)

V=1 Existe overflow. (si sumando 2 nros. de igual signo y resultado cambia de signo, quedando el resultado truncado)

BANDERAS (FLAGS)

Ejemplo:

$$\begin{array}{r} \\ + \\ \hline 1 \end{array}$$

V	S	C	Z
1	0	1	0

BCD – Código de Representación Decimal

BCD es un acrónimo de *Código Binario Decimal*

- Los Códigos BCD son convenciones que permiten la representación de números decimales (0 a 9) en bloques binarios de 4 bits.
- Son códigos de representación de números denominados “**Códigos Ponderados**” porque adjudican cierto peso a los “**1**” binarios según la posición que ocupan en el bloque. (La suma de los pesos de cada combinación debe ser igual al número decimal representado).
- En esta notación se utilizan **grupos de 4 bits** para representar cada dígito decimal del **0** al **9**. Con este método podemos representar dos dígitos por byte de información.
- Este método es mucho mas práctico para la representación de números en la memoria en comparación al método ASCII, todavía se queda por debajo del binario, ya que con un byte en el método BCD solo podemos representar dígitos del 0 al 99, en cambio, en formato binario podemos representar todos los dígitos desde 0 hasta 255.

BCD – Código de Representación Decimal

Casos de uso:

- Este formato es utilizado principalmente para representar números muy grandes en aplicaciones mercantiles ya que facilita las operaciones con los mismos evitando errores de redondeo.
- Sigue siendo ampliamente utilizado para almacenar datos, en aritmética binaria o en electrónica. Los números se pueden mostrar fácilmente en visualizadores de siete segmentos enviando cada cuarteto BCD a un visualizador.
- La BIOS de un ordenador personal almacena generalmente la fecha y la hora en formato BCD.

BCD – Código de Representación Decimal

La ventaja del código **BCD** frente a la representación binaria clásica es que no hay límite para el tamaño de un número.

Los números que se representan en formato binario están generalmente limitados por el número mayor que se pueda representar con 8, 16, 32 o 64 bits.

Por el contrario, utilizando **BCD**, añadir un nuevo dígito solo implica añadir una nueva secuencia de 4 bits.

$$137_{10} = 10001001_2 \quad \text{Binario}$$

$$137_{10} = 0001 \ 0011 \ 0111 \quad \text{BCD}$$

BCD – Código de Representación Decimal

Existen varias clases de códigos **BCD**, veremos el código **BCD Puro (8421)**

Se denomina así porque los pesos en cada bloque coinciden con los 4 primeros pesos del sistema binario puro.

VALOR DECIMAL	BCD PURO 8421
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Sistemas de Representación

Unidades de medida para almacenamiento de datos – Bytes

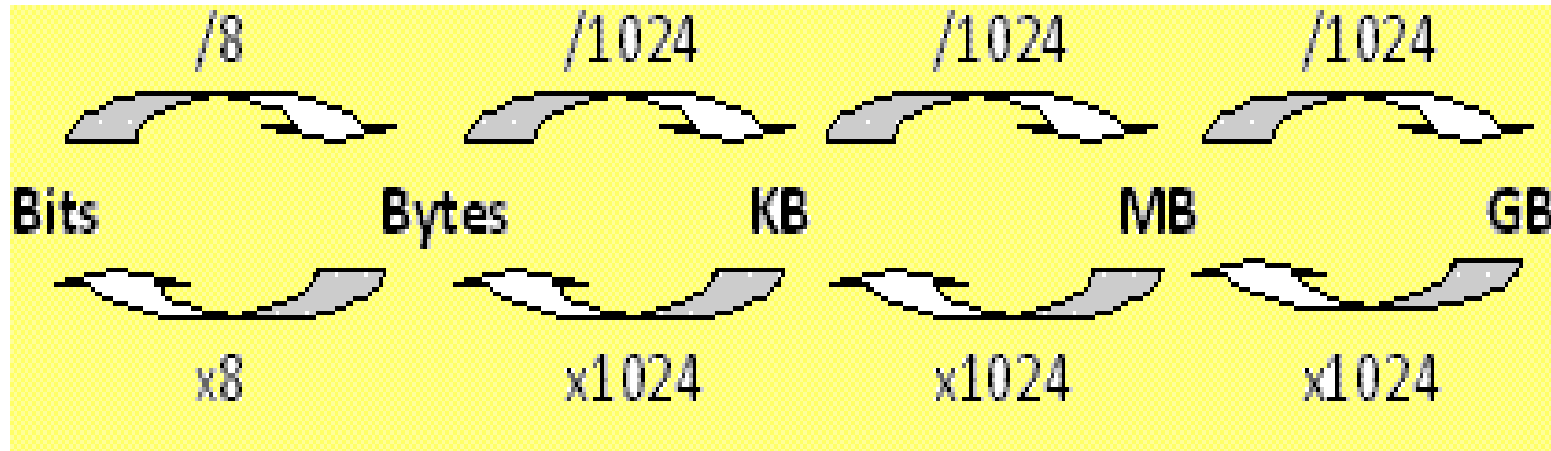
El sistema de medición para almacenamiento de datos se fundamenta en:

- **Un Byte como medida base.**
- **1024 (2^{10}) como factor multiplicador para el incremento.**
- **Los prefijos: Kilo, Mega, Giga, Tera, etc.**

Así:

- **1 KiloByte = 1024 Bytes = 2^{10}**
- **1 MegaByte = 1024 KiloBytes = 2^{20}**
- **1 GigaByte = 1024 MegaBytes = 2^{30}**
- **1 TeraByte = 1024 GigaBytes = 2^{40}**

Conversión entre las Unidades de Información



Ejemplo:

Una película ocupa 1550 MB. ¿Cuántas películas caben en un DVD que tiene 4,7 GB de capacidad?

Solución:

Para pasar de MB a GB se divide una vez por 1024. **$1550 \text{ MB} / 1024 = 1,513671875 \text{ GB}$**
(Obtenemos lo que ocupa una película pero en GB)

Ahora dividimos la capacidad del DVD por lo que ocupa la película.

$4,7 \text{ GB} / 1,513671875 \text{ GB} = 3,1$ (Por lo tanto caben **3 películas en el DVD**)

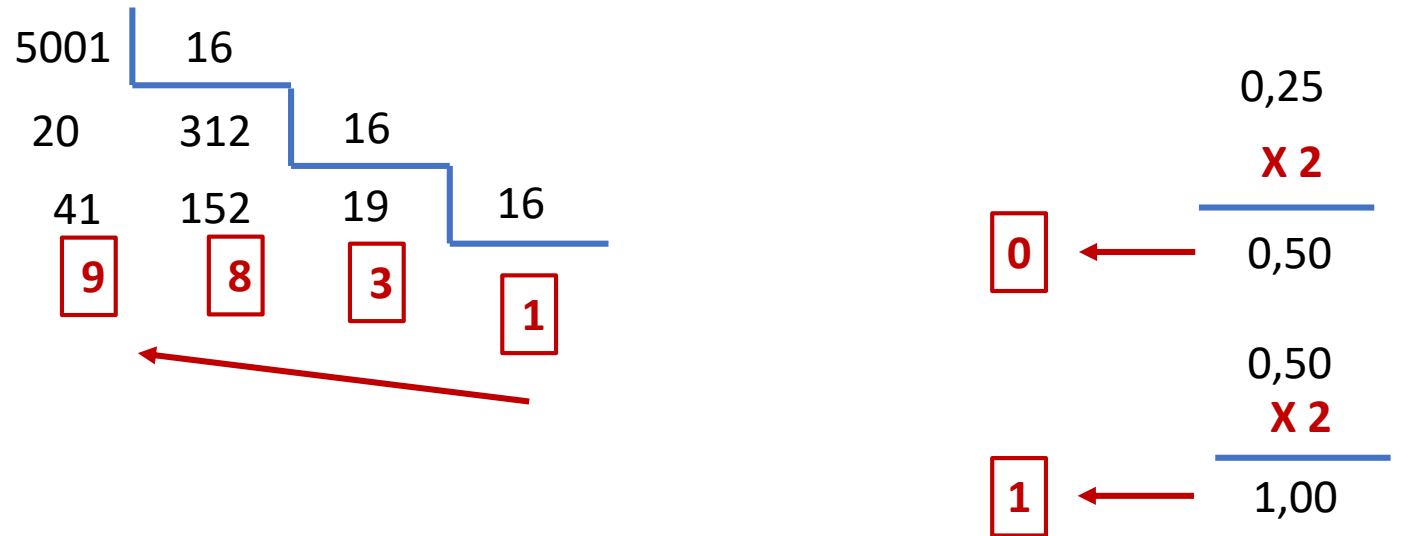
Bibliografía Sugerida

- **Arquitectura de Computadoras. Patricia Quiroga. 1ra Edición**
 - ✓ Capítulos 2, 3 y 4
- **Organización y Arquitectura de Computadores. Stallings, William. 7ma Edición**
 - ✓ Capítulo 9
- **Introducción a la Organización y Arquitectura de la Computadora. Osio, Morales. 1ra Edición. UNAJ.**
 - ✓ Capítulo 1
- **Link de interés**
 - ✓ <http://coderstoolbox.net/number/>
 - ✓ <http://babbage.cs.qc.edu/IEEE-754/>

PUNTO FLOTANTE

EXCESO 128

EJEMPLO: - 5001,25



$$5001)_{10} = 1389)_{16} = \underline{0001} \underline{0011} \underline{1000} \underline{1001}$$

$$- \underline{0001} \underline{0011} \underline{1000} \underline{1001,01})_2$$

PUNTO FLOTANTE

EXCESO 128

EJEMPLO: - 5001,25 = **- 0001001110001001,01**₂

NORMALIZACIÓN

$$\pm m \times 10^{\pm P}$$

$$0,1 < m < 1$$

$$0, X_1 X_2 X_3 \dots X_n$$

$$X_1 \neq 0$$

+ 13

$$\textbf{- 0,100111000100101} \times \textbf{10}^{+1101}$$

PUNTO FLOTANTE

EXCESO 128

+

128

13

141

+

100000000

000001101

100001101

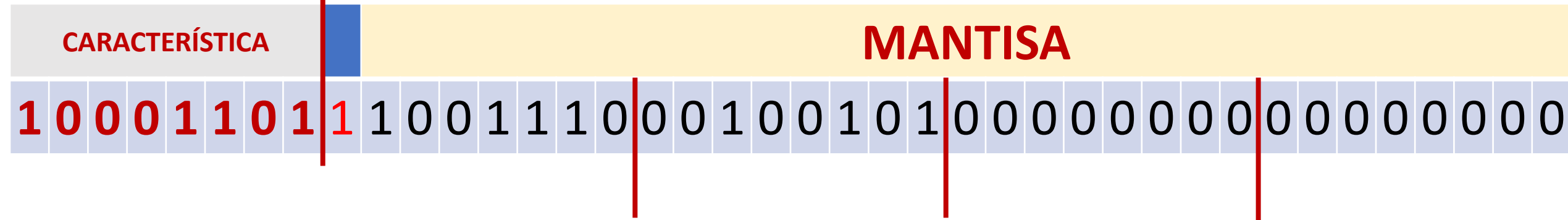
EJEMPLO: - 5001,25= - **0001001110001001,01**)₂

- 0,100111000100101 x **10⁺¹¹⁰¹**

SIGNO DE LA MANTISA

CARACTERÍSTICA

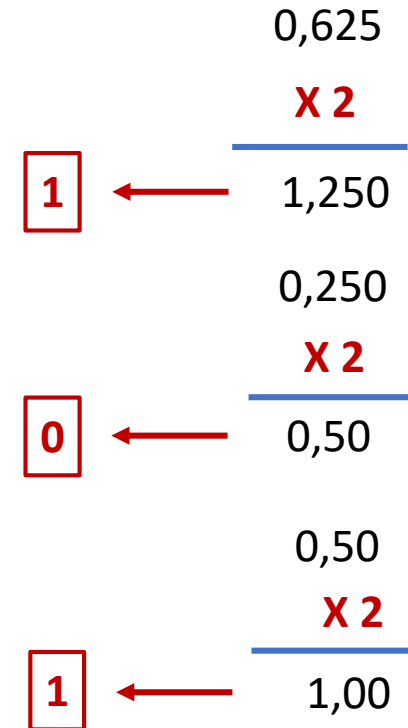
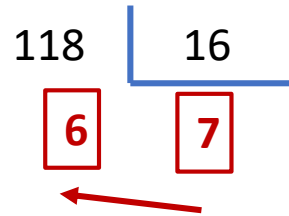
MANTISA



PUNTO FLOTANTE

EXCESO 127

EJEMPLO: - 118,625



$$118)_{10} = 76)_{16} = \mathbf{01110110}$$
$$- \mathbf{01110110,101})_2$$

PUNTO FLOTANTE
EXCESO 127 IEEE COMA FLOTANTE

EJEMPLO: - 118,625 =

NORMALIZACIÓN

$$\pm m \times 10^{\pm P}$$

$$0,1 < m < 1$$

$$0,X_1X_2X_3\ldots\ldots\ldots X_n$$

$$X_1 \neq 0$$

$$\mathbf{0,1}X_2X_3\ldots\ldots\ldots X_n$$

$$\mathbf{1,}X_2X_3\ldots\ldots\ldots X_n$$

$$- \mathbf{01110110,101}_2 \longrightarrow - \mathbf{1,110110101} \times \mathbf{10}^{+110}$$

EXCESO 127 IEEE COMA FLOTANTE

$$\begin{array}{r}
 127 \\
 + \quad 6 \\
 \hline
 133
 \end{array}
 \qquad
 \begin{array}{r}
 01111111 \\
 + \quad 00000110 \\
 \hline
 10000101
 \end{array}$$

$$\begin{array}{r}
 \text{133} \\
 - \text{127} \\
 \hline
 \text{6}
 \end{array}
 \qquad
 \begin{array}{r}
 \text{10000101} \\
 - \text{01111111} \\
 \hline
 \text{00000110}
 \end{array}$$

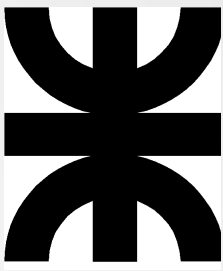
SIGNO DE LA MANTISA



PUNTO FLOTANTE

EXCESO 127 IEEE COMA FLOTANTE

EJEMPLO PARA RESOLVER: $+10,1)_{10}$



UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL LA PLATA

ARQUITECTURA DE COMPUTADORA

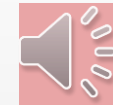
CONCEPTOS

- TRANSMISIÓN DIGITAL EN UN MEDIO GUIADO
- UNIDADES DE ALMACENAMIENTO
- VELOCIDAD DE TRANSMISIÓN



CONCEPTOS BÁSICOS DE LA TRANSMISIÓN DIGITAL

❑ TRANSMISIÓN DIGITAL EN UN MEDIO ALÁMBRICO:



AUDIO

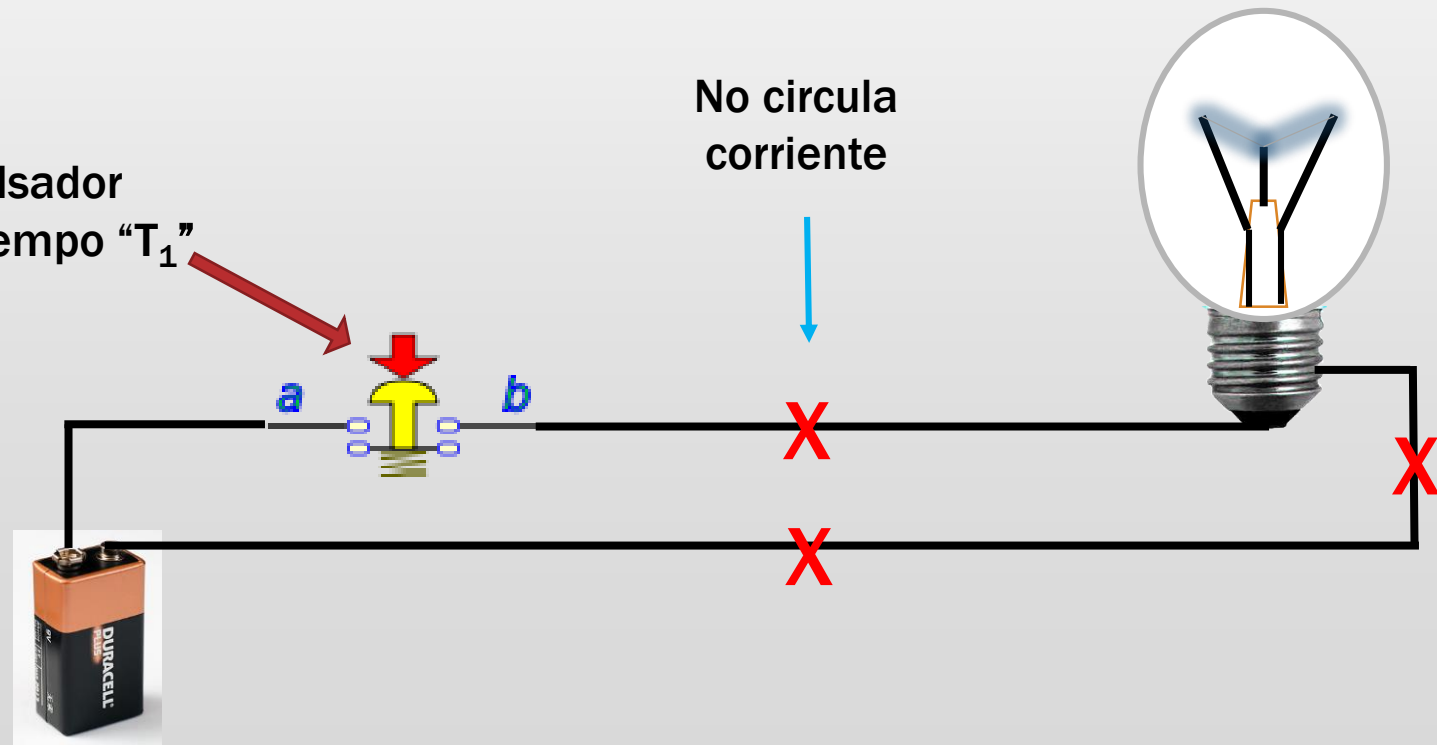
- ✓ ES LA TRANSFERENCIA FÍSICA DE DATOS (UN FLUJO DIGITAL DE BITS) POR UN CANAL DE COMUNICACIÓN. LOS DATOS SE REPRESENTAN COMO UNA SEÑAL DE TENSIÓN ELÉCTRICA.

CONCEPTOS BÁSICOS DE LA TRANSMISIÓN DIGITAL

❑ EJEMPLO 1:

- ✓ SI NO PRESIONAMOS EL PULSADOR DURANTE UN TIEMPO T_1 . NO CIRCULARÁ CORRIENTE Y LA LÁMPARA PERMANECERÁ APAGADA DURANTE ESE TIEMPO T_1 .

NO presionar el pulsador
Durante el intervalo tiempo " T_1 "



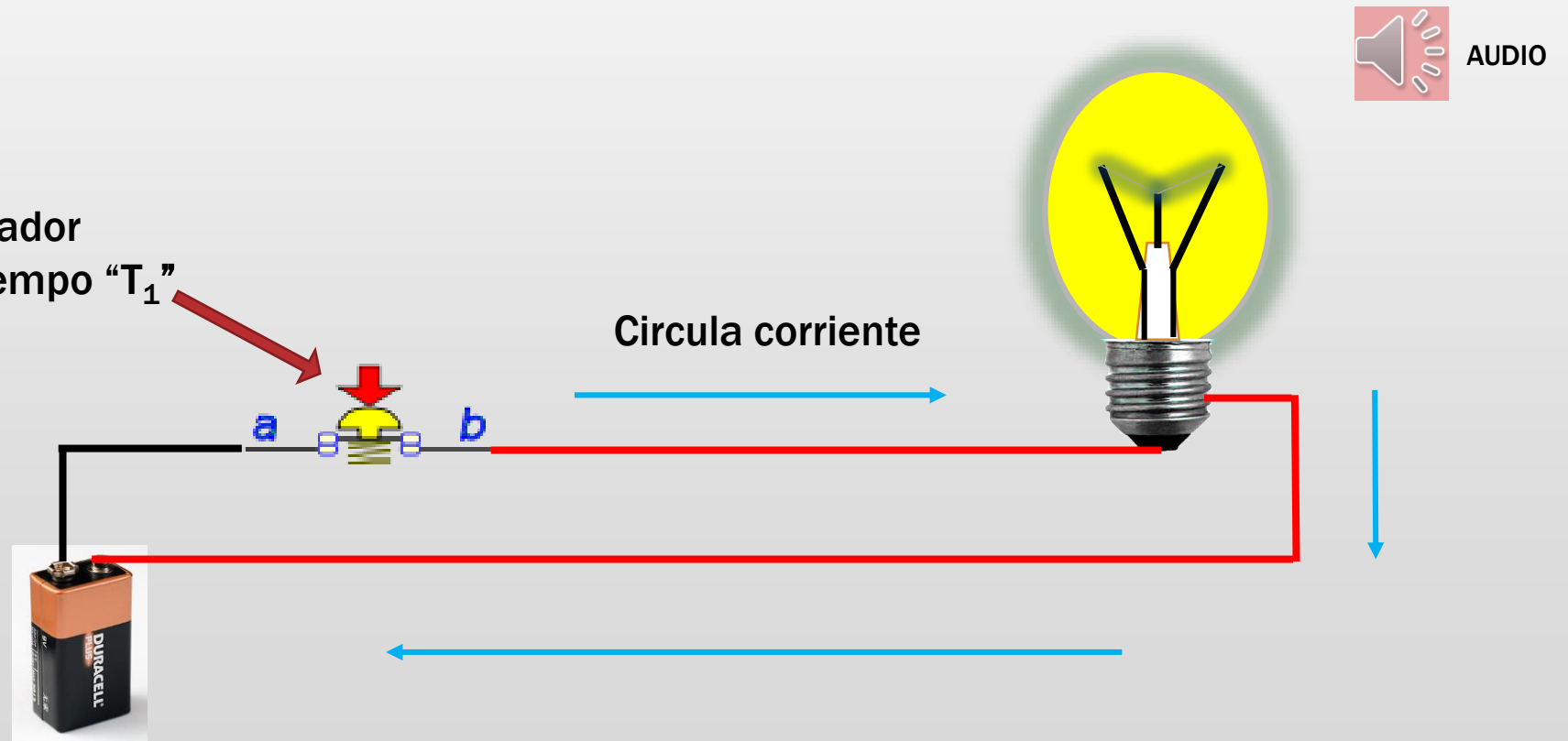
AUDIO

CONCEPTOS BÁSICOS DE LA TRANSMISIÓN DIGITAL

❑ EJEMPLO 2:

- ✓ SI PRESIONAMOS EL PULSADOR DURANTE UN TIEMPO T_1 . CIRCULARÁ CORRIENTE Y LA LÁMPARA PERMANECERÁ ENCENDIDA DURANTE ESE TIEMPO T_1 .

SI Presionar pulsador durante el intervalo tiempo " T_1 "



CONCEPTOS BÁSICOS DE LA TRANSMISIÓN DIGITAL

❑ EJEMPLO 3: SINCRONIZAR LA OPERATORIA:

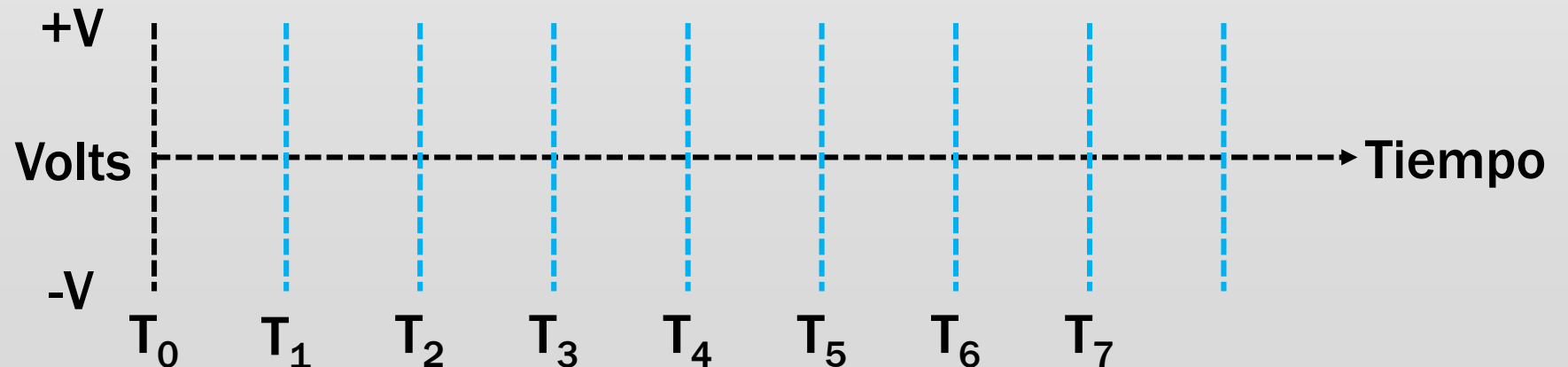
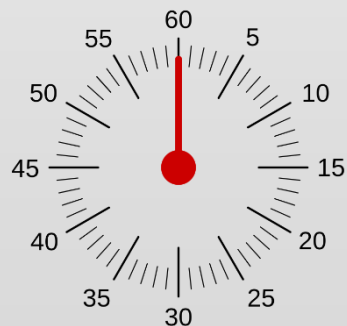
- ✓ LA ACCIÓN QUE DECIDAMOS REALIZAR, YA SEA PRESIONAR EL PULSADOR O DEJARLO SIN PRESIONAR, DEBERÁ MANTENERSE CONSTANTE DURANTE UN TIEMPO T_i . AL FINALIZAR, ARBITRAMOS LA SIGUIENTE ACCIÓN Y DEBEREMOS MANTENERLA CONSTANTE DURANTE EL TIEMPO T_{i+1} Y ASÍ SUCESIVAMENTE.
- ✓ LOS INTERVALOS DE TIEMPO SON FIJOS, O SEA $T_1 = T_2 = \dots T_N$.



AUDIO

SUPONGAMOS:

$$0 - 60 = T_i$$

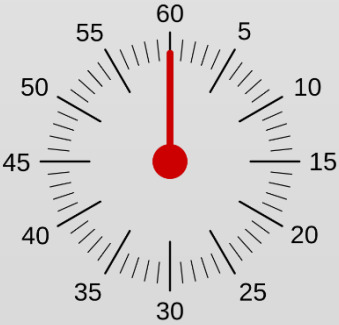


CONCEPTOS BÁSICOS DE LA TRANSMISIÓN DIGITAL

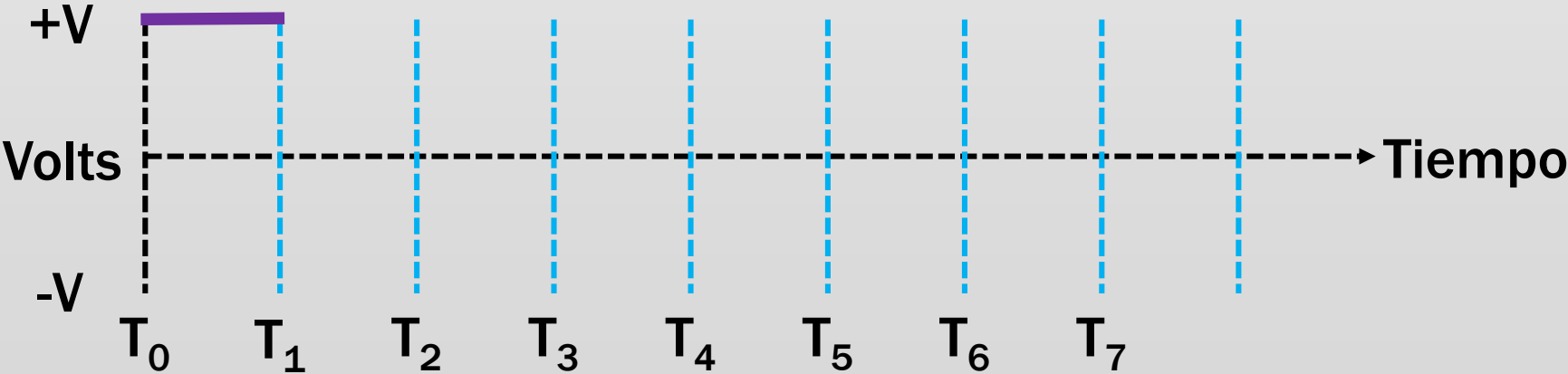
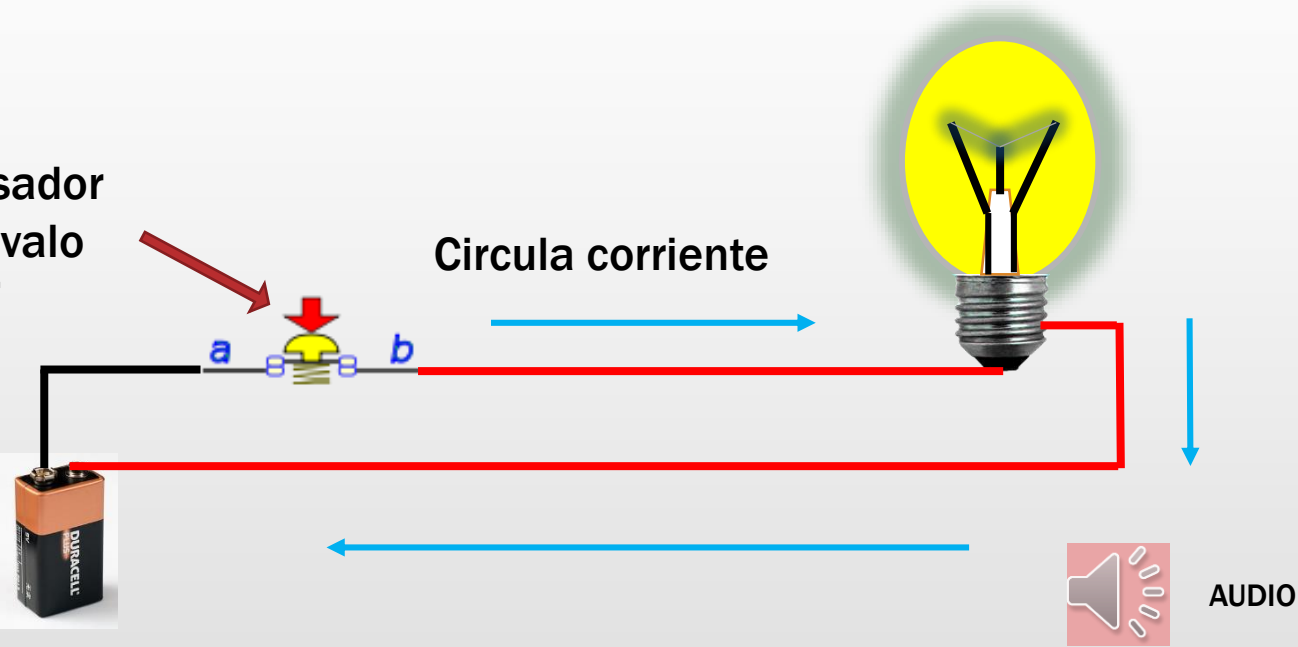
☐ ✓ ACCIÓN 1:

PULSAR SI / NO	PASO DE CORRIENTE
SI	✓

SUPONGAMOS:
 $0 - 60 = T_i$



SI Presionar pulsador
durante el intervalo
tiempo “ T_1 ”

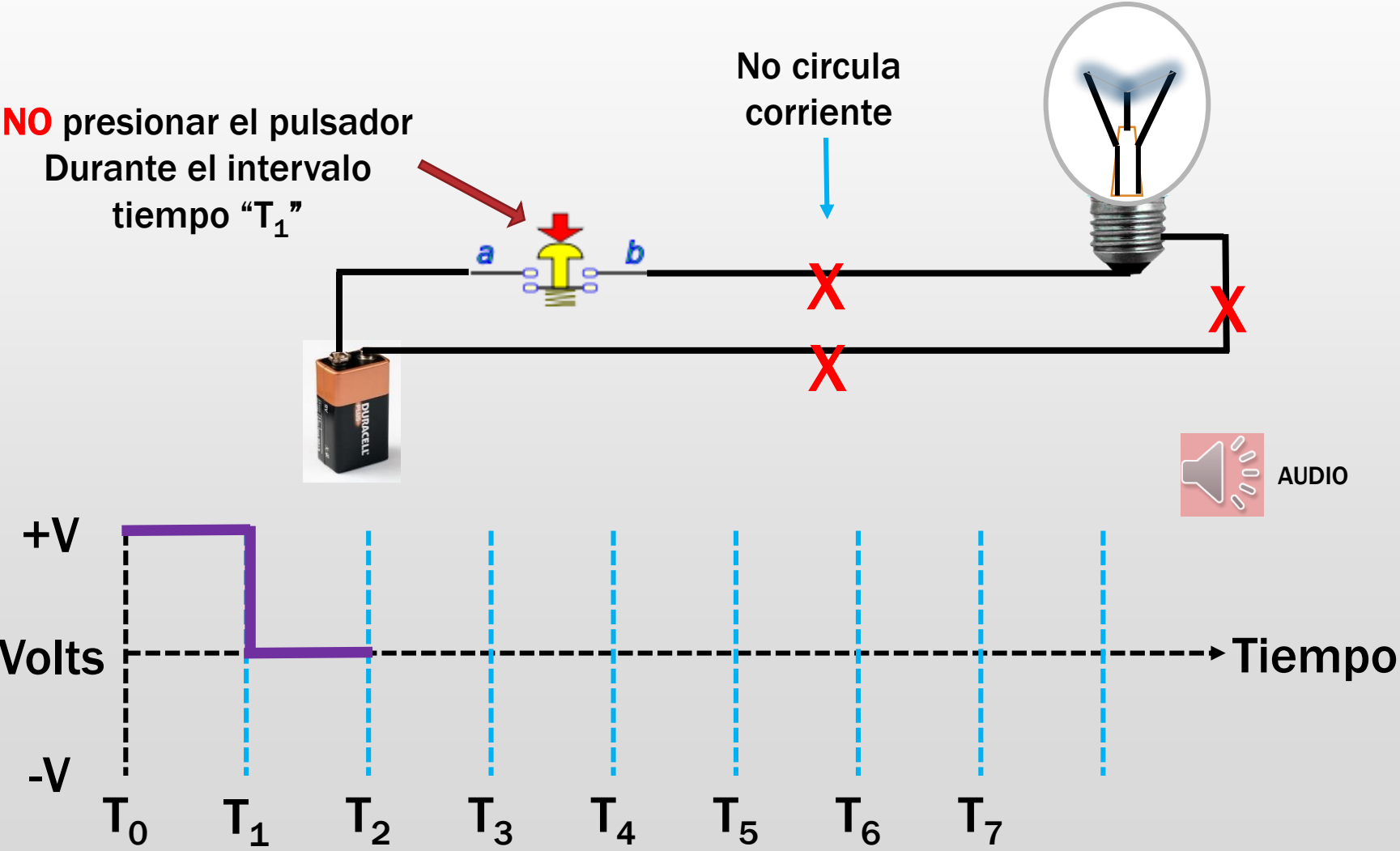
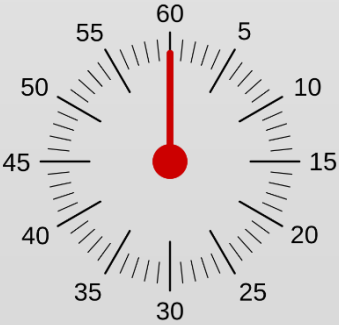


CONCEPTOS BÁSICOS DE LA TRANSMISIÓN DIGITAL

☐ ✓ ACCIÓN 2:

PULSAR SI/ NO	PASO DE CORRIENTE
SI	✓
NO	X

SUPONGAMOS:
 $0 - 60 = T_i$

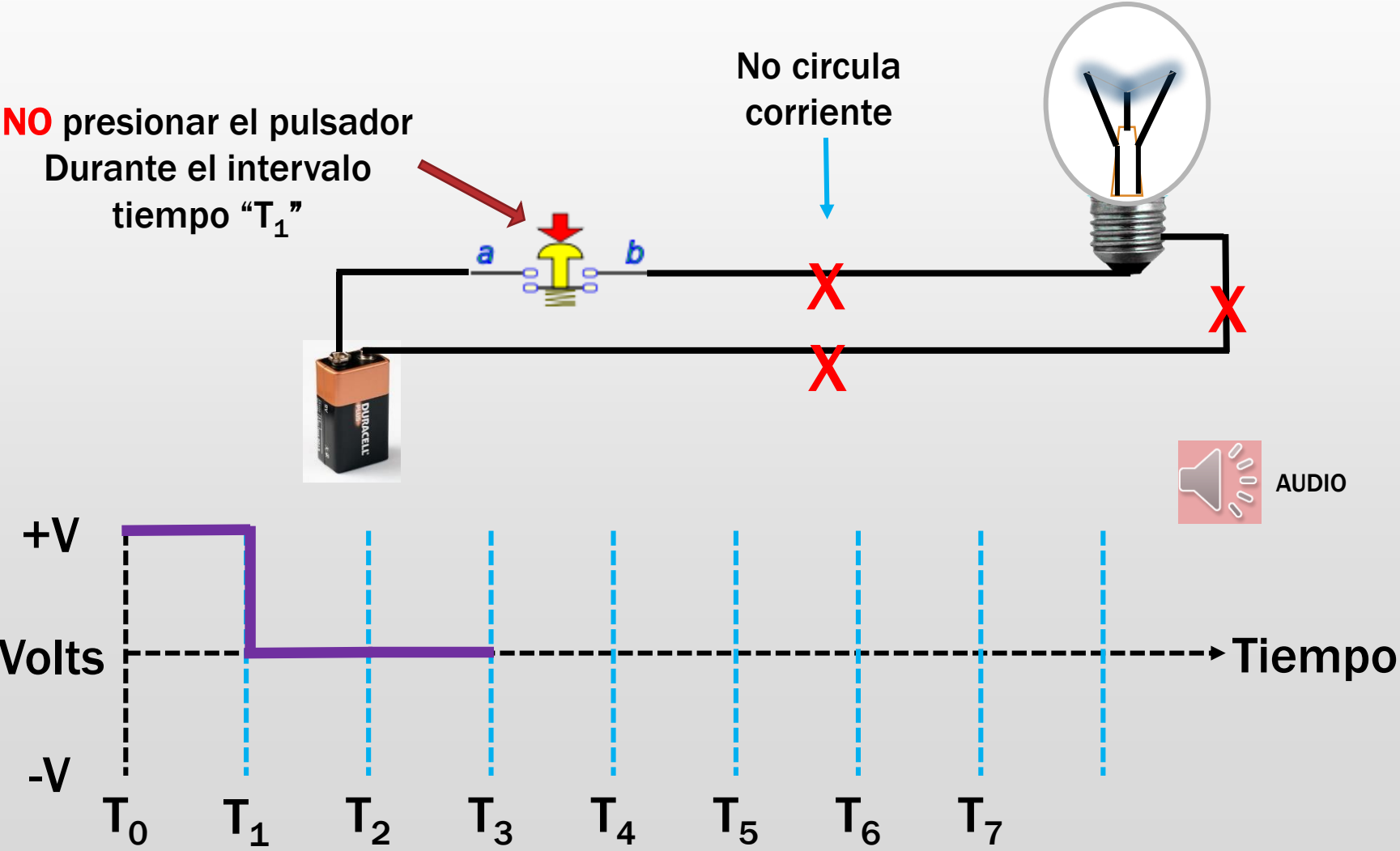
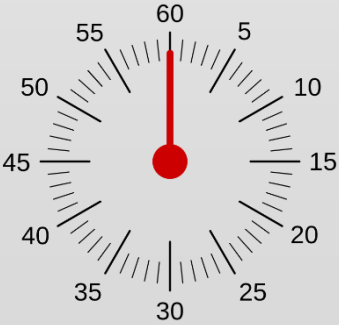


CONCEPTOS BÁSICOS DE LA TRANSMISIÓN DIGITAL

☐ ✓ ACCIÓN 3:

PULSAR SI/ NO	PASO DE CORRIENTE
SI	✓
NO	X
NO	X

SUPONGAMOS:
 $0 - 60 = T_i$

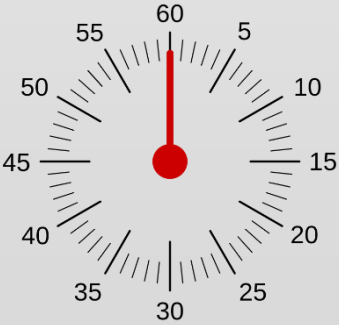


CONCEPTOS BÁSICOS DE LA TRANSMISIÓN DIGITAL

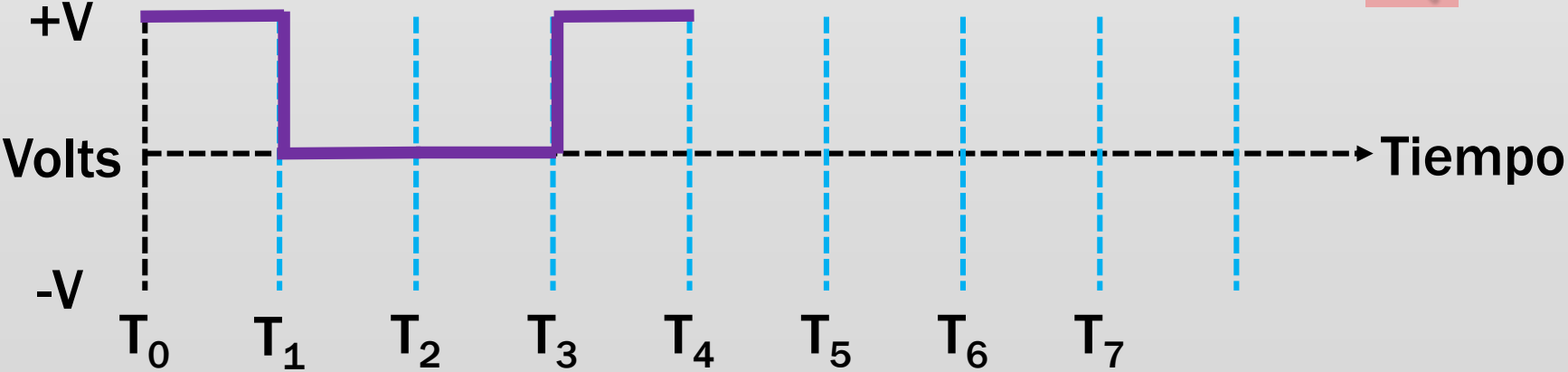
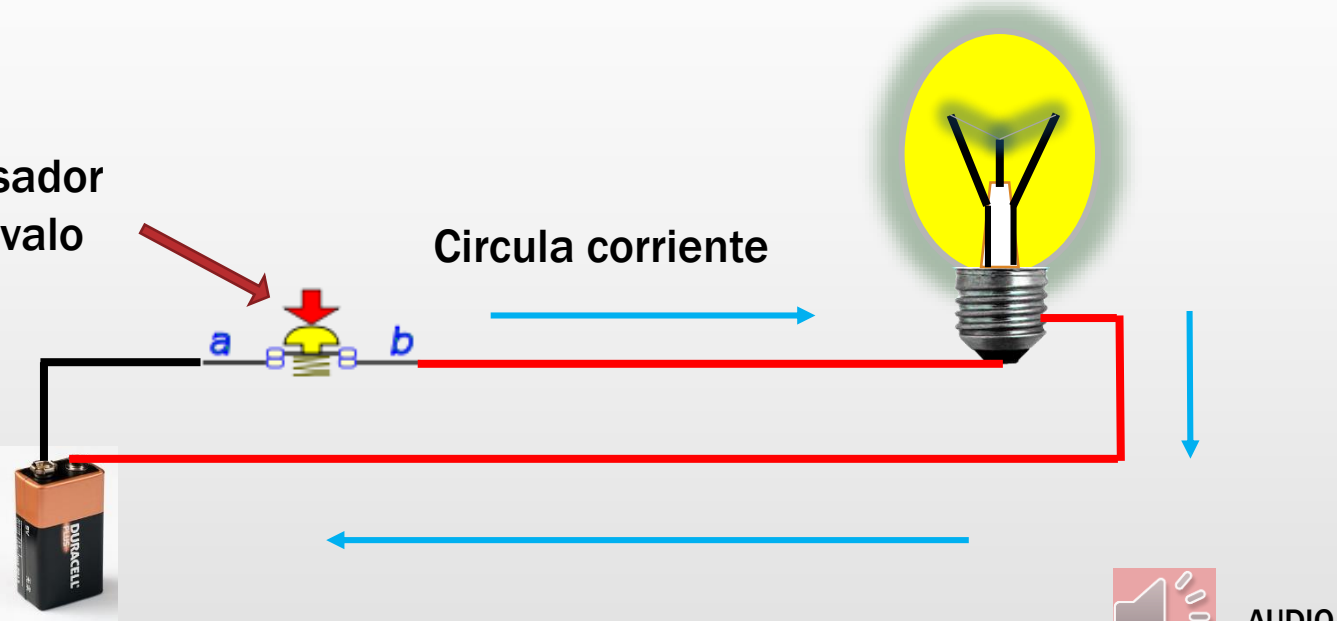
☐ ✓ ACCIÓN 4:

PULSAR SI/ NO	PASO DE CORRIENTE
SI	✓
NO	X
NO	X
SI	✓

SUPONGAMOS:
 $0 - 60 = T_i$



SI Presionar pulsador
durante el intervalo
tiempo “ T_1 ”

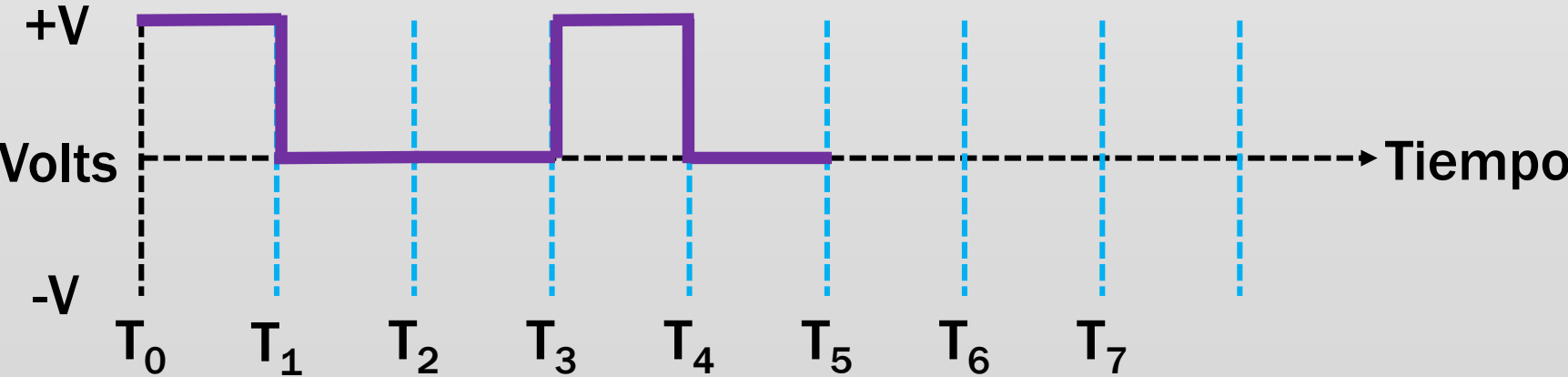
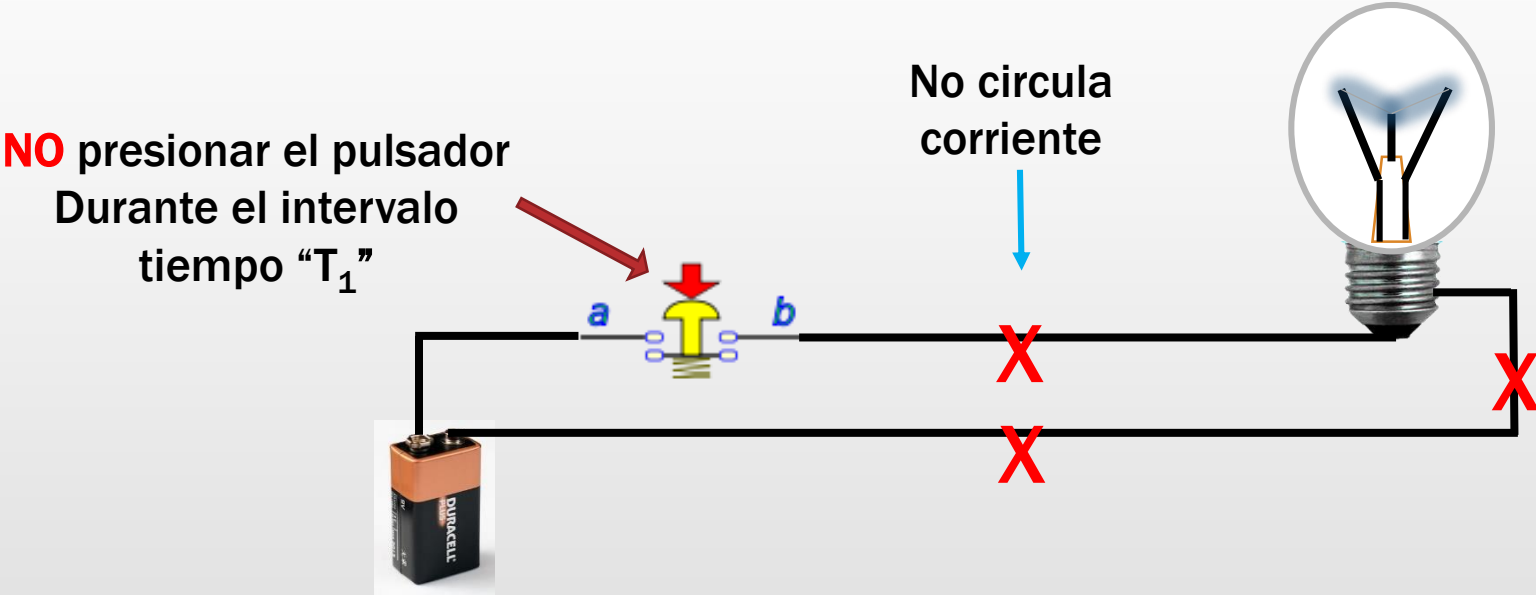
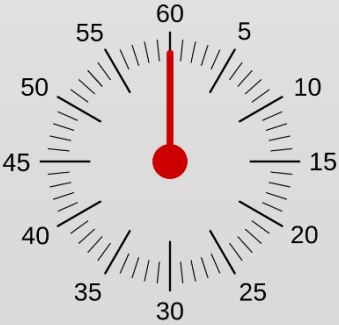


CONCEPTOS BÁSICOS DE LA TRANSMISIÓN DIGITAL

☐ ✓ ACCIÓN 5:

PULSAR SI/ NO	PASO DE CORRIENTE
SI	✓
NO	X
NO	X
SI	✓
NO	X

SUPONGAMOS:
 $0 - 60 = T_i$

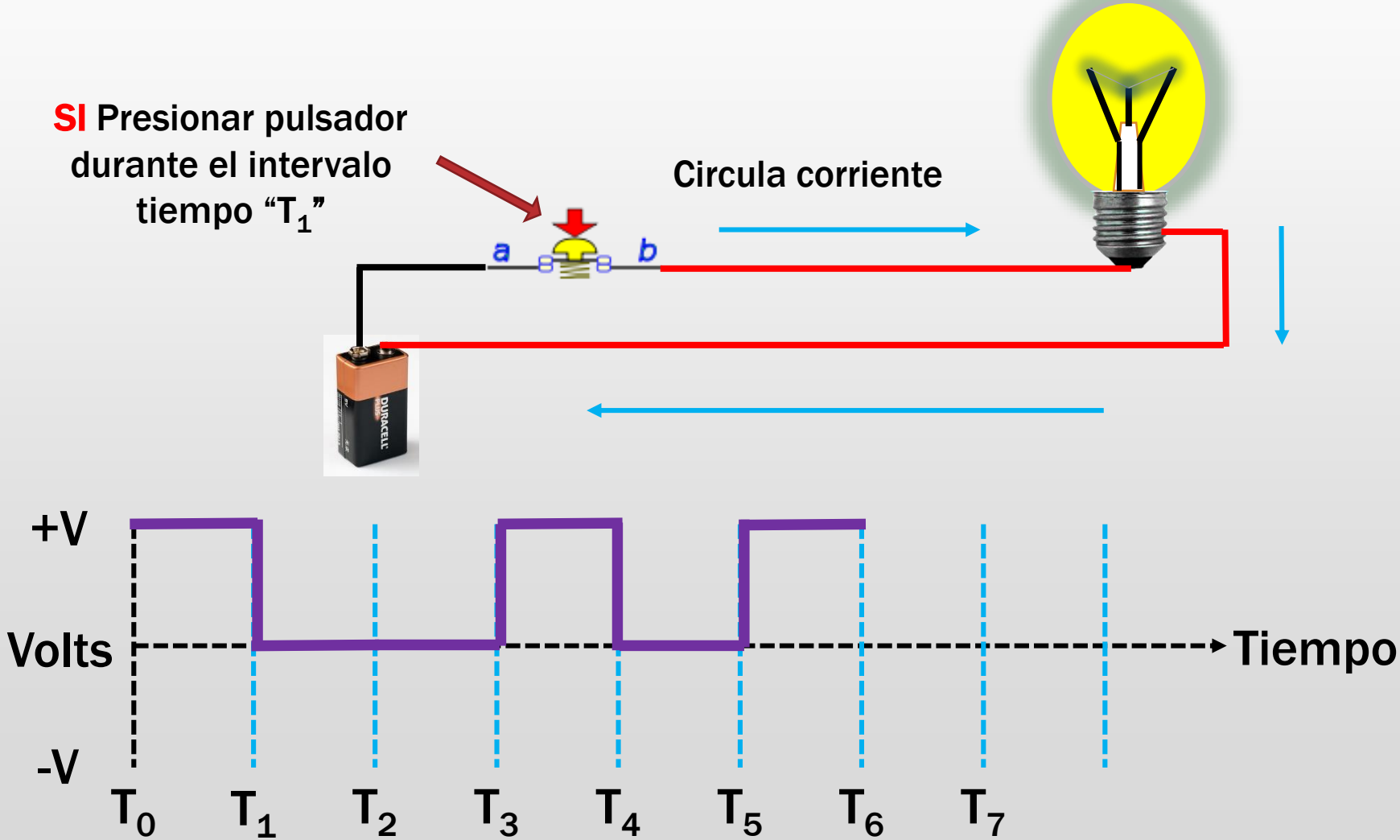
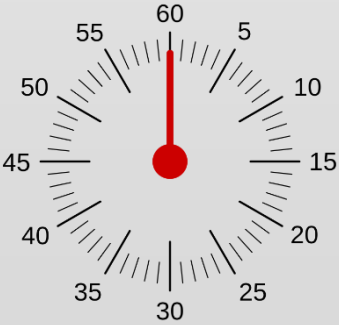


CONCEPTOS BÁSICOS DE LA TRANSMISIÓN DIGITAL

☐ ✓ ACCIÓN 6:

PULSAR SI/ NO	PASO DE CORRIENTE
SI	✓
NO	X
NO	X
SI	✓
NO	X
SI	✓

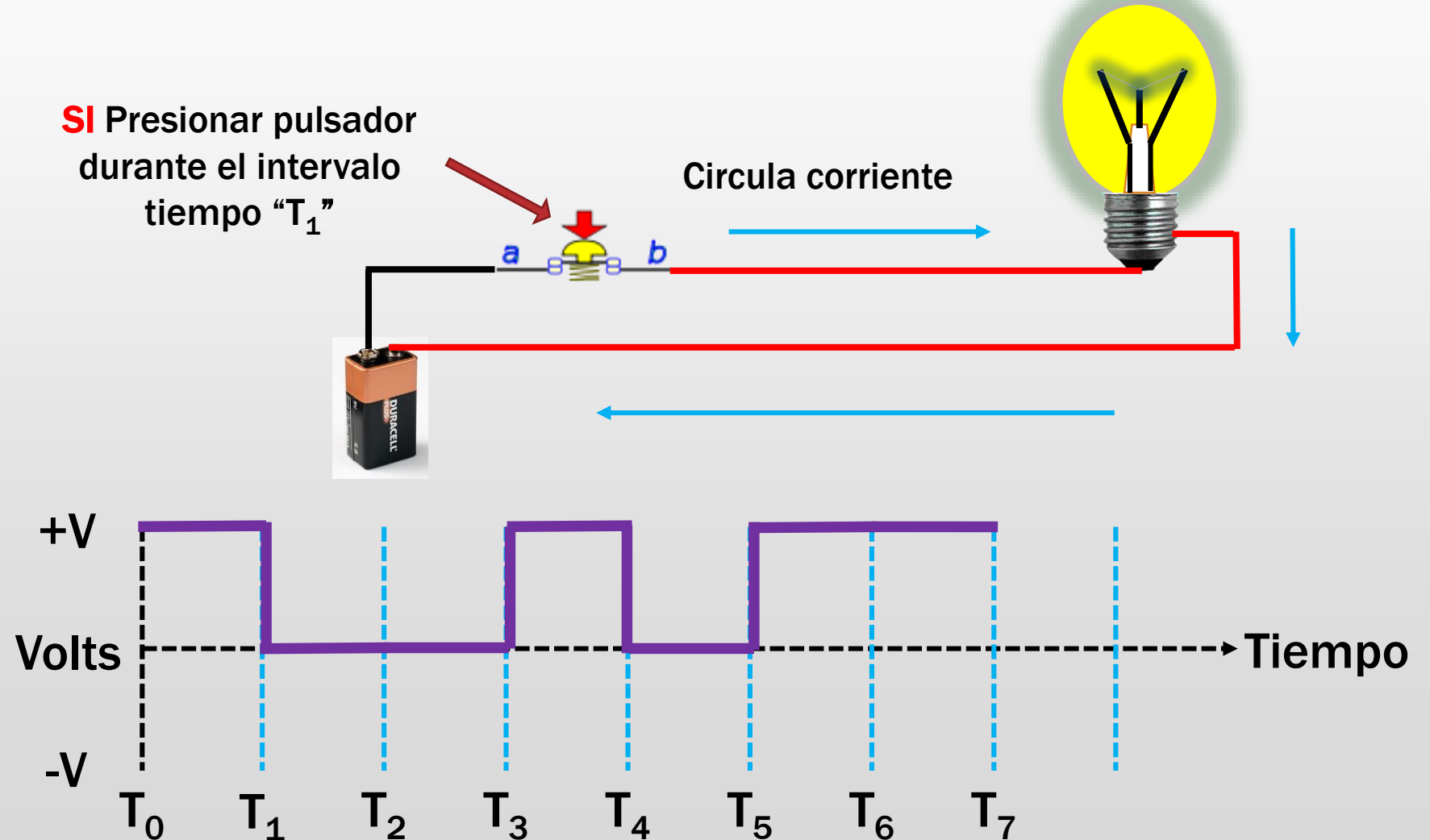
SUPONGAMOS:
 $0 - 60 = T_i$



CONCEPTOS BÁSICOS DE LA TRANSMISIÓN DIGITAL

☐ ✓ ACCIÓN 7:

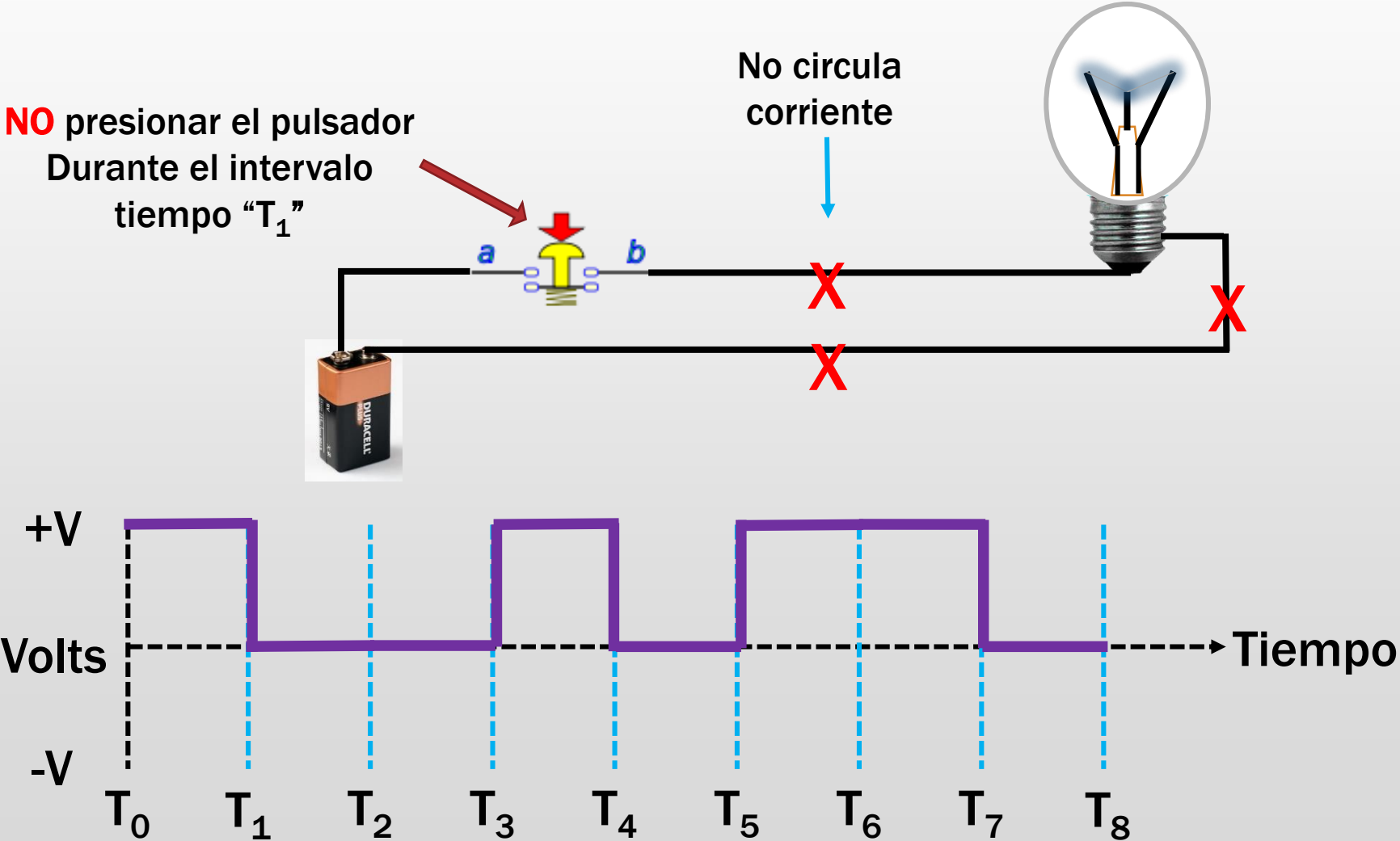
PULSAR SI/ NO	PASO DE CORRIENTE
SI	✓
NO	X
NO	X
SI	✓
NO	X
SI	✓
SI	✓



CONCEPTOS BÁSICOS DE LA TRANSMISIÓN DIGITAL

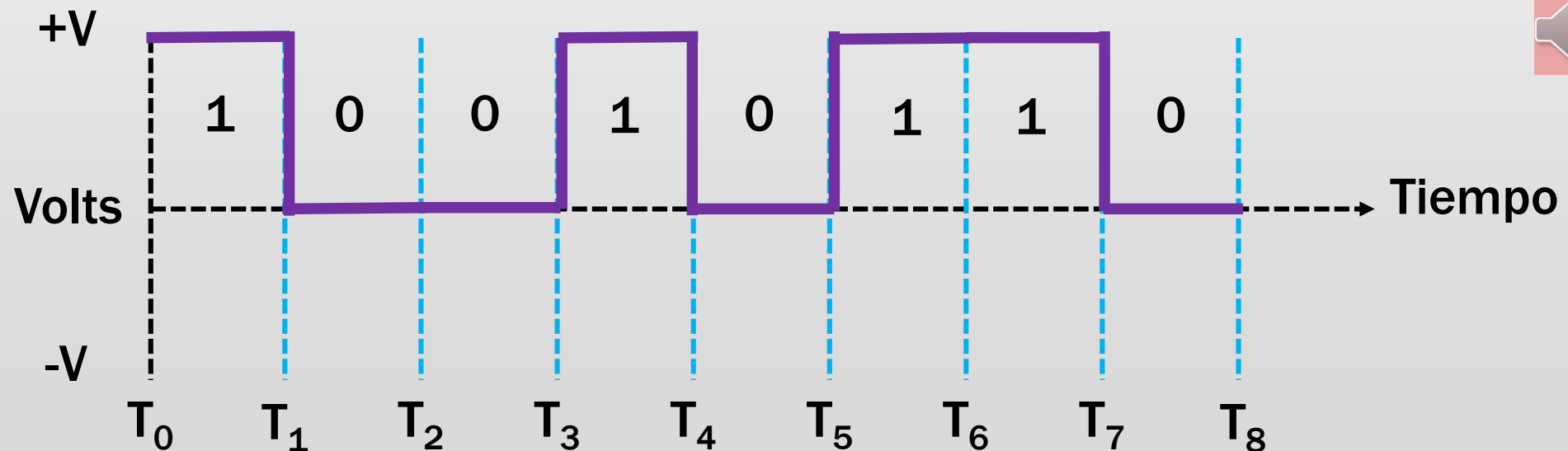
☐ ✓ ACCIÓN 8:

PULSAR SI/ NO	PASO DE CORRIENTE
SI	✓
NO	X
NO	X
SI	✓
NO	X
SI	✓
SI	✓
NO	X



CONCEPTOS BÁSICOS DE LA TRANSMISIÓN DIGITAL

- ❑ ✓ EL MÉTODO DE TRANSMISIÓN, EN ESTE CASO, ES A TRAVÉS DE PULSOS ELÉCTRICOS QUE VARÍAN ENTRE DOS NIVELES DISTINTOS DE VOLTAJE:
- ❑ ✓ COMO EN ESTE CASO SE UTILIZAN DOS ESTADOS, APLICAMOS PARA SU REPRESENTACIÓN EL SISTEMA BINARIO, O SEA EL SISTEMA NUMÉRICO DE DOS DIGITOS :
 - **DIGITO "0" PARA INDICAR AUSENCIA DE LA CORRIENTE DURANTE UN INSTANTE T_i .**
 - **DIGITO "1" PARA INDICAR PRESENCIA DE LA CORRIENTE DURANTE UN INSTANTE T_i .**

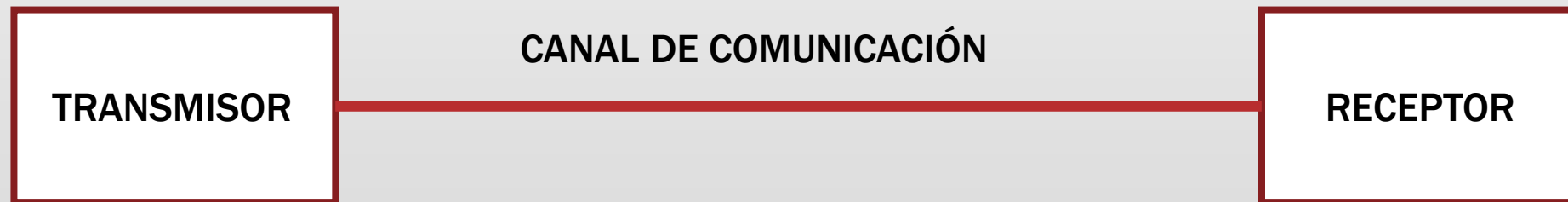


AUDIO

CONCEPTOS BÁSICOS DE LA TRANSMISIÓN DIGITAL

☐ ✓ DADO NUESTRO EJEMPLO DEL CIRCUITO:

- ✓ SI CONSIDERAMOS AL PULSADOR COMO UN TRANSMISOR, EL CABLE COMO UN CANAL DE COMUNICACIÓN Y LA LÁMPARA COMO UN RECEPTOR. ENTONCES PODEMOS DECIR QUE SON LAS TRES COMPONENTES BÁSICAS PARA CONSTITUIR UN **SISTEMA DE COMUNICACIÓN** ELÉCTRICA.

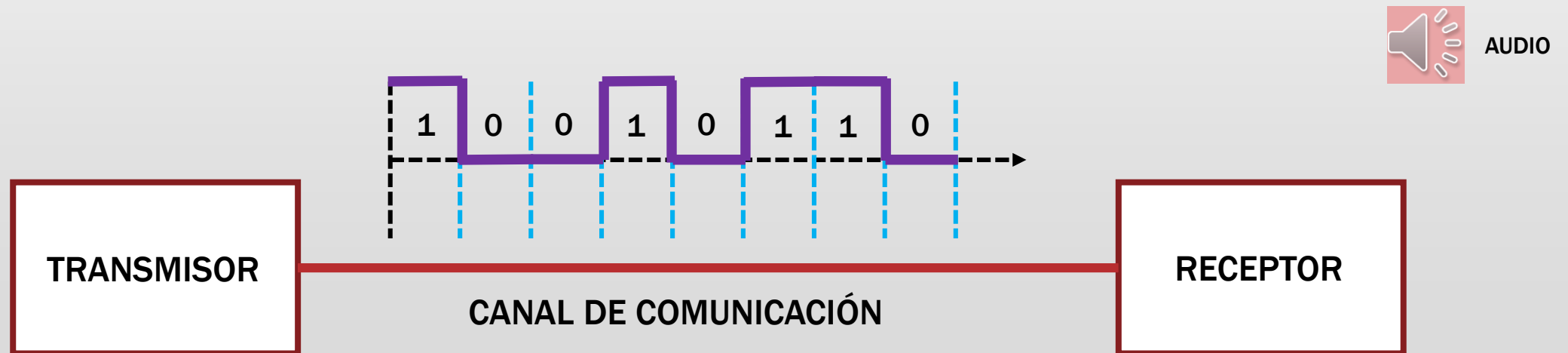


AUDIO

CONCEPTOS BÁSICOS DE LA TRANSMISIÓN DIGITAL

❑ ✓ **ALMACENAMIENTO:**

- ✓ CUANDO SE TRANSFIERE UNA CIERTA CANTIDAD DE DIGITOS BINARIOS DESDE EL TRANSMISOR AL RECEPTOR, ESTOS DEBEN DER ALMACENADOS.
- ✓ LA CANTIDAD DE DATOS QUE PUEDE ALMACENARSE EN UN DISPOSITIVO DE ALMACENAMIENTO ES LA **CAPACIDAD DE ALMACENAMIENTO**.



CONCEPTOS BÁSICOS DE LA TRANSMISIÓN DIGITAL

☐ ✓ UNIDADES ALMACENAMIENTO



AUDIO

NOMBRE	ABREVIATURA	MEDIDA BINARIA	CANTIDAD DE BYTES	EQUIVALENCIA
BIT	b			
BYTE	B	2^0	1	8 BITS
KILOBYTE	KB	2^{10}	1024	1024 BYTES
MEGABYTE	MB	2^{20}	1048576	1024 KB
GIGABYTE	GB	2^{30}	1.073.741.824	1024 MB
TERABYTE	TB	2^{40}	1.099.511.627.776	1024 GB
PETABYTE	PB	2^{50}	1.125.899.906.842.624	1024 TB
EXABYTE	EB	2^{60}	1.152.921.504.606.846.976	1024 PB
ZETTABYTE	ZB	2^{70}	1.180.591.620.717.411.303.424	1024 EB
YOTTABYTE	YB	2^{80}	1.208.925.819.614.629.174.706.176	1024 ZB

CONCEPTOS BÁSICOS DE LA TRANSMISIÓN DIGITAL

☐ ✓ EJEMPLOS DE UNIDADES ALMACENAMIENTO

- SI UN ARCHIVO LLAMADO **PRUEBA.JPG** ESTA ALMACENADO EN UN DISPOSITIVO CON UN TAMAÑO DE **1.610.612.736** BYTES.

¿CUAL ES SU TAMAÑO EQUIVALENTE EXPRESADO EN **GB**?

$$\begin{array}{l} 1024 \text{ BYTES} \underline{\hspace{1.5cm}} 1 \text{ KB} \\ 1.610.612.736 \text{ BYTES} \underline{\hspace{1.5cm}} x = \frac{1.610.612.736}{1024} = \mathbf{1572864 \text{ KB}} \end{array}$$

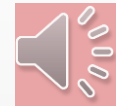
$$\begin{array}{l} 1024 \text{ KB} \underline{\hspace{1.5cm}} 1 \text{ MB} \\ 1572864 \text{ KB} \underline{\hspace{1.5cm}} x = \frac{1572864}{1024} = \mathbf{1536 \text{ MB}} \end{array}$$

$$\begin{array}{l} 1024 \text{ MB} \underline{\hspace{1.5cm}} 1 \text{ GB} \\ 1536 \text{ MB} \underline{\hspace{1.5cm}} x = \frac{1536}{1024} = \mathbf{1,5 \text{ GB}} \end{array}$$

✓ **PRUEBA.JPG = 1,5 GB**

CONCEPTOS BÁSICOS DE LA TRANSMISIÓN DIGITAL

❑ ✓ VELOCIDAD DE TRANSMISIÓN:



AUDIO

- ✓ LA VELOCIDAD DE TRANSMISIÓN DE DATOS, SERÍA LA CANTIDAD DE BITS QUE SE TRANSFIEREN ENTRE DOS DISPOSITIVOS POR UNIDAD DE TIEMPO, USUALMENTE SEGUNDOS.
- ✓ SI HIPOTÉTICAMENTE PUDIERAMOS OBSERVAR POR UNA VENTANA QUE DARÍA SOBRE UN CANAL DE COMUNICACIÓN, OBSERVARÍAMOS COMO UNA CIERTA CANTIDAD DE DIGITOS BINARIOS PASARÍAN POR UNA UNIDAD DE TIEMPO, DADO EN SEGUNDO.
- ✓ **ENTONCES, DIRÍAMOS QUE SE TRANSFIERE A UNA VELOCIDAD DE X BITS POR SEGUNDO**



VELOCIDAD LA TRANSMISIÓN DE DATOS

❑ VELOCIDAD DE TRANSMISIÓN:



AUDIO

- ✓ EN EL CASO DE DEFINIR LAS VELOCIDADES DE TRANSMISIÓN SE PUEDE USAR COMO BASE EL BIT, Y MÁS CONCRETAMENTE EL BIT POR SEGUNDO.
- ✓ LOS MÁS UTILIZADOS SON EL KILOBITS/s O **Kbps**, MEGABITS/s O **Mbps** Y GIGABITS/s O **Gbps**.
- ✓ LAS ABREVIATURAS SE DIFERENCIAN DE LOS TÉRMINOS DE ALMACENAMIENTO EN QUE SE EXPRESAN CON **b** MINÚSCULAS . ESTAS ABREVIATURAS SON:

Kbps = 1.000 bps

Mbps = 1.000 Kbps = 1.000.000 bps

Gbps = 1.0000 Mbps = 1.000.000.000 bps

VELOCIDAD LA TRANSMISIÓN DE DATOS

❑ VELOCIDAD DE TRANSMISIÓN:



AUDIO

- ✓ EN ESTE SENTIDO HAY QUE TENER EN CUENTA QUE LAS VELOCIDADES QUE EN LA MAYORÍA DE LAS OCASIONES SE MUESTRA EN INTERNET ESTÁN EXPRESADAS EN **KB/s (KILOBYTE POR SEGUNDO)**, LO QUE REALMENTE SUPONE QUE NOS DICE LA CANTIDAD DE BYTES (UNIDAD DE ALMACENAMIENTO) QUE HEMOS RECIBIDO EN UN SEGUNDO, NO A LA VELOCIDAD DE TRANSMISIÓN.
- ✓ PODEMOS CALCULAR ESA VELOCIDAD DE TRANSMISIÓN (PARA PASARLA A **Kbps O KILOBITS POR SEGUNDO**) SIMPLEMENTE MULTIPLICANDO EL DATO QUE SE NOS MUESTRA POR 8.

VELOCIDAD LA TRANSMISIÓN DE DATOS

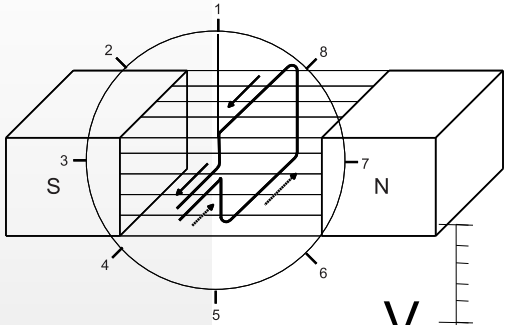
❑ EJEMPLO:

- ✓ SUPONGAMOS QUE LA TRANSMISIÓN QUE SE NOS INDICA POR INTERNET ES DE 308 KB/s. CUAL ES SU VELOCIDAD DE TRANSMISIÓN EXPRESADA EN Kbps Y Mbps ?

$$308 \text{ KB/s} \times 8 = \underline{\underline{2.464 \text{ Kbps}}}$$

$$2.464 \text{ Kbps} = \frac{2.464}{1000} = \underline{\underline{2,46 \text{ Mbps}}}$$

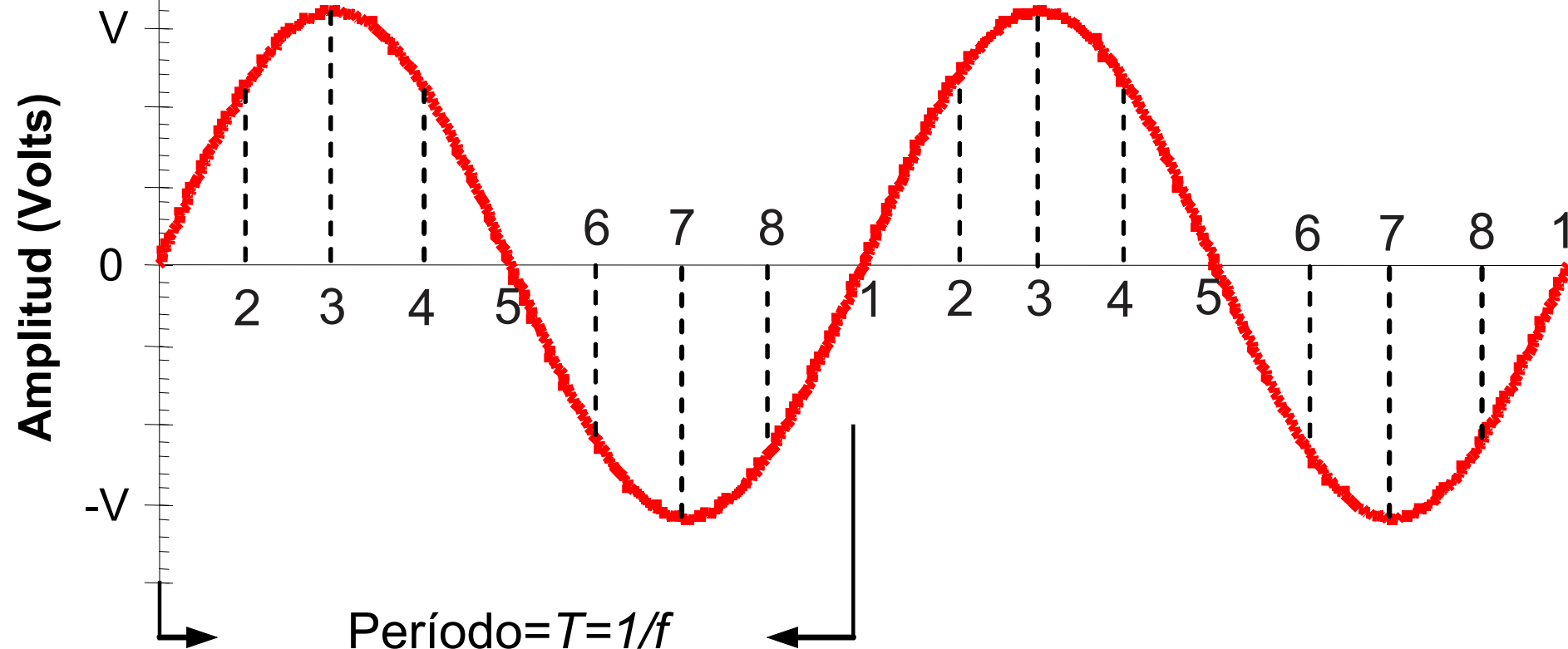
FUERZA ELECTROMOTRIZ GENERADA POR LA ROTACIÓN DE UNA ESPIRA



Señal Analógica

$$f(t) = V \operatorname{sen} (\omega t + \theta)$$

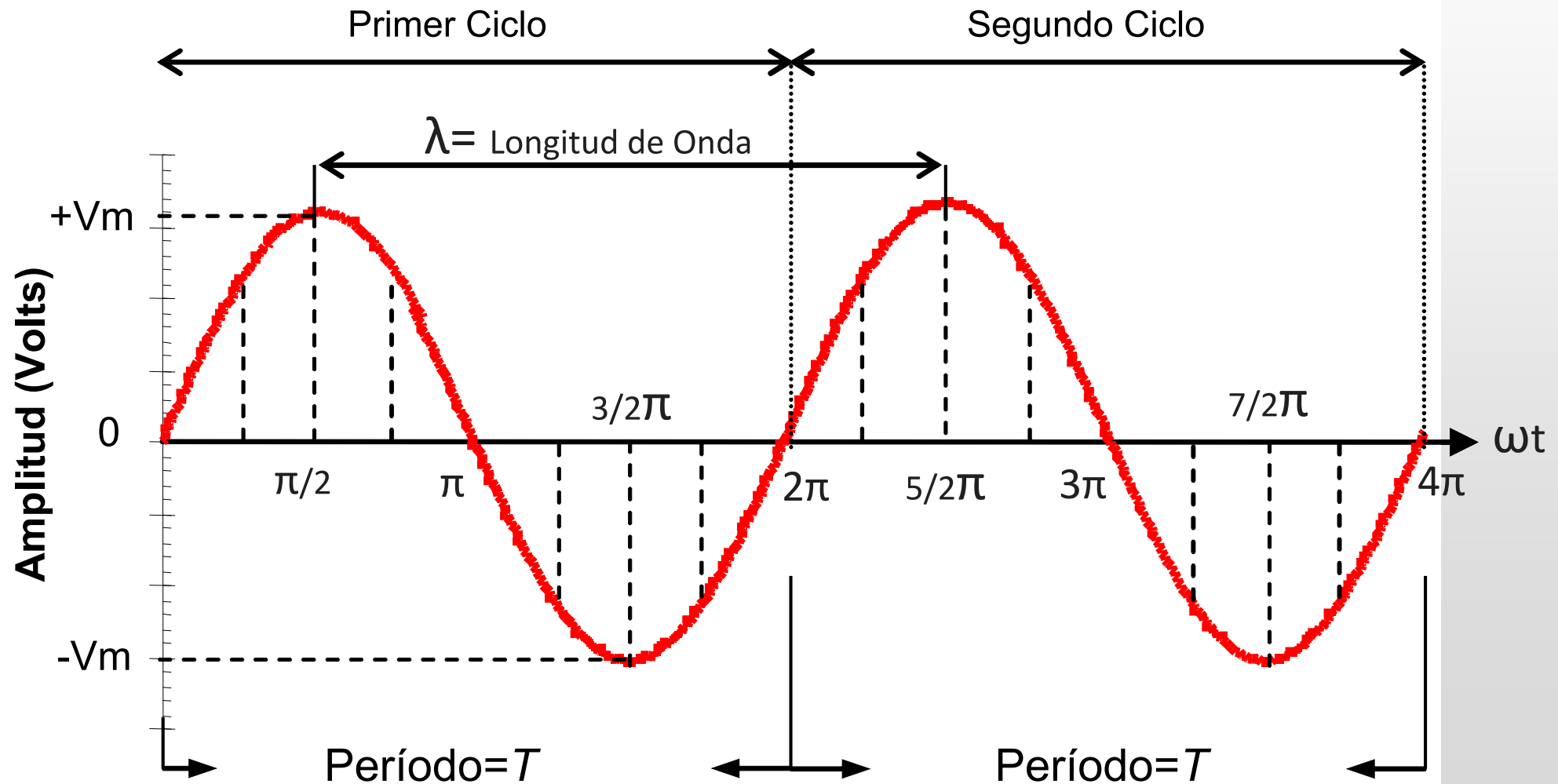
V = Voltaje pico (Volts)
 ωt = Ángulo barrido
 θ = Ángulo fase inicial



Posición

VELOCIDAD LA TRANSMISIÓN DE DATOS

Señal Analógica



VELOCIDAD LA TRANSMISIÓN DE DATOS

V = Voltaje pico (Volts)

ωt = Ángulo barrido

θ = Ángulo fase inicial

λ = Longitud de onda

ω = velocidad angular $\omega = \frac{2\pi}{T} = 2\pi f \left(\frac{\text{radianes}}{\text{segundos}} \right)$

En un tiempo t segundos la espira habrá recorrido :

$\omega t = \text{Ángulo barrido}$ $\omega t = 2\pi f t (\text{radianes})$

La longitud de onda λ , es la longitud de un ciclo de una onda electromagnética.

$$\lambda = \frac{c}{f} \frac{(\text{velocidad de la luz})}{(\text{frecuencia})} = \frac{3 \times 10^8 \left(\frac{\text{metros}}{\text{segundos}} \right)}{f} = [\text{metros}]$$

FIN

TEOREMA FUNDAMENTAL DEL ALGEBRA DE BOOLE

$$1) A + 1 = 1$$

$$2) A + 0 = A$$

$$3) A * 1 = A$$

$$4) A * 0 = 0$$

$$5) A + \overline{A} = 1$$

$$6) A + A = A$$

$$7) A * \overline{A} = 0$$

$$8) A * A = A$$

ASOCIATIVA

$$9) (A+B) + C = A + (B + C)$$

$$10) (A * B) * C = A * (B * C)$$

CONMUTATIVA

$$11) A + B = B + A$$

$$12) A * B = B * A$$

DISTRIBUTIVA

$$13) A * (B + C) = A * B + A * C$$

$$14) A + (B * C) = (A + B) * (A + C)$$

TEOREMA DE MORGAN

$$15) \overline{\overline{A}} = A$$

$$16) A + A * B = (A + A) * (A + B) = A * (A + B) = A * A + A * B = A + A * B = A * (1 + B) = \mathbf{A}$$

$$17) A * (A + B) = A * A + A * B = A + A * B = \mathbf{A}$$

$$18) \overline{A + B} = \overline{A} * \overline{B}$$

$$19) \overline{A * B} = \overline{A} + \overline{B}$$

UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL LA PLATA

CÁTEDRA :
ARQUITECTURA DE COMPUTADORAS

REGLAS PARA SIMPLIFICAR POR EL MAPA DE KARNAUGH

DIAGRAMA DE VEITCH - KARNAUGH

- UNA CELDA ES ADYACENTE DE OTRA CUANDO LA INTERSECCIÓN DE LAS VARIABLES, VARÍA UNA SOLA VARIABLE, EN ESTE CASO SE SIMPLIFICARÍA UNA VARIABLE.
- EN GENERAL, TENDREMOS QUE AGRUPAR 2^n CELDAS ADYACENTES PARA PODER ELIMINAR n VARIABLES.
- ESTOS MAPAS DEJAN DE SER PRÁCTICOS PARA MÁS DE SEIS VARIABLES

Construcción con 2 variables

AB \ CD				
	00	01	11	10
00	1	0	0	1
01	1	1	1	1
11	1	0	0	1
10	1	0	0	1

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	0

Mapa K		
A \ B	0	1
	0	1
1	1	0

REGLAS PARA SIMPLIFICACIÓN POR CEROS O POR UNOS:

1. SI TODAS LAS CASILLAS CONTIENEN “1”, COMO SE AGRUPARÍAN TODAS (SE ELIMINAN TODAS LAS VARIABLES), EL RESULTADO DE LA FUNCIÓN ES $= 1$.
2. SI SE TRATA DE AGRUPAR LA MAYOR CANTIDAD DE CELDAS POSIBLES, AUNQUE ALGUNA CELDA YA SE HUBIEREN TOMADO CON OTRO GRUPO (LA MISIÓN ES LOGRAR LA MENOR CANTIDAD DE GRUPOS QUE CONTENGAN LA MAYOR CANTIDAD DE CELDAS). HABRÁ QUE HACER TANTOS GRUPOS HASTA CUBRIR TODOS LOS “1”.
3. SE TRATA DE NO REPETIR GRUPOS YA CUBIERTOS, SALVO QUE SIRVA PARA HACER EL GRUPO MÁS GRANDE DE LAS CELDAS QUE VAN QUEDANDO.
4. SI AÚN QUEDARAN CASILLAS LIBRES Y NO PUDIERAN AGRUPARSE (NI DE A 2), SE TOMARÁN SOLAS (EQUIVALENTE AL MINITÉRMINO).
5. SI EXISTIERA ALGUNA “X” (PUEDE SER “0” o “1”), ESTA SE TOMARÁ DEL VALOR MÁS CONVENIENTE PARA LA SIMPLIFICACIÓN, ES DECIR, QUE SI HAY QUE TOMARLA ES A NUESTRO PROVECHO Y NO CREAR UN NUEVO GRUPO PARA CUBRIRLA.
6. EN GENERAL, SE TRATARÁ DE CUMPLIR CON EL “PUNTO 2” POR LO CUAL, A VECES, CONVENDRÁ TRABAJAR POR LOS “0”, USANDO LA CONVENCION DE LOS MAXITÉRMINOS.

MAPA DE 4 VARIABLES

cd ab					
		00	01	11	10
00	00	0	0	0	0
01	01	0	0	0	0
11	11	0	0	0	0
10	10	0	0	0	0

MAPA DE 5 VARIABLES

cde ab									
		000	001	011	010	110	111	101	100
00	00	0	0	0	0	0	0	0	0
01	01	0	0	0	0	0	0	0	0
11	11	0	0	0	0	0	0	0	0
10	10	0	0	0	0	0	0	0	0

CATEDRA DE ARQUITECTURA DE COMPUTADORAS

CIRCUITOS DIGITALES

UNIDAD TEMÁTICA III

CONTENIDOS:

- Circuitos digitales. Descripción de los circuitos combinacionales de base. Empleo de los circuitos combinacionales. Sumador. Multiplexores, demultiplexores.
- Decodificador, codificador. Memoria ROM. Tipos de ROM.
- Descripción de los circuitos secuenciales. Empleo de los circuitos secuenciales. Flip-Flops.
- Contadores. Registros.
- Tipos y estructura básica de una RAM.

Sistemas Digitales

Podemos definir un sistema digital como un conjunto de elementos binarios que se encuentran relacionados entre si de alguna manera.

En todo sistema digital podemos distinguir dos tipos de variables:

✓ **Variables de entrada**

✓ **Variables de salida (dependientes de las variables de entrada)**

Funcionalmente las variables de entrada se dividen en dos grupos:

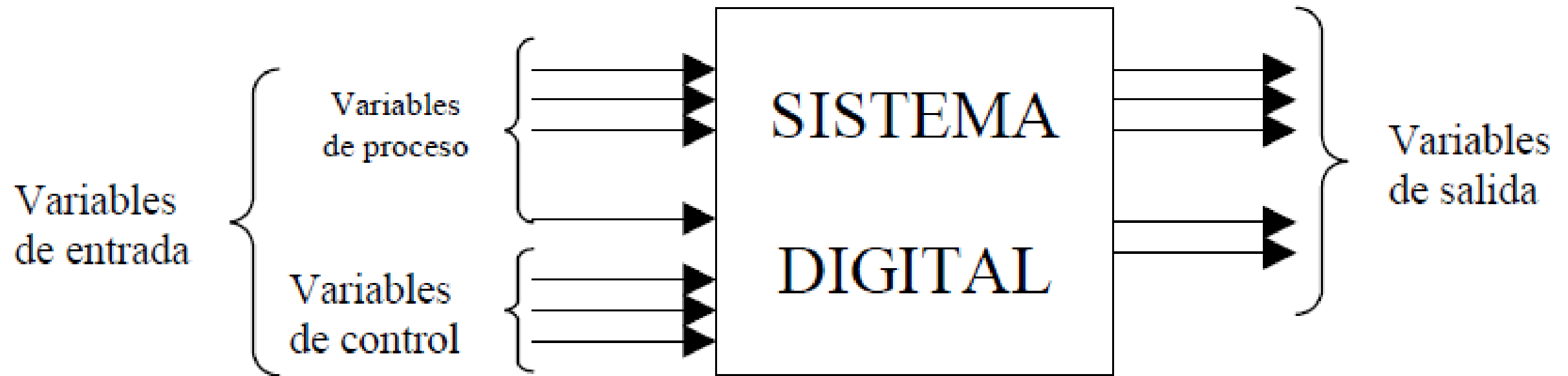
✓ **Variables de proceso:**

- Son aquellas con las cuales el circuito puede realizar una determinada función u operación.

✓ **Variables de control:**

- Son aquellas que influyen en la forma en que el circuito actúa sobre las variables de proceso o sobre el nivel lógico de las variables de salida del circuito.

Sistemas Digitales

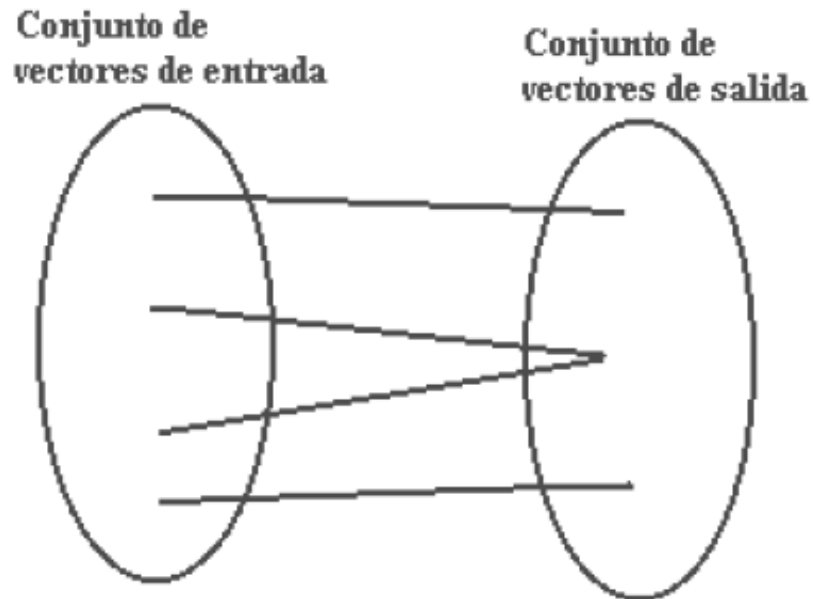


Sistemas Digitales

Sistema Combinacional:

Cuando cada combinación de las variables de entrada (**Vector de entrada**) se corresponde con una y sólo una combinación de las variables de salida (**Vector de salida**).

Es decir, siempre que se repita un conjunto de valores de las variables de entrada, se repetirá la salida de dicho sistema.



**Correspondencias entre entradas y salidas
de un Sistema Combinacional**

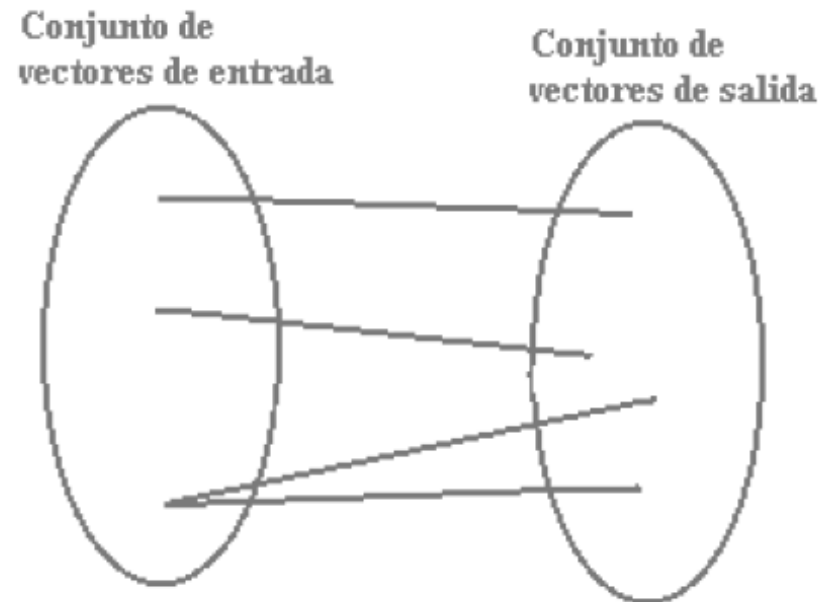
Sistemas Digitales

Sistema Secuencial:

Cuando a un mismo vector de entrada puede corresponder más de uno de salida.

Es decir, cuando se repite un conjunto de valores de las variables de entrada, no necesariamente se repetirá la salida.

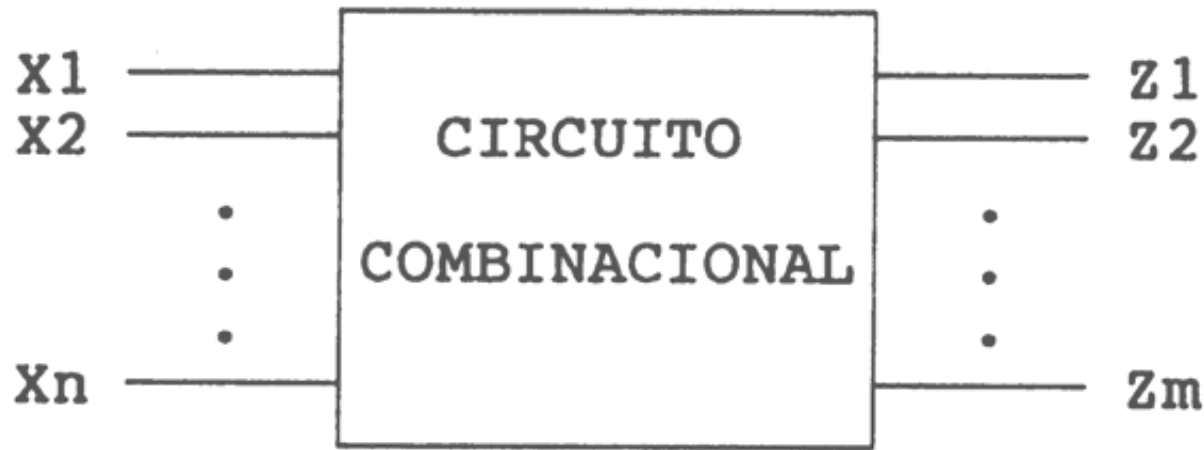
Los sistemas secuenciales deben poseer memoria interna ya que sus salidas son consecuencia de la evolución anterior de sus entradas.



Correspondencias de un Sistema Secuencial

Circuitos Combinacionales

Son aquellos cuyas salidas en un determinado instante son función exclusivamente del valor de las entradas en dicho instante. Dicho en otras palabras, las salidas no son otra cosa que funciones lógicas de las entradas.



En la figura se puede observar el diagrama en bloque de un circuito combinacional con “*n*” entradas y “*m*” salidas. Podemos definir una función Z_i tal que $Z_i = f_i (X_1, X_2, \dots, X_n)$

De lo anterior se deduce que para diseñar un circuito combinacional bastará con minimizar las funciones requeridas y finalmente implementar con compuertas lógicas.

Circuitos Combinacionales

Estos circuitos están compuestos por operaciones booleanas básicas (**AND**, **OR**, **NOT**)

Bloques Combinacionales Estándares Principales:

- **SUMADORES Y RESTADORES**
- **DECODIFICADORES**
- **MULTIPLEXORES**

Sumadores

Semisumador Elemental (HALF ADDER)

Es un circuito que suma dos bits de entrada **a** y **b** y devuelve un bit de resultado **s** y un bit de acarreo **C_{out}**

Operación:

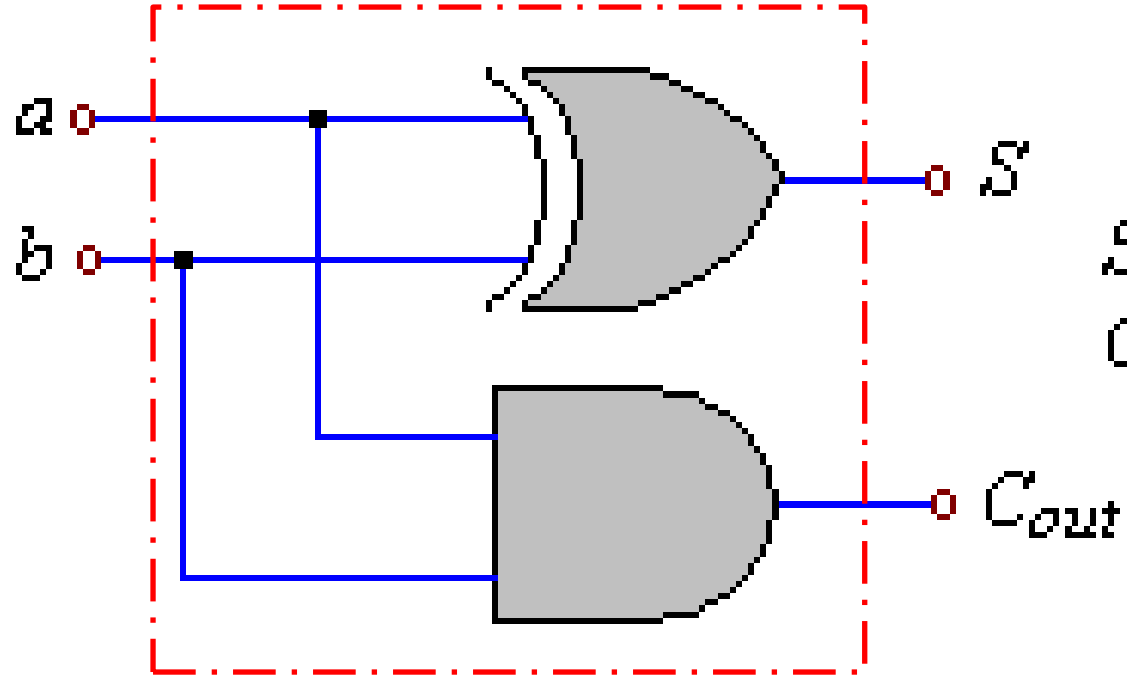
$$\begin{array}{r} a + b = S \\ \hline 0 + 0 = 0 \\ 0 + 1 = 1 \\ 1 + 0 = 1 \\ 1 + 1 = 0 \text{ Llevo } 1 \end{array}$$

La tabla de la verdad quedará de la siguiente forma:

a	b	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Donde **S** es la suma de **a** y **b**, y la letra **C** es el acarreo (**Carry**)

Circuito con Puertas Lógicas



$$S = a \oplus b$$
$$C_{out} = a * b$$

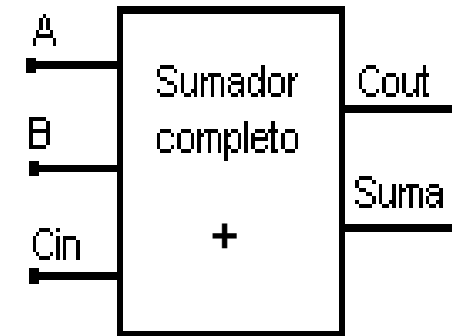
a	b	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Sumador Elemental Completo (FULL ADDER)

Es un circuito que suma dos bits de entrada **a** y **b**, más un acarreo de entrada **C_{in}** y devuelve un bit de resultado **S** y un bit de acarreo **C_{out}**

Tabla de verdad

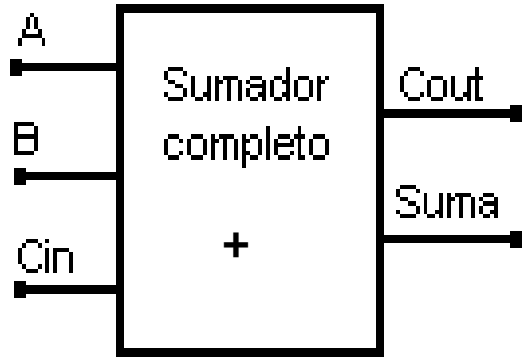
a	b	C _{in}	C _{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



$$C_{out} = a \cdot b + a \cdot C_{in} + b \cdot C_{in}$$

$$S = a \oplus b \oplus C_{in}$$

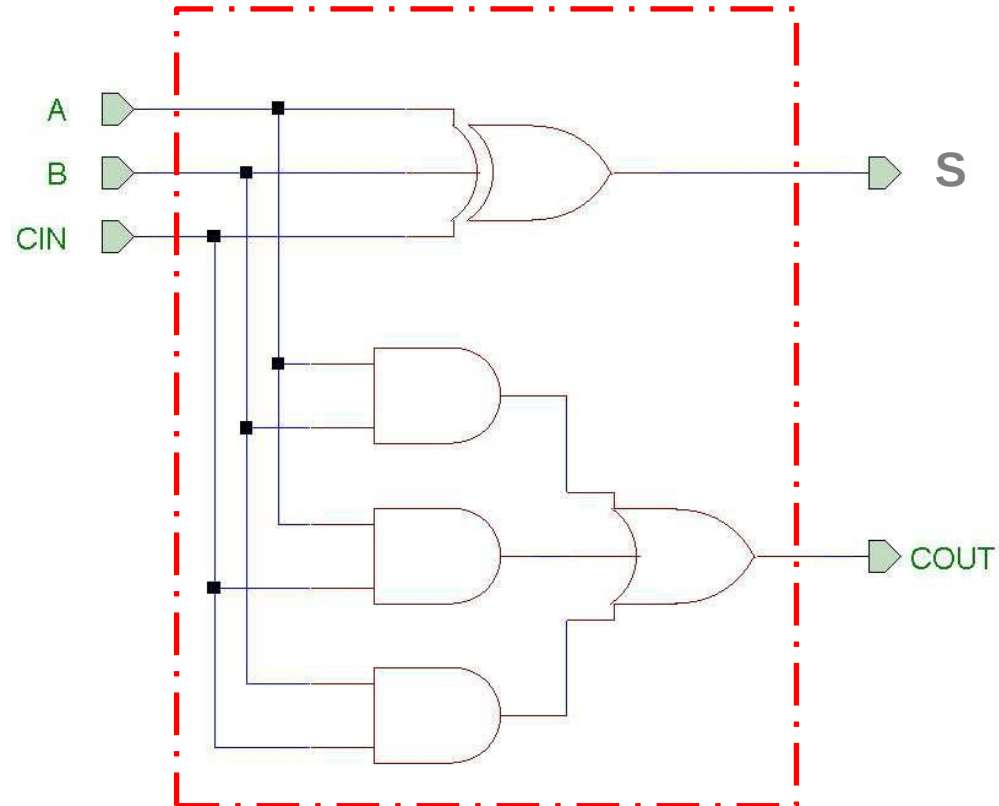
Sumador Elemental Completo (FULL ADDER)



$$C_{out} = a \cdot b + a \cdot C_{in} + b \cdot C_{in}$$

$$S = a \oplus b \oplus C_{in}$$

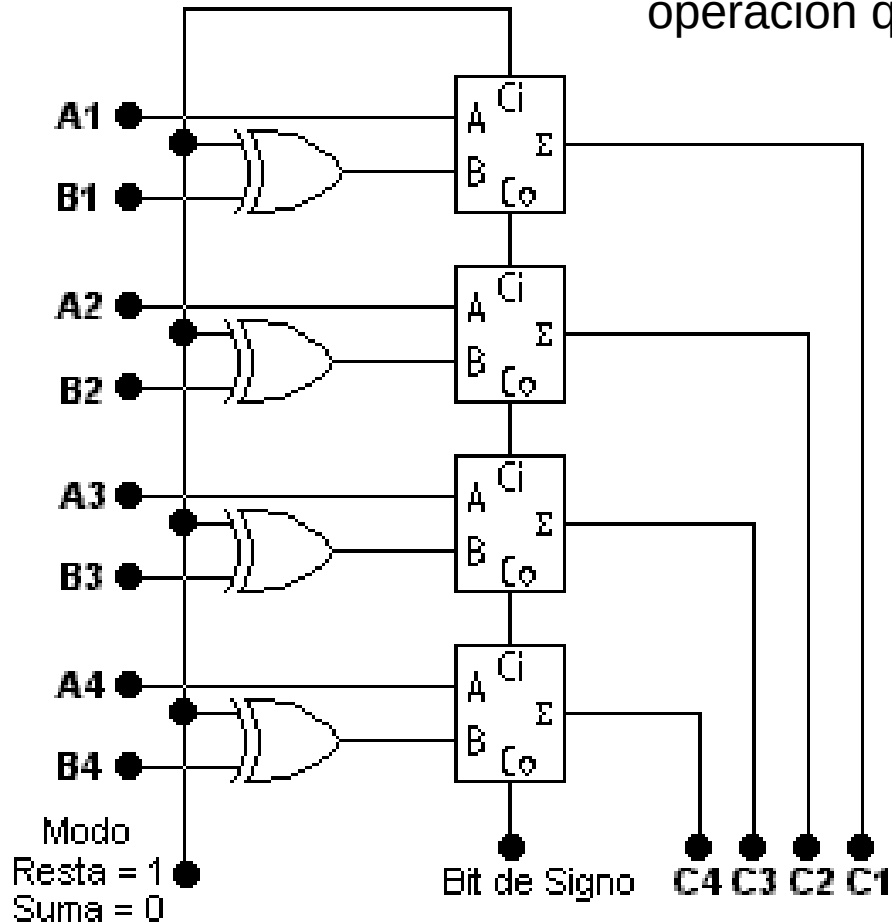
Si lo implementamos mediante compuertas:



Sumador / Restador

Los circuitos que realizan operaciones en paralelo son mas rápidos en sus respuestas, casi inmediatos para dar un resultado.

Al invertir los bits de “ B_i ” queda en C_{a1} , pero como “**Modo**” alimenta $C_{in-1} \Rightarrow$ la operación queda en C_{a2}



Modo	B_i	$X = \text{Modo} \oplus B_i$
0	0	$0 \Rightarrow B_i$
0	1	$1 \Rightarrow B_i$
1	0	$1 \Rightarrow \overline{B_i}$
1	1	$0 \Rightarrow \overline{B_i}$

Circuitos Combinacionales

El diseño de funciones lógicas, o de circuitos lógicos que vimos se ha realizado mediante el uso exclusivo de compuertas básicas.

Los circuitos que contienen estas compuertas lógicas son conocidos como **SSI (*Small Scale of Integration*)** porque contienen un número pequeño de transistores.

Un paso más profundo en el diseño HARDWARE es realizar un diseño basado en el uso de bloques constructores más complejos.

Esto se puede realizar mediante el uso de sistemas o circuitos **MSI (*Medium Scale of Integration*)** dónde el número de compuertas lógicas que lo componen es de entre **12** y **100**.

Más avanzados son los sistemas:

LSI (*Large Scale of Integration ~1.000*)

VLSI (*Very Large Scale of Integration >1.000*)

ULSI (*Ultra Large Scale of Integration >100.000*)

DECODIFICADOR

En las computadoras digitales, se emplean códigos binarios para representar cantidades discretas de información.

Un código binario de **n bits** es capaz de representar hasta **2^n** elementos distintos de información codificada.

Decodificar es convertir un código de entrada de n bits en un código de salida de m bits, donde **$n \leq m \leq 2^n$** , tal que para cada palabra válida codificada en la entrada exista un único código de salida.

DECODIFICADOR

Es un módulo Combinacional con **n** entradas y **2^n** salidas.

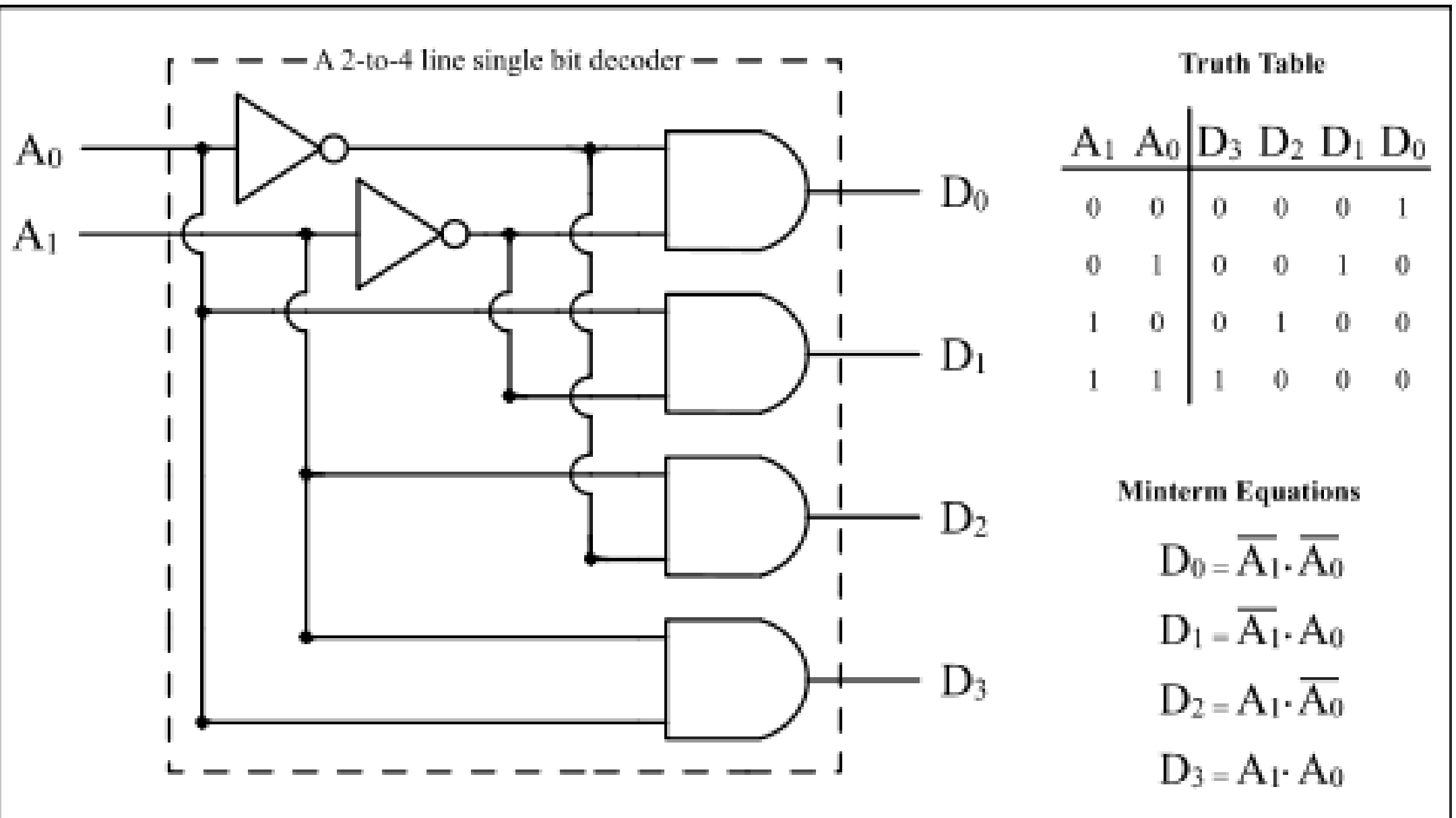
Cuando en sus **n** entradas se coloca una combinación de **n bits (dirección)**, una sola de sus **2^n** salidas se activa, permitiendo seleccionar la posición o celda de memoria de 8 bits deseada.

Un circuito Decodificador permite reconocer qué combinación está presente en sus n entradas, y con ello activará alguno de sus 2^n cables de salida.

En General, un Decodificador Completo de **n** entradas consta de **2^n** compuertas **AND** de **n** entradas.

Cada salida de una de las compuertas es una salida del circuito.

DISEÑO CON COMPUERTAS LÓGICAS



MULTIPLEXOR

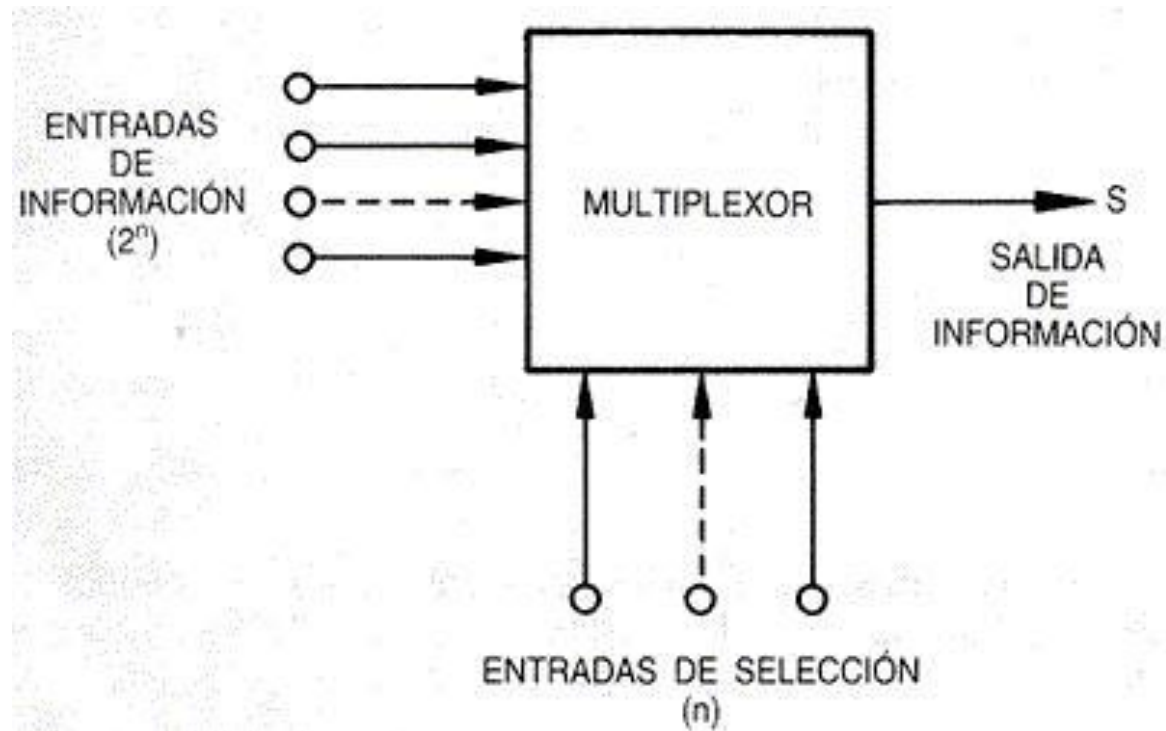
Un multiplexor es un circuito combinacional que selecciona información binaria de una de muchas líneas de entrada y la envía a una sola línea de salida. La selección de una línea de entrada dada se controla con un conjunto de líneas de selección. Normalmente, hay **2^n líneas de entrada** y **n líneas de selección** cuyas combinaciones de bits determinan cuál entrada se selecciona.

Permite seleccionar el valor lógico que aparecerá en su salida, entre los valores lógicos presentes en sus “ k ” (2^n) entradas de datos.

Su actuación consiste en canalizar múltiples fuentes de información binaria hacia una misma salida. La entrada de información activa en cada instante es la seleccionada a través de la configuración aplicada a las “ **n** ” líneas de selección.

La caracterización de los multiplexores obedece al número de entradas de información o canales que posea el sistema. Así, hay multiplexores de **2, 4, 8, 16... canales**

Multiplexor



$k = 2^n$ (ENTRADAS)

$n = n^\circ$ de VARIABLES DE SELECCIÓN

$s =$ SALIDA

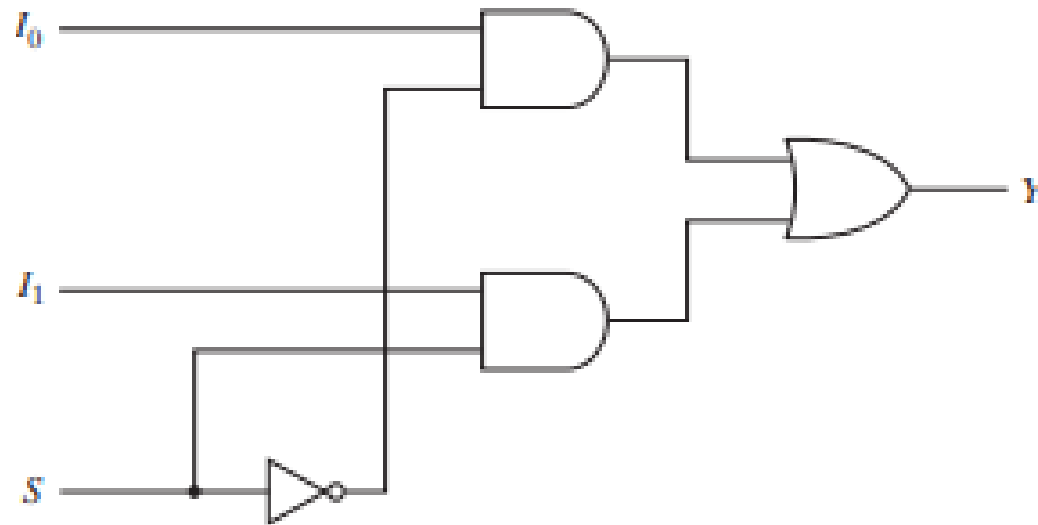
Multiplexor

Un multiplexor de **2 líneas a 1** conecta una de dos fuentes de un bit a un destino común, como se indica en el siguiente ejemplo. El circuito tiene **dos líneas de entrada de datos, una línea de salida y una línea de selección S**.

Cuando **$S=0$** , se habilita la compuerta **AND superior** e **I_0** cuenta con una trayectoria hacia la salida.

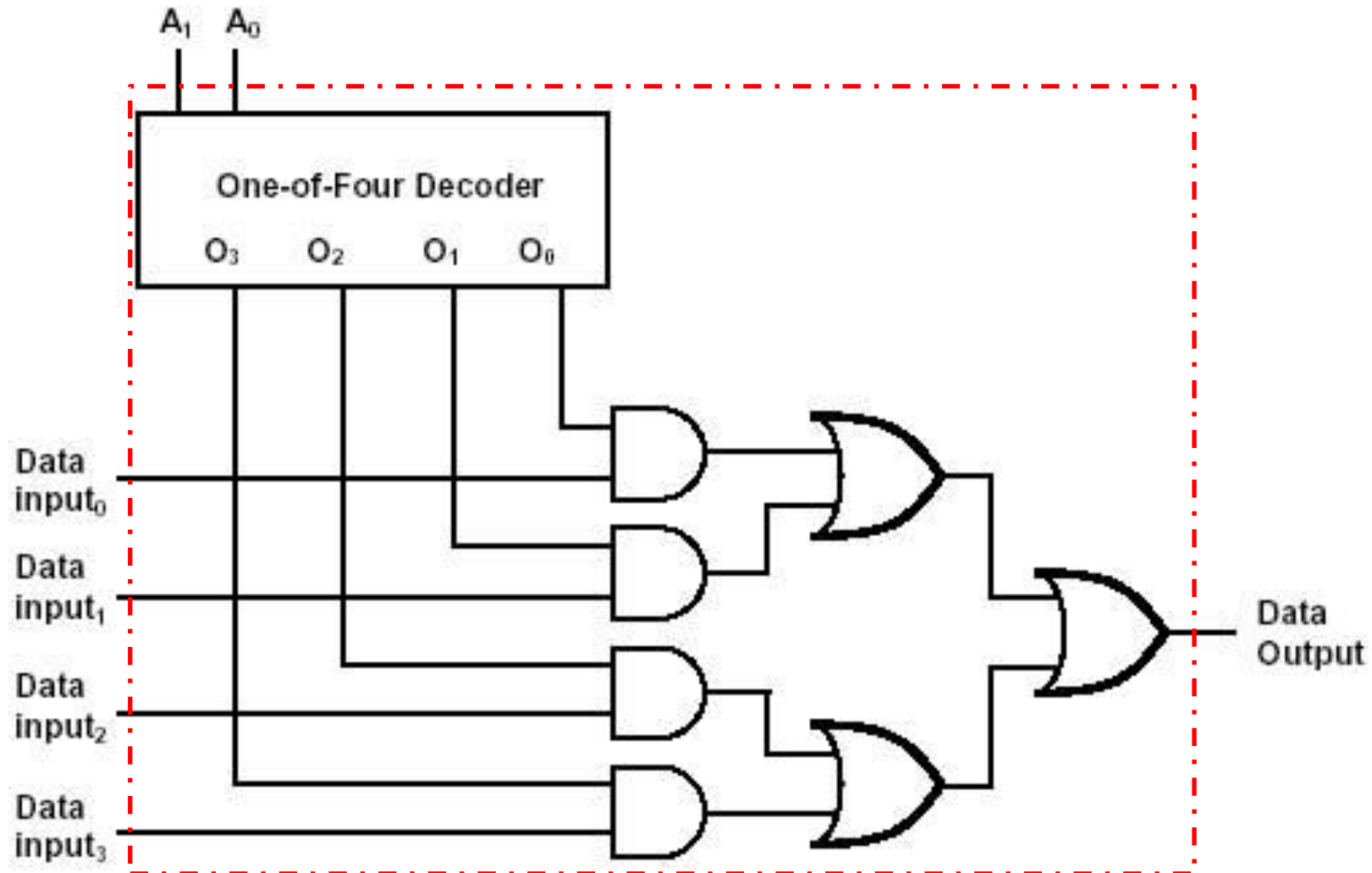
Cuando **$S=1$** , la compuerta **AND inferior** está habilitada e **I_1** tiene una trayectoria hacia la salida.

El **multiplexor** actúa como un interruptor electrónico que selecciona una de dos fuentes



Diseño con Compuertas Lógicas y Decodificador

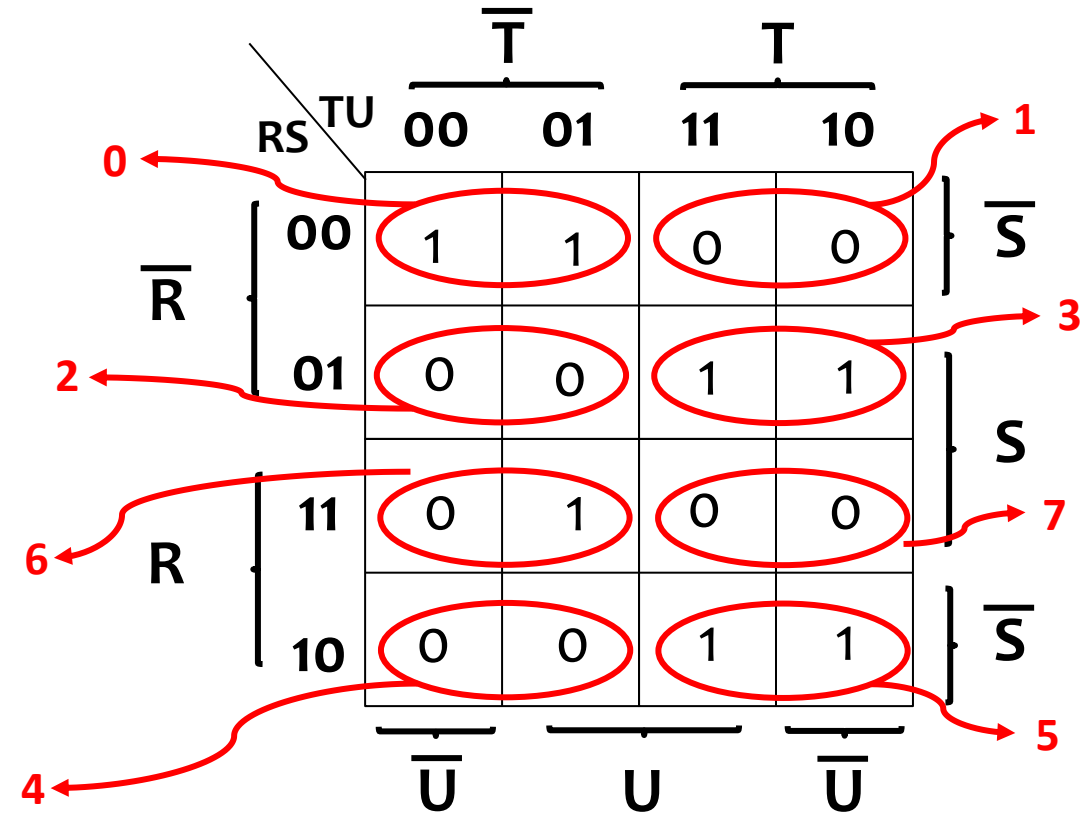
MULTIPLEXOR 4x2



Ej.: Dada la siguiente función, realizar un Multiplexor con 3 variables de control

$$F(R,S,T,U) = \sum m(0,1,6,7,10,11,13)$$

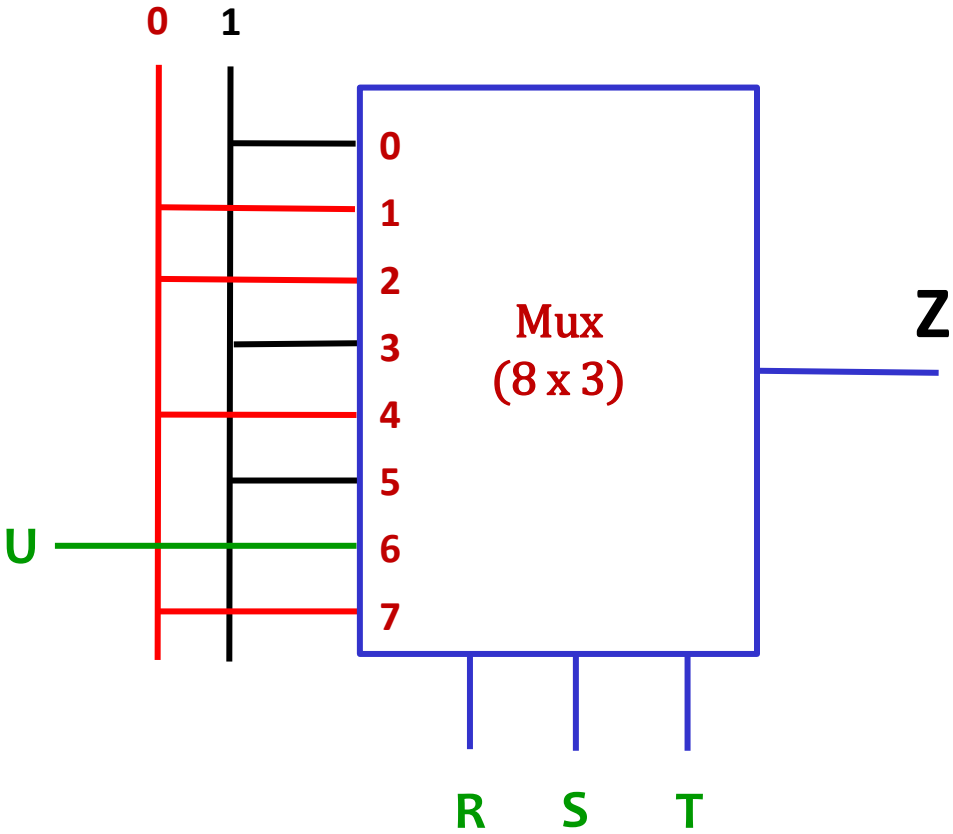
↑ ↑ ↑ ↑ ↑ ↑ ↑



Mux (8 x 3)

1 Variable de Residuo: U

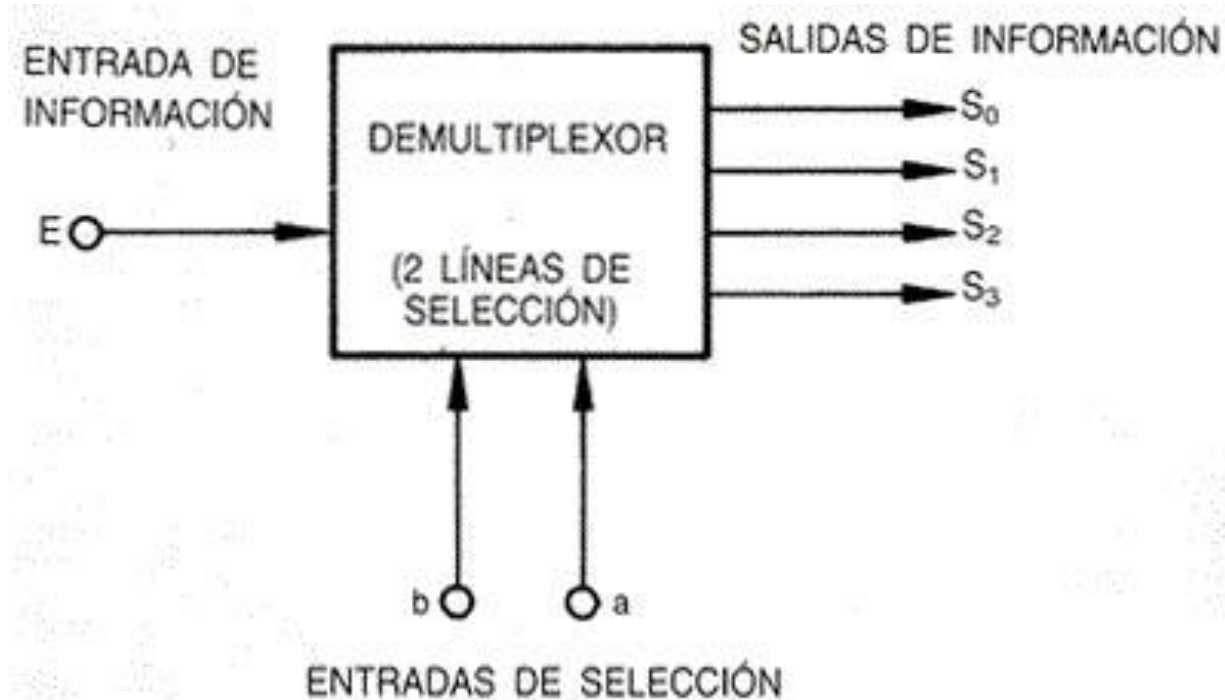
3 Variables de Control: R, S, T



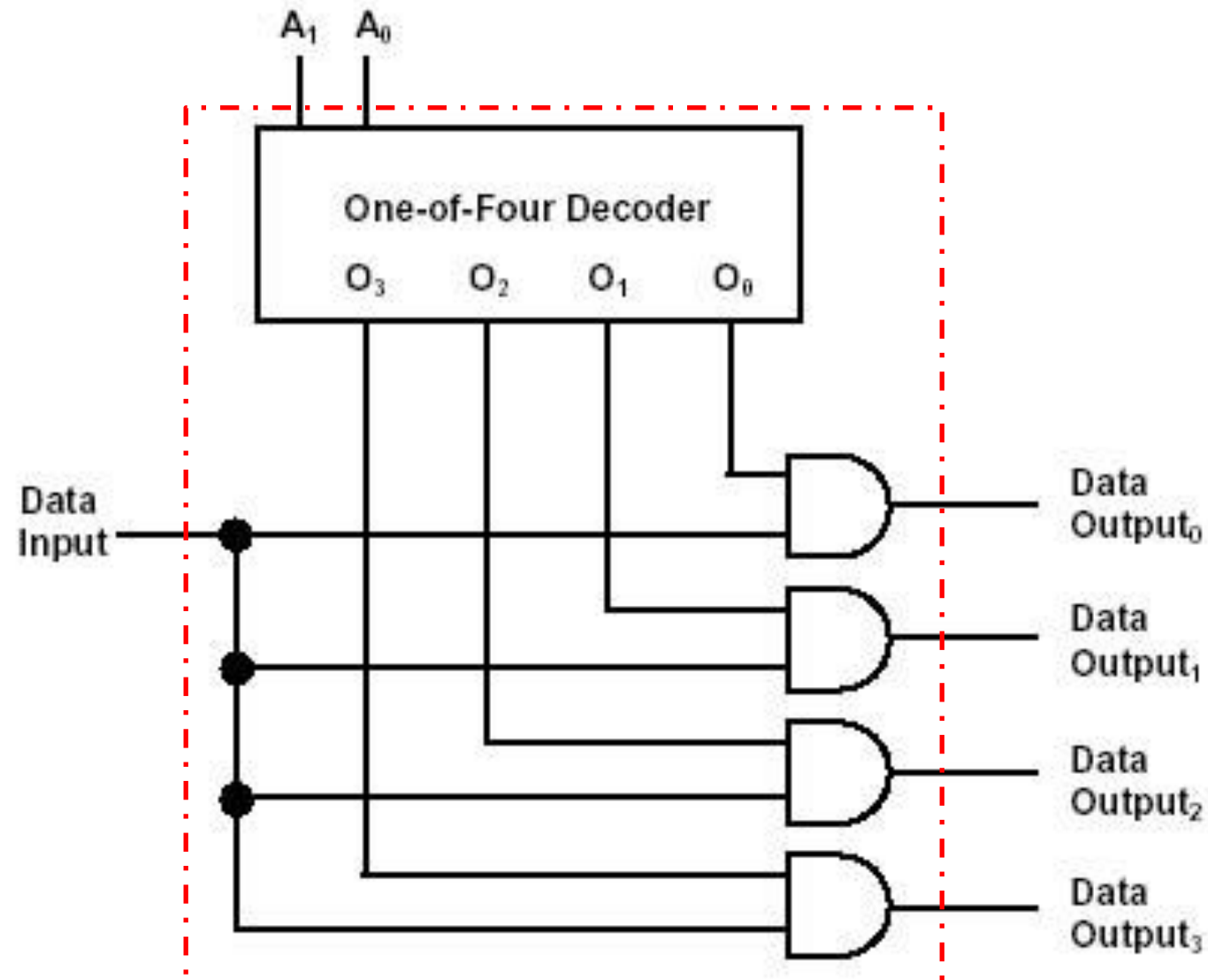
DEMULTIPLEXOR

Realiza la función inversa de un Multiplexor.

El valor lógico de la línea de entrada puede ser direccionado mediante “ n ” entradas de selección, para que aparezca en una de sus “ 2^n ” líneas de salida.



Diseño con Compuertas Lógicas y Decodificador



Mayor información

➤ Circuitos Secuenciales

- ✓ Apuntes 3 de Cátedra. Circuitos Integrados. Circuitos Combinacionales. Circuitos secuenciales. Memoria.

➤ Capítulo 6 (6.4)

- ✓ Quiroga patricia 1^{ra} Ed

➤ Capítulo 5 y 6

- ✓ Stallings, 7^{ma}. Ed.

➤ Apéndice B: Lógica digital (B.4 - B.5- B.6)

- ✓ Stallings, 7^{ma}. Ed.

➤ Links de Interés:

- ✓ Demo circuitos digitales:

http://www.play-hookey.com/digital/combinational/derived_gates.html

CATEDRA DE ARQUITECTURA DE COMPUTADORAS

**CIRCUITOS SECUENCIALES:
FLIP FLOP**

Introducción - Lógica Secuencial

- Las salidas dependen tanto de las entradas como del estado interno del circuito.
- Tienen la característica de “**almacenar**” valores lógicos internamente.
- Estos valores se almacenan aunque las entradas no estén.
- La salida es también entrada (**lazo de realimentación**).
- En ningún circuito combinatorio una salida transportaba información hacia la entrada.

Introducción - Lógica Secuencial

➤ La mayor parte de los sistemas encontrados en la práctica también incluyen elementos de almacenamiento que requieren que el sistema se describa en términos de circuitos secuenciales.

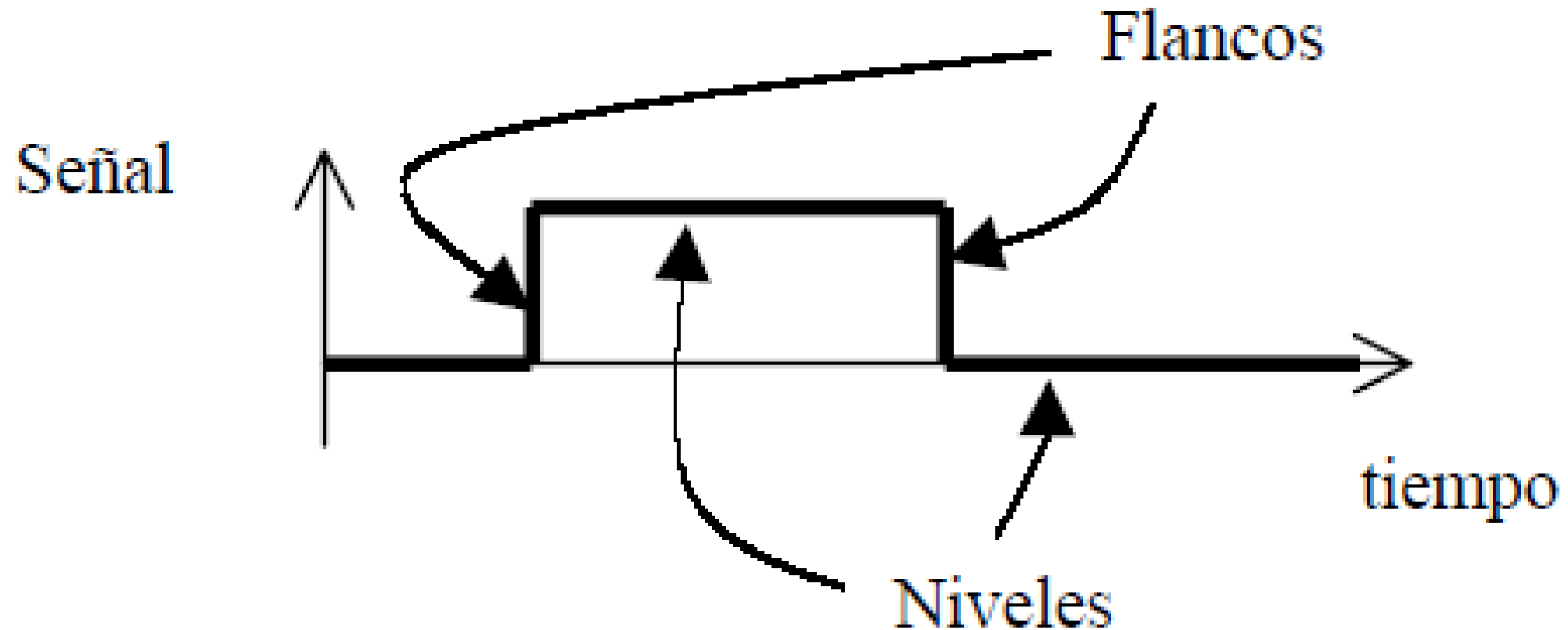
➤ Elementos de memoria: **Flip-Flop**

➤ Sincronismo:

- ✓ El tipo de circuito secuencial más común es el tipo **síncrono**, esto significa que los elementos de almacenamiento se afectan sólo en instantes de tiempo discretos.
- ✓ La sincronización se genera en un dispositivo denominado **Reloj (clock)** que produce un tren periódico de pulsos

Circuitos Secuenciales

En dicho tren de pulsos se pueden indicar las siguientes partes:



Circuitos Secuenciales

Sus salidas dependen del valor de las entradas y del estado anterior del circuito, puede tener una o más salidas. Tiene uno o más lazos de realimentación.

Tipos de Circuitos Secuenciales:

Asincrónicos: (RS)

Los cambios solo se producen cuando están presentes las entradas, sin necesidad de Señal de Reloj (Clock).

Sincrónicos: (RS, D, JK y T)

Los cambios se producen cuando se establecen las entradas y, además, se genera una transición de Señal de Reloj.

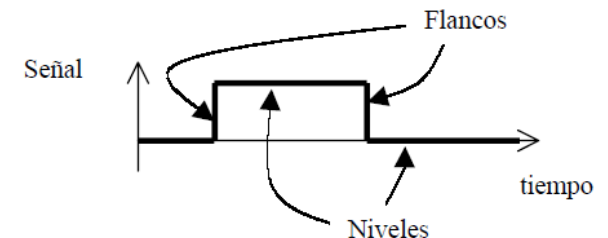
PUEDEN SER ACTIVADOS:



POR NIVEL (ALTO O BAJO)



POR FLANCO (ASCENDENTE O DESCENDENTE)



Circuitos Secuenciales

BIESTABLES o FLIP-FLOP

Un **BIESTABLE** es una celda binaria capaz de almacenar un bit. Tiene dos salidas, una para el bit almacenado y otra que representa su complemento.

Se llaman así porque poseen dos estados posibles de funcionamiento. Permaneciendo en cualquiera de ellos si los niveles lógicos de entrada no fuerzan un cambio.

Generalmente es un arreglo de compuertas, caracterizado por tener lazos de Realimentación que permiten mantener o memorizar el efecto de combinaciones anteriores en las entradas.

Para que un circuito sea un elemento de memoria debe cumplir con dos condiciones:

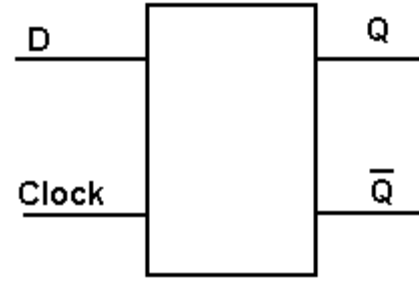
- **“Retener un bit”, aún después de la desactivación de las entradas.**
- **Poder ser “puesto a 0 o puesto a 1”, independientemente del valor del bit almacenado con anterioridad.**

Circuitos Secuenciales

Características de los FLIP-FLOPS

- Un **Flip-Flop** es una celda binaria capaz de almacenar un bit de información. Tiene dos salidas, una para el valor normal y otra para el valor complementario.
- La diferencia entre los diversos tipos de **Flip-Flop** está en el número de entradas que posean y la manera en la cual las entradas afectan el estado de memoria (**RS**, **JK**, **D** y **T**)
- **Tablas Característica:** Especifican el estado siguiente cuando se conocen las entradas y el estado presente, por lo general, durante el diseño de un circuito secuencial, se conoce la transición requerida del estado presente al siguiente estado y requeriremos encontrar las condiciones de entrada del **Flip-Flop** que causen esa transición, para esto se usan las tablas de excitación.

BIESTABLE “D” (DATA O DELAY)



El dispositivo de almacenamiento temporal es de dos estados (**alto** y **bajo**), cuya salida adquiere el valor de la entrada **D** cuando se activa la entrada de sincronismo (**Clock**). En función del modo de activación de dicha entrada de sincronismo, existen dos tipos:

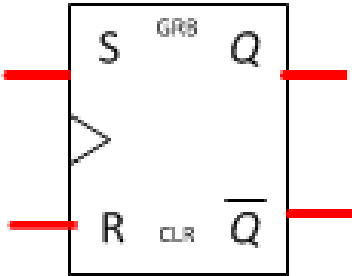
- ✓ **Activo por nivel (alto o bajo)**, también denominado **registro** o **cerrojo** (***latch*** en inglés).
- ✓ **Activo por flanco (de subida o de bajada)**.

La ecuación característica del biestable **D** que describe su comportamiento es:

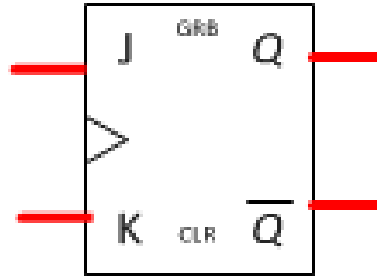
$$Q_{siguiente} = D$$

Tablas de excitación de Flip - Flops

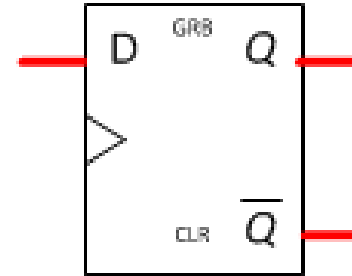
Tipos de Flip-Flop:



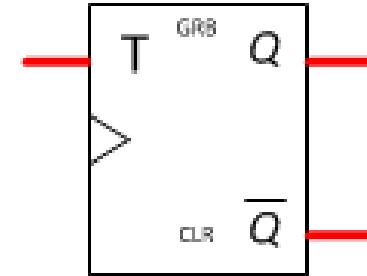
S_t	R_t	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	X



J_t	K_t	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	$\overline{Q}_t$



D_t	Q_{t+1}
0	0
1	1



T_t	Q_{t+1}
0	Q_t
1	$\overline{Q}_t$

Flip – Flops : Karnaugh - Ecuación

S_t	R_t	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	X

Q_t $S_t R_t$		0	1
00	0	0	1
01	2	0	3
11	6	X	7
10	4	1	5

$$Q_{t+1} = S_t + \bar{R}_t \cdot Q_t$$

J_t	K_t	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	$\bar{Q}_t$

Q_t $J_t K_t$		0	1
00	0	0	1
01	2	0	3
11	6	1	7
10	4	1	5

$$Q_{t+1} = J_t \cdot \bar{Q}_t + \bar{K}_t \cdot Q_t$$

D_t	Q_{t+1}
0	0
1	1

Q_t D_t		0	1
0	0	0	1
1	2	1	3

$$Q_{t+1} = D_t$$

T_t	Q_{t+1}
0	Q_t
1	$\bar{Q}_t$

Q_t T_t		0	1
0	0	0	1
1	2	1	3

$$Q_{t+1} = \bar{T}_t \cdot Q_t + T_t \cdot \bar{Q}_t$$

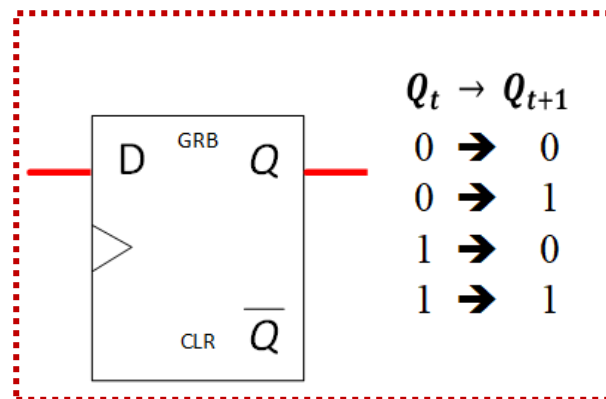
Tablas de excitación de Flip - Flops

S_t	R_t	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	X

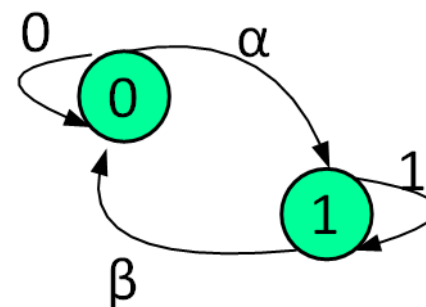
J_t	K_t	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	$\overline{Q}_t$

D_t	Q_{t+1}
0	0
1	1

T_t	Q_{t+1}
0	Q_t
1	$\overline{Q}_t$



$Q_t \rightarrow Q_{t+1}$	Trans.	D_t	T_t	J_t	K_t	S_t	R_t
$0 \rightarrow 0$	0	0	0	0	X	0	X
$0 \rightarrow 1$	α	1	1	1	X	1	0
$1 \rightarrow 1$	1	1	0	X	0	X	0
$1 \rightarrow 0$	β	0	1	X	1	0	1



Mayor información

➤ Circuitos Secuenciales

- ✓ Apuntes 3 de Cátedra. Circuitos Integrados. Circuitos Combinacionales. Circuitos secuenciales. Memoria.

➤ Capítulo 6 (6.4)

- ✓ Quiroga patricia 1^{ra} Ed

➤ Capítulo 5 y 6

- ✓ Stallings, 7^{ma}. Ed.

➤ Apéndice B: Lógica digital (B.4 - B.5- B.6)

- ✓ Stallings, 7^{ma}. Ed.

➤ Links de Interés:

- ✓ Demo Flip-Flop:

http://www.play-hookey.com/digital/alt_flip_flops/

CATEDRA DE ARQUITECTURA DE COMPUTADORAS

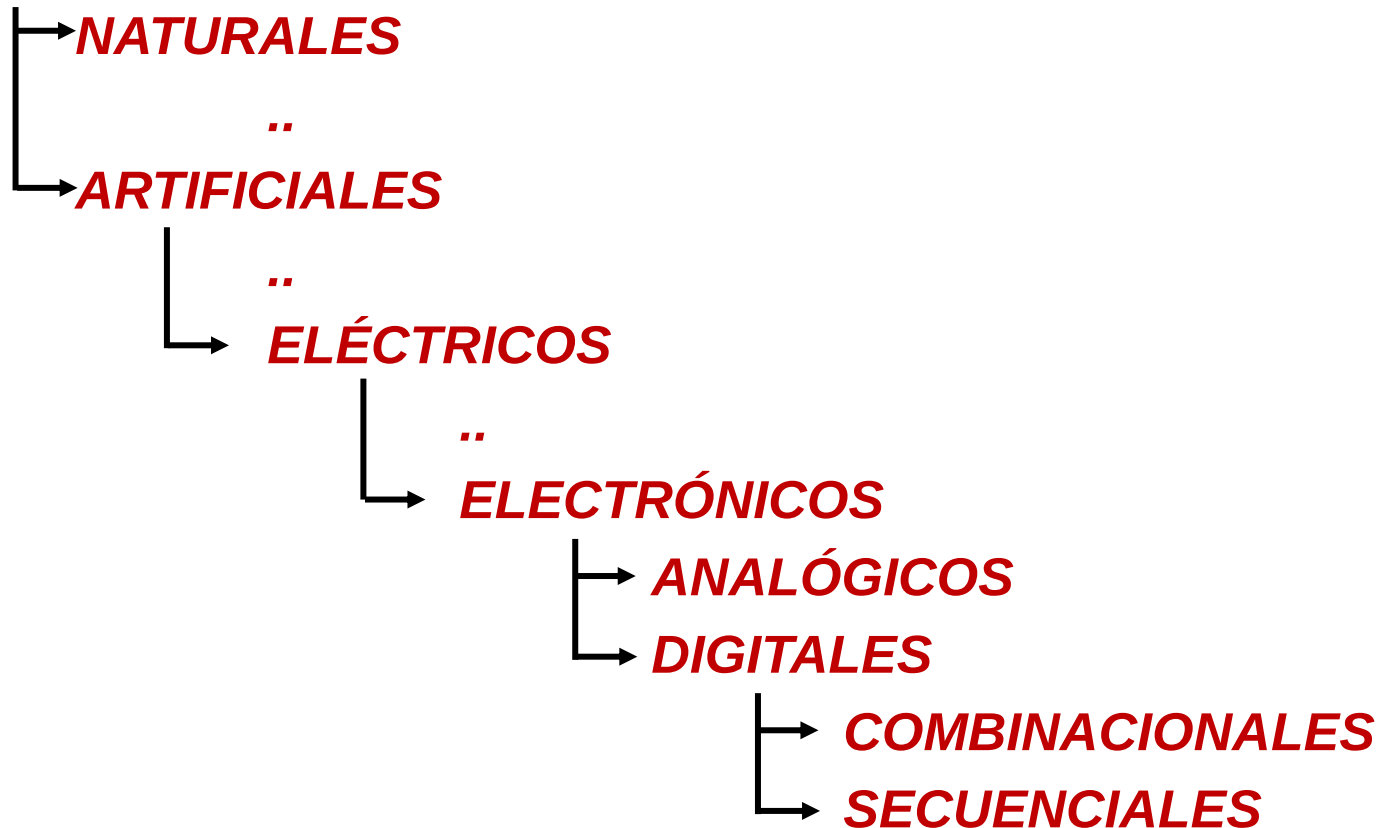
CIRCUITOS LÓGICOS

INTRODUCCION

Un **Sistema** es un conjunto de elementos que guardan una relación entre sí, a su vez un elemento del sistema puede ser otro sistema (**subsistema**).

Los Sistemas se clasifican de la siguiente manera:

SISTEMAS



INTRODUCCION

Sistemas Digitales: Sus elementos sólo pueden adoptar valores discretos.

Por cuestiones de confiabilidad y costo la **base 2** es la más adecuada para la elección de un sistema de numeración.

Por esta razón los **Sistemas Digitales** emplean **elementos binarios** (aquellos que sólo pueden adoptar dos valores).

Por lo expuesto, conocer el **álgebra binaria** es fundamental para poder estudiar los **Sistemas Digitales**.

El **Álgebra de George Boole (1854)** resulta la más apropiada para nuestro fin. Claude Shannon en **1938** adaptó esta álgebra para la aplicación en sistemas digitales.

Seguidamente se estudia el **álgebra de Boole**, las **funciones booleanas**, las **compuertas lógicas**, los **Sistemas Combinacionales** y, finalmente, los **Sistemas Secuenciales**.

ÁLGEBRA DE BOOLE

Constituye una formalización apropiada para representar información digital y permite expresar en forma algebraica las operaciones lógicas que realizan los circuitos digitales y determinar su respuesta.

Nos permite:

- ✓ **Realizar el análisis y síntesis de circuitos digitales combinacionales.**
- ✓ **La minimización del número de cables y compuertas de los circuitos.**
- ✓ **Simplificar las Tablas de Decisión.**

El Álgebra de Boole consiste en un Conjunto de **Axiomas** (Postulados) y **Teoremas**.

- Los **Axiomas** son premisas que se aceptan como base del sistema y no se deducen de otras.
- Los **Teoremas** son una proposición que afirma una verdad demostrable.

El álgebra es un conjunto de elementos binarios relacionados entre sí mediante las operaciones lógicas **producto “.”** y **suma “+”**, que cumplen con los siguientes postulados (las letras **a**, **b**, **c**, etc., indican variables binarias):

1) Existe el elemento identidad

$$a + 0 = a$$

$$a . 1 = a$$

2) Las dos operaciones cumplen con la propiedad conmutativa

$$a + b = b + a$$

$$a . b = b . a$$

3) Propiedad distributiva

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$$

$$a + (b \cdot c) = (a + b) \cdot (a + c)$$

$$(a \cdot b) + c = (a + c) \cdot (b + c)$$

$$a + \bar{a} \cdot b = a + b \quad \Rightarrow \quad \text{Viene de: } a + \bar{a} \cdot b = \underbrace{(a + \bar{a})}_{=1} \cdot (a + b) = a + b$$

4) Complementación o inversión lógica

$$a + \bar{a} = 1$$

$$a \cdot \bar{a} = 0$$

Algunos teoremas importantes:

1) Principio de Dualidad:

Toda igualdad lógica sigue siendo válida si se intercambian los operadores (“+” y “.”) y los elementos de identidad (**0** y **1**). La simetría de los postulados demuestra este teorema.

$$a + \bar{a} = 1 \quad \Rightarrow \quad a \cdot \bar{a} = 0$$

2) El álgebra es un conjunto cerrado; es decir, los resultados de aplicar las operaciones lógicas a las variables, pertenecen al álgebra.

3) En el álgebra se cumple lo siguiente:

$$a + 1 = 1$$

$$a \cdot 0 = 0$$

4) Ley de Idempotencia

$$a + a = a$$

$$a \cdot a = a$$

5) Ley de involución

$$\overline{\overline{a}} = a$$

6) Las operaciones lógicas son asociativas

$$a + (b + c) = (a + b) + c$$

$$a \cdot (b \cdot c) = (a \cdot b) \cdot c$$

7) Absorción

$a + (a \cdot b) = a \quad \Rightarrow \quad a \cdot (1 + b) = a \cdot 1 = a$

$a \cdot (a + b) = a \quad \Rightarrow \quad a \cdot a + a \cdot b = a + a \cdot b = a \cdot (1 + b) = a \cdot 1 = a$

8) Leyes de De Morgan

$\overline{(a + b + c + d + + n)} = \overline{a} \cdot \overline{b} \cdot \overline{c} \cdot \overline{d} \overline{n}$

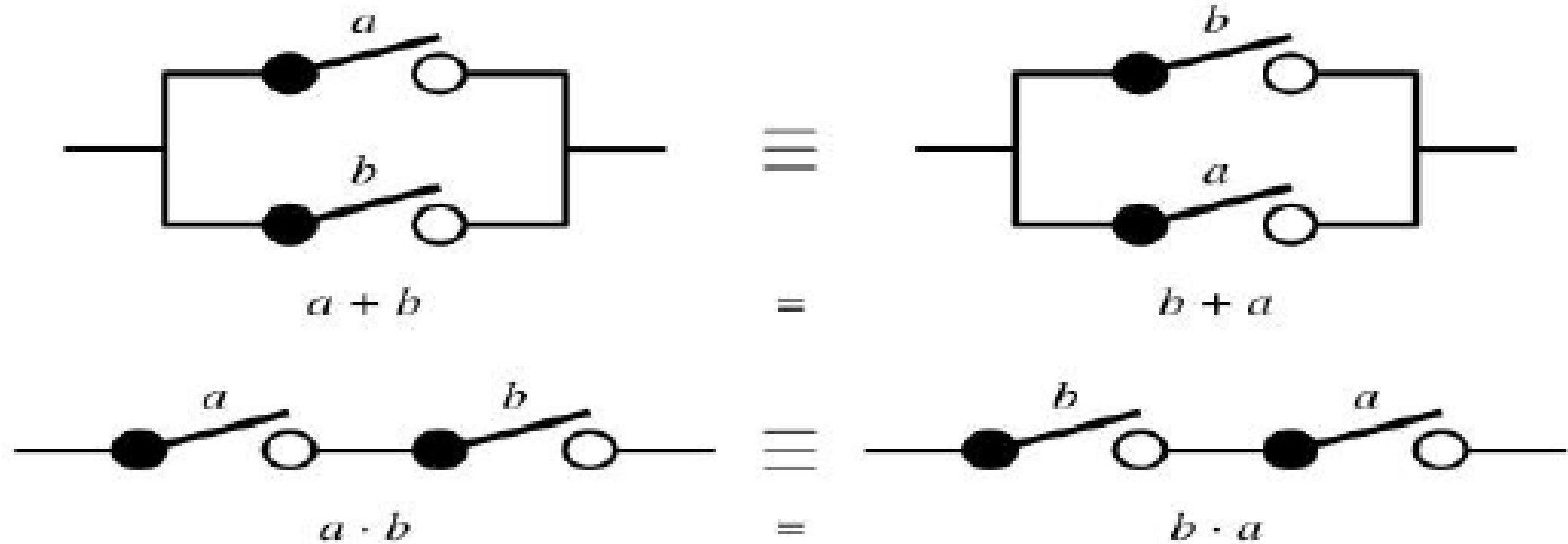


$\overline{(a \cdot b \cdot c \cdot d n)} = \overline{a} + \overline{b} + \overline{c} + \overline{d} + + \overline{n}$



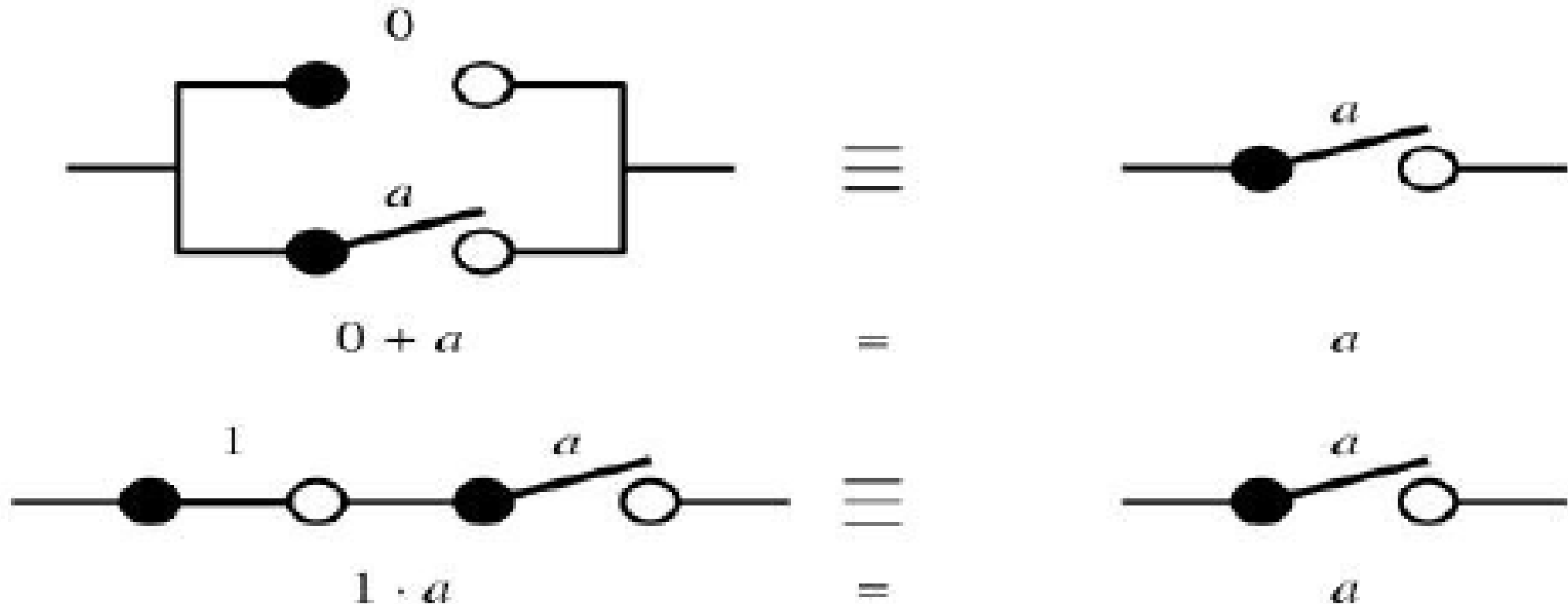
El Álgebra de Boole en Lenguaje de Contactos – Circuitos

Propiedad Conmutativa



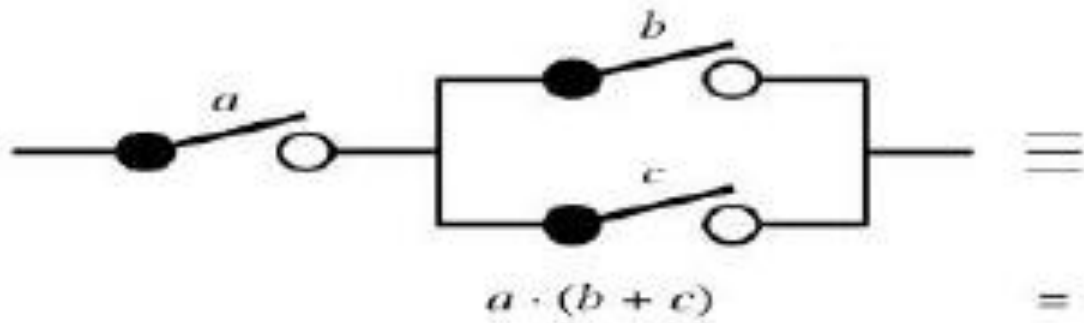
El Álgebra de Boole en Lenguaje de Contactos – Circuitos

Identidad

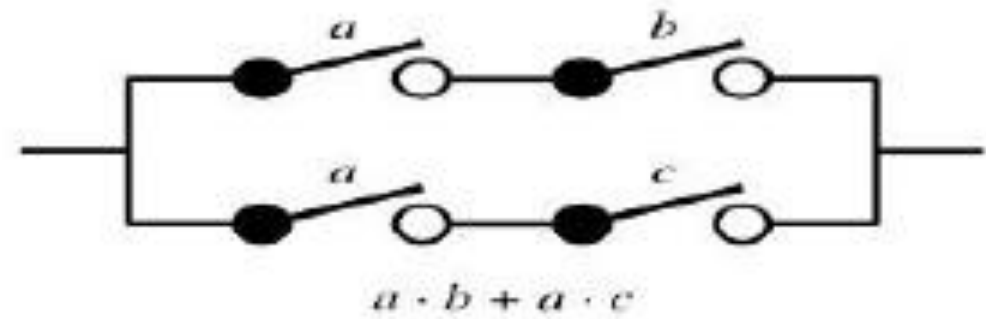


El Álgebra de Boole en Lenguaje de Contactos – Circuitos

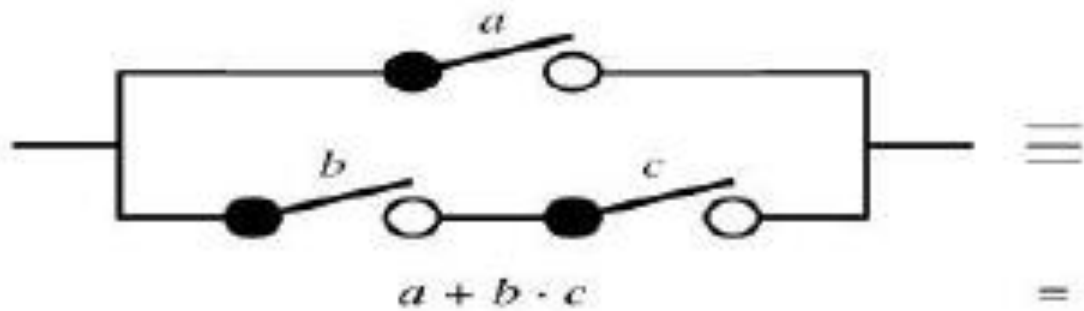
Propiedad Distributiva



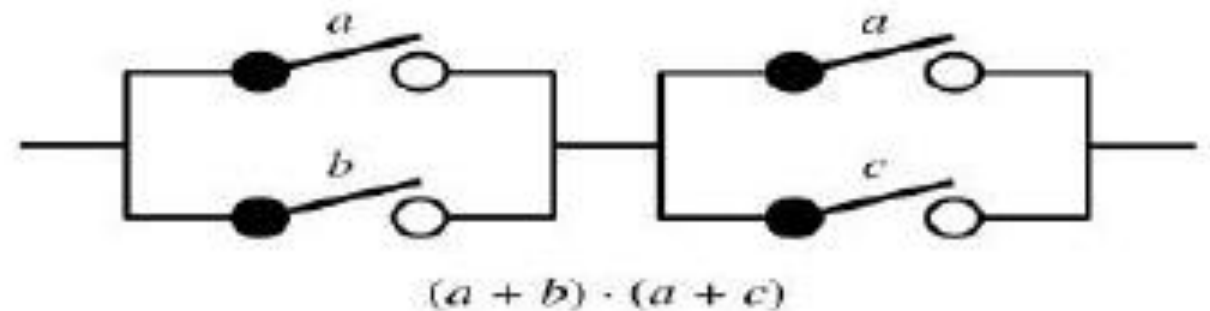
$\equiv$



$=$



$\equiv$



$=$

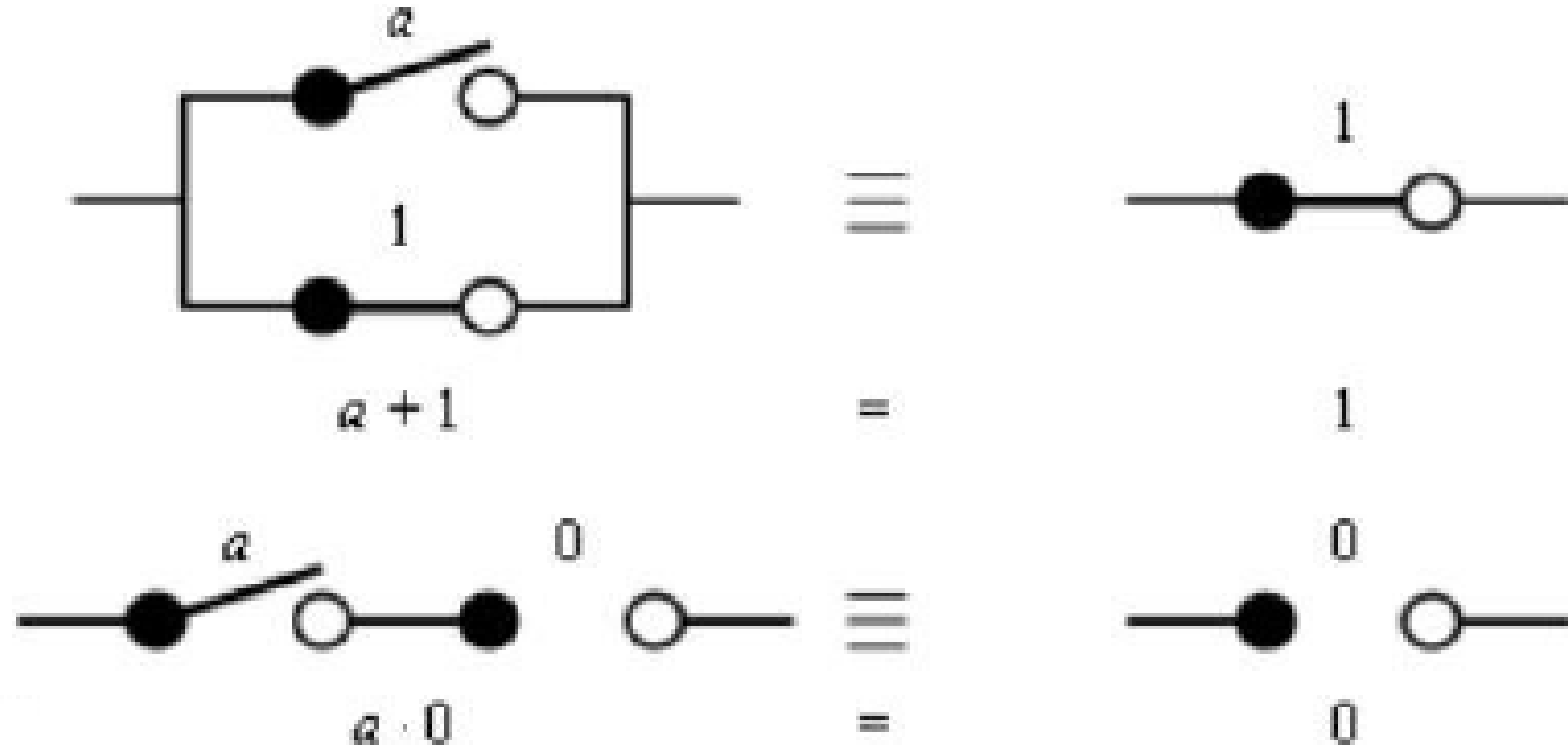
El Álgebra de Boole en Lenguaje de Contactos – Circuitos

Complemento o Inversión



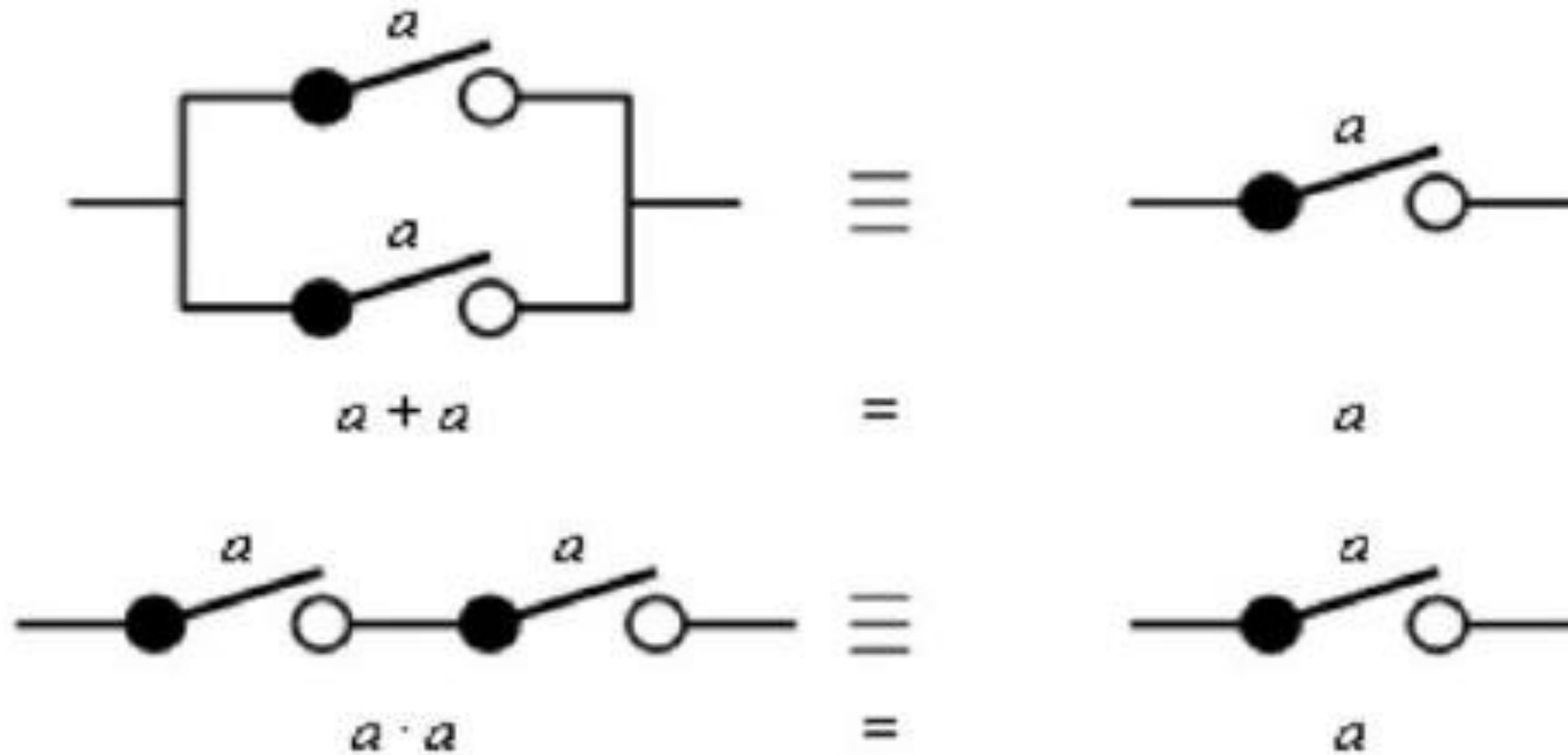
El Álgebra de Boole en Lenguaje de Contactos – Circuitos

Teorema 3 - En el álgebra se cumple lo siguiente



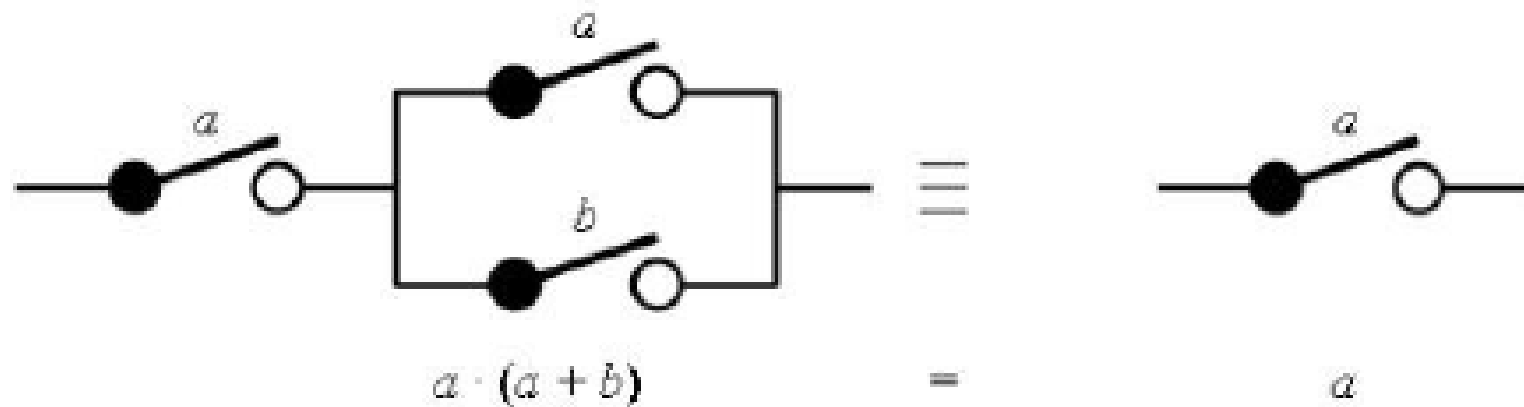
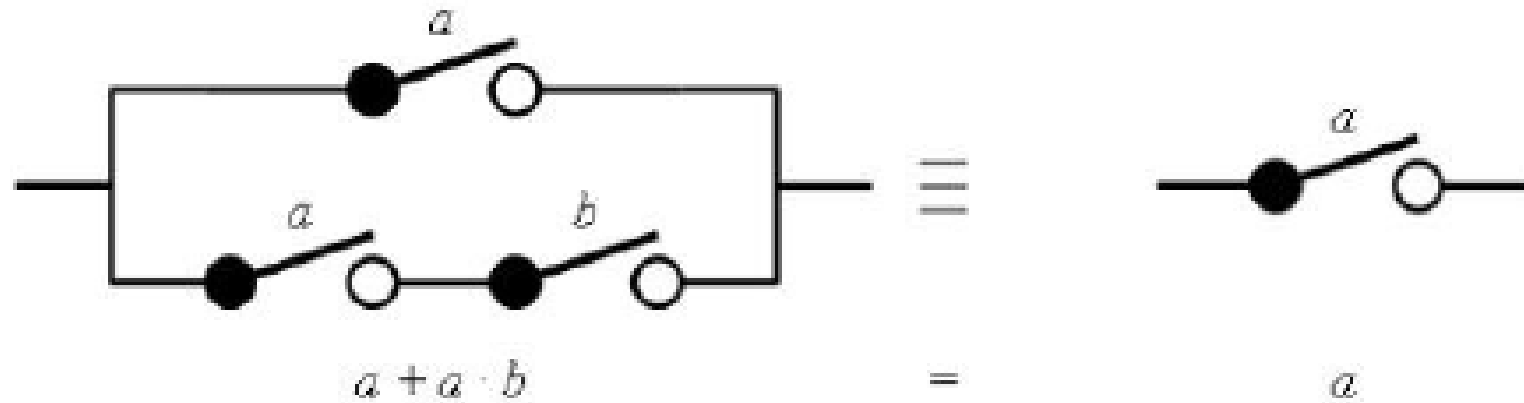
El Álgebra de Boole en Lenguaje de Contactos – Circuitos

Teorema 4 - Ley de Idempotencia



El Álgebra de Boole en Lenguaje de Contactos – Circuitos

Teorema 7 – Ley de Absorción



Jerarquía de Operadores

Al evaluar una expresión booleana, deben realizarse las operaciones de acuerdo a su nivel jerárquico.

- Si existen paréntesis, de adentro hacia afuera.
- Si no existen paréntesis:

1°) NOT (NEGACIÓN)

2°) AND (PRODUCTO)

3°) OR (SUMA)

Compuertas Lógicas

Podemos conmutar el nivel de tensión de un cable si se conmuta el estado de una llave.

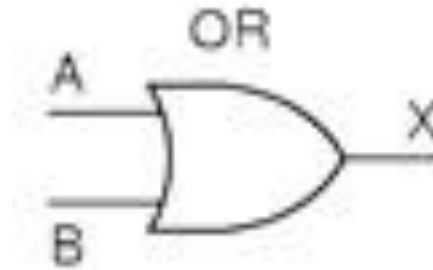
Mediante un dispositivo electrónico denominado “**COMPUERTA LÓGICA**” se puede conmutar el nivel de tensión de un cable conectado a su salida.

COMPUERTA: Porque se relaciona con el hecho de que este dispositivo puede usarse para permitir o no que el nivel alto o bajo de un cable que llega a una de sus entradas se repita en su salida.

LÓGICA: En esencia realiza electrónicamente una operación lógica.

Para cada combinación de valores lógicos existentes en las entradas, resultará un valor lógico en su salida.

Compuerta "OR"

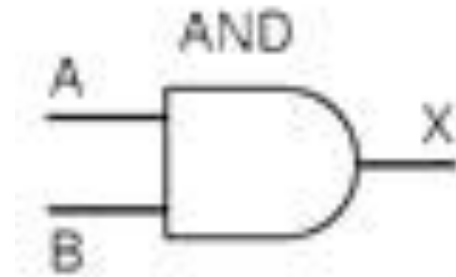


$$X = A + B$$

Tabla de Verdad:

A	B	$X = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

Compuerta “AND”

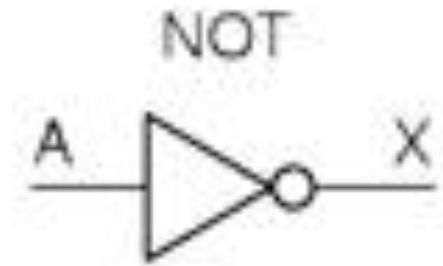


$$X = A \cdot B$$

Tabla de Verdad:

A	B	$X = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

Compuerta “INVERSOR”

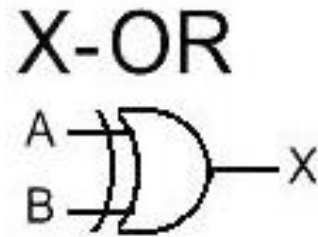


$$X = \bar{A}$$

Tabla de Verdad:

A	$X = \bar{A}$
0	1
1	0

Compuerta “XOR (OR EXCLUSIVO)”

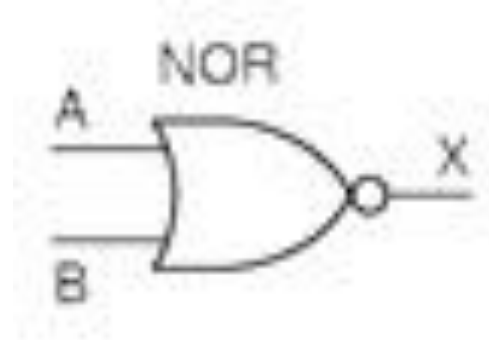


$$X = A \oplus B = \bar{A}.B + A.\bar{B}$$

Tabla de Verdad:

A	B	$X = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Compuerta “NOR”

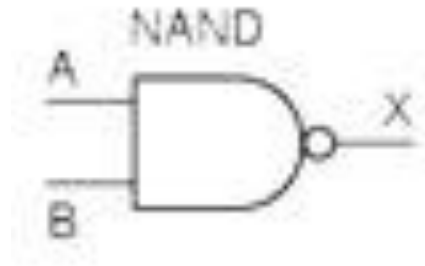


$$X = \overline{A + B}$$

Tabla de Verdad:

A	B	$X = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

Compuerta “NAND”



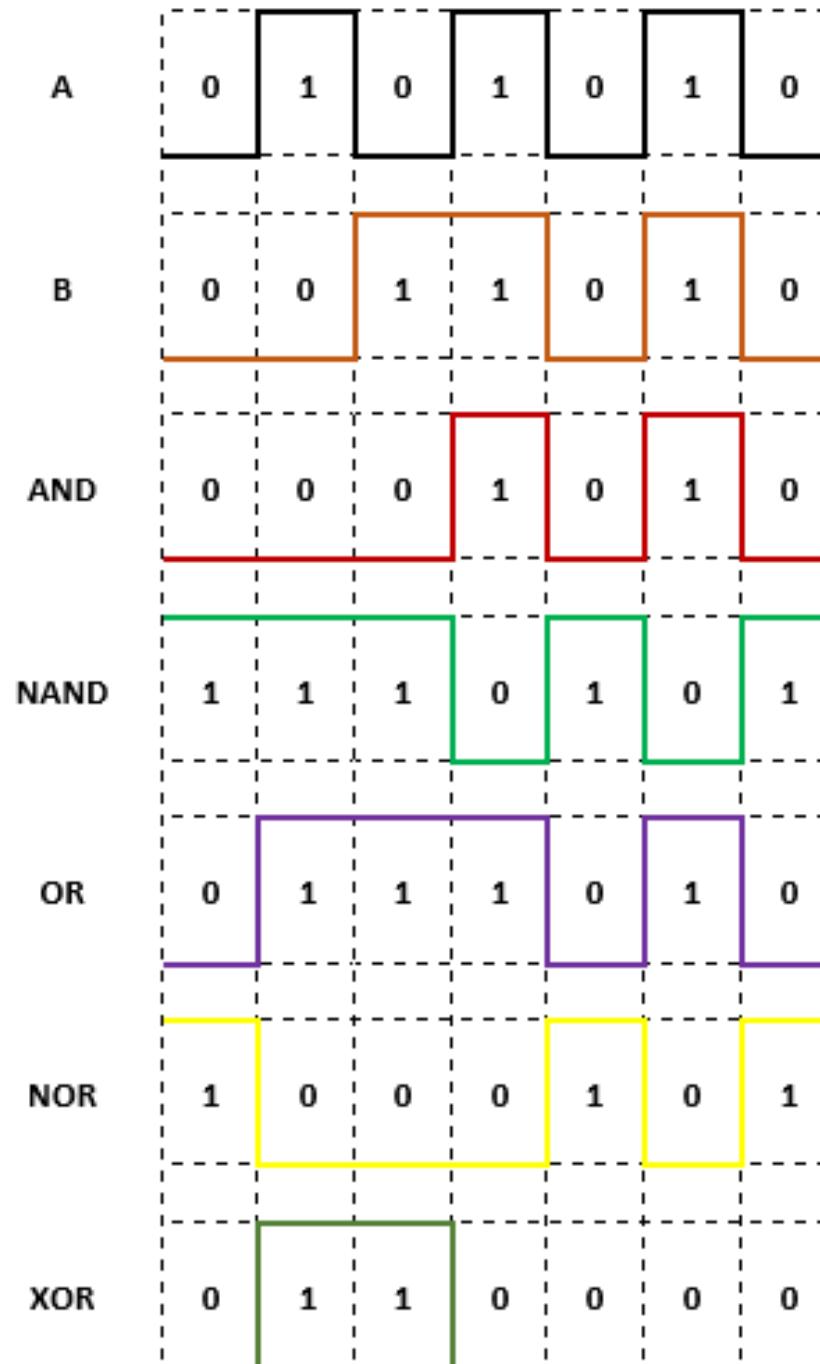
$$X = \overline{A \cdot B}$$

Tabla de Verdad:

A	B	$X = \overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

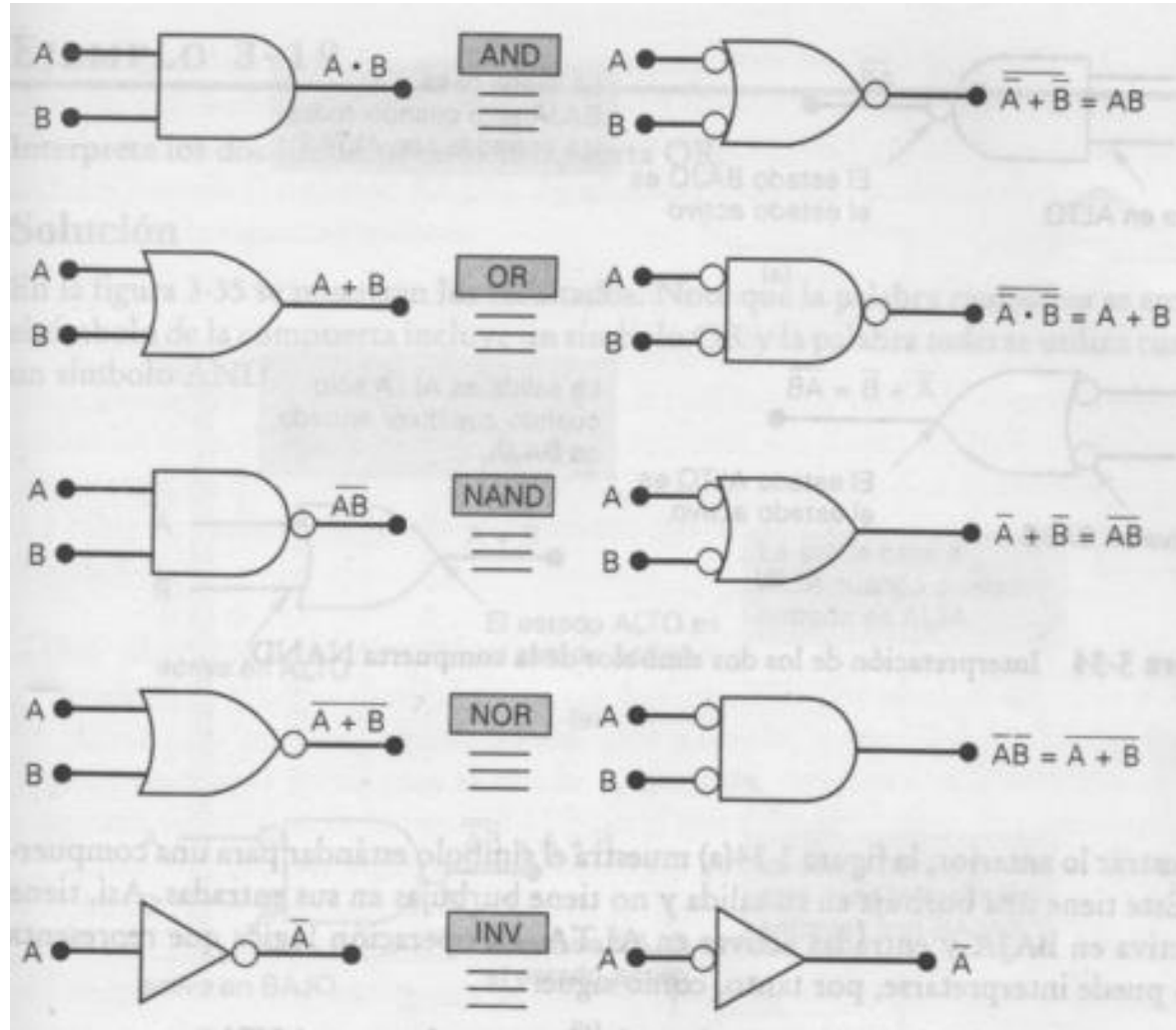
Diagramas Temporales

Los Diagramas Temporales representan el comportamiento de un circuito digital a través del tiempo.



Equivalencia entre Funciones Lógicas

Dos expresiones booleanas o funciones son **EQUIVALENTES** entre sí si poseen igual tabla de verdad.



Formas Canónicas

Término canónico de una función lógica:

Es todo producto o suma en los cuales aparecen todas las variables en su forma directa o inversa.

Las funciones lógicas son expresables en **Forma Canónica**, ya sea como **“SUMA DE MINTÉRMINOS”** o como **“PRODUCTO DE MAXTÉRMINOS”**.

MINTÉRMINOS

Dado un número de “**n**” variables, un **MINTÉRMINO** es un Producto Lógico, cuyos factores son “**TODAS**” las variables, negadas o no.

Ejemplo:

$$\overline{A} . \overline{B} . C = 001 \text{ (3 variables)}$$

$$\overline{A} . \overline{B} . \overline{C} . D = 0001 \text{ (4 variables)}$$

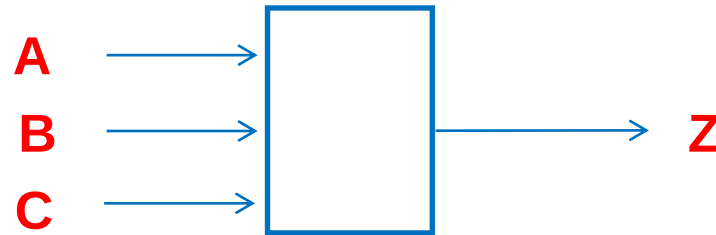
Para cada **MINTÉRMINO**, existe una sola combinación para la cual el producto resulta igual a **1** y en forma recíproca, dada una combinación de valores de las variables, existe un solo **MINTÉRMINO** que resulta igual a **1** para dicha combinación.

MINTÉRMINOS

a	b	<i>Minitérminos</i>
0	0	$\overline{a} \cdot \overline{b}$
0	1	$\overline{a} \cdot b$
1	0	$a \cdot \overline{b}$
1	1	$a \cdot b$

Expresión de una Función como Suma de Mintérminos

Supongamos que se necesita un circuito con **3 entradas (A, B y C)** y una salida “**Z**”, tal que esta valga ‘**1**’ si en las entradas se aplica la combinación “**011**”, o la “**101**”, o la “**111**”.



La tabla resultante de dicho circuito sería la siguiente:

A	B	C	Z
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

$$Z = \bar{A}.B.C + A.\bar{B}.C + A.B.C$$

→ Para 011: $\bar{A}.B.C$, la suma seria $Z = 1 + 0 + 0 = 1$

→ Para 101: $A.\bar{B}.C$, la suma seria $Z = 0 + 1 + 0 = 1$

→ Para 111: $A.B.C$, la suma seria $Z = 0 + 0 + 1 = 1$

En General: Dada una Tabla de Verdad de una función, si se suman los MINTÉRMINOS correspondientes a las combinaciones de variables donde la función vale 1, dicha Suma de MINTÉRMINOS responde a la Tabla de Verdad.

MAXTÉRMINOS

Es una expresión lógica de “n” variables, que consiste únicamente en la **Disyunción Lógica** y el operador **complemento** o **negación**. Son una **expresión dual** de los **Mintérminos**.

En lugar de **AND** utilizamos **OR**.

Ejemplo:

$$A + \overline{B} + C$$

$$\overline{A} + B + C$$

Dualización: El complemento de un **Mintérmino** es su respectivo **Maxtérmino**.

Ej.: $\overline{(\overline{A}.B)} = A + \overline{B}$

“n”	Maxtérmino	Mintérmino	A B C
0	$M_0 = A+B+C$	$m_0 = \overline{A}.\overline{B}.\overline{C}$	0 0 0
1	$M_1 = A+B+\overline{C}$	$m_1 = \overline{A}.\overline{B}.C$	0 0 1

MAXTÉRMINOS

a	b	$Maxitérminos$
0	0	$a + b$
0	1	$a + \overline{b}$
1	0	$\overline{a} + b$
1	1	$\overline{a} + \overline{b}$

Minimización de Funciones : Forma Gráfica

Mapas de Karnaugh

- Los mapas de Karnaugh son formas modificadas de Tablas de Verdad que permiten minimizar funciones.
- Los mapas de Karnaugh permiten el diseño rápido de circuitos combinacionales de mínimo costo, es decir, con el mínimo número de compuertas.

DIAGRAMA DE KARNAUGH

Es una matriz de “**n x m**” con variables adecuadamente dispuestas que permite obtener funciones mínimas a partir del agrupamiento de celdas adyacentes.

Los grupos resultantes pueden ser de **1, 2, 4, 8 ó 16 celdas**, cuanto mayor sea el agrupamiento, más variables serán eliminadas.

Agrupando **1** celda **no se elimina ninguna variable** => valor grupo = **4 variables**

Agrupando **2** celdas adyacentes **se elimina 1 variable**

Agrupando **4** celdas adyacentes **se eliminan 2 variables**

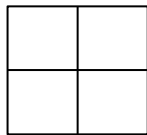
Agrupando **8** celdas adyacentes **se eliminan 3 variables**

Agrupando **16** celdas adyacentes **el resultado es “1”**

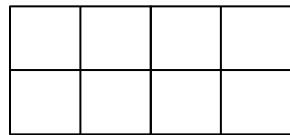
Construcción de Mapas de Karnaugh

Para construir mapas de Karnaugh se siguen los siguientes pasos:

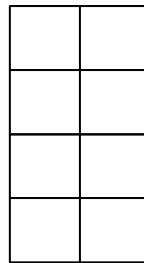
- 1) Para una función de n variables, el mapa de Karnaugh tiene 2^n celdas



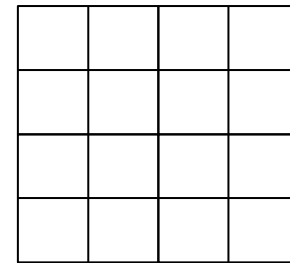
$n=2$



$n=3$



$n=3$



$n=4$

Construcción de Mapas de Karnaugh

Se asigna un 1 a una variable sin complementar y un cero a una variable complementada.

De esta forma, cada celda queda determinada por una combinación de unos y ceros

A \ B	0	1
0	$\overline{A} \cdot \overline{B}$	$\overline{A} \cdot B$
1	$A \cdot \overline{B}$	$A \cdot B$

A \ B	0	1
0	00	01
1	10	11

Construcción de Mapas de Karnaugh

En el marco del mapa se escribe el código de gray, para que las **filas** y las **columnas** del mapa sean **adyacente o vecinas** (cambian entre ellas una sola variable de estado lógico).

De esta forma las celdas son vecinas, dentro del mapa y **agrupándolas en lazos que contengan la mayor cantidad de términos en potencia de dos y realizando el OR de todos los lazos** se obtiene la función mínima en **Suma de Producto** si en cada celda se encuentra un **uno** y si realizamos lazos de **ceros** y hago **el AND de todos los lazos de ceros obtenemos la función lógica mínima en Producto de Suma**.

Construcción de Mapas de Karnaugh

A cada combinación de unos y ceros de una celda se le asigna el equivalente decimal de la representación binaria y se coloca en el extremo derecho superior.

	A	B	C	
0	0	0	0	
1	0	0	1	
3	0	1	1	
2	0	1	0	
6	1	1	0	
7	1	1	1	
5	1	0	1	
4	1	0	0	

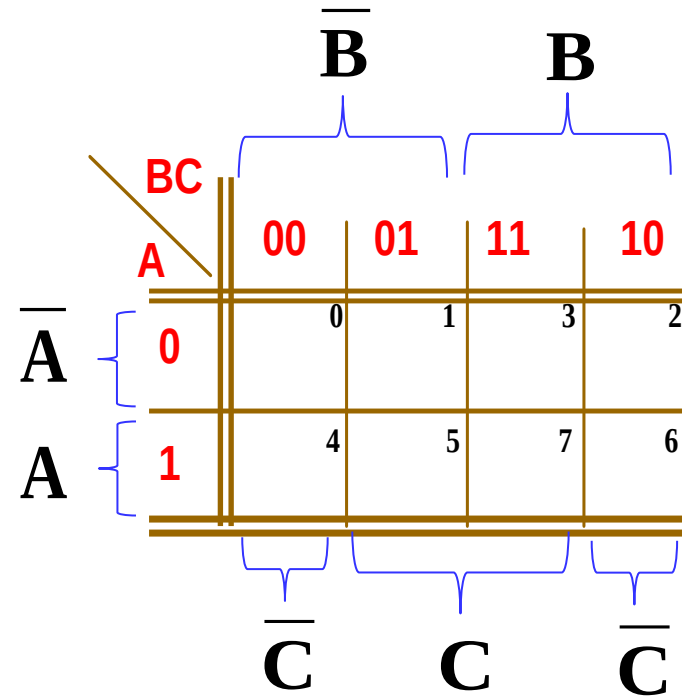


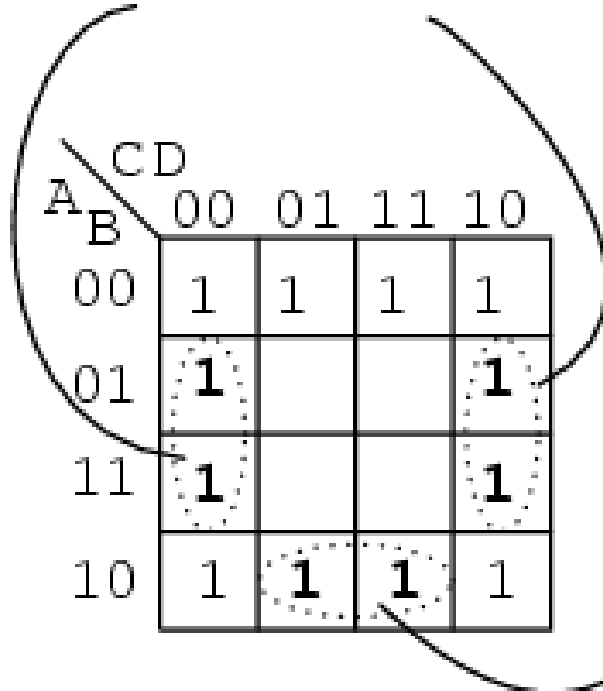
DIAGRAMA DE KARNAUGH - EJEMPLO

AB \ CD				
	00	01	11	10
00				
01				
11				
10				

AB \ CD				
	00	01	11	10
00	$\overline{A}\overline{B}\overline{C}\overline{D}$	$\overline{A}\overline{B}\overline{C}D$	$\overline{A}\overline{B}CD$	$\overline{A}\overline{B}C\overline{D}$
01	$\overline{A}B\overline{C}\overline{D}$	$\overline{A}B\overline{C}D$	$\overline{A}BCD$	$\overline{A}BC\overline{D}$
11	$AB\overline{C}\overline{D}$	$AB\overline{C}D$	$ABCD$	$ABC\overline{D}$
10	$A\overline{B}\overline{C}\overline{D}$	$A\overline{B}\overline{C}D$	$A\overline{B}CD$	$A\overline{B}C\overline{D}$

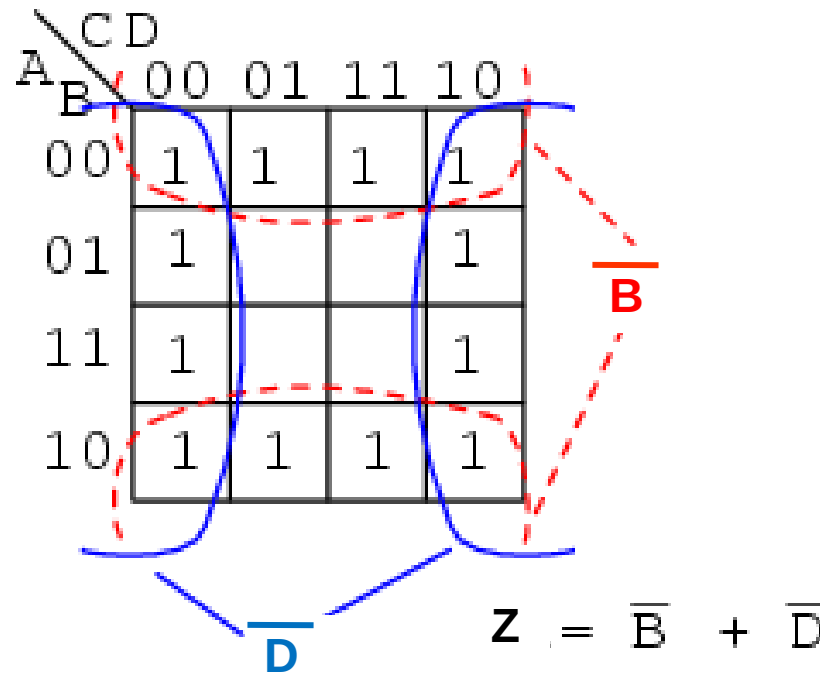
DIAGRAMA DE KARNAUGH - EJEMPLOS

$$Z := \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}C\overline{D} + \overline{A}\overline{B}CD + \overline{B}\overline{C}\overline{D} + B\overline{C}\overline{D} + A\overline{B}\overline{C}\overline{D} + A\overline{B}D + A\overline{B}C\overline{D}$$



A Karnaugh map for the function Z. The vertical axis is labeled A/B with values 00, 01, 11, 10. The horizontal axis is labeled C/D with values 00, 01, 11, 10. The map contains 1s in the following cells: (00,00), (00,01), (00,11), (00,10), (01,00), (01,01), (01,11), (01,10), (11,00), (11,01), (11,11), (11,10), (10,00), (10,01), (10,11), (10,10). The 1s are grouped into two main groups: a group of 8 cells (all 1s) and a group of 4 cells (all 1s). The group of 8 cells is circled in black and labeled with a blue $\overline{D}$. The group of 4 cells is circled in red and labeled with a red $\overline{B}$.

A \ B \ C/D	00	01	11	10
00	1	1	1	1
01	1			1
11	1			1
10	1	1	1	1



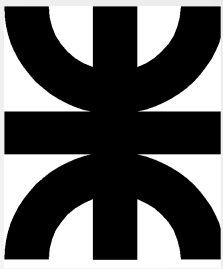
A Karnaugh map for the function Z. The vertical axis is labeled A/B with values 00, 01, 11, 10. The horizontal axis is labeled C/D with values 00, 01, 11, 10. The map contains 1s in the following cells: (00,00), (00,01), (00,11), (00,10), (01,00), (01,01), (01,11), (01,10), (11,00), (11,01), (11,11), (11,10), (10,00), (10,01), (10,11), (10,10). The 1s are grouped into two main groups: a group of 8 cells (all 1s) and a group of 4 cells (all 1s). The group of 8 cells is circled in blue and labeled with a blue $\overline{D}$. The group of 4 cells is circled in red and labeled with a red $\overline{B}$.

A \ B \ C/D	00	01	11	10
00	1	1	1	1
01	1			1
11	1			1
10	1	1	1	1

$Z = \overline{B} + \overline{D}$

BIBLIOGRAFÍA SUGERIDA

- 1) Organización y arquitectura de Computadoras. William Stallings. 7ta Edición. PEARSON. Apéndice B: Lógica digital (B.1 y B.2).**
- 2) Arquitectura de Computadoras. Patricia Quiroga. 1ra Edición. ALFAOMEGA. Capítulo 5.**
- 3) Introducción a la Organización y Arquitectura de la Computadora. Osio, Morales. 1ra Edición. UNAJ. Capítulo 2.**



UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL LA PLATA

ARQUITECTURA DE COMPUTADORA

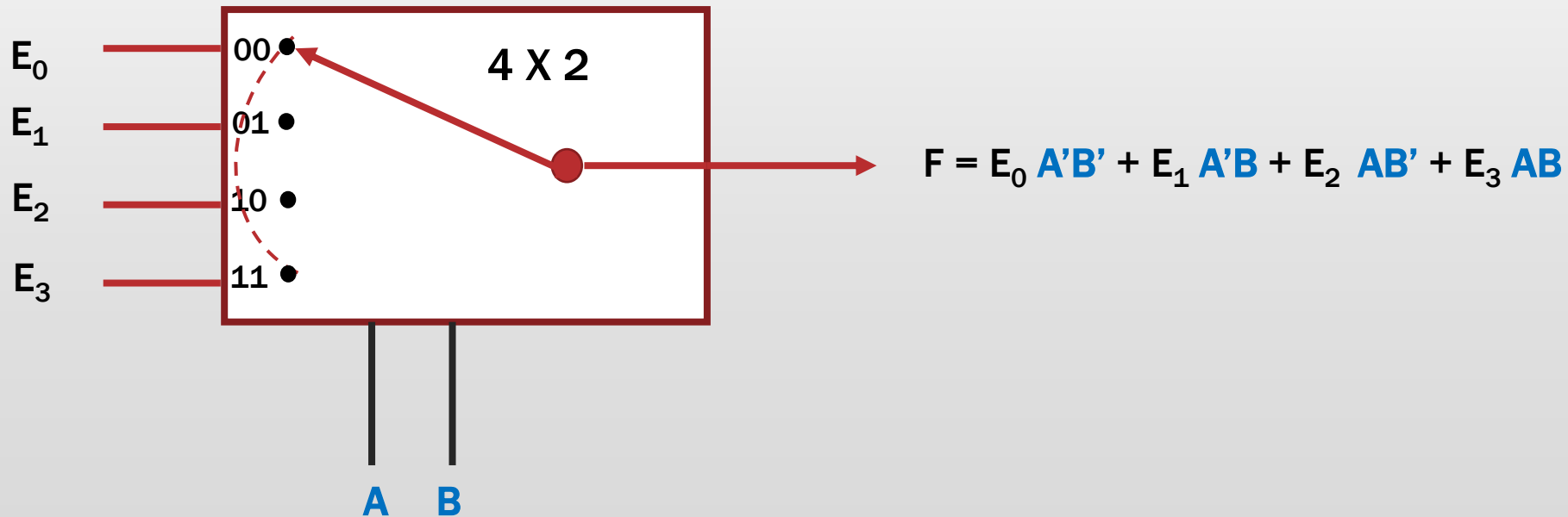
CONCEPTOS

- MULTIPLEXOR



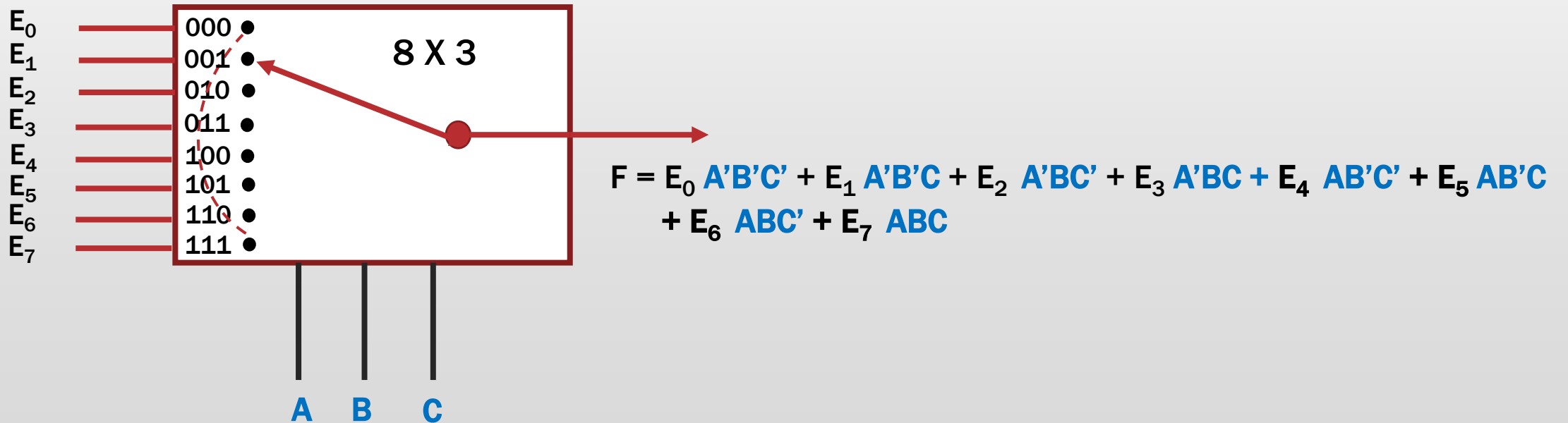
MULTIPLEXOR

LOS MULTIPLEXORES SON CIRCUITOS COMBINACIONALES CON VARIAS ENTRADAS Y UNA ÚNICA SALIDA DE DATOS. ESTÁN DOTADOS DE ENTRADAS DE CONTROL CAPACES DE SELECCIONAR UNA, Y SOLO UNA, DE LAS ENTRADAS DE DATOS PARA PERMITIR SU TRANSMISIÓN DESDE LA ENTRADA SELECCIONADA HACIA DICHA SALIDA.

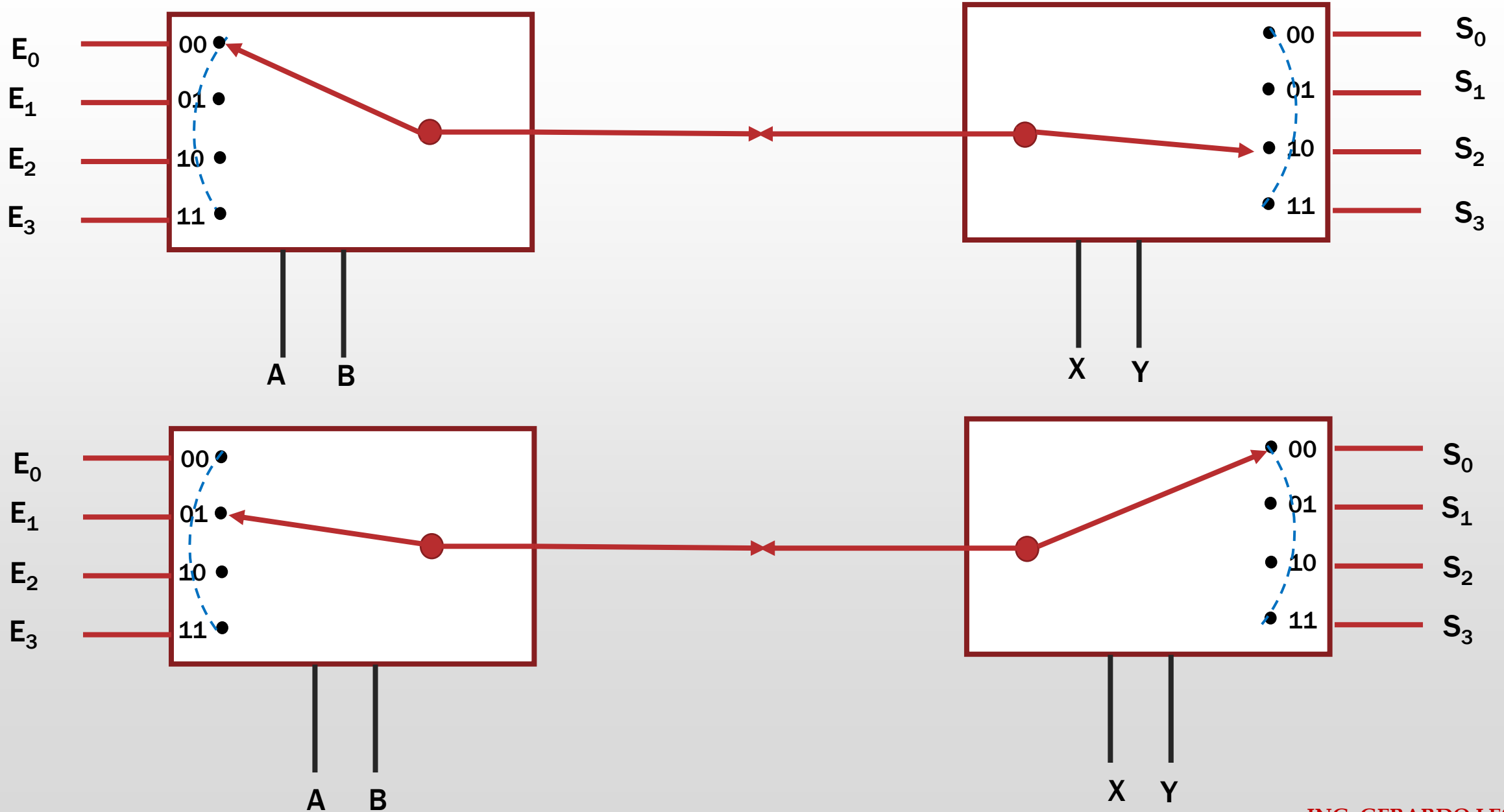


MULTIPLEXOR

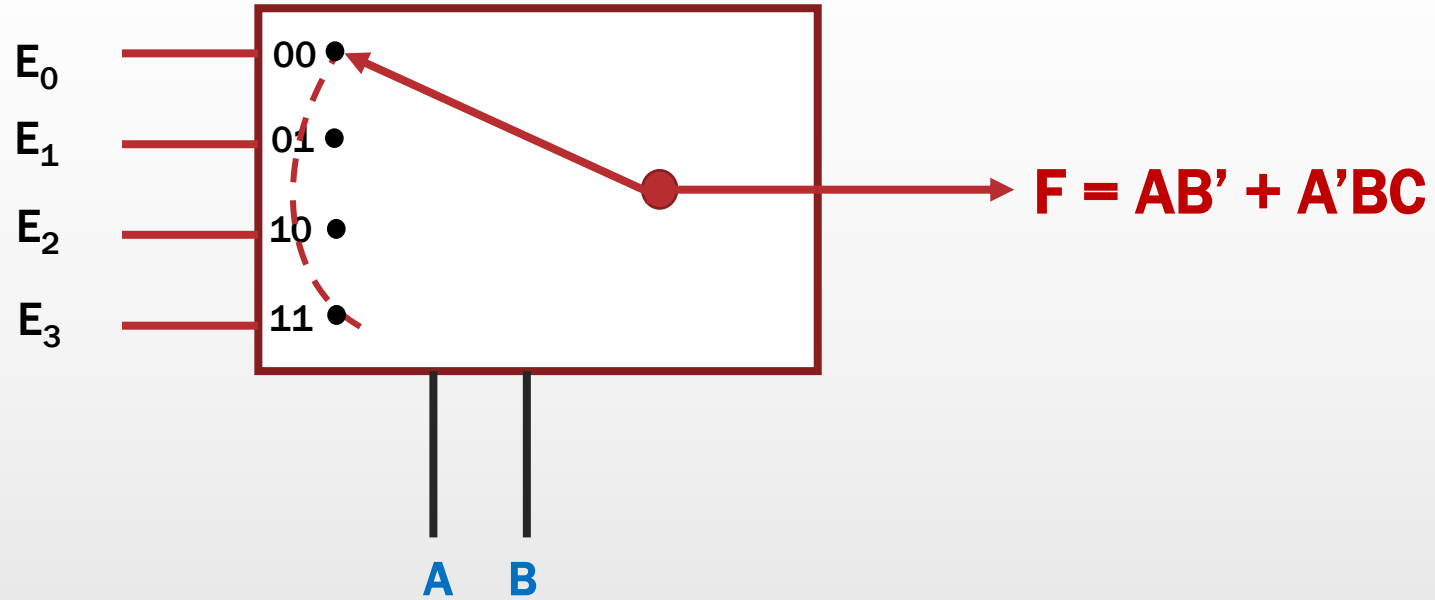
LOS MULTIPLEXORES SON CIRCUITOS COMBINACIONALES CON VARIAS ENTRADAS Y UNA ÚNICA SALIDA DE DATOS. ESTÁN DOTADOS DE ENTRADAS DE CONTROL CAPACES DE SELECCIONAR UNA, Y SOLO UNA, DE LAS ENTRADAS DE DATOS PARA PERMITIR SU TRANSMISIÓN DESDE LA ENTRADA SELECCIONADA HACIA DICHA SALIDA.



MULTIPLEXACIÓN POR DIVISIÓN DE TIEMPO



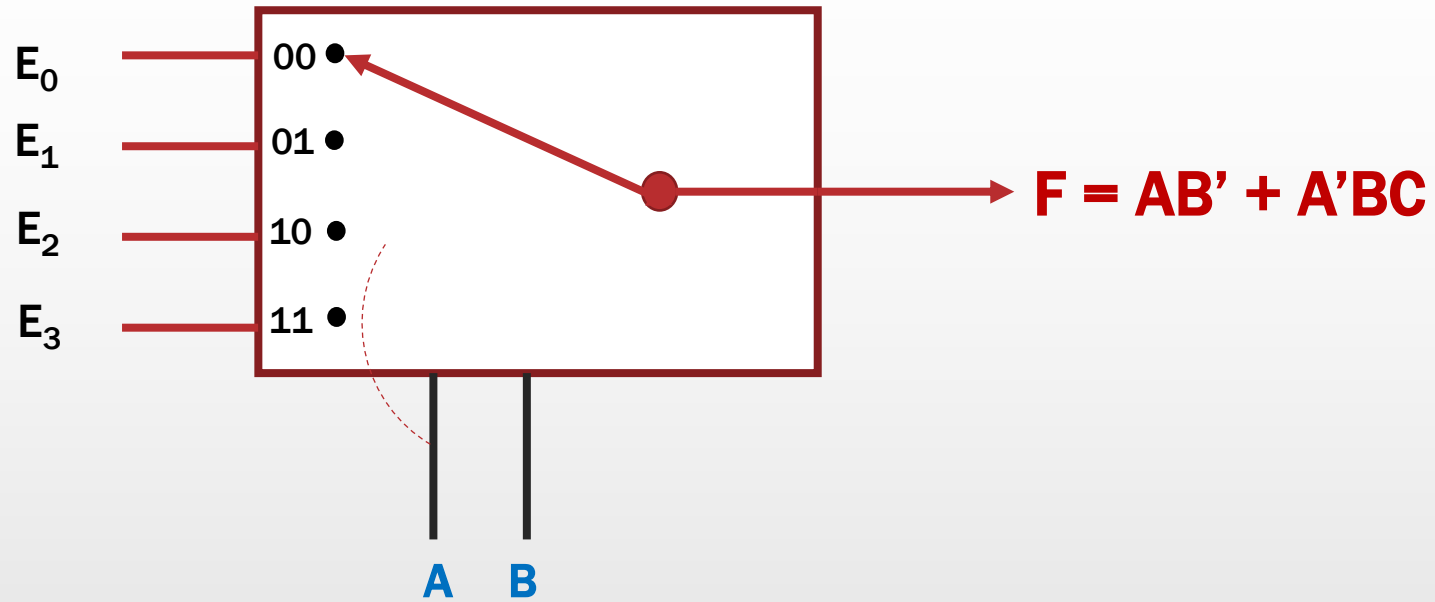
MULTIPLEXOR



PRIMER FORMA CANÓNICA

$$F = AB'(C + C') + A'BC = \underline{AB'C} + \underline{AB'C'} + A'BC$$

MULTIPLEXOR

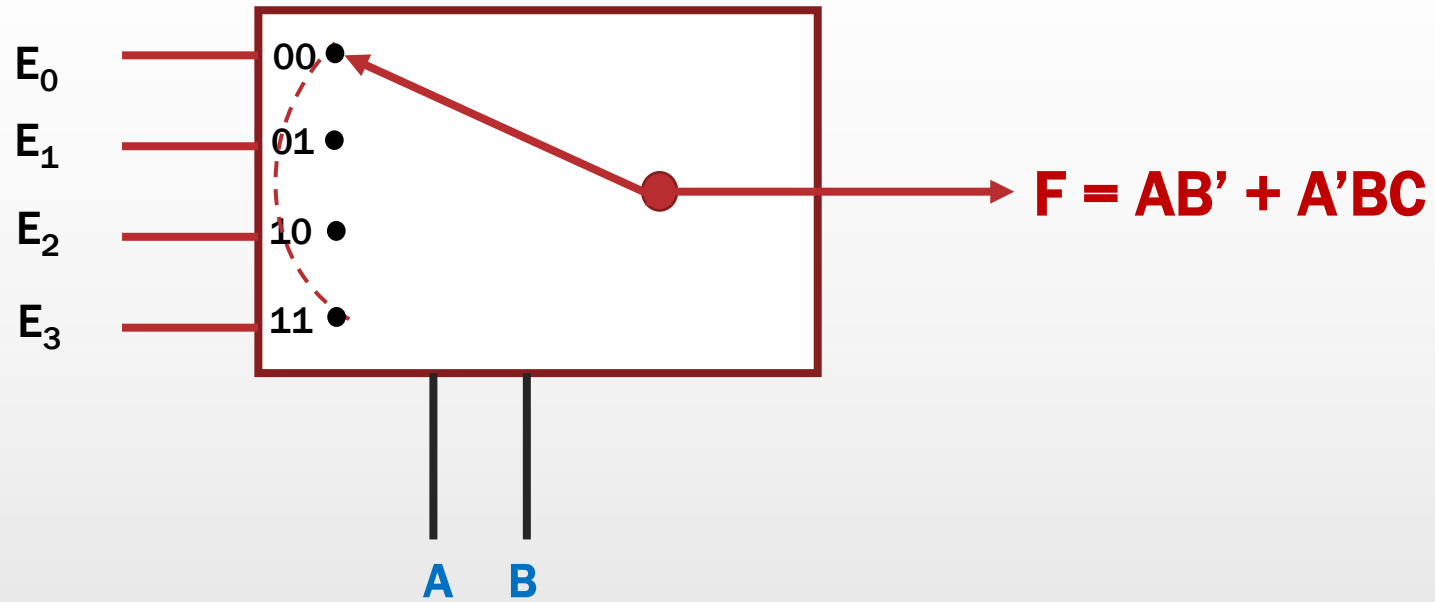


PRIMER FORMA CANÓNICA

$$F = AB'(C + C') + A'BC = \underline{AB'C} + \underline{AB'C'} + A'BC$$

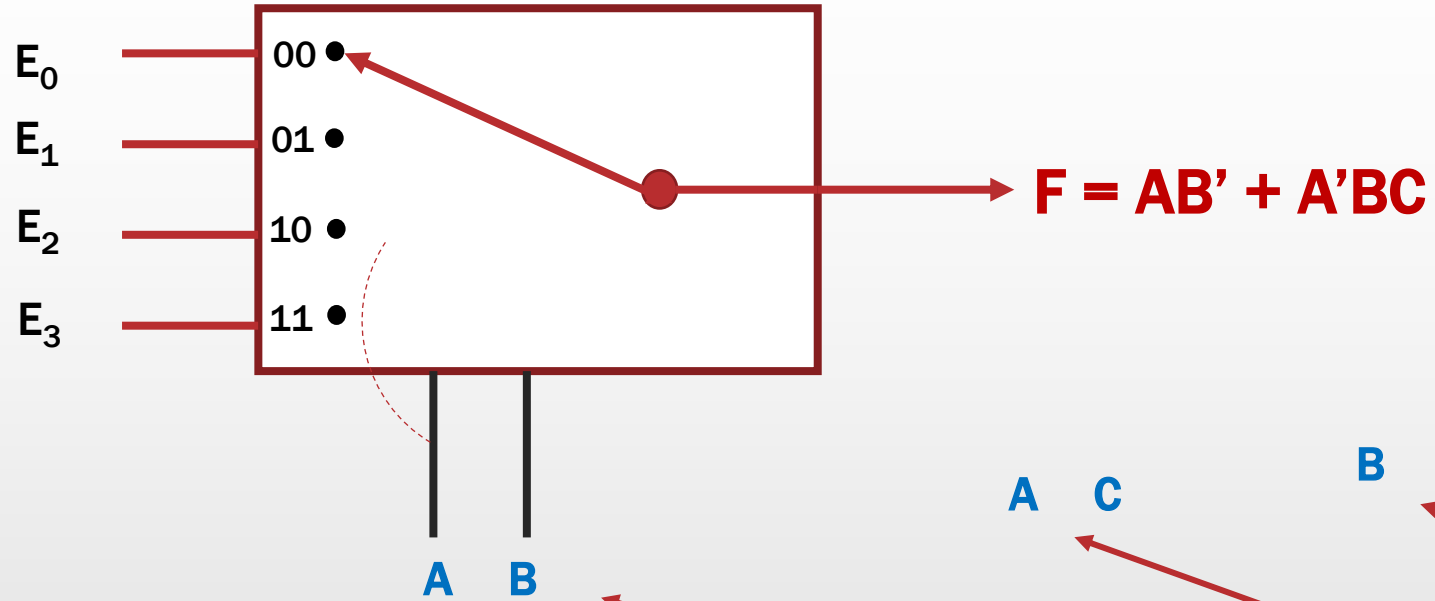
A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

MULTIPLEXOR



A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

MULTIPLEXOR



A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

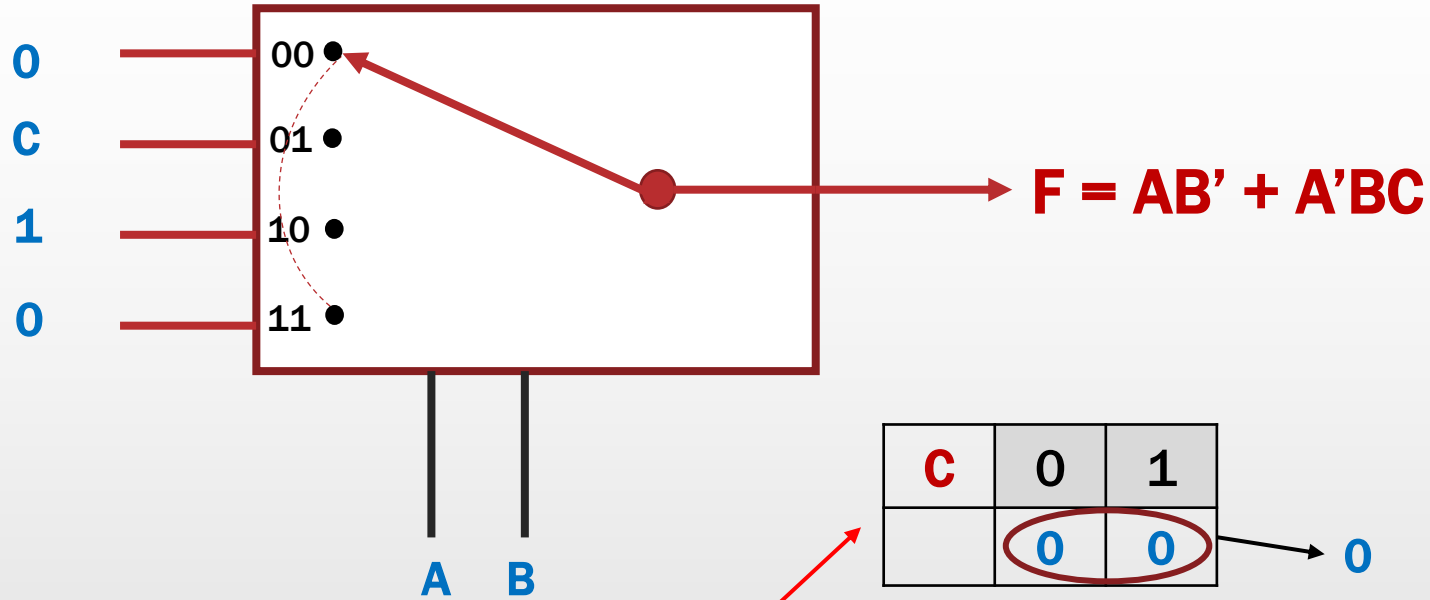
AB \ C	0	1
00	0	0
01	0	1
11	0	0
10	1	1

AB \ C	0	1
00	0	0
01	0	1
11	0	0
10	1	1

AB \ C	0	1
00	0	0
01	0	1
11	0	0
10	1	1

AB \ C	0	1
00	0	0
01	0	1
11	0	0
10	1	1

MULTIPLEXOR



AB \ C	0	1
00	0	0
01	0	1
11	0	0
10	1	1

C	0	1
	0	0

→ 0

C	0	1
	0	1

→ C

C	0	1
	0	0

→ 0

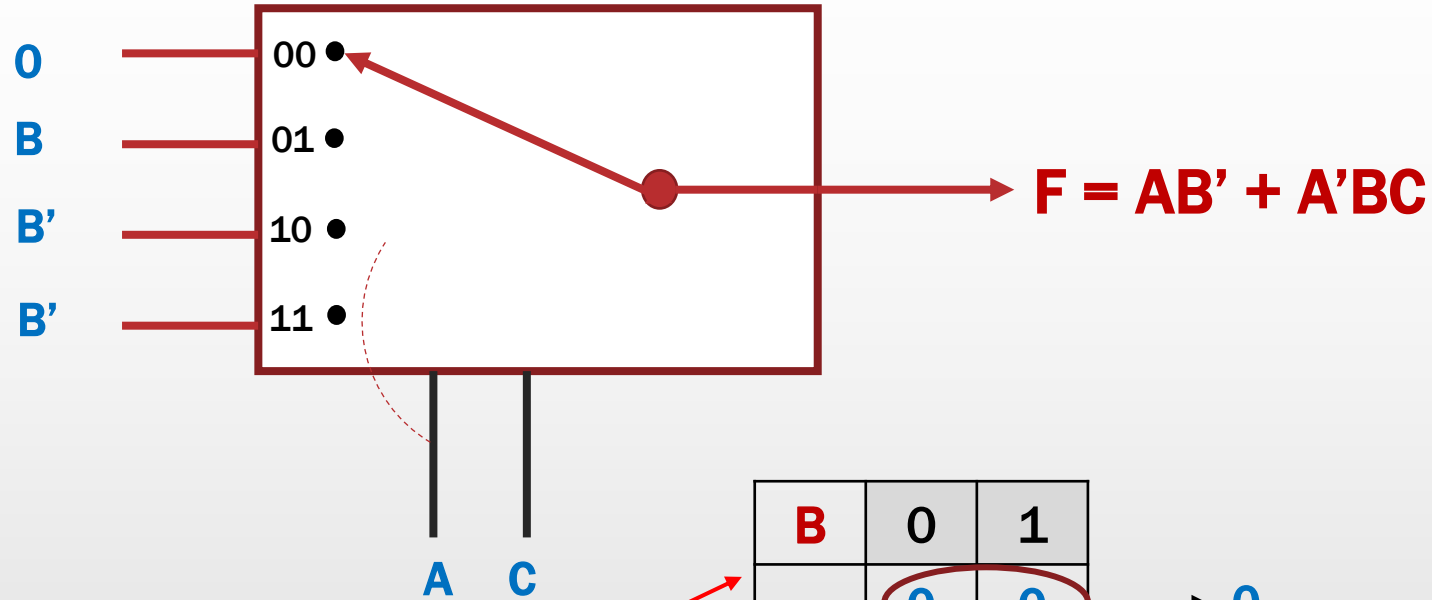
C	0	1
	1	1

→ 1

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$$F = A'B'0 + A'BC + AB'1 + AB0 = \underline{AB'} + \underline{A'BC}$$

MULTIPLEXOR



A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

AB \ C	0	1
00	0	0
01	0	1
11	0	0
10	1	1

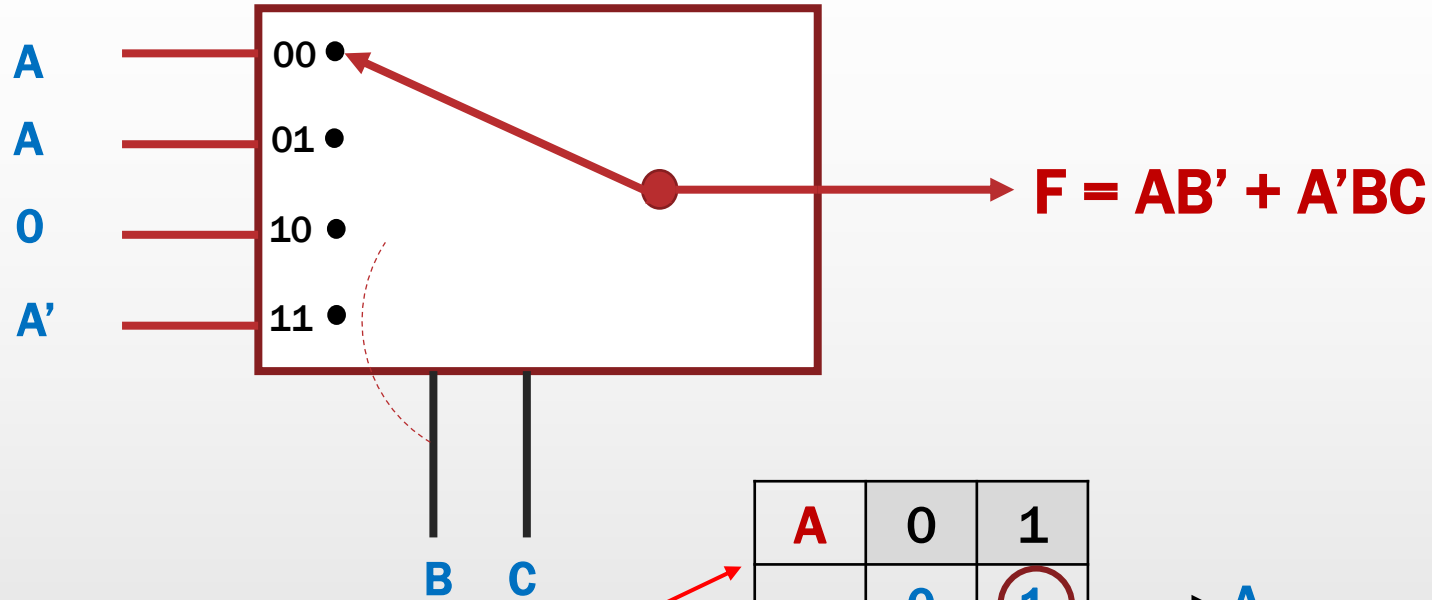
B	0	1
	0	0
	0	1
	1	0
	1	0

$$F = A'C'0 + A'CB + AC'B' + ACB' = \underline{AB'} + \underline{A'BC}$$

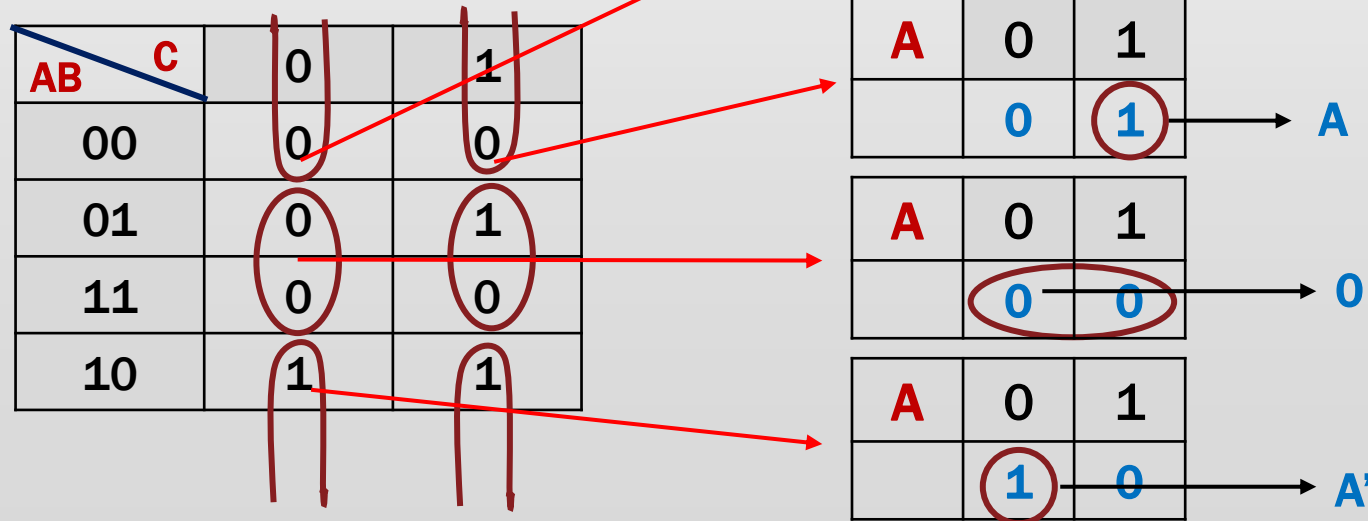
$$F = A'BC + AB'C' + AB'C = A'BC + AB'(C' + C)$$

$$F = \underline{AB'} + \underline{A'BC}$$

MULTIPLEXOR

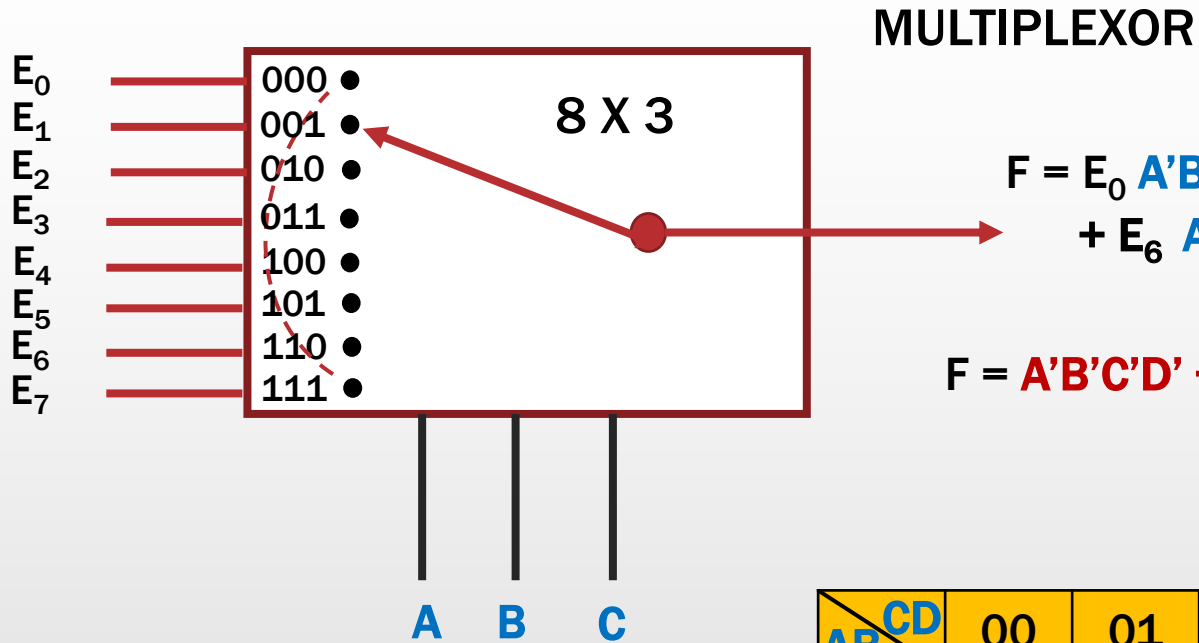


A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0



$$F = B'C'A + B'CA + BC'0 + BCA'$$

$$F = AB'(C' + C) + A'BC = \underline{AB'} + A'BC$$



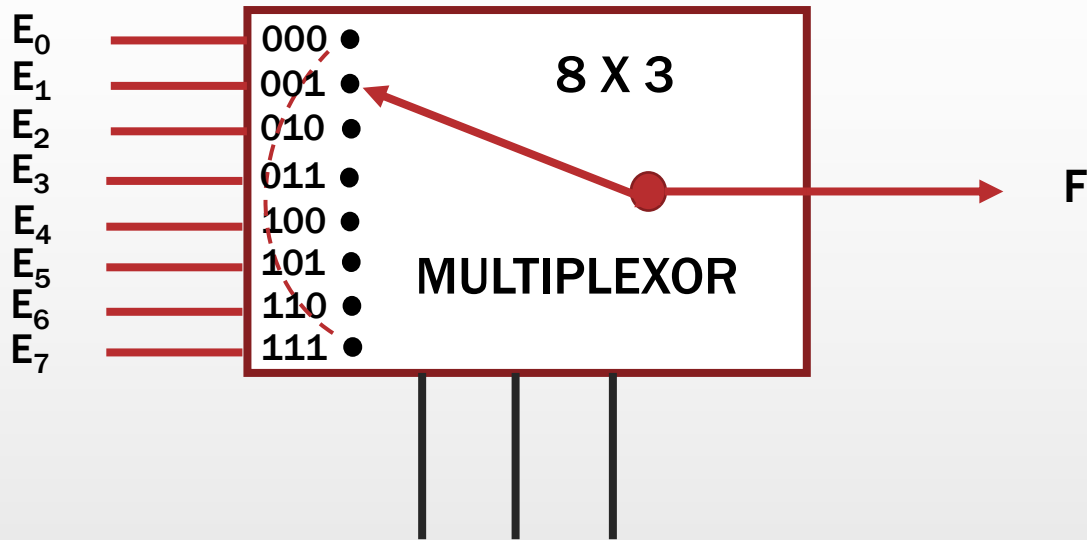
$$F = E_0 A'B'C' + E_1 A'B'C + E_2 A'BC' + E_3 A'BC + E_4 AB'C' + E_5 AB'C + E_6 ABC' + E_7 ABC$$

$$F = A'B'C'D' + A'B'C'D + A'BCD + A'BCD' + AB'C'D + AB'CD + AB'CD' + ABCD$$

AB \ CD	00	01	11	10
00	1	1	0	0
01	0	0	1	1
11	0	1	0	0
10	0	0	1	1

$$1) F = E_0 R'S'T' + E_1 R'S'T + E_2 R'ST' + E_3 R'ST + E_4 RS'T' + E_5 RS'T + E_6 RST' + E_7 RST$$

$$2) F = E_0 R'T'U' + E_1 R'T'U + E_2 R'TU' + E_3 R'TU + E_4 RT'U' + E_5 RT'U + E_6 RTU' + E_7 RTU$$



1)

?

$RS \backslash TU$	00	01	11	10
00	1	1	0	0
01	0	0	1	1
11	0	1	0	0
10	0	0	1	1

Variable de Control: **RST**

Variable de Residuo: **U**

2)

$RS \backslash TU$	00	01	11	10
00	1	1	0	0
01	0	0	1	1
11	0	1	0	0
10	0	0	1	1

Variable de Control: **RTU**

Variable de Residuo: **S**

4)

$RS \backslash TU$	00	01	11	10
00	1	1	0	0
01	0	0	1	1
11	0	1	0	0
10	0	0	1	1

Variable de Control: **STU**

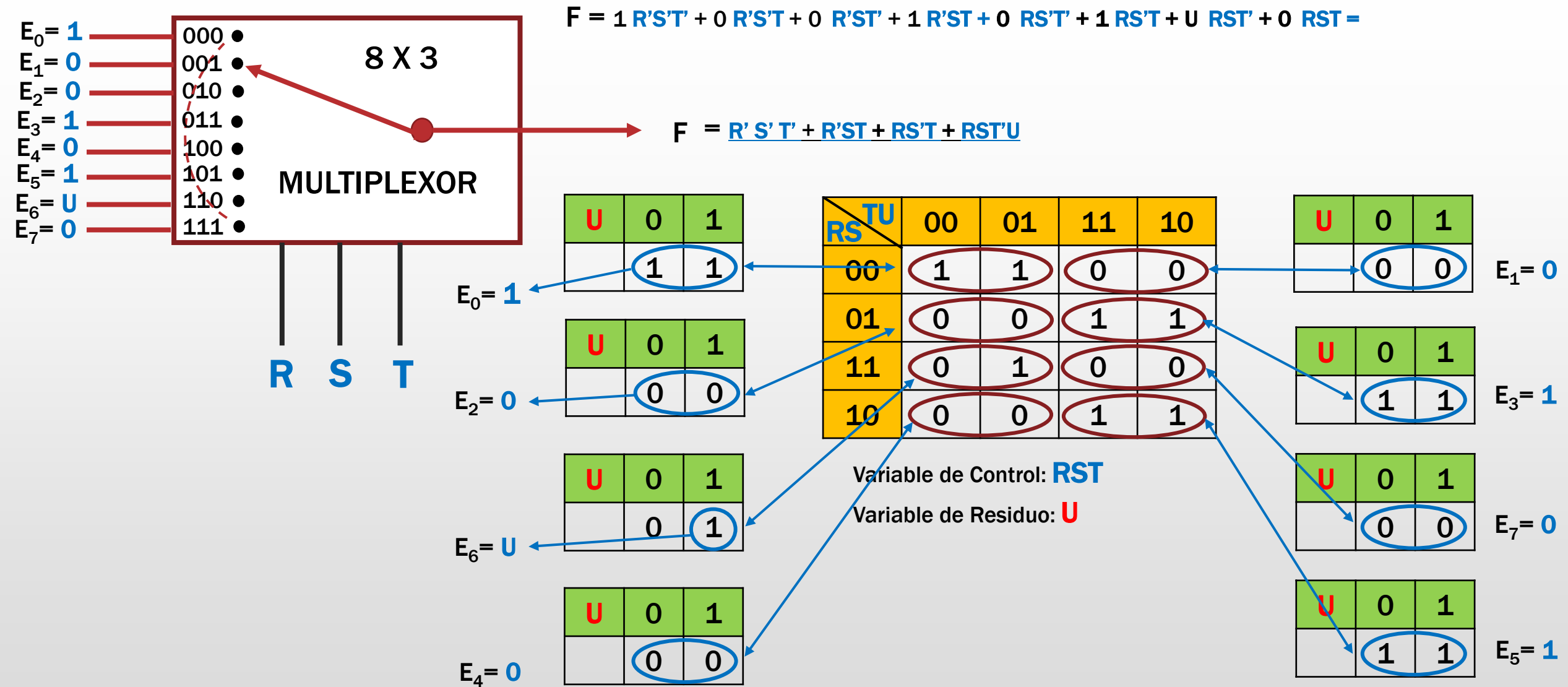
Variable de Residuo: **R**

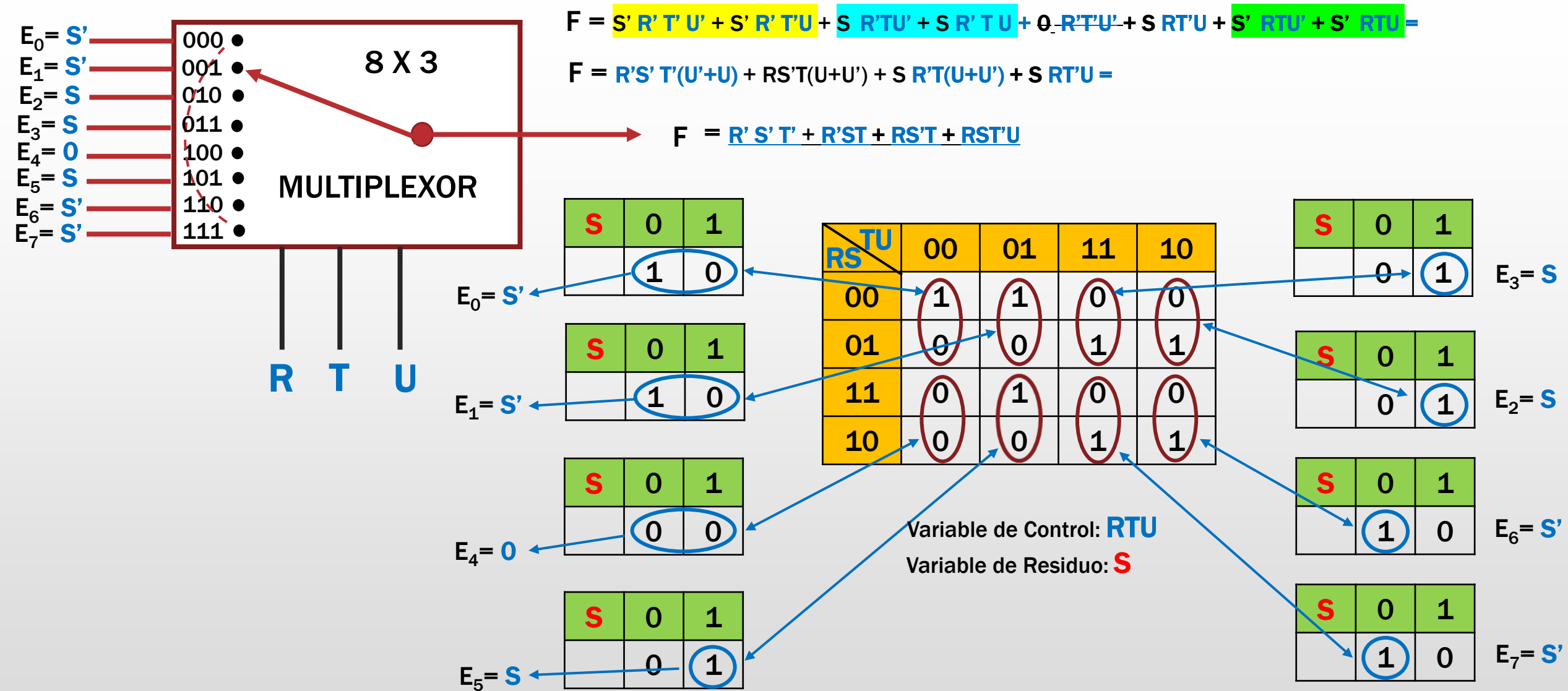
3)

$RS \backslash TU$	00	01	11	10
00	1	1	0	0
01	0	0	1	1
11	0	1	0	0
10	0	0	1	1

Variable de Control: **RSU**

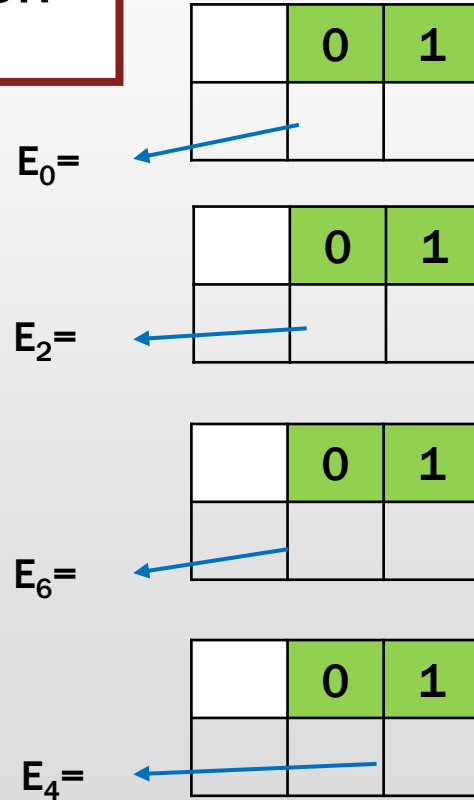
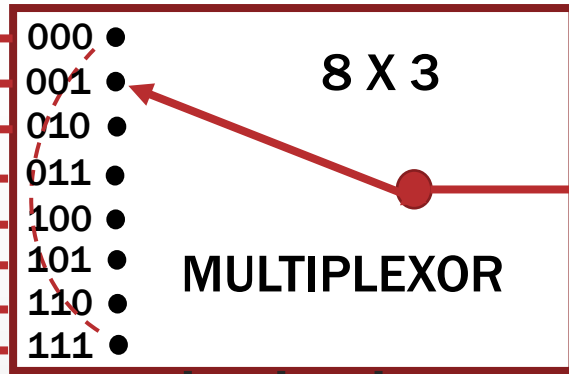
Variable de Residuo: **T**





RESOLVER

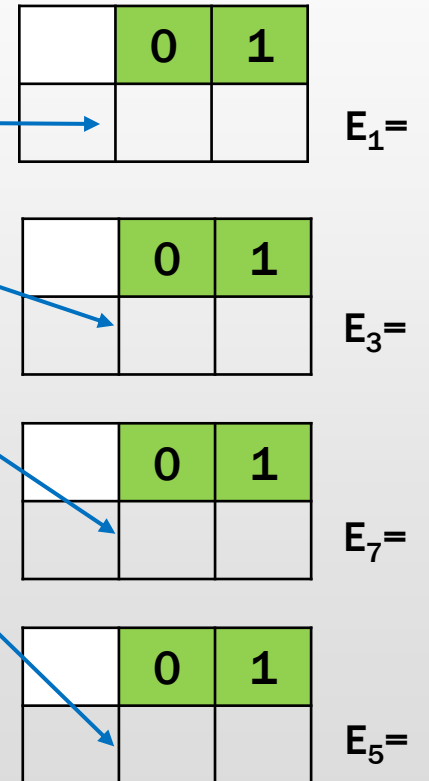
$$F = \underline{R' S' T'} + \underline{R' S T} + \underline{R S' T} + \underline{R S T' U}$$



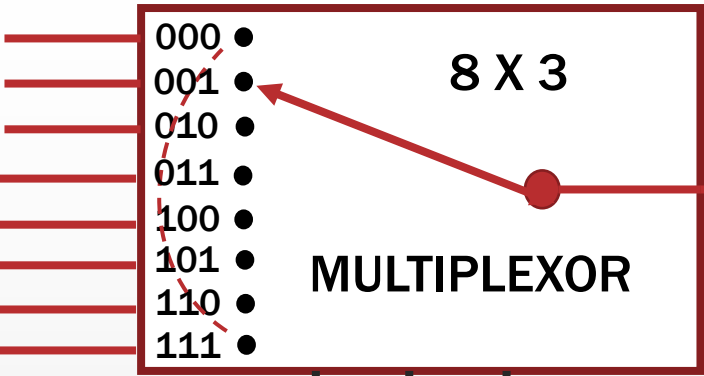
RS \ TU	00	01	11	10
00	1	1	0	0
01	0	0	1	1
11	0	1	0	0
10	0	0	1	1

Variable de Control: ☐ ☐ ☐

Variable de Residuo: ☐



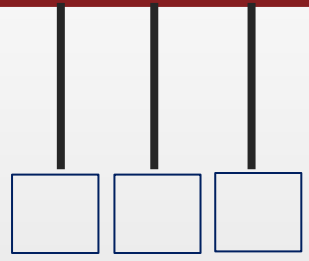
E₀=
E₁=
E₂=
E₃=
E₄=
E₅=
E₆=
E₇=



$F = R'S'T' + R'ST + RST' + RST'U$

RESOLVER

RS \ TU	00	01	11	10
00	1	1	0	0
01	0	0	1	1
11	0	1	0	0
10	0	0	1	1

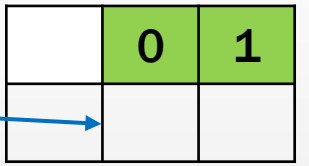


E₀=

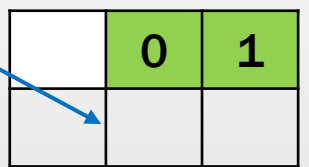
E₁=

E₄=

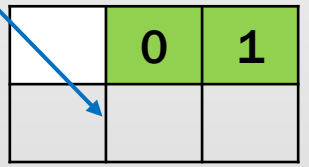
E₅=



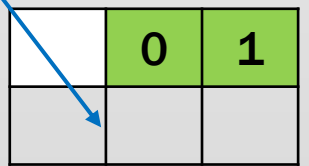
E₂=



E₃=



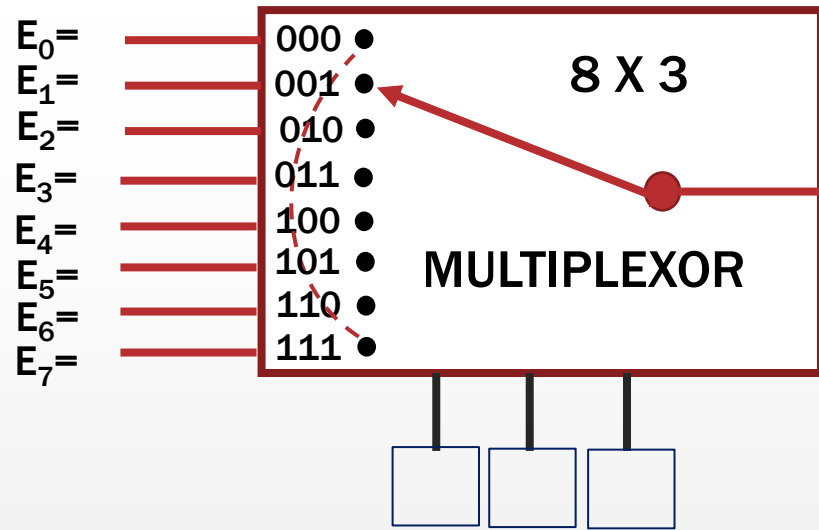
E₆=



E₇=

Variable de Control: ☐ ☐ ☐

Variable de Residuo: ☐



RESOLVER: CONVERTIR EN 4X2 CON **A** y **B** DE CONTROL Y **C** y **D** RESIDUO

$$F = \underline{A' B' C'} + \underline{A' B C} + \underline{A B' C} + \underline{A B C' D}$$

$AB \backslash CD$	00	01	11	10
00	1	1	0	0
01	0	0	1	1
11	0	1	0	0
10	0	0	1	1

$C \backslash D$	0	1
0	1	1
1	0	0

$$E_0 = C'$$

$C \backslash D$	0	1
0	0	0
1	1	1

$$E_1 = C$$

$C \backslash D$	0	1
0	0	1
1	0	0

$$E_2 = C' + D$$

$C \backslash D$	0	1
0	0	0
1	1	1

$$E_3 = C$$

Variable de Control:

Variable de Residuo:

FIN

CATEDRA DE ARQUITECTURA DE COMPUTADORAS

**MEMORIA ROM - TIPOS DE ROM
TIPOS Y ESTRUCTURA BÁSICA DE UNA RAM**

MEMORIAS ELECTRÓNICAS

Son los dispositivos de almacenamiento de datos e instrucciones en una computadora.

UNIDAD DE INFORMACION: Se utiliza para identificar un grupo de bytes a los que se accede en forma conjunta según el tipo de soporte.

Para las memorias de semiconductores (RAM o ROM) también se utiliza el termino **palabra o posición de memoria y para las memorias** sobre medio magnético, por lo general, **bloque**.

Semiconductor: Es un **material aislante** que, cuando se le añaden ciertas sustancias o en un determinado contexto, se vuelve **conductor**. Esto quiere decir que, de acuerdo a determinados factores, el semiconductor actúa a modo de aislante o como conductor.

SISTEMA DE MEMORIA

Conjunto de estos dispositivos y los algoritmos de hardware y/o software de control de los mismos.

Existen diversos dispositivos capaces de almacenar información, lo deseable es lograr la mayor velocidad de procesamiento.

Desafortunadamente, memorias de velocidades similares al procesador son muy caras. Por ello, la información almacenada se distribuye en forma compleja en una variedad de memorias diferentes, con características físicas distintas.

Clasificación de Memorias

Clasificadas según ciertos parámetros que las diferencian:

- 1. El modo de acceso a la unidad de información**
- 2. Las operaciones que aceptan por cada acceso**
- 3. La duración de la información en el soporte**

1. El modo de acceso a la unidad de información

Acceso Aleatorio:

Cuando un componente de selección **habilita** una palabra (o posición) e **inhabilita** a las demás. En cada acceso el componente de selección de palabra (o **decodificador de direcciones**) recibe “**el identificador**” (que es **único**) de la unidad de información implicada. El tiempo de acceso es independiente del lugar físico que ocupa la unidad de información en el soporte.

Acceso Secuencial:

Cuando para acceder a una unidad de información se establece una posición de referencia, a partir de la cual comienza un rastreo de la unidad de información que consiste en la lectura de todas las unidades que la precedan, hasta lograr la buscada. En este caso, el **tiempo de acceso** depende de la **distancia** entre la posición inicial y la unidad de información.

1. El modo de acceso a la unidad de información

Acceso Asociativo:

Cuando la búsqueda de la unidad de información implica la comparación de un grupo de bits de la unidad de información con el contenido de una posición de memoria.

Las palabras no se identifican por su dirección y el objetivo de su lectura es verificar si este grupo de bits coincide con el contenido de alguna de ellas. Para lograr esto se envía a la memoria el grupo de bits que actúa de rotulo (***label***), *que se compara* con todas las unidades de información con el fin de hallar una coincidencia.

Esta se logra si se verifica la igualdad entre los bits del rotulo y los comparados.

2. Las operaciones que aceptan por cada acceso

Una memoria es de **lectura/escritura** cuando admite ambas operaciones y es **sólo de lectura** cuando permite únicamente esta operación.

3. La duración de la información en el soporte

Las memorias son **volátiles** cuando pierden su información con el corte de suministro de corriente y **perennes, permanentes o no volátiles**, en caso contrario.

La clasificación también comprende una tecnología de memoria RAM denominada **dinámica**, que aunque sus celdas binarias degradan la información con el tiempo, el proceso de recuperación es gestionado por hardware, y, por lo tanto, la "perdida momentánea" de información se hace transparente al entorno.

Parámetros generales aplicables a todas las memorias

Tiempo de acceso (t_a):

Es el que se tarda en leer o escribir una palabra en la memoria desde el momento que se direcciona.

Cálculo de la velocidad de acceso:

$$V_a = 1/t_a$$

Tiempo de ciclo (t_c):

Indica el mínimo tiempo entre dos accesos sucesivos a la memoria. Es el tiempo de recuperación de la memoria antes del próximo acceso.

$$t_c = t_a + \text{Tiempo de Recuperación de la memoria}$$

Parámetros generales aplicables a todas las memorias

Medio Físico

- **Electrónicas:** Construidas con semiconductores.
- **Magnéticas:** Basadas en materiales ferromagnéticos.
- **Ópticas:** Utilizan la tecnología láser.

Estabilidad

- **Volatilidad:** El contenido de la memoria se pierde cuando se suspende la alimentación eléctrica.
- **Almacenamiento dinámico:** La información se pierde cuando el capacitor se descarga lo que hace necesario un refresco periódico para restaurar el contenido antes que se deteriore.
- **Lectura destructiva:** Al efectuar la lectura se pierde la información, por lo cual dicho proceso debe acompañarse de una restauración.

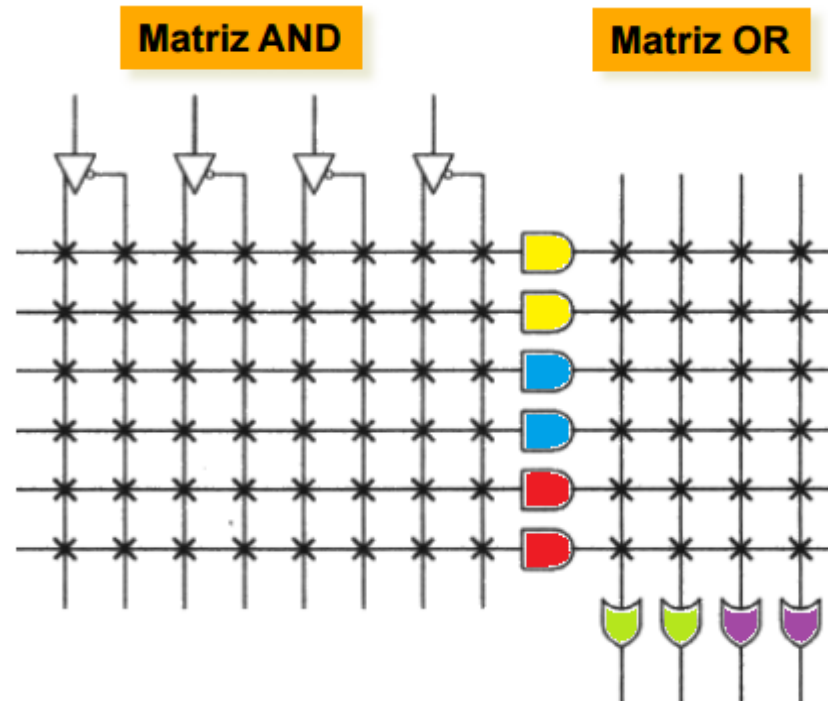
MEMORIA ROM

Conocida como “Firmware”, es un circuito integrado programado con datos específicos cuando es fabricado.

Los datos almacenados en estos chips son **“NO VOLATILES”** (permanecen al cortar el suministro de energía).

Se construye mediante la combinación de un **“DECODIFICADOR”** y **“COMPUERTAS OR”**

Matrices programables



Al inicio las salidas de una ROM están todas en '1' (enlace intacto), la programación de los '0' es un procedimiento físico que genera la destrucción de los enlaces

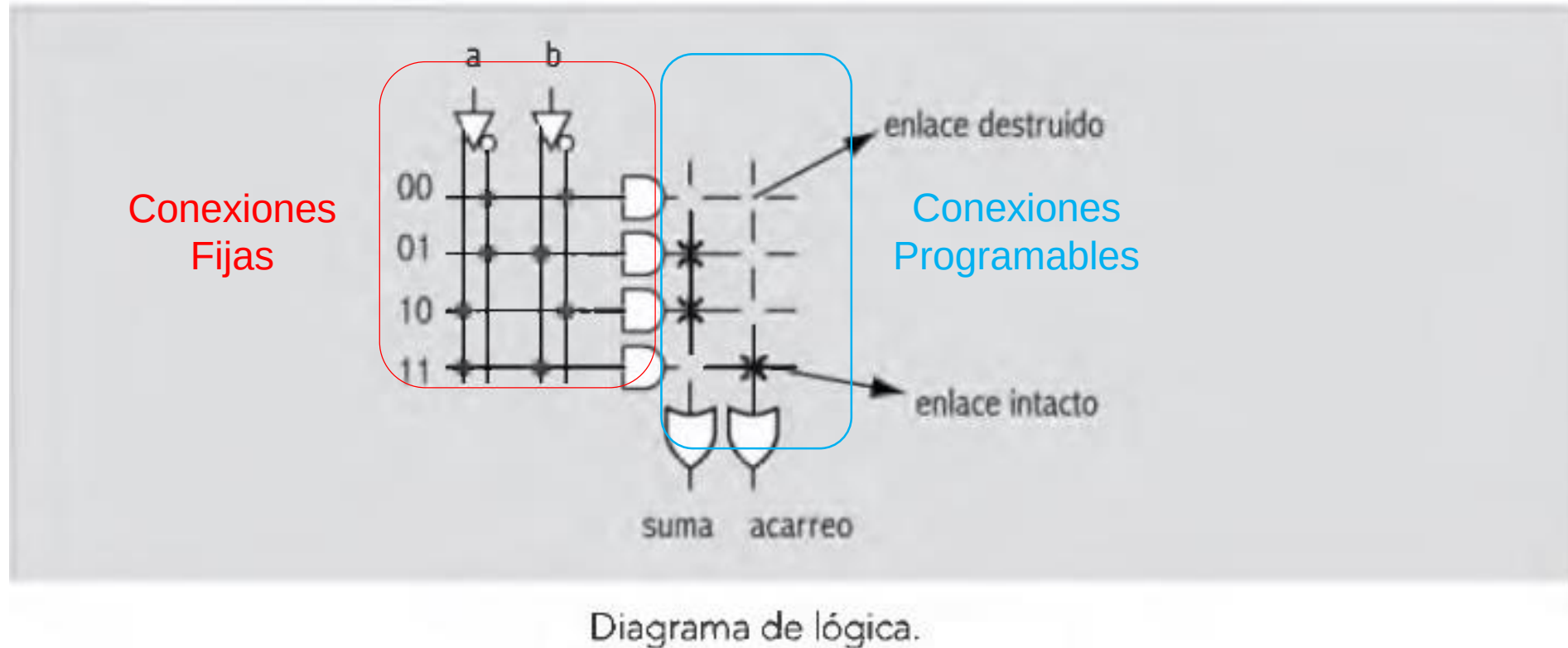
Circuitos “programables” para múltiples funciones:

Los dispositivos de lógica programable (**PLD**) (**Programmable Logic Device**), son circuitos integrados con cientos de compuertas **AND-OR** que se relacionan por medio de enlaces electrónicos y conexiones fijas.

Supongamos que queremos programar la función digital que responde al circuito semisumador a partir de su tabla de verdad y que las variables de este circuito son las entradas a un **decodificador 2 a 4**.

Tabla 6-10. Tabla de verdad			
<i>a</i>	<i>b</i>	<i>Suma</i>	<i>Acarreo</i>
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Para programar el circuito se debe destruir el enlace, según indique la tabla de verdad, de la siguiente manera:



En la red **OR**, las cruces representan la programación de '1' o “**enlace intacto**”.

Los espacios vacíos representan la programación de los '0' ó “**enlace destruido**”.

Un arreglo de compuertas **AND-OR** programado de esta manera es un ejemplo simple de memoria **ROM**. Una vez establecida la combinación binaria de las salidas, estas permanecen inalterables.

Cuando una **ROM** se utiliza para implementar una función lógica, ***las entradas al decodificador AND constituyen las variables de la expresión booleana.***

Cuando se utiliza como programa fijo, constituyen la dirección de memoria para acceder.

Cada **línea de salida del decodificador** se considera una **PALABRA DE MEMORIA** y **cada enlace (intacto o no)** se considera una **celda de almacenamiento de 1 bit**.

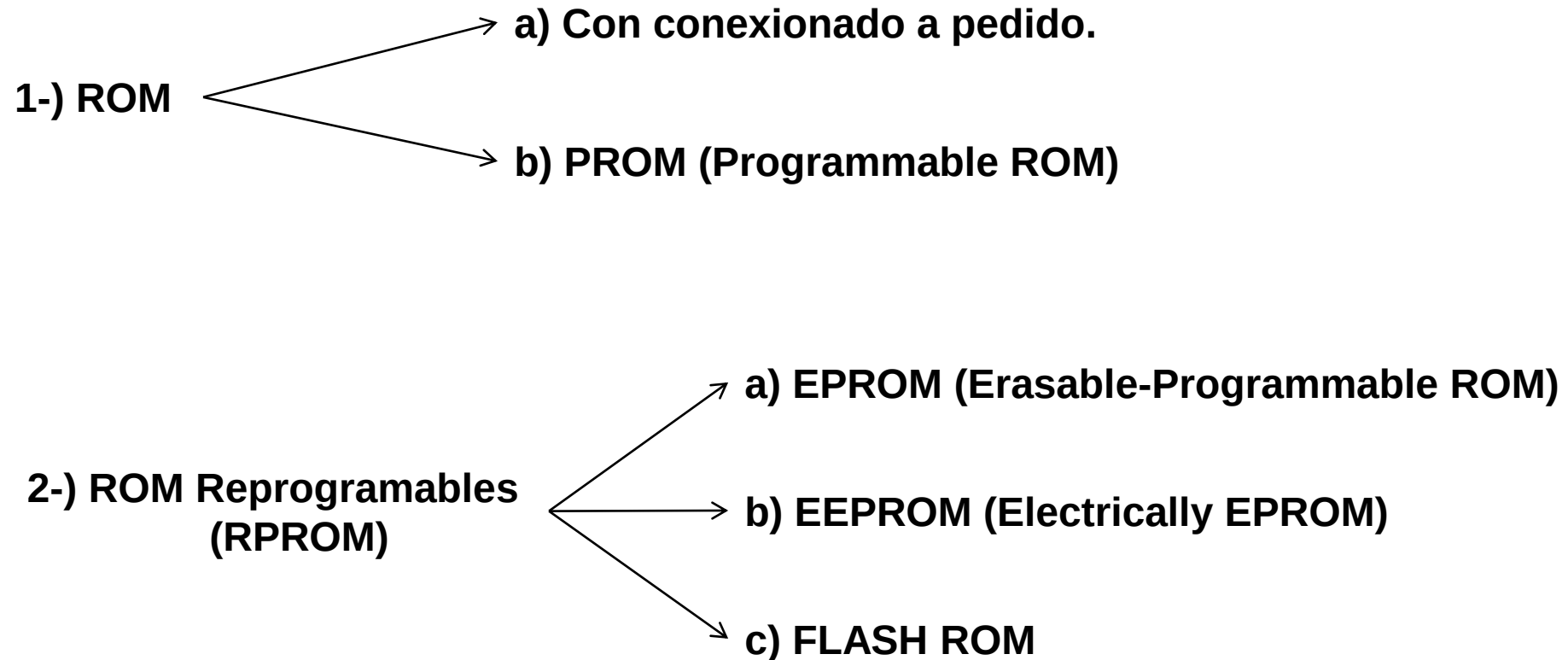
LA CANTIDAD DE COMPUERTAS “OR” DETERMINA LA CANTIDAD DE BITS POR PALABRA.

De esta manera se construye una matriz de “**m x n**” (“**m**” palabras de “**n**” bits cada una).

Para identificar **1** de las “**m**” palabras existe un valor “**p**” tal que **$2^p = m$** , entonces “**p**” es el número de entradas del decodificador.

TIPOS DE ROM

Según las características de este procedimiento, se fabrican distintos tipos de dispositivos:



1-) ROM Programables una sola vez e inalterables

Una vez realizado el conexionado programado por el usuario no puede ser modificado.

a) ROM con Conexionado a Pedido (Masked ROM)

El usuario programa las conexiones que tendrá la matriz y el conexionado lo realiza el fabricante.

b) PROM con Conexionado a Pedido

El usuario, además de especificar la programación de la matriz lleva a cabo la misma, en un proceso posterior a la fabricación del Chip. (la valoración de un bit esta sujeta al estado de un fusible, el cual tiene la posibilidad de quemarse una única vez).

2-) ROM Reprogramables (RROM)

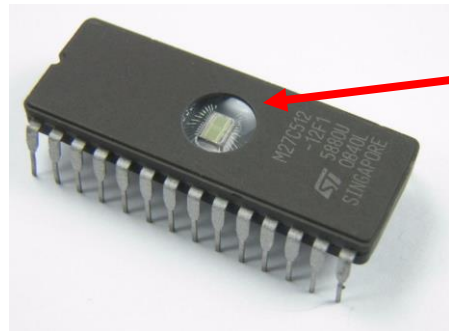
Una vez que el usuario configura la matriz de conexionado, puede modificarla de modo de cambiar la tabla que cumple la ROM.

a) EPROM (Erasable-Programmable ROM)

Se refiere a un chip de memoria ROM constituido por celdas de transistores de puerta flotante. La programación de estos chips se realiza por medio de un dispositivo electrónico especial.

Al ser programada, sólo cuenta con la posibilidad de borrarse a través del contacto con una luz ultravioleta de alta potencia. Esto sucede debido que los fotones de la luz son capaces de afectar a los electrones propios de las celdas, provocando su descarga.

Memoria EPROM

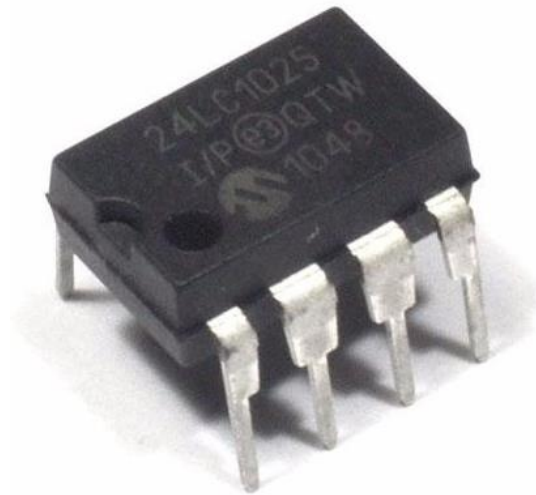


Ventana de cuarzo donde se recibe luz UV durante el borrado.

b) EEPROM (Electrically Erasable-Programmable ROM)

Como su nombre lo indica, esta clase de memoria ROM dispone de la capacidad de programación, borrado y reprogramación de forma eléctrica. No son memorias volátiles. La cantidad de veces que una memoria EEPROM puede borrarse y reprogramarse oscila entre cien mil y un millón de veces. Sin embargo son ilimitadas las veces que la misma puede leerse.

Memoria EEPROM



c) FLASH ROM

Constituye una clase de EEPROM, la cual puede ser reprogramada en bloques.

Es un dispositivo de almacenamiento que admite la lectura y escritura de un gran número de posiciones de memoria en una misma acción.

Tiene una velocidad de funcionamiento superior al resto de las memorias ROM.

La tecnología flash es utilizada en pendrives, cámaras de fotos, etc.

Ejemplos de Memorias FLASH

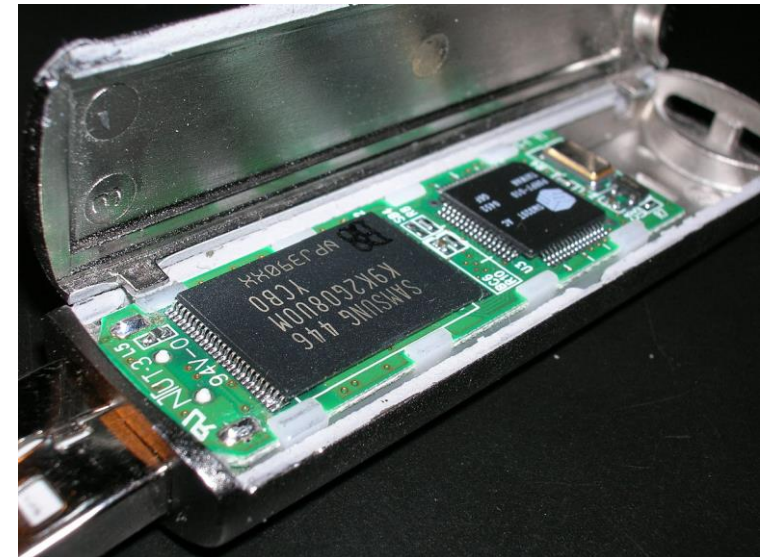


TABLA COMPARATIVA

TIPO	CATEGORIA	BORRADO
ROM	SOLO LECTURA	NO
PROM	SOLO LECTURA	NO
EPROM	PRINCIPALMENTE LECTURA	LUZ U.V.
EEPROM	PRINCIPALMENTE LECTURA	ELÉCTRICO
FLASH	LECTURA / ESCRITURA	ELÉCTRICO

Tipos de Memorias



Memorias de Acceso Aleatorio

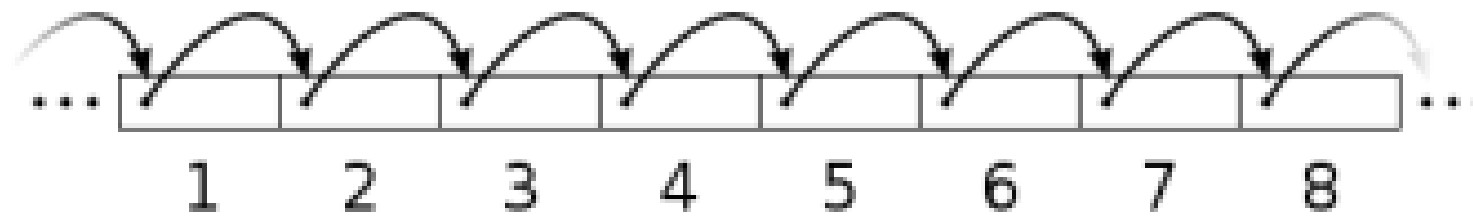
RAM

- ✓ Las memorias de Acceso Aleatorio son conocidas como memorias *RAM* de la sigla en inglés *Random Access Memory*.
- ✓ Se caracterizan por ser memorias de lectura/escritura y contienen un conjunto de variables de dirección que permiten seleccionar cualquier dirección de memoria de forma directa e independiente de la posición en la que se encuentre.
- ✓ Estas memorias son volátiles, es decir, que se pierde la información cuando no hay energía y se clasifican en dos categorías básicas: la *RAM* *estática* y la *RAM* *dinámica*.

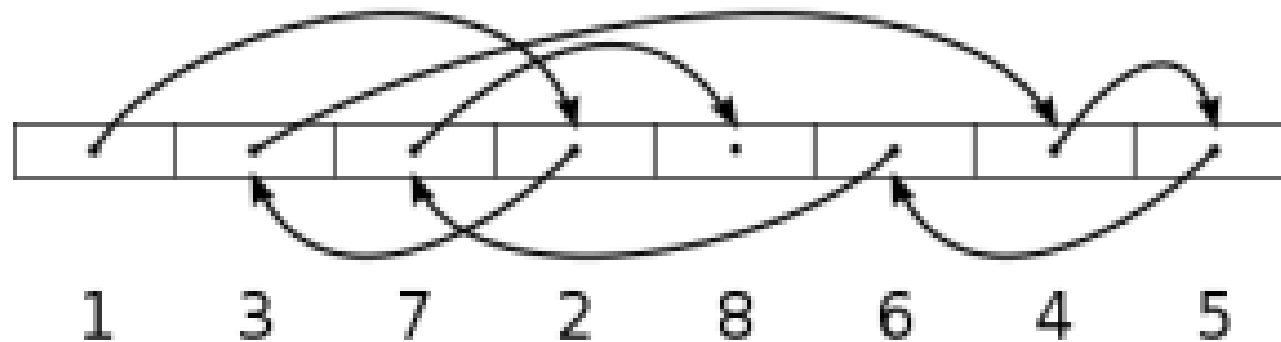
Comparación esquemática de ambos tipos de acceso a datos:

Aleatorio y Secuencial

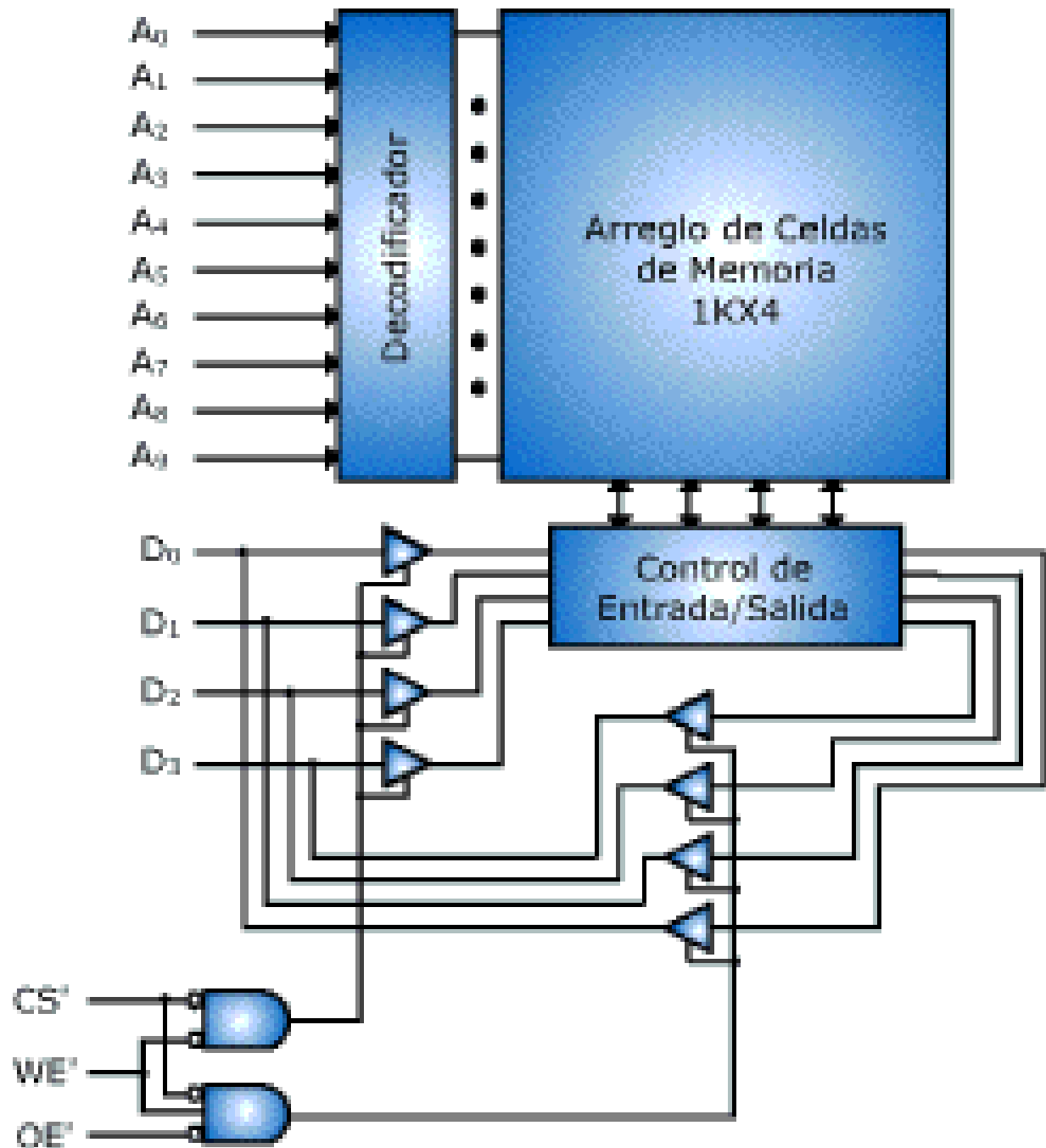
Acceso secuencial



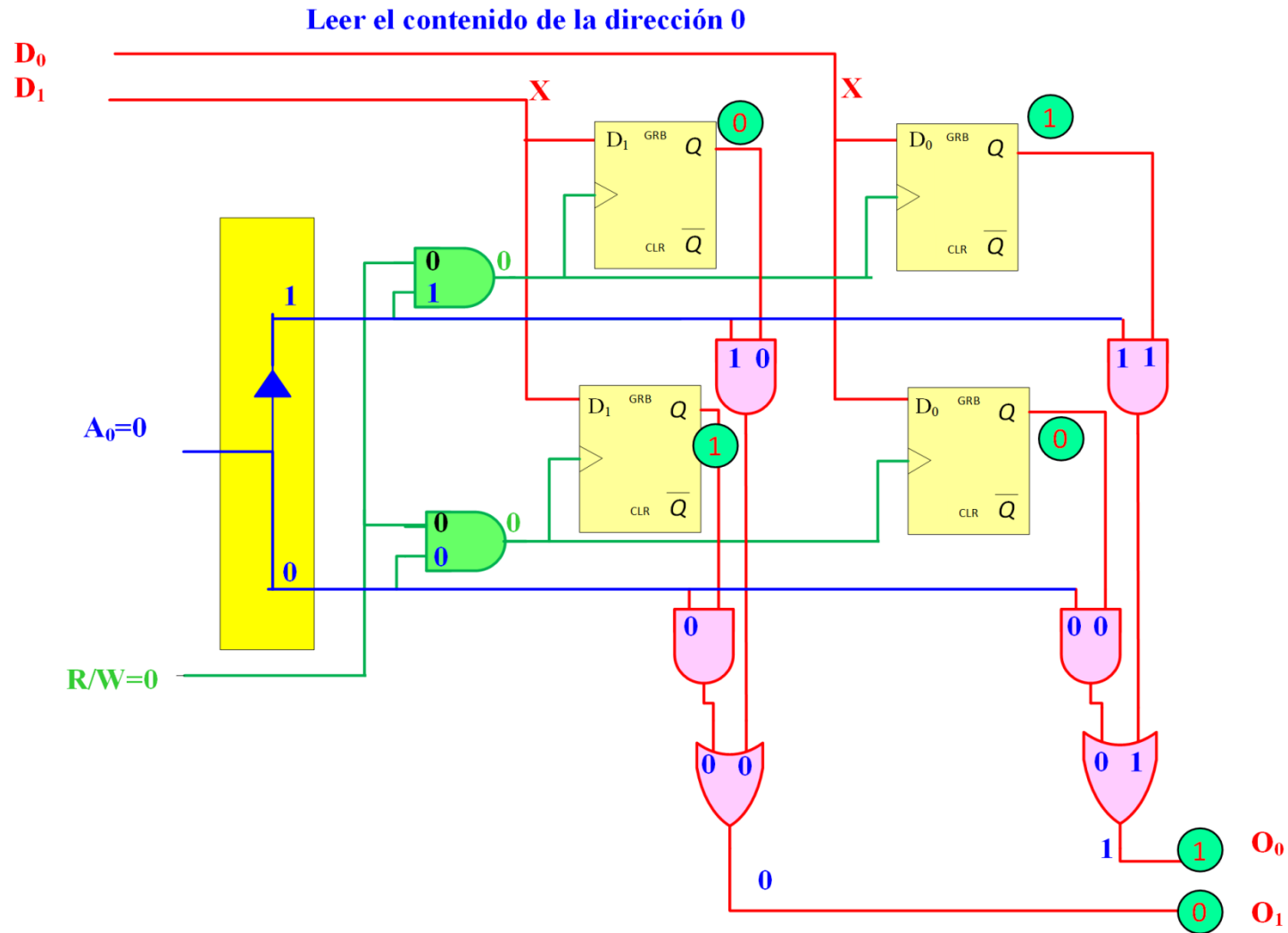
Acceso aleatorio



Esquema descriptivo de una Memoria

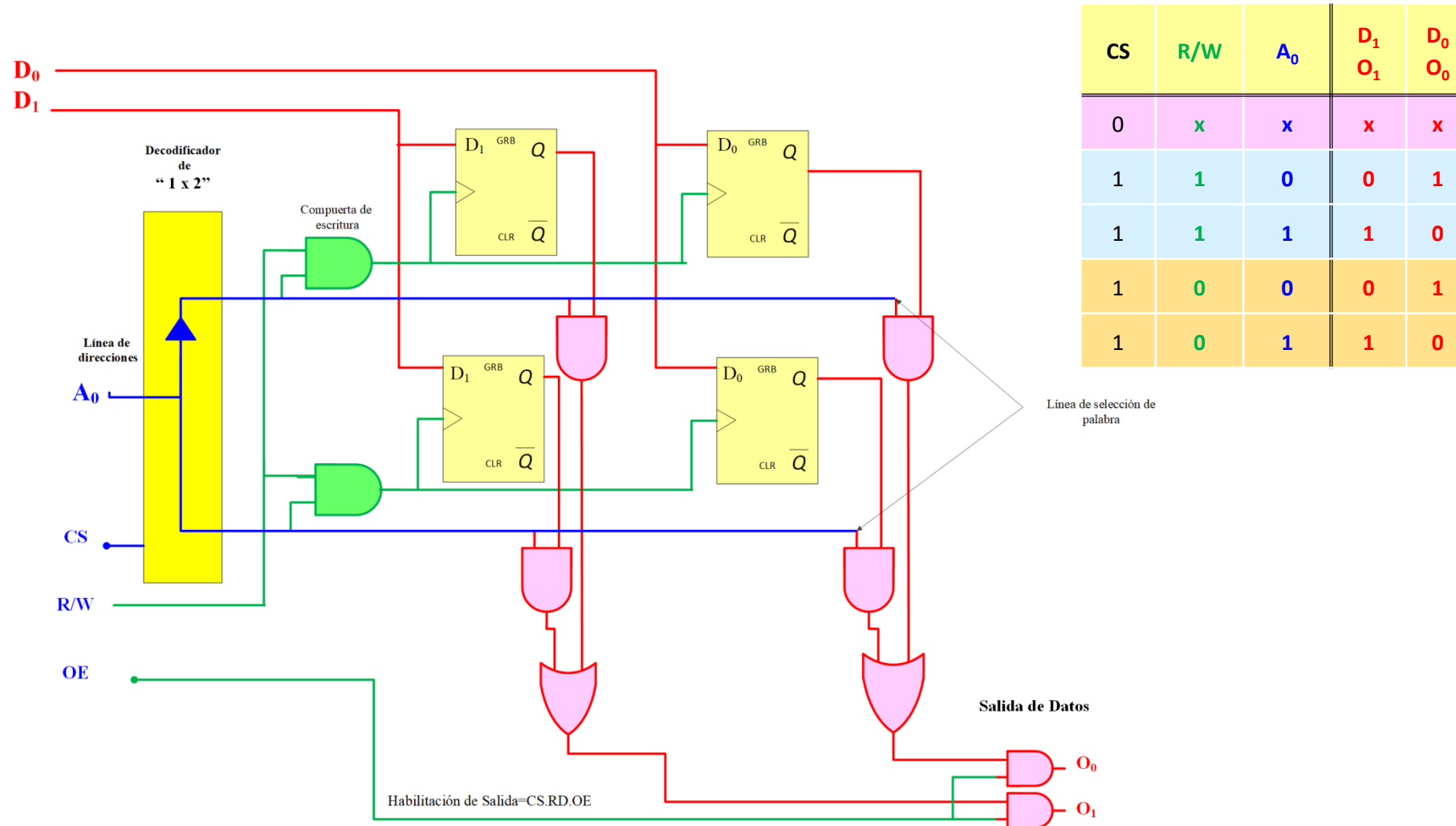


Memoria Vólátil – Cache (RAM)

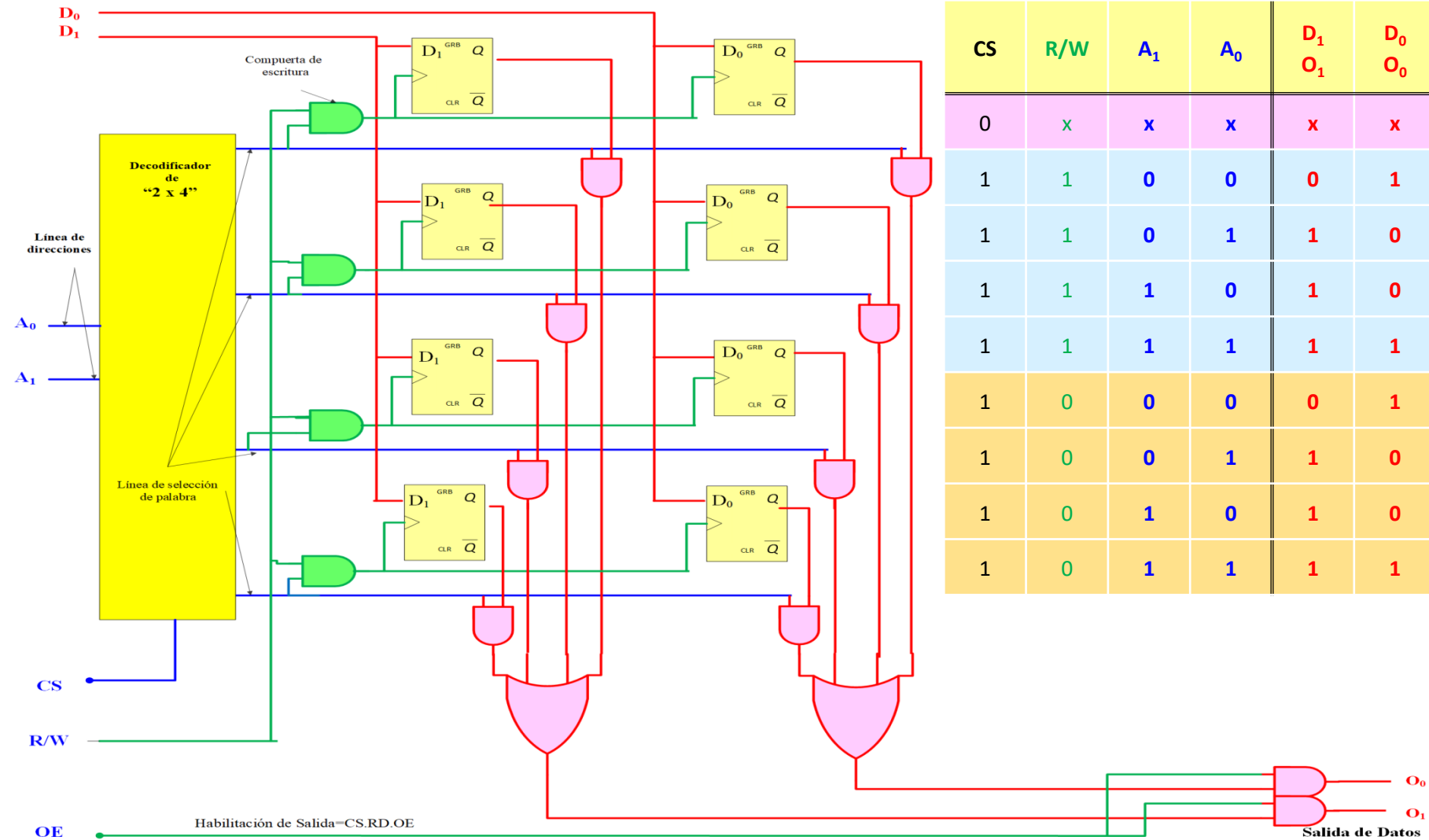


R/W	A_0	D_1 O_1	D_0 O_0
1	0	0	1
1	1	1	0
0	0	0	1
0	1	1	0

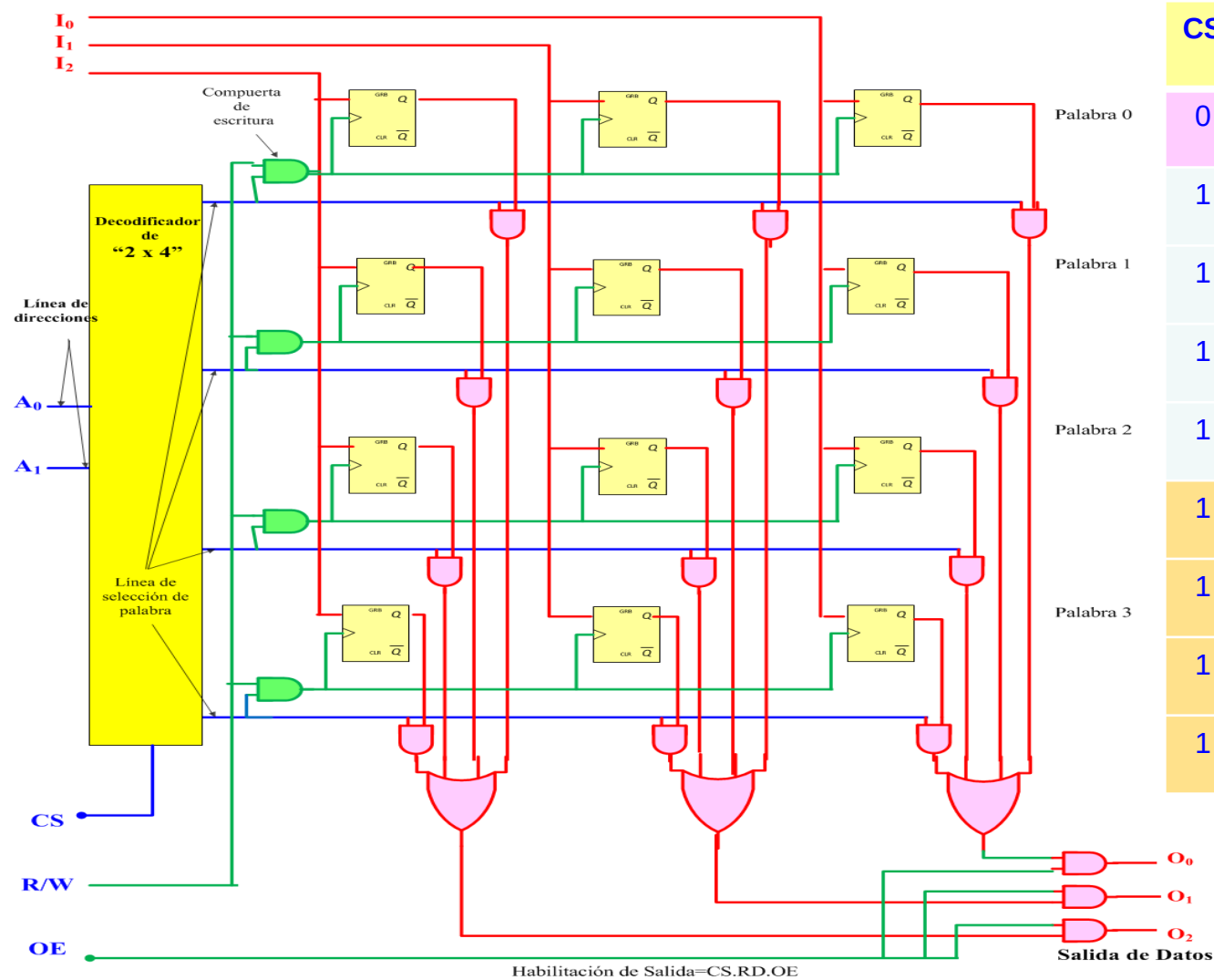
Memoria Vólátil – Cache (RAM)



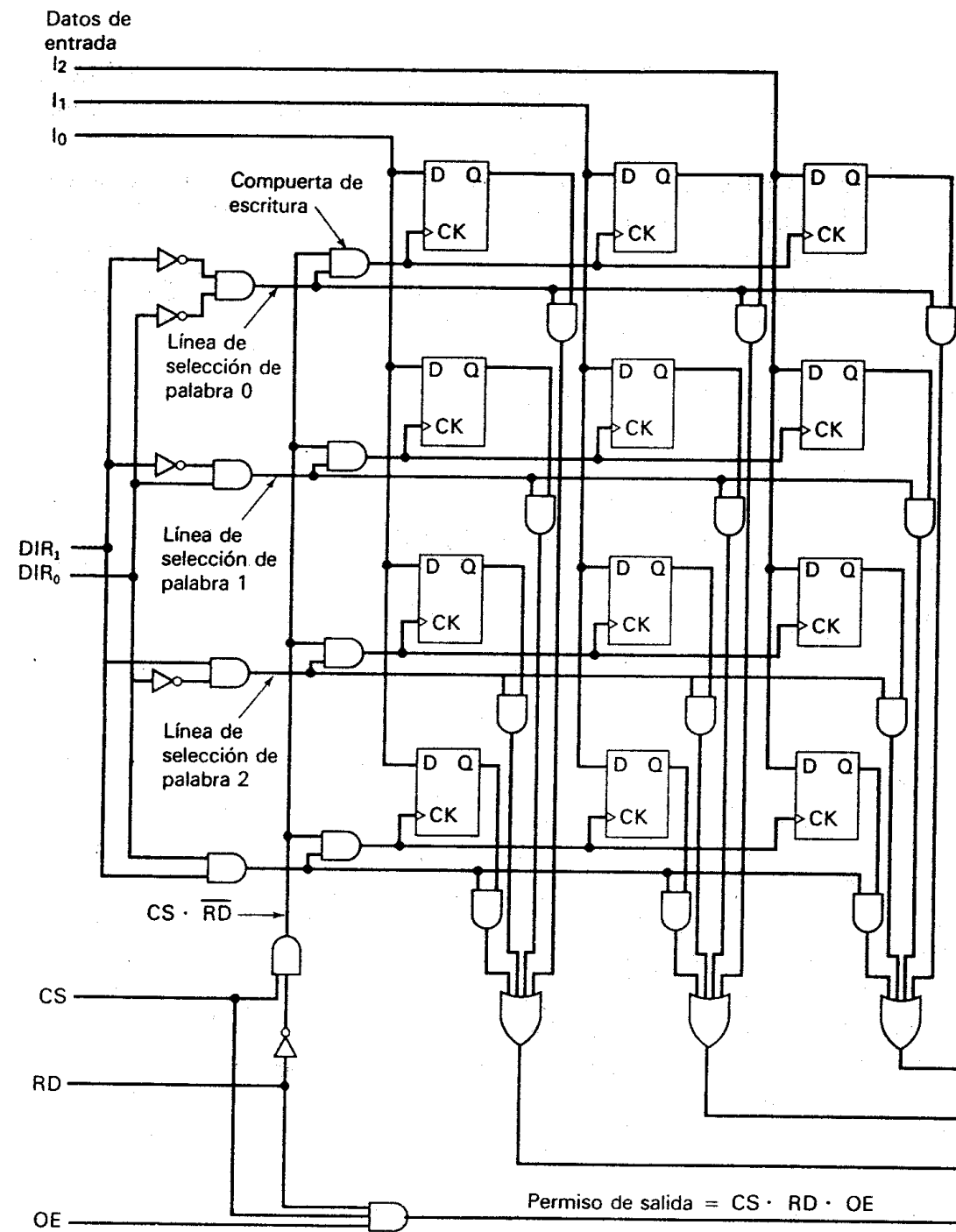
Memoria Vólátil – Cache (RAM)



Memoria Vólátil – Cache (RAM)

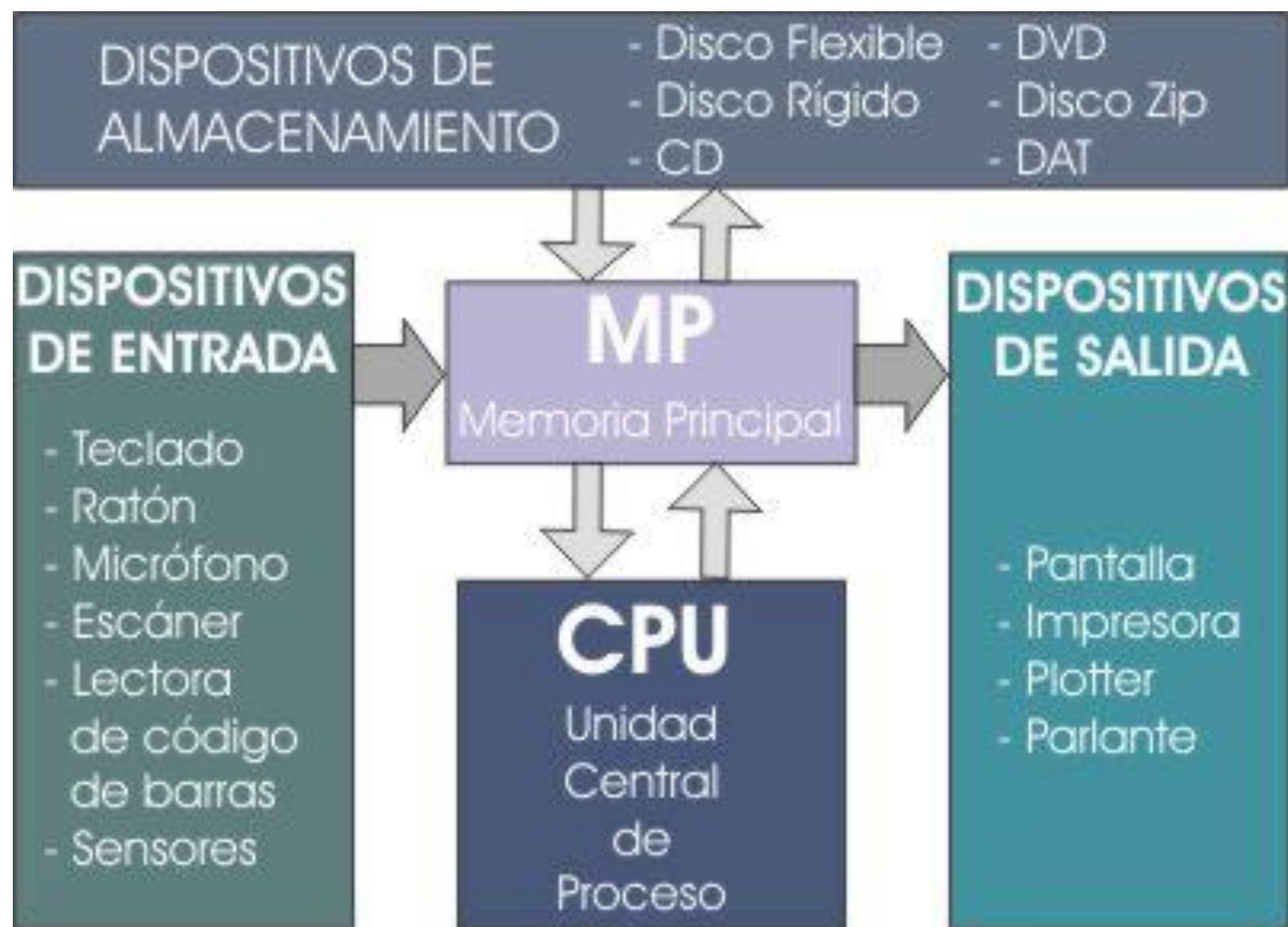


CS	R/W	A ₁	A ₀	O ₂	O ₁	O ₀
0	X	X	X	X	X	X
1	1	0	0	0	1	1
1	1	0	1	1	0	0
1	1	1	0	1	0	1
1	1	1	1	1	1	0
1	0	0	0	0	1	1
1	0	0	1	1	0	0
1	0	1	0	1	0	1
1	0	1	1	1	1	0



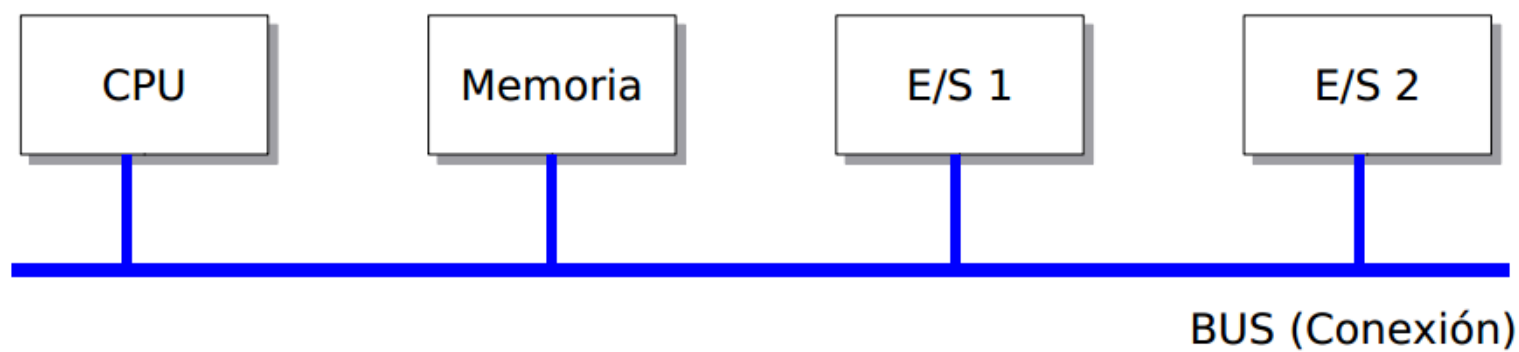
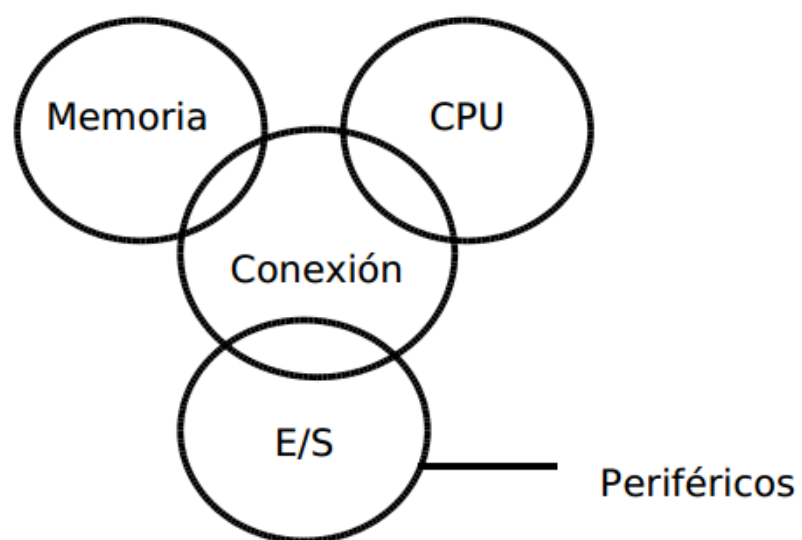
CATEDRA DE ARQUITECTURA DE COMPUTADORAS

ESTRUCTURA DEL COMPUTADOR



* Las flechas corresponden a flujos de datos e información

Estructura del computador



Descripción de los componentes de un sistema

El computador está constituido por **dispositivos** clasificados en:

- **Dispositivos de Entrada (Teclado, Scanner)**
- **Dispositivos de Salida (Monitor, Impresora)**
- **Dispositivos de Comunicación (Tarjeta de Red)**
- **Dispositivos de Almacenamiento (Disco Rígido, el CD, el DVD, Memoria USB)**
- **Dispositivos de Cómputo (Unidad Central de Procesamiento)**

Interfaz de Entrada - Salida

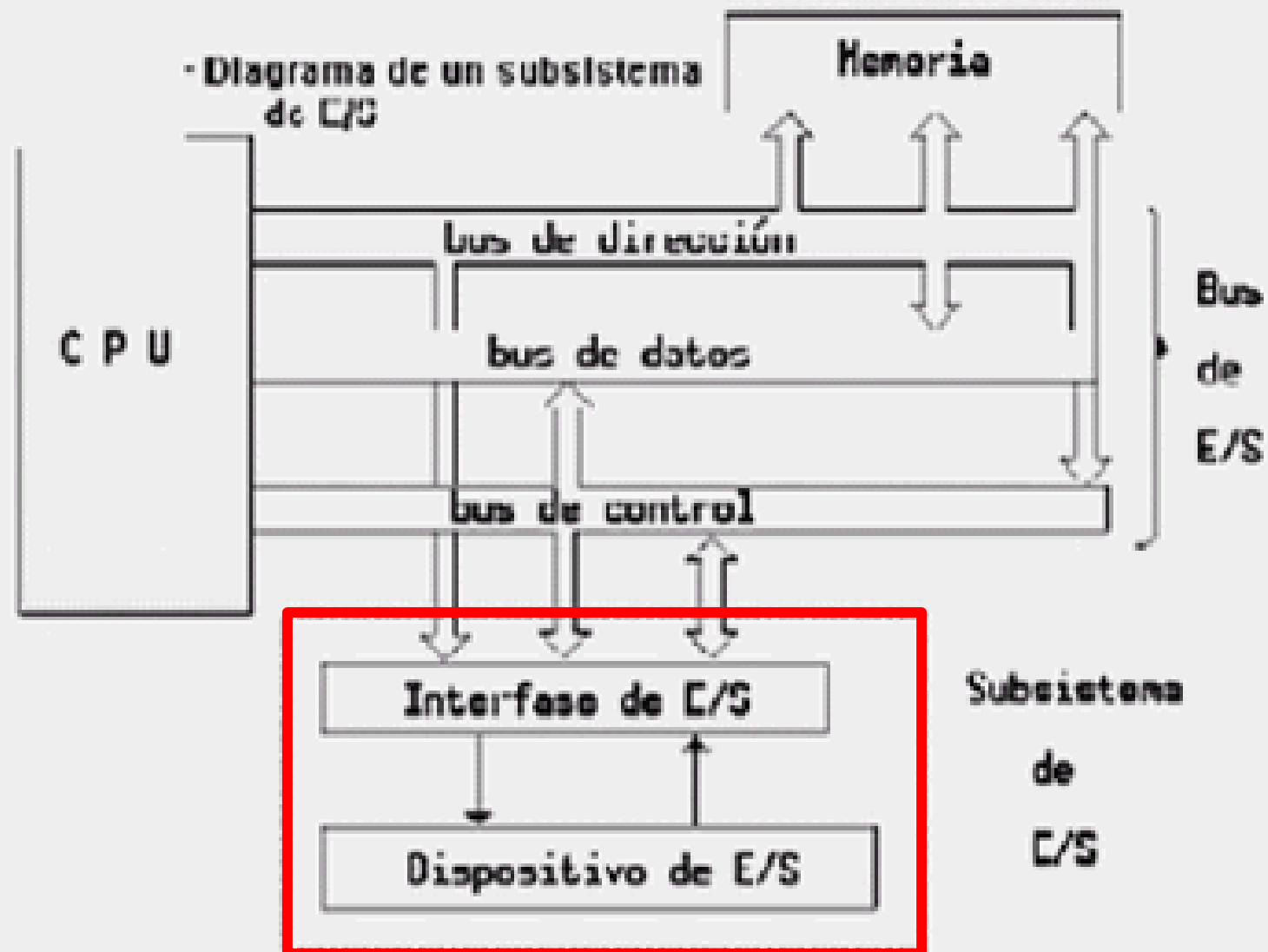
La interfaz **establece la comunicación** entre la **unidad central** y el **proceso**, filtrando, adaptando y codificando de forma comprensible para dicha unidad las señales procedentes de los elementos de entrada, y decodificando y amplificando las señales generadas durante la ejecución del programa antes de enviarlas a los elementos de salida.

La interfaz E-S **proporciona un método para transferir información** entre dispositivos de (E/S) de almacenamiento interno y de (E/S) externas. Los periféricos conectados a una computadora necesitan un enlace de comunicación especial para funcionar como una interfaz con la unidad de procesamiento central.

El **propósito** del enlace de comunicación es **resolver las diferencias** que existen entre la computadora central y cada periférico.

Se llama interfaz porque se comunica tanto con el canal del procesador como con el dispositivo periférico.

- Diagrama de un subsistema de E/S



Funciones de la interfaz

- **Almacenar** los datos y realizar las conversiones que se requieran.
- **Detectar** errores en la transmisión.
- Ser capaz de **reiniciar** la transacción en casos de error.
- Puede **testear**, **arrancar** y **detener** el dispositivo según las directivas impartidas por la CPU.
- En algunos casos la interfaz puede **consultar** a la CPU si algún dispositivo está requiriendo atención urgente

Almacenamientos Auxiliares

El ***almacenamiento secundario*** (memoria secundaria, memoria auxiliar o memoria externa) es el ***conjunto*** de ***dispositivos*** (aparatos) y ***medios*** (soportes) de almacenamiento, que conforman el subsistema de memoria de una computadora, junto a la memoria principal.

No deben confundirse las "unidades o dispositivos de almacenamiento" con los "medios o soportes de almacenamiento", pues los primeros son los aparatos que leen o escriben los datos almacenados en los soportes.

La ***memoria secundaria*** es un tipo de almacenamiento masivo y permanente (no volátil), a diferencia de la memoria RAM que es volátil; pero posee mayor capacidad que la memoria principal, aunque es más lenta que ésta.

En la actualidad para almacenar información se utilizan principalmente tres **'tecnologías'**:

- **Magnética** (ej. disco duro, cintas magnéticas);
- **Óptica** (ej. CD, DVD, etc.)
- **Memoria Flash** (Tarjetas de Memorias Flash y Unidades de Estado sólido SSD)

Características del almacenamiento secundario

- Gran capacidad de almacenamiento.
- No se pierde la información por falta de alimentación.
- Altas velocidades de transferencia de información.
- Mismo formato de almacenamiento que en memoria principal.
- Siempre es independiente del CPU y de la Memoria Principal. Debido a esto, los dispositivos de almacenamiento secundario también son conocidos como “**Dispositivos de Almacenamiento Externo**”.

Dispositivos (Unidades) y soportes (medios) de almacenamiento:	
<u>Dispositivos o Unidades</u>	<u>Soportes o Medios</u>
Unidad de Disco Rígido (<i>Hard Disk Drive, HDD</i>)	Discos Rígidos
Unidad de Cintas Magnéticas (<i>Tape Drive</i>)	Cintas magnéticas de Datos
Lectora o Grabadora de Discos Ópticos o Unidad de Discos Ópticos	CD (CD-ROM, CD-R, CD-RW), DVD (DVD±R, DVD±RW), Blu-Ray
Lector de tarjetas de memoria	Tarjetas de Memorias Flash
Unidades Flash	Unidades de Estado Sólido (SSD)

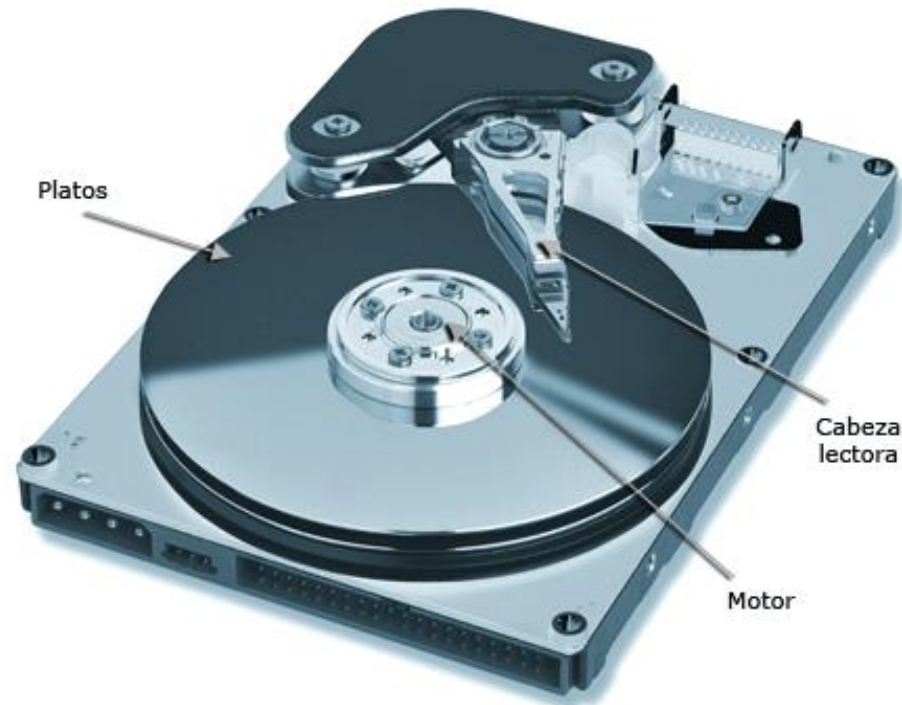
Tipos de Almacenamiento

En función de la tecnología utilizada por los dispositivos y medios (soportes), el almacenamiento se clasifica en:

- **Almacenamiento magnético.**
- **Almacenamiento óptico.**
- **Almacenamiento electrónico o de estado sólido (Memoria Flash)**

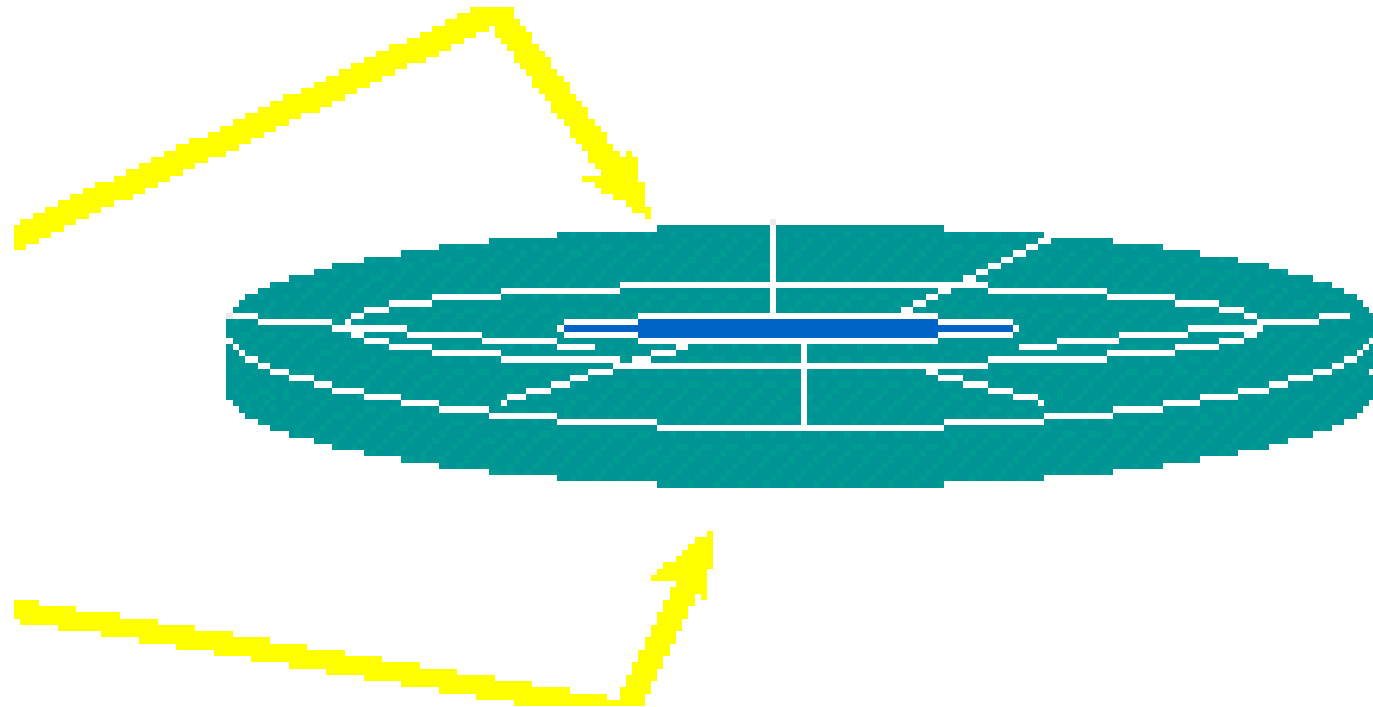
Almacenamiento magnético

Se aplican campos magnéticos sobre materiales capaces de reaccionar frente a esta influencia y orientarse en determinadas posiciones (Antes se cubrían los platos de los discos con partículas de óxido férrico como material a magnetizar para el almacenamiento de datos; actualmente se utilizan partículas de una aleación de cobalto.).
Ejemplo: disco duro, cinta magnética.

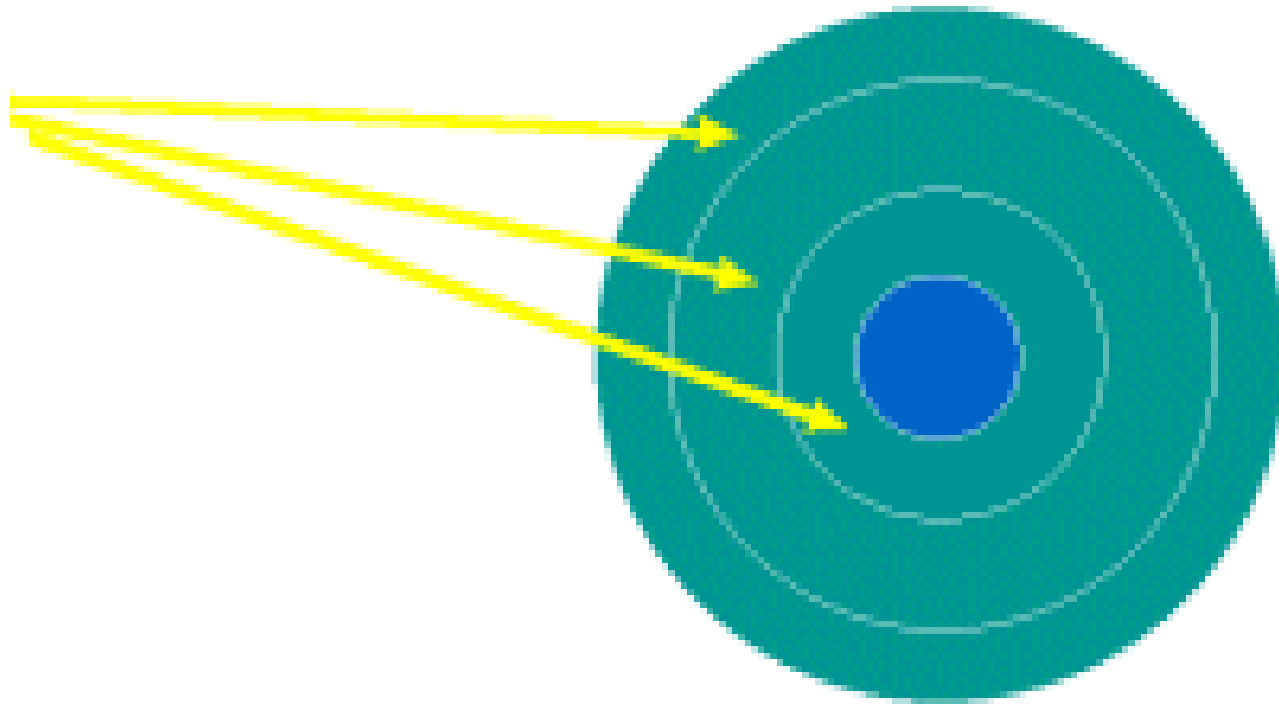


COMPOSICION INTERNA

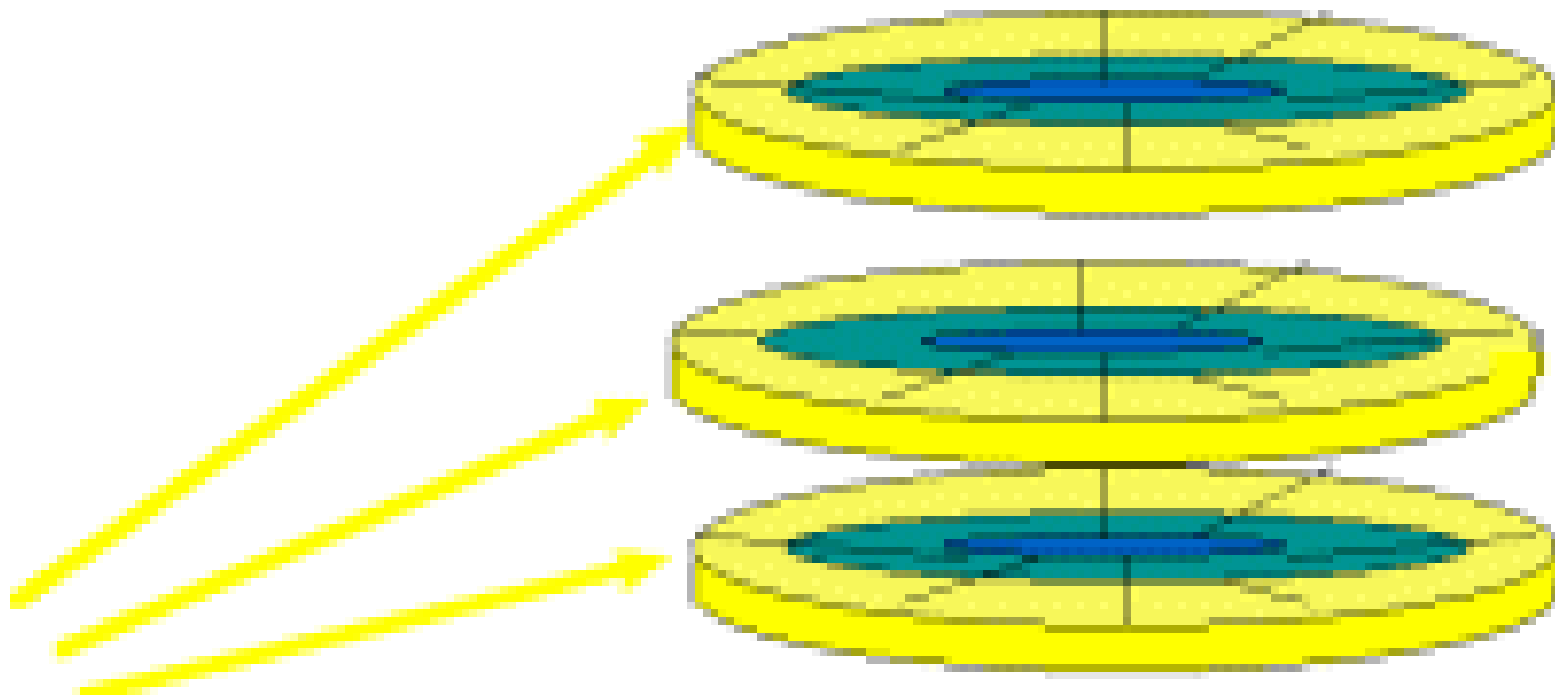
- **CARAS:** En un disco es posible leer y escribir información en sus dos caras (la primera cara es la numero 0 y la segunda el 1). Los discos duros pueden tener múltiples platos, cada uno de ellos con dos caras, numeradas 0, 1, 2, 3, etc.



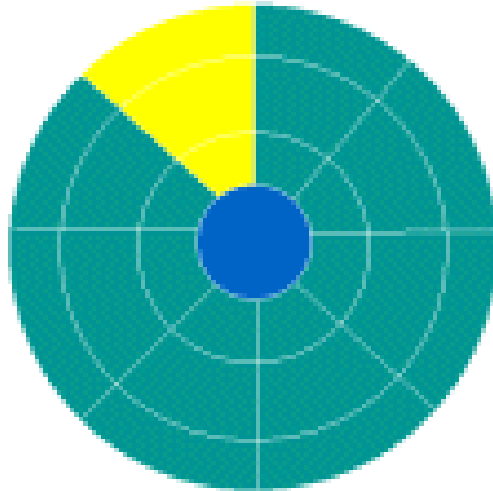
- **PISTAS:** Las caras de un disco se dividen según círculos concéntricos. La pista de mayor radio en cada cara es la 0. Así tenemos la pista 0 cara 0, y la de la otra cara pista 0 cara 1.



- **CILINDROS:** Las pistas de igual radio de un disco reciben el nombre colectivo de cilindro. Si se colocan uno encima del otro se obtiene un cuerpo que semeja la forma de una lata sin tapa ni fondo - un cilindro.



- **SECTORES:** Se llama sector de un disco a una parte en forma de cuña del mismo. Cada sector está numerado. Todas las pistas de un disco se dividen en un mismo número de sectores, todos de igual área. El número de sectores por pistas varía con el tamaño de disco.



Los sectores se agrupan en **clústeres** ó **unidades de asignación**.

¿Qué es un Clúster?

Un clúster es la cantidad mínima de espacio que se puede asignar a un archivo.

Ejemplo: Si el clúster es de 4Kb, significa que para guardar un archivo de 1Kb se usan 4Kb (el clúster entero).

Para guardar 5Kb se emplean 8Kb (un clúster de 4K completo y otro con 1kB del que se desaprovechan 3KB), por eso el clúster debe ser del menor tamaño posible.

Se podría decir que es la unidad de almacenamiento del disco duro.

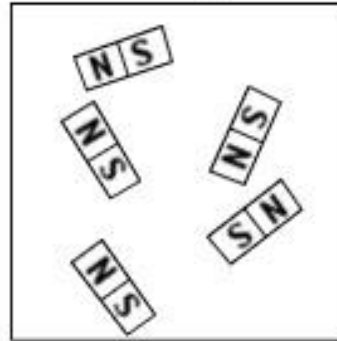
El tamaño de clúster representa la menor cantidad de espacio en disco que puede utilizarse para contener un archivo.

Video:

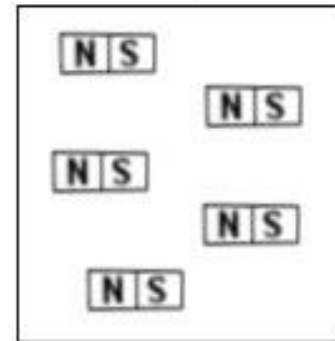
Funcionamiento de un Disco Rígido

¿Cómo se guarda la información en un Disco Rígido?

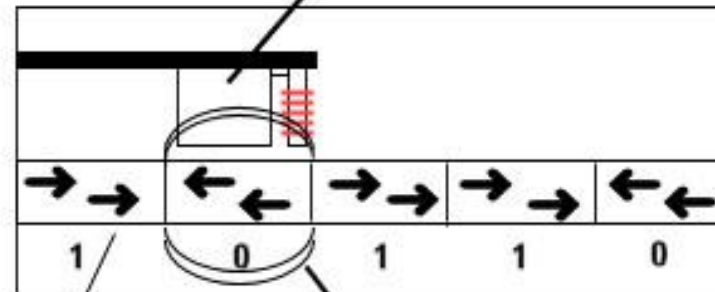
Partículas no magnetizadas



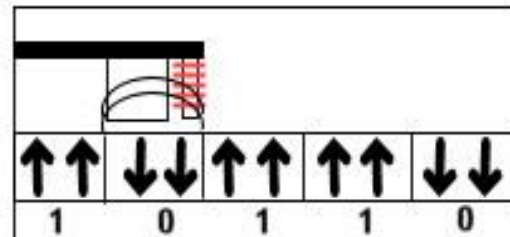
Partículas magnetizadas



Cabezal de lectura



Partículas magnetizadas



Método avanzado de partículas verticales

DISCO DURO (IBM 350)

- El 13 de septiembre de 1956 IBM presentó el primer disco duro de la historia. Podía almacenar casi 4,4 MB, tenía 50 discos de 24 pulgadas de diámetro con 100 superficies de grabación, giraban a 1200 RPM y la tasa de transferencia de datos de 8800 caracteres por segundo. Medía 1,52 mts de largo, 1,72 mts de alto y 74 cms. de ancho.

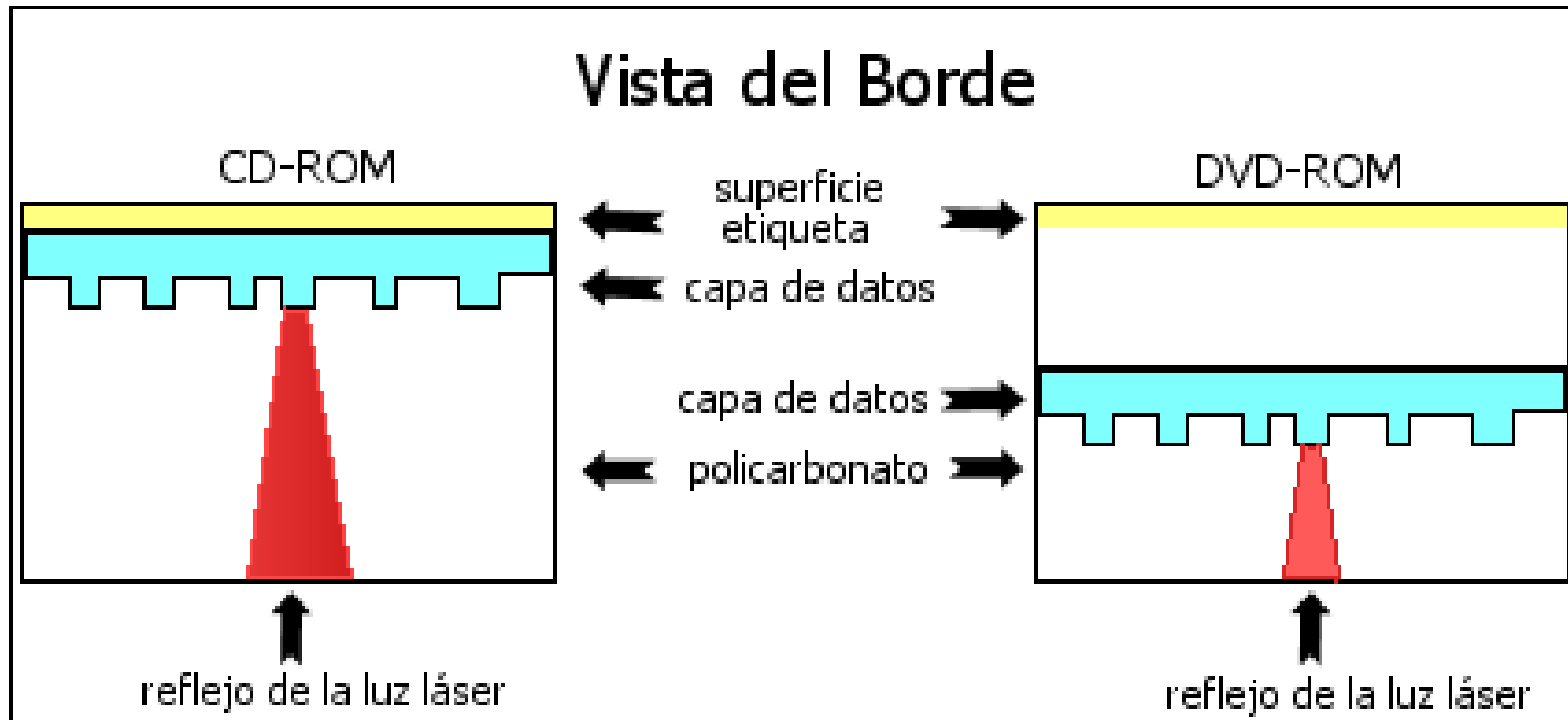




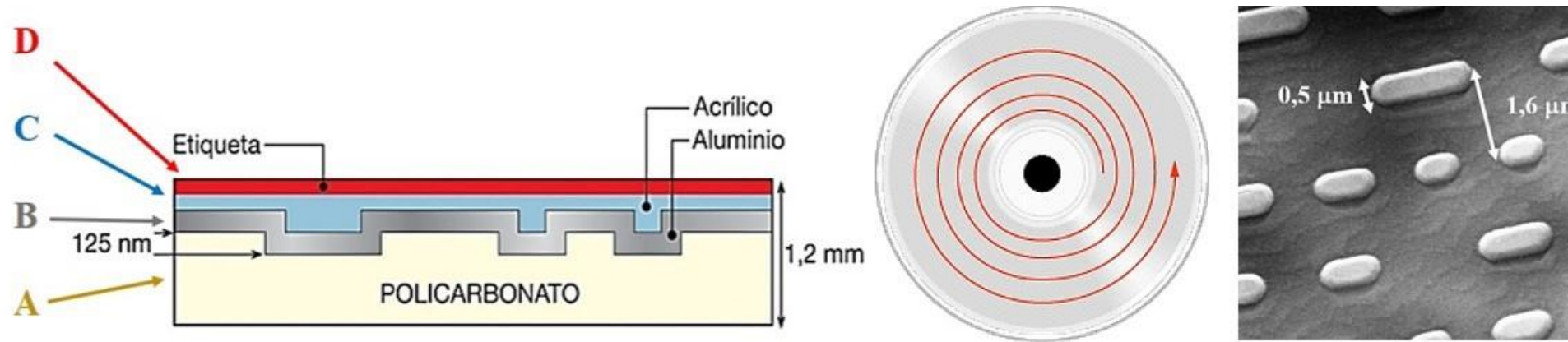


ALMACENAMIENTO ÓPTICO

En los discos ópticos la información se guarda de una forma secuencial en una espira que comienza en el centro del disco. Además de la capacidad, estos discos presentan ventajas como la fiabilidad, resistencia a los arañazos, la suciedad y a los efectos de los campos magnéticos.



ESTRUCTURA DE UN DISCO OPTICO

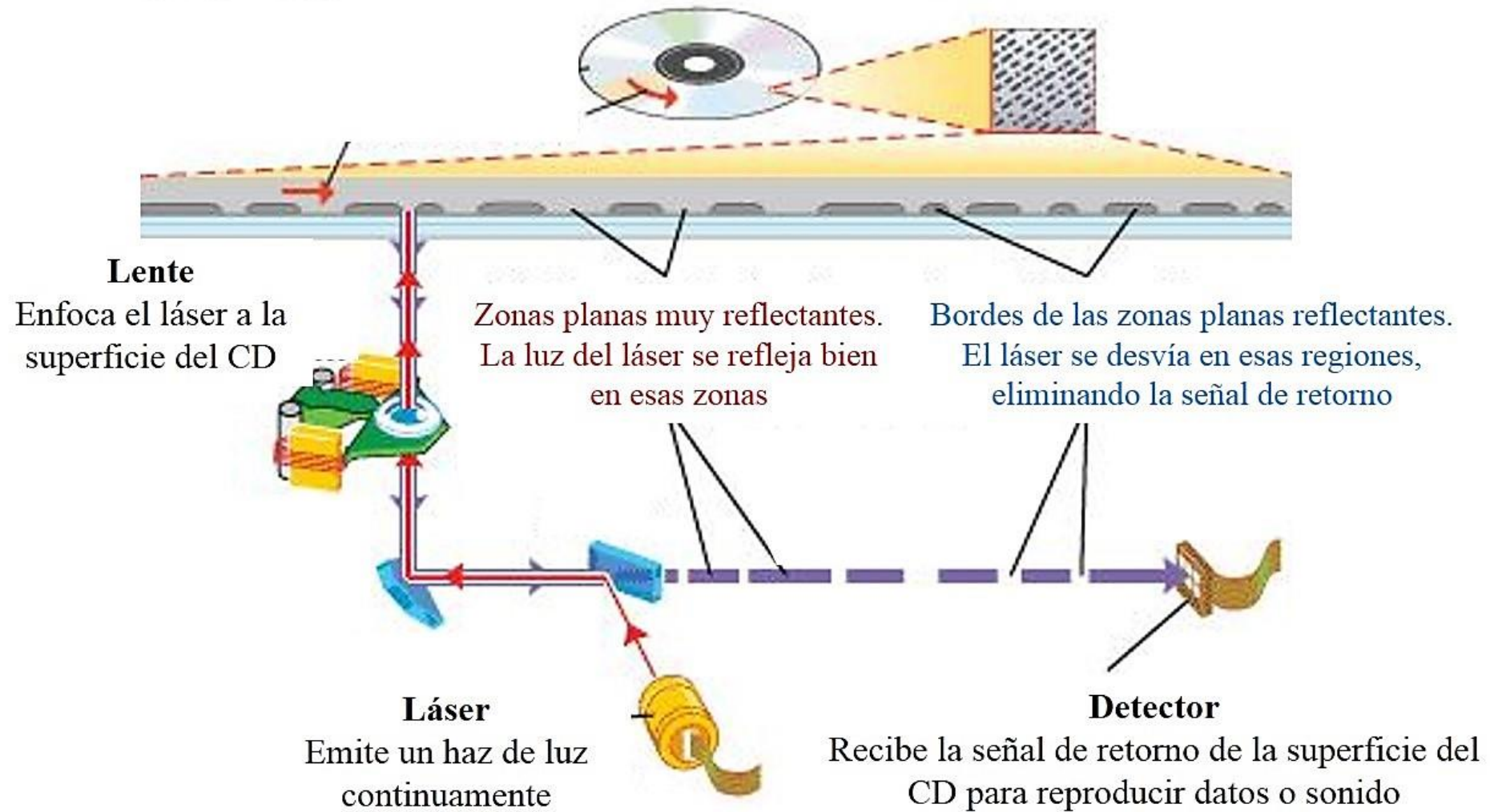


Izquierda: Las distintas capas que componen un CD.

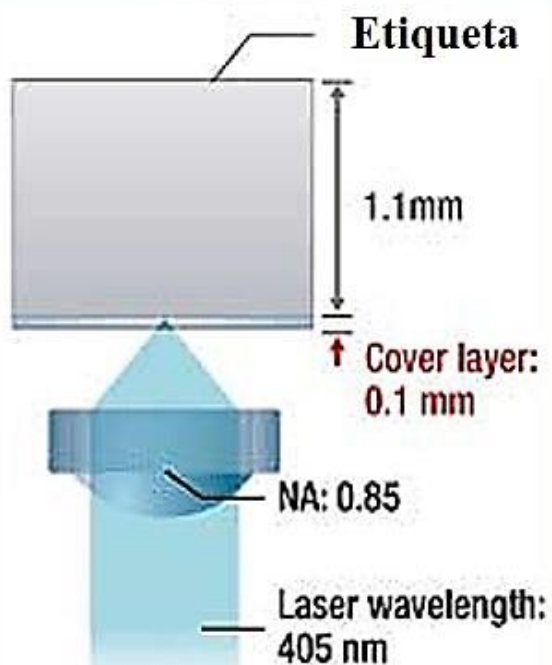
Centro: La superficie de un CD visto por la superficie donde se graban los datos. La espiral señala el camino que hacen estos.

Derecha: Imagen tomada con un microscópico electrónico de barrido de la superficie de un CD, con la anchura de las protuberancias y de la separación entre dos pistas consecutivas

ESQUEMA DE FUNCIONAMIENTO



Blu-ray Disc

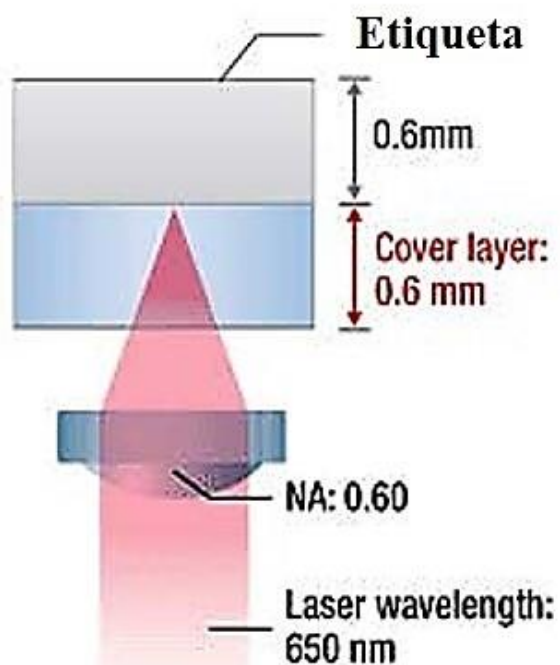


2.32 Gb/cm²

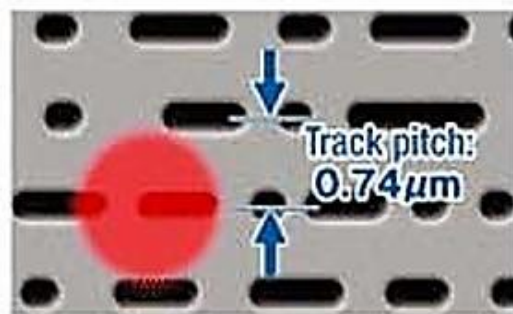


Capacity: **25GB**

DVD

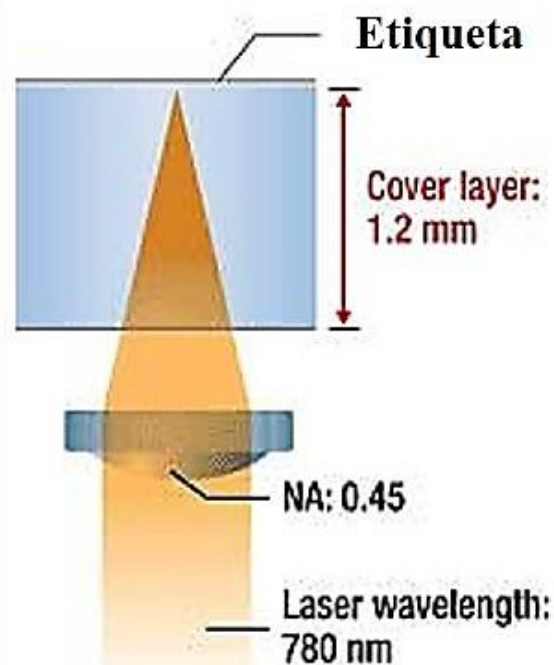


0.43 Gb/cm²

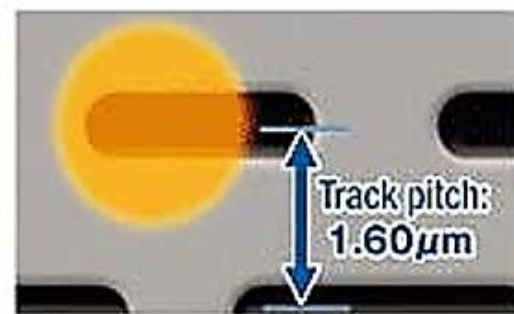


Capacity: **4.7GB**

CD



0.065 Gb/cm²



Capacity: **700MB**

EJEMPLOS

- **CD-ROM** Discos de solo lectura.
- **CD-R** Discos de escritura y múltiples lecturas.
- **CD-RW** Discos de múltiples escrituras y lecturas.
- **DVD+/-R** Discos de capacidad de 4.5GB, hasta 9.4GB, de escritura y múltiples lecturas.
- **DVD+/-RW** Discos de capacidad de 4.5GB, hasta 9.4GB, de múltiples escritura y múltiples lecturas.
- **Blu-Ray** Tecnología de disco de alta densidad, desarrollada por Sony. Su superioridad se debe a que hace uso de un laser con una longitud de onda "Azul", en vez de "Roja", tecnología que ha demostrado ser mucho más rápida y eficiente que la implementada por el DVD de alta definición.

Jerarquía de Memoria

Se trata de la organización piramidal de la memoria en niveles.

Objetivo:

Lograr el rendimiento de una memoria de gran velocidad al costo de una memoria de baja velocidad (basándose en el principio de cercanía de referencias)

La determinación de una Jerarquía de Memoria se basa en tres atributos:

- ✓ **VELOCIDAD DE ACCESO**
- ✓ **COSTO DE LA CELDA (BIT)**
- ✓ **CAPACIDAD DE ALMACENAMIENTO**

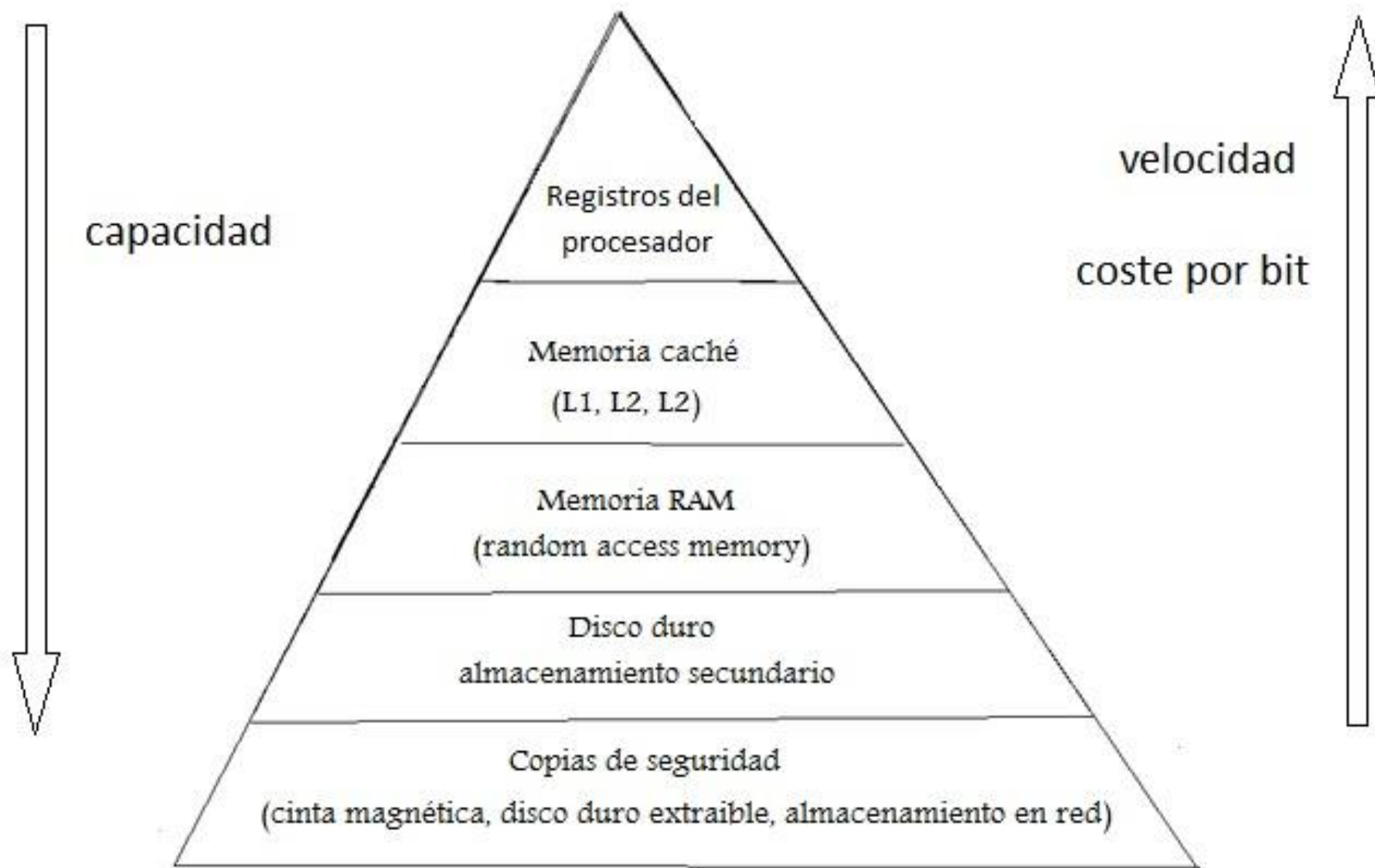
Los tres factores compiten entre sí Hay que encontrar un equilibrio. Las siguientes afirmaciones son válidas:

- **A MENOR TIEMPO DE ACCESO MAYOR COSTO.**
- **A MAYOR CAPACIDAD MAYOR COSTO.**
- **A MAYOR CAPACIDAD MENOR VELOCIDAD.**

Gracias a un principio llamado **cercanía de referencias**, es factible utilizar una mezcla de los distintos tipos y lograr un rendimiento cercano al de la memoria más rápida.

Según el principio de cercanía, las referencias a los datos y al programa dentro de un proceso tiende a agruparse y después de cortos períodos de tiempo, se necesitarán sólo unos pocos fragmentos de un proceso.

JERARQUÍA DE MEMORIA DEL COMPUTADOR



TIPOS DE MEMORIA EN LA JERARQUÍA

✓ REGISTROS INTERNOS DEL PROCESADOR:

- **Rápidas**
- **Pequeñas**
- **Costosas**
- **Volátiles**

✓ CACHE:

- **Entre los registros y la Memoria Principal.**
- **No accesibles por el usuario**
- **Volátiles**
- **Menor Capacidad de Almacenamiento**

✓ **MEMORIA PRINCIPAL:**

- **Alta densidad**
- **Volátiles**
- **Mayor Capacidad de Almacenamiento**
- **Velocidad media**

✓ **MEMORIA SECUNDARIA:**

- **Almacenamiento permanente de datos y programas**
- **Dispositivos externos**
- **Información en forma de archivos y registros**
- **Discos, cintas y ópticos**
- **Disco como extensión de M.P., da lugar a la virtual**
- **Mayor capacidad de almacenamiento**

Registros Internos del Procesador

Un registro es una memoria de alta velocidad y poca capacidad, integrada en el microprocesador.

Los registros están en la cumbre de la jerarquía de memoria, y son la manera más rápida que tiene el sistema de almacenar datos.

Los registros se miden generalmente por el número de bits que almacenan, por ejemplo, "**registro de 8 bits**" ó "**registro de 16 bits**".

El término es usado generalmente para referirse al grupo de registros que pueden ser directamente indexados como operandos de una instrucción, como está definido en el conjunto de instrucciones.

Sin embargo, los microprocesadores poseen además muchos otros registros que son usados con un propósito específico, como el contador de programa.

Memoria Caché

Una caché es un sistema especial de almacenamiento de alta velocidad.

Puede ser tanto un área reservada de la memoria principal como un dispositivo de almacenamiento de alta velocidad independiente.

Hay **dos tipos de caché** frecuentemente usados en las computadoras personales: memoria caché y caché de disco.

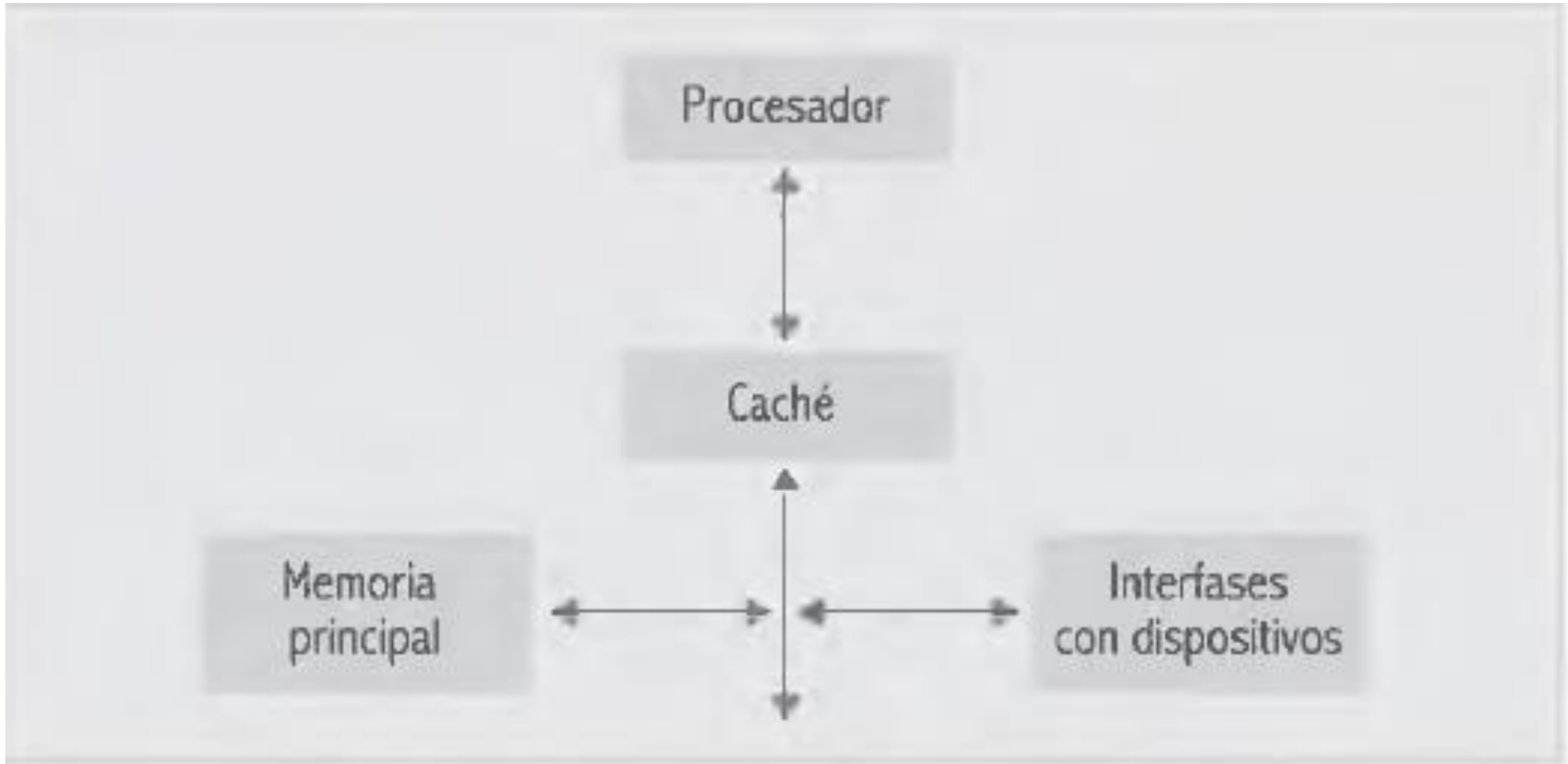
Una memoria caché, llamada también a veces almacenamiento **caché** o **RAM caché**, es una parte de memoria RAM estática de alta velocidad (**SRAM**) más rápida que la lenta y económica que la RAM dinámica (**DRAM**) usada como memoria principal. La memoria caché es efectiva dado que los programas acceden una y otra vez a los mismos datos o instrucciones.

Por lo tanto, Guardando esta información en **SRAM**, la computadora evita acceder a la **DRAM** que es mas lenta.

Los sistemas de memoria caché usan una tecnología conocida como **caché inteligente** en la cual el sistema puede **reconocer** cierto tipo de **datos usados frecuentemente**.

Las estrategias para determinar qué información debe de ser puesta en el caché constituyen uno de los problemas más interesantes en la ciencia de las computadoras. Algunas memorias caché están construidas en la arquitectura de los microprocesadores

Esquema de Relación Funcional



Una caché es más que un simple área de almacenamiento, se trata de un subsistema constituido por una memoria de etiquetas, una memoria de datos y un controlador.

El "cerebro" de un sistema de memoria caché se llama controlador de memoria caché. Cuando éste trae una instrucción de la memoria principal, también trae las próximas instrucciones y las almacena en caché. Esto ocurre porque hay una alta probabilidad de que la próxima instrucción también se necesite. Esto incrementa la posibilidad de que la CPU encuentre la instrucción que precise en la memoria más rápida y, por lo tanto, pueda optimizar el tiempo.

NIVELES DE CACHÉ

En el caso de los microprocesadores, estos integran de 1 a 3 tipos de memoria caché denominadas **L1**, **L2** y **L3**, que significan (“**Level X**”) o Traducido: **Nivel 1**, **Nivel 2** y **Nivel 3**.

+ **Caché L1**: Se encuentra **integrada dentro de los circuitos del microprocesador** y eso la hace más costosa y más complicada en el diseño, pero también mucho más eficiente ya que funciona a la misma velocidad que él. Es de tamaño reducido y se ubica funcionalmente, interceptando los datos que entran en el microprocesador desde la memoria o los que salen ya procesados hacia la memoria.

– **L1 DC**: (“**Level 1 Data Cache**”): **Encargada de almacenar datos usados frecuentemente** y cuando sea necesario volver a utilizarlos, inmediatamente los accede, por lo que se agilizan los procesos.

– **L1 IC**: (“**Level 1 Instruction Cache**”): **Encargada de almacenar instrucciones usadas frecuentemente** y cuando sea necesario volver a utilizarlas, inmediatamente las recupera, por lo que se agilizan los procesos.

+ Memoria L2: esta anteriormente se encontraba en tarjetas de memoria, para ser insertada en una ranura especial de la tarjeta principal (***Motherboard***) y funciona a la velocidad de trabajo de la misma. Actualmente la memoria L2 viene integrada en el microprocesador, se encarga de almacenar datos de uso frecuente y agilizar los procesos; determina por mucho si un microprocesador es la versión completa ó un modelo austero.

+ Memoria L3: La memoria caché L3, o de nivel 3, es una memoria que está integrada en la placa base. Se utiliza para alimentar a la memoria caché L2, y generalmente es más rápida que la memoria principal del sistema.

- Las ventajas de contar con varios niveles se empiezan a notar cuando las aplicaciones son muy grandes o requieren acceso frecuente a datos en la memoria principal.
- Si la computadora es un servidor en una red de computadoras, el acceso a su memoria principal es elevado, así como el acceso a sus dispositivos de E/S; esto produce un alto tráfico en el bus, que se beneficia con la implementación de estos niveles de memoria.
- En el caso de sistemas multiprocesadores su utilización se justifica en gran medida, ya que en estos sistemas muchos comparten la misma memoria, por lo tanto, el acceso frecuente al bus puede llegar a condiciones de saturación

La Memoria como en un Espacio Lógico

El procesador ve a la memoria como hardware (como una matriz de $m \times n$), que contiene los datos y programas necesarios para ejecutar los procesos.

El procesador no hace diferencia entre las distintas tecnologías que pueden constituir una memoria principal, simplemente envía la dirección física del byte vía bus de direcciones.

La manera en que una memoria se administra y se gestionan sus accesos depende fundamentalmente del sistema operativo.

La Memoria como en un Espacio Lógico

Una computadora puede tener menos memoria real que su posible capacidad de direccionamiento.

También puede ocurrir que la cantidad de bits de una referencia a memoria no alcance para acceder a toda la memoria.

En ambos casos, la capacidad de direccionamiento lógico no es congruente con la capacidad física real de la memoria.

Como las direcciones en los programas no son direcciones físicas sino lógicas, para lograr el acceso deberán traducirse (**mapearse**).

El procedimiento de “**mapeo**” consiste en aplicar un algoritmo para establecer la correspondencia entre direcciones **lógicas** y **físicas**, que a veces incluyen una o más tablas que contienen "una parte" de la dirección.

Almacenamiento de bytes en Memoria

Ya sea que se trate de datos o código de programa, el almacenamiento en memoria debe seguir un orden específico.

Dependiendo de la manera en que sean almacenados en memoria los bytes podemos clasificarlos en dos formatos diferentes:

LITTLE ENDIAN

BIG ENDIAN

LITTLE ENDIAN

El byte de menor peso se almacena en la dirección más baja de memoria y el byte de mayor peso en la más alta. Así, **un Long Int de 4 bytes**:

Byte3	Byte2	Byte1	Byte0
-------	-------	-------	-------

Se almacenará en memoria de la siguiente manera:

Dirección Base +0 ==> Byte0

Dirección Base +1 ==> Byte1

Dirección Base +2 ==> Byte2

Dirección Base +3 ==> Byte3

Los procesadores **Intel** (usados en la mayoría de los ordenadores personales) y los **DEC Alpha RISC** son “**Little Endian**”.

BIG ENDIAN

El byte de mayor peso se almacena en la dirección más baja de memoria y el byte de menor peso en la dirección más alta.

Byte3	Byte2	Byte1	Byte0
-------	-------	-------	-------

El **Long Int** anterior, se almacenaría ahora así:

Dirección Base +0 ==> Byte3

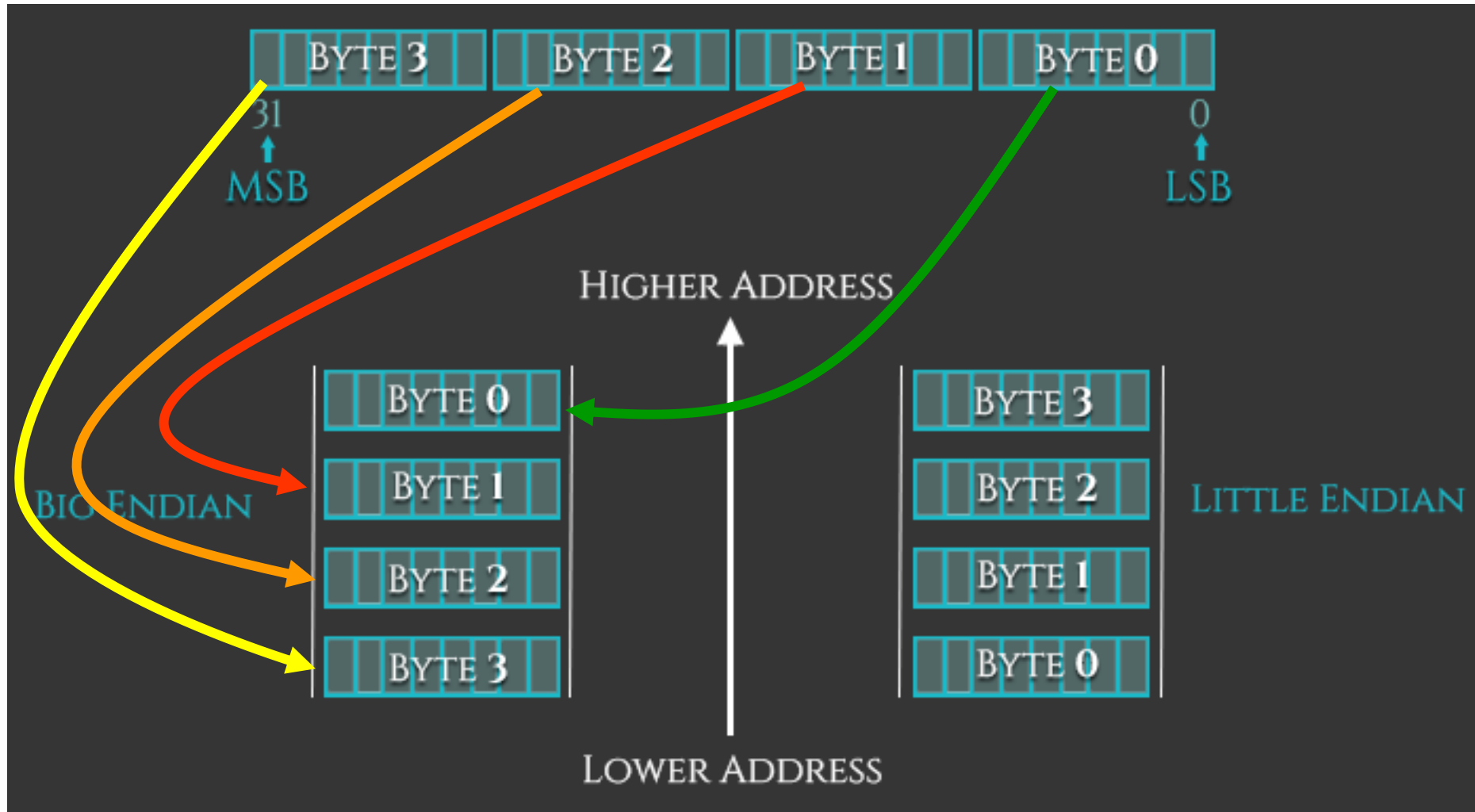
Dirección Base +1 ==> Byte2

Dirección Base +2 ==> Byte1

Dirección Base +3 ==> Byte0

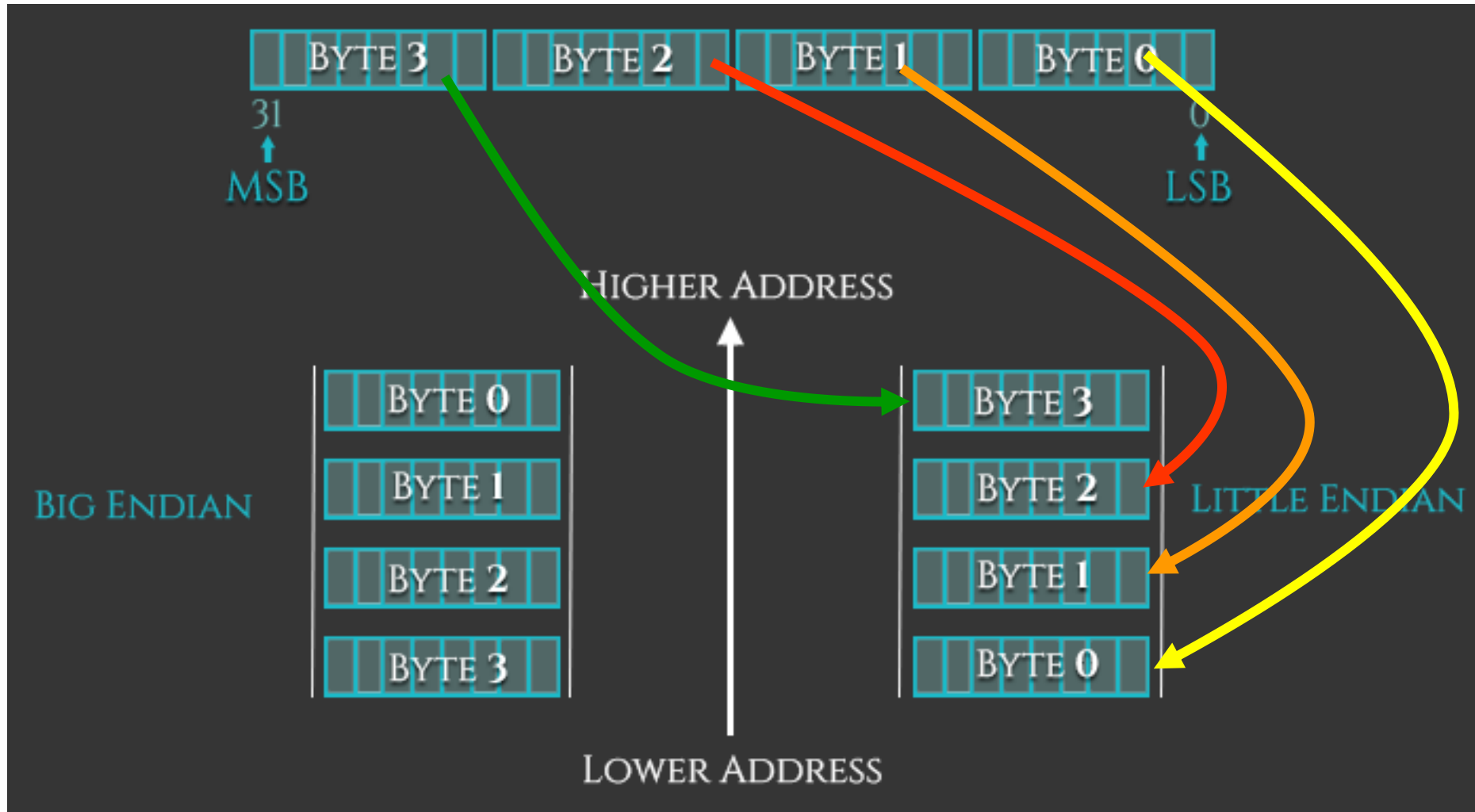
La mayoría de los sistemas **UNIX**, el protocolo de Internet **TCP**, los procesadores **Motorola 680×0** (y, por lo tanto, los **Macintosh**), **Hewlett-Packard**, **PA-RISC**, y **Sun SuperSPARC** son “**Big Endian**”.

BIG ENDIAN vs LITTLE ENDIAN



La diferencia sustancial de estos dos formatos de almacenamiento es simplemente el orden de almacenamiento acuerdo a su valor o equivalencia.

BIG ENDIAN vs LITTLE ENDIAN



La diferencia sustancial de estos dos formatos de almacenamiento es simplemente el orden de almacenamiento acuerdo a su valor o equivalencia.

Gestión de memoria y modos de operación de los procesadores

Memoria: Es uno de los recursos más valiosos que gestiona el sistema operativo.

Uno de los elementos principales que caracterizan un proceso es la memoria que utiliza.

Un proceso no puede acceder, al espacio de memoria asignado a otro proceso, lo cual es imprescindible para la seguridad y estabilidad del sistema.

Objetivos del sistema de gestión de memoria:

- Ofrecer a cada proceso un espacio lógico propio.
- Proporcionar protección entre procesos.
- Permitir que los procesos compartan memoria.
- Dar soporte a las distintas regiones del proceso.
- Maximizar el rendimiento del sistema.
- Proporcionar a los procesos mapas de memoria muy grandes.

Espacio de direcciones de un proceso: Es el conjunto de direcciones a las que hace referencia.

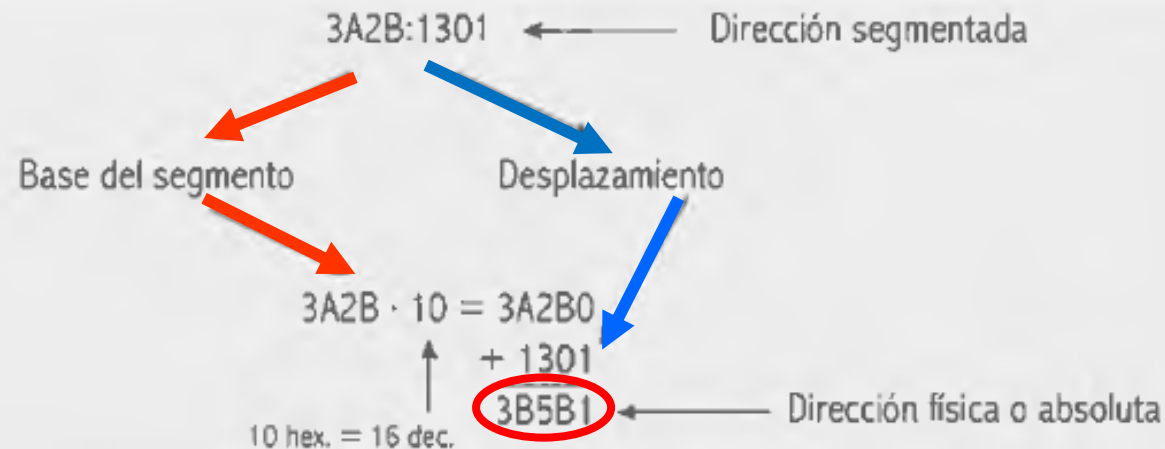
Los espacios de direcciones involucrados en la gestión de la memoria son de tres tipos:

- **Espacio de direcciones físicas:** Las direcciones físicas son aquellas que referencian alguna posición de la memoria física. Utilizado para acceder a las celdas de memoria RAM del sistema.
- **Espacio de direcciones lógicas:** Las direcciones lógicas son las direcciones utilizadas por los procesos. Las direcciones se componen de un **segmento (segment)** y un **desplazamiento (offset)**.
- **Espacio de direcciones lineales:** Las direcciones lineales se obtienen a partir de las direcciones lógicas tras haber aplicado una transformación dependiente de la arquitectura. (**direccionamiento virtual**). **Permite acceder a la memoria como un todo** (Permite referenciar por ej.: a $2^{32} = 4\text{GB}$ de memoria).



Denominaremos dirección lógica a la entidad binaria que identifica una locación de memoria a nivel software, que no se corresponde con la dirección física y, por lo tanto, **necesita ser mapeada**. Para direccionar una posición de memoria dentro de un segmento, cada dirección lógica se compone de dos entidades: una para **identificar el segmento** y otra para **identificar un desplazamiento dentro del segmento**.

Caso ejemplo: algoritmo de cálculo que realiza la MMU (Memory Manager Unit) del procesador en modo real



Gestión de memoria y modos de operación de los procesadores

Una de las formas de administrar la memoria es hacerlo sin realizar una adecuada protección de la misma  “**MODO REAL**”.

En los sistemas operativos multitarea, uno de los recursos a compartir es la memoria, por lo tanto se deberían prevenir conflictos entre las distintas áreas locales de cada una de ellas.

Esto es responsabilidad en parte de la administración del sistema operativo, y en parte del soporte hardware que desde el procesador gestiona el mapeo de la memoria.

El modo de operación del procesador, denominado “**MODO PROTEGIDO**”, implica entre otras cosas

Resguardo de la integridad de la memoria

Gestión de memoria y modos de operación de los procesadores

En ambos modos la memoria no tiene la misma vista lógica, es por ello que ambos modos de operación no pueden estar activados en forma simultánea;

El procesador puede conmutar de un modo al otro.

Por lo tanto existe una dependencia entre la forma de administrar la memoria y el soporte que brinda el hardware del procesador para hacerlo posible. Éste es el motivo por el que se incluyen algunos conceptos de administración de memoria en relación con el soporte hardware que requiere.

Memoria Virtual

Desde las primeras computadoras, la memoria principal era insuficiente en relación con los requerimientos del programa.

El concepto de un programa almacenado en su totalidad en memoria principal estaba lejos de poder ser implementado.

Por aquel entonces, las memorias secundarias de tipo disco ya existían y los programadores **simulaban el concepto original de programa almacenado en memoria fragmentando el programa.**

De esta manera, un fragmento cabía en la memoria física y el programador provocaba la búsqueda del fragmento siguiente cuando el actual ya se había ejecutado.

Con el tiempo se fue perfeccionando una técnica para que tanto la fragmentación como el intercambio de fragmentos entre el disco y la memoria se hicieran en forma automática.

Actualmente, la gestión esta a cargo del sistema operativo sin intervención del programador. Esta técnica se conoce como **memoria virtual** y posibilita la ilusión de contar con una memoria principal lo suficientemente amplia como para contener el programa y los datos, aunque esto no sea real.

La técnica de **memoria virtual** involucra por un lado memoria y discos y, por el otro, al procesador y al sistema operativo que la administra.

Una dirección lógica perteneciente al espacio de direccionamiento virtual es mayor que una dirección física, y la llamaremos dirección virtual.

Al crear una tarea se asigna en disco un archivo temporal, que **es copia del archivo ejecutable dividido en secciones**, es un área de almacenamiento separada de la que ocupa el archivo ejecutable.

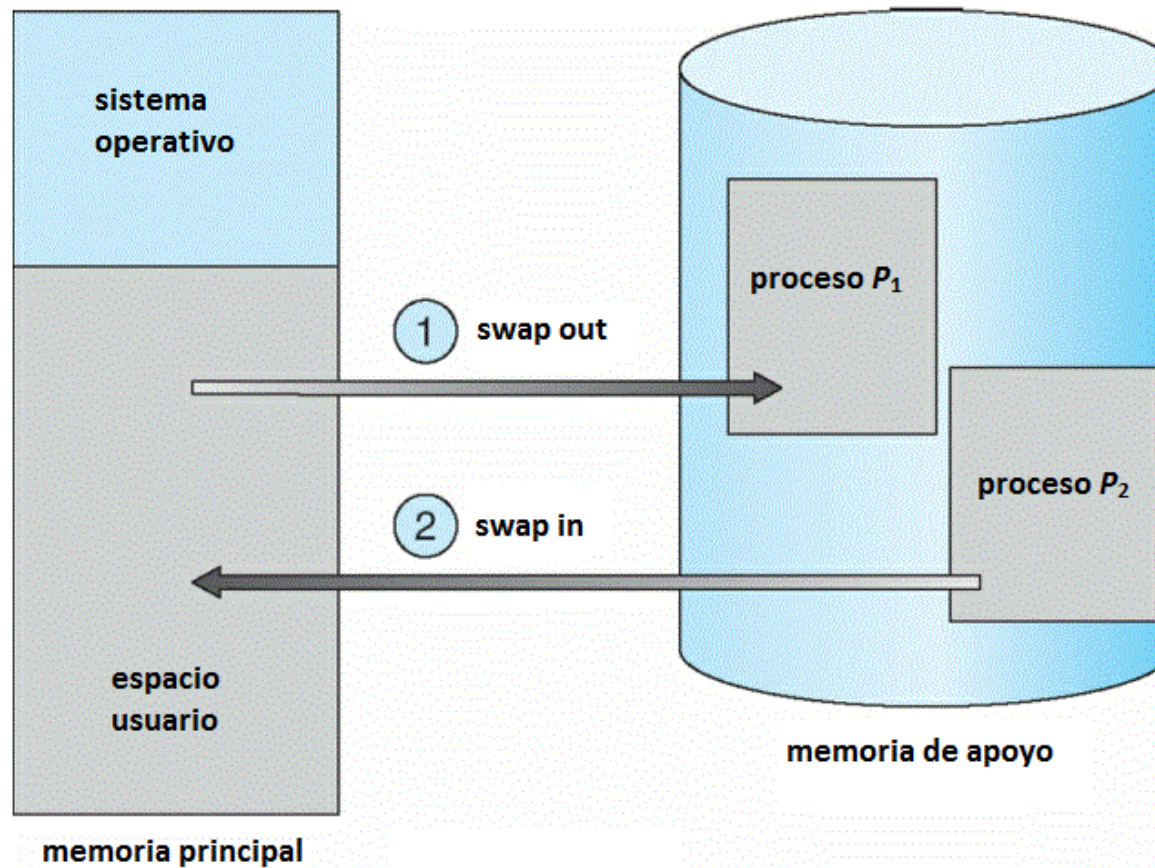
Cada sección puede ser un segmento, una página, o un segmento paginado.

La idea de “**fraccionar**” el ejecutable es que se puedan llevar a memoria secciones de código, o datos, mas pequeñas para alojarlas en memoria principal en un menor espacio de almacenamiento.

Como no se llevan todas las secciones, cada vez que se agote el uso de cada una se deberá demandar por otras alojadas en disco.

De este modo los modelos de memoria que utilizan memoria virtual se conocen como segmentación por demanda y paginación por demanda.

Swapping



Swapping (del inglés "swap", que significa intercambiar) es mover un proceso o parte de él temporalmente desde la memoria principal a un dispositivo secundario de almacenamiento (memoria de apoyo) para luego devolverlo a la memoria principal. El propósito de esta técnica es que el sistema operativo sea capaz de asignar más memoria de la que tiene físicamente. De esta forma, en el caso que algún proceso requiera de memoria y ya esté toda asignada o no haya suficiente, el kernel se preocupa de intercambiar páginas de memoria desde y hacia la memoria de apoyo para hacerle espacio.

La memoria virtual es una forma de administrar la memoria para asignar mas de ella a cada tarea, utilizando almacenamiento en disco, que suele ser el dispositivo de acceso directo mas rápido; así, se simula que se dispone de una memoria mucho mas amplia que la que el sistema tiene en realidad.

Un segmento es un bloque lógico de tamaño variable

Cada segmento contiene información de la misma clase (código, datos, pila).

Administración de Memoria

Existen dos formas de organizar la memoria de un computador:

a- En páginas

b- En segmentos

a- Paginación

Las páginas tienen tamaño fijo. Ej: 4 Kbytes = 4096 bytes

Son contiguas

Ventajas:

- * Administración simple de la memoria (tamaño fijo)

Desventajas:

- * Si requiero memoria para una variable de 1 byte el sistema me asigna una página de 4 K completa
- * Si se requiere reubicar código el algoritmo, en principio, es complejo.

Administración de Memoria

b- Segmentación

- * Los segmentos son de tamaño variable
- * No necesariamente son contiguos
- * Se cuenta con un registro de desplazamiento que haciendo referencia al principio del segmento (dirección base del segmento) permite desplazarse a lo largo del mismo
- * Utiliza un **Contador de Programa** a través de un segmento de código.
- * Utiliza un **Stack Pointer** a través de un Segmento de pila

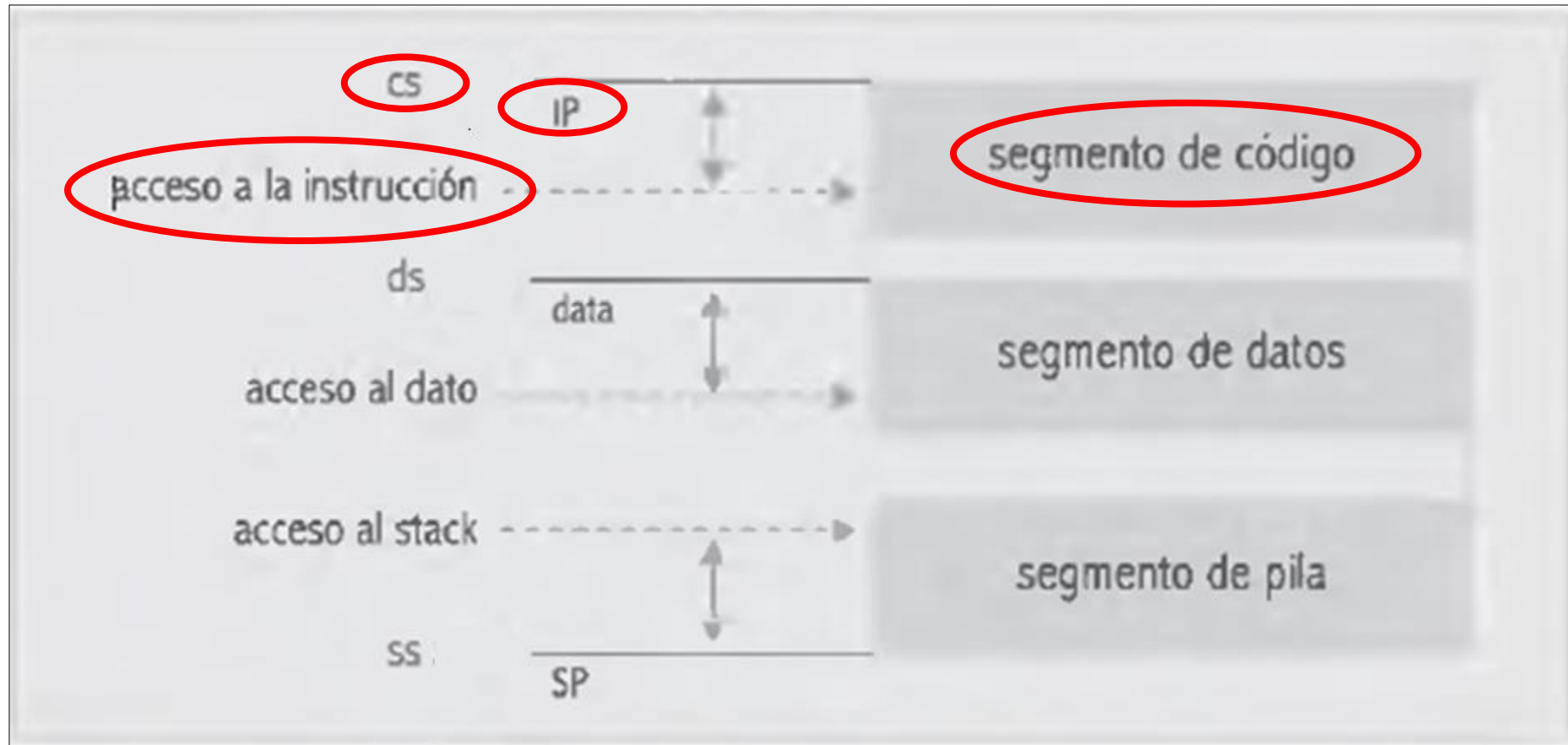
Ventajas:

- * Manteniendo fijo el valor inicial del segmento el movimiento a través del mismo se realiza solo con el registro de desplazamiento.
- * Reubicación en memoria simple de código, pilas y bloques de datos.
- * Si se necesitan pocos bytes para una variable pido al sistema un segmento de por ejemplo 16 bits. Se desperdicia mucho menos memoria

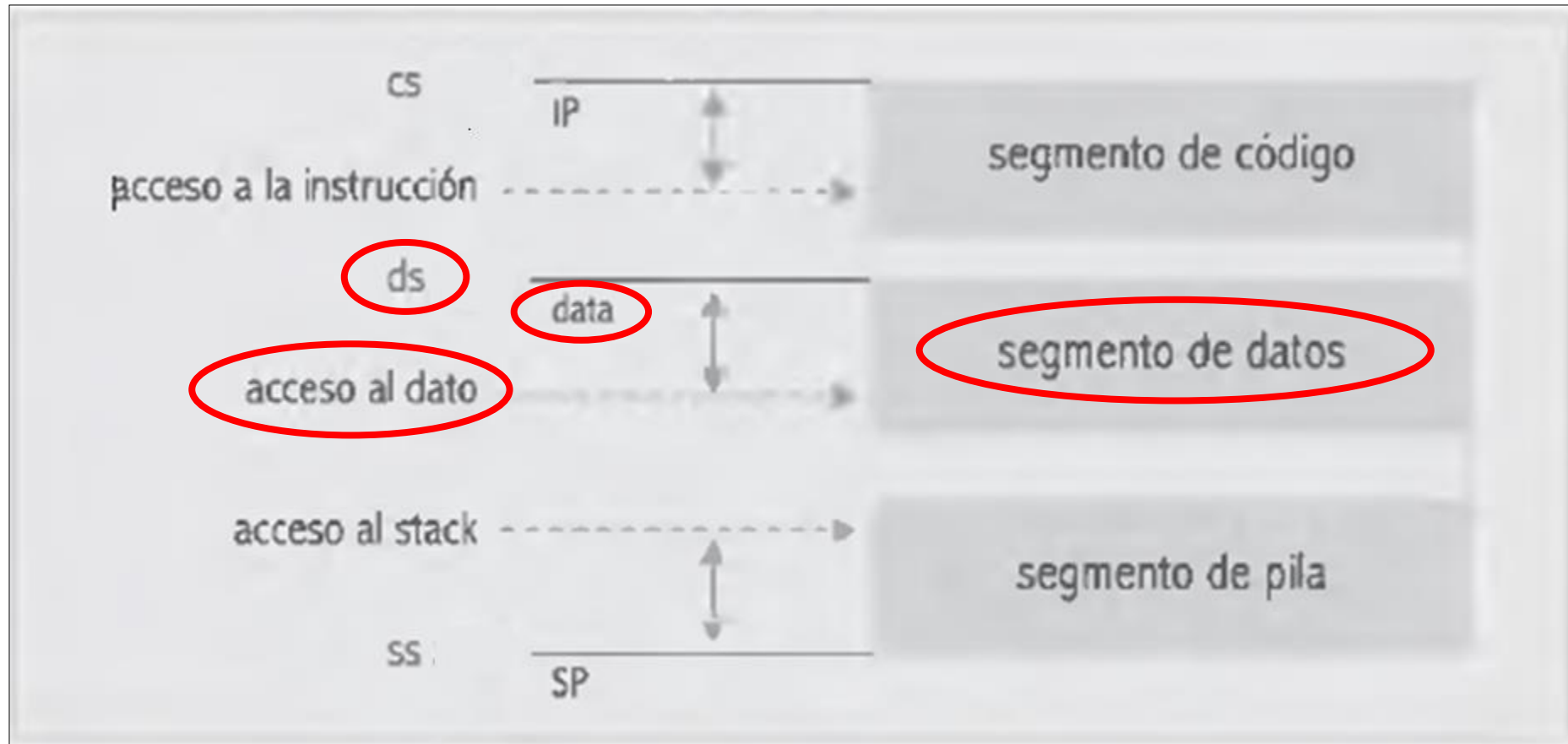
Desventajas:

- * Es engorroso administrar memoria con bloques de tamaño variable

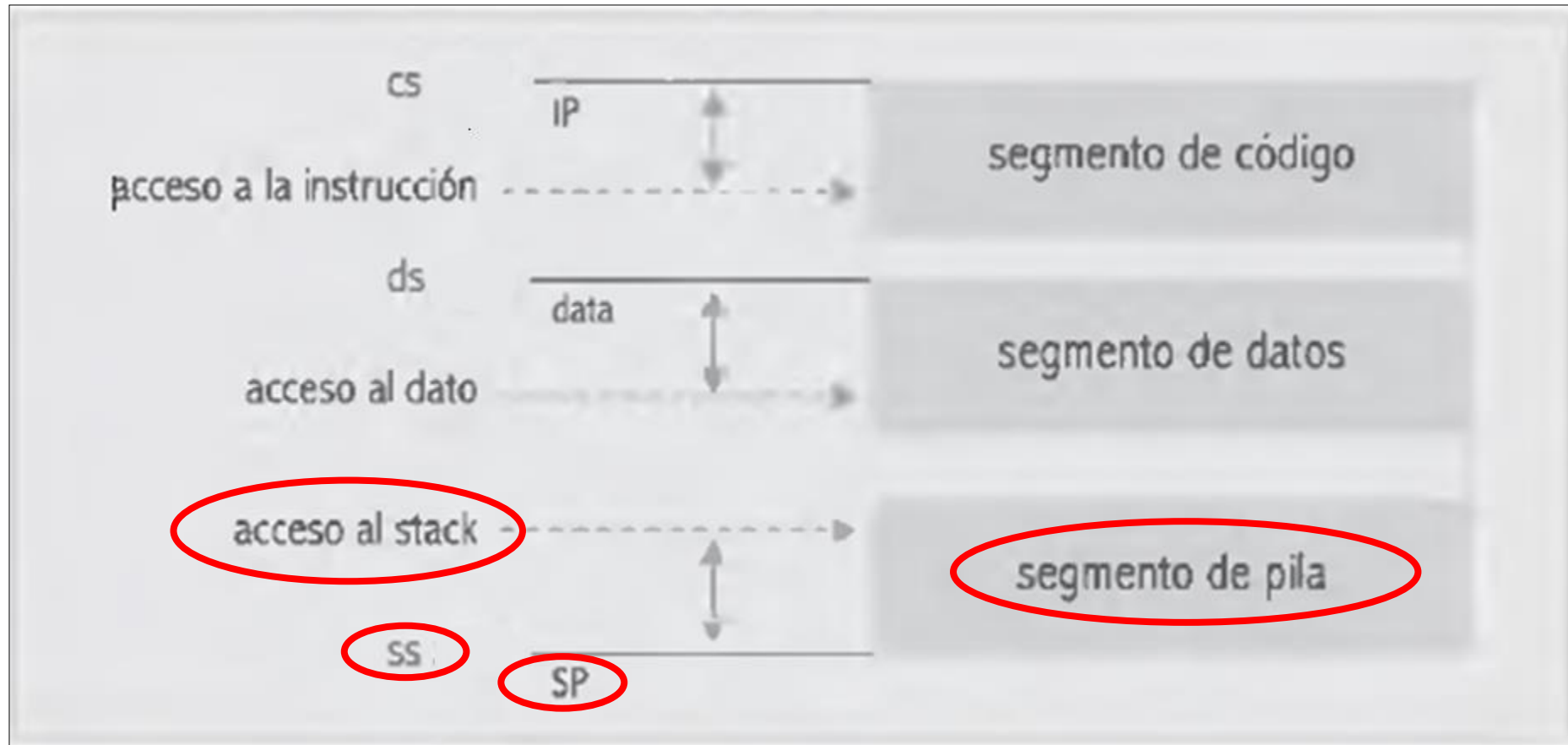
En el siguiente esquema se representan tres segmentos



En el siguiente esquema se representan tres segmentos



En el siguiente esquema se representan tres segmentos



Modelo de Memoria Segmentada Pura

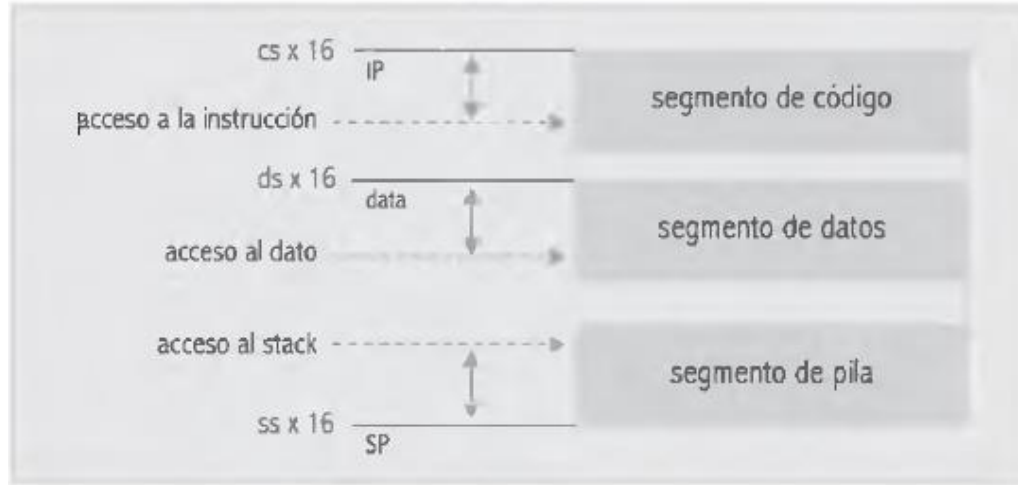
Para acceder a la memoria física se puede utilizar una técnica denominada **segmentación**, consistente en dividirla en territorios pequeños con límites determinados por el software.

Un segmento es un **bloque lógico en memoria de tamaño variable**. El control del tamaño de los segmentos es responsabilidad del sistema operativo que los administra. Los segmentos contienen un solo tipo de objeto, por ejemplo, en una memoria se puede organizar un segmento para el código ejecutable de un programa, otro para los datos y otro para una pila que utilice.

Segmentos distintos no necesitan estar en zonas contiguas, pero es necesario que un segmento como entidad lógica si lo este.

Modelo de Memoria Segmentada Pura

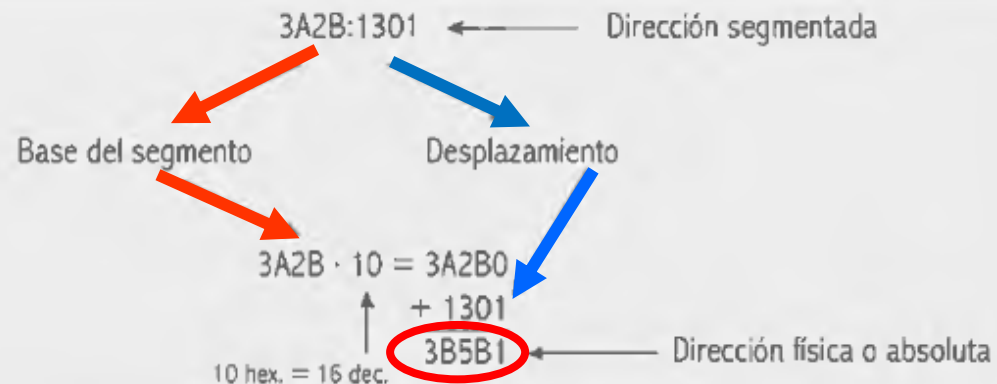
En el siguiente esquema se representan tres segmentos



Dirección Lógica: Entidad binaria que identifica una locación de memoria a nivel software, que no se corresponde con la dirección física y, por lo tanto, necesita ser mapeada. Para direccionar una posición de memoria dentro de un segmento, cada dirección lógica se compone de dos entidades: una para identificar el segmento y otra para identificar un desplazamiento dentro del segmento.

El acceso se referencia de acuerdo al siguiente algoritmo:

Caso ejemplo: algoritmo de cálculo que realiza la MMU (Memory Manager Unit) del procesador en modo real



GESTION DE MEMORIA EN LA FAMILIA 8086/8088

Estructura de memoria de segmentación:

El 8086/8088 usa un esquema ingenioso llamado **segmentación**, para acceder correctamente a un megabyte completo de memoria, con referencias de direcciones de sólo 16 bits.

GESTION DE MEMORIA EN LA FAMILIA 8086/8088

Estructura de memoria de segmentación:

Hay dos registros de segmento que tienen usos especiales:

El microprocesador utiliza el registro **CS** (con el **offset** almacenado en el **puntero de instrucción IP**) cada vez que se debe acceder a un byte de instrucción de programa

Las instrucciones que utilizan la pila (llamados a procedimientos, retornos, interrupciones y las instrucciones PUSH y POP) siempre utilizan el registro de segmento **SS** (con el offset almacenado en el registro **puntero de pila SP**)

De ahí los nombres que toman:

CS es el **segmento de código (Code Segment)**

SS es el registro **Segmento de Pila (Stack Segment)**

GESTION DE MEMORIA EN LA FAMILIA 8086/8088

Para acceder a datos en la memoria se puede utilizar cualquiera de los cuatro registros de segmento, pero uno de ellos provoca que la instrucción ocupe un byte menos de memoria: es el llamado segmento por defecto, por lo que en lo posible hay que tratar de usar dicho segmento para direccionar datos.

Dicho segmento es el **DS** (registro de **Segmento de Datos** o **Data Segment**).

Mayor información

➤ Capítulo 9

✓ Quiroga patricia 1^{ra} Ed

➤ Capítulo 4 y 5

✓ Stallings, 7^{ma}. Ed.

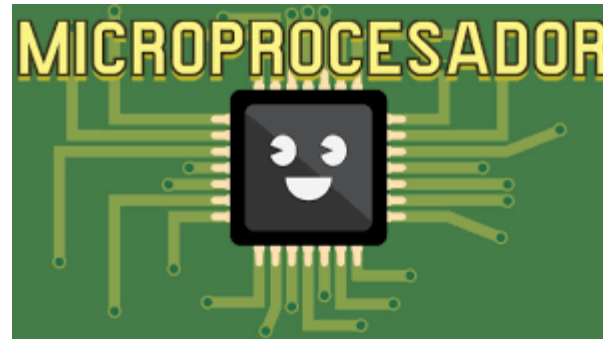
➤ Link video

<https://photos.app.goo.gl/zJ27jpBiwRdhHot18>

➤ Capítulo 7

✓ Osio, Morales, 1^{ma}. Ed.

CÁTEDRA DE ARQUITECTURA DE COMPUTADORAS



**REGISTROS INTERNOS
DEL PROCESADOR
8086/8088**

MICROPROCESADORES



BREVE HISTORIA DE LOS MICROPROCESADORES

1971: Microprocesador 4004: El 4004 fue el primer microprocesador de INTEL. Este fue el que impulsó la calculadora de BUSICOM.

1972: Microprocesador 8008: Este fue pedido a INTEL por COMPUTER TERMINAL CORPORATION para ser usado en el terminal programable DATAPOINT 2200. Pero debido a que INTEL concluyó tarde el proyecto finalmente no fue utilizado.

1978: Microprocesador 8086-8088: El éxito del 8088 propulso a INTEL a la lista de las 500 mejores compañías de la prestigiosa revista Fortune. Esta revista nombró a esta compañía como uno de los triunfos comerciales de los setenta.

1982: Microprocesador 286: Era el primer procesador de INTEL que podría ejecutar todo el software escrito para su predecesor.

BREVE HISTORIA DE LOS MICROPROCESADORES

1985: El microprocesador INTEL 386™: El INTEL 386™ microprocesador ofreció 275.000 transistores, más de 100 veces tantos como en el original 4004.

1993: Procesador de PENTIUM: El procesador de PENTIUM poseía una arquitectura capaz de ejecutar dos operaciones a la vez gracias a sus dos pipeline de datos de 32bits cada uno equivalente al 486DX (U) y el otro equivalente a 486SX (U).

1995: Procesador PENTIUM profesional: Lanzado al mercado para el otoño de 1995 el procesador PENTIUM PRO se diseña con una arquitectura de 32 bits, su uso en servidores, los programas y aplicaciones para estaciones de trabajo (redes).

1998: El procesador PENTIUM® II XEON™: Los procesadores PENTIUM II XEON™ se diseñan para cumplir con los requisitos de desempeño en computadoras de medio-rango, servidores más poderosos y estaciones de trabajo.

MODELO MICRO	VELOCIDAD	CACHE	BUS DE DATOS
4004	108 KHz	N/D	4 bits
4040			4 bits
8008			8 bits
8080	2 MHz		8 bits
8086	5 a 10 MHz		16 bits
8088	5 a 8 MHz		8 + 8 bits =16 bits
80286	8 a 12 MHz		16 bits
i386 DX	16 a 33 MHz		32 bits
i386 SX	16 a 20 MHz		
i486	25 a 133 MHz	4 + 4 = 8KB	32 bits
PENTIUM	60 A 200 MHz	L1 16 KB	32 bits
PENTIUM PRO	150 A 200 MHz	L1 16 KB L2 256 o 512 KB	32 bits
PENTIUM MMX	166 A 233 MHz	L1 32 KB	32 bits
PENTIUM II	233 A 300 MHz	L1 512 KB	32 bits
PENTIUM CELERON	266 a 466 MHz	L1 32 KB L2 256 KB	32 bits
PENTIUM XEON	400 A 450 MHz	L1 32 KB L2 512 KB a 2 MB	32 bits
PENTIUM III	450 a 600 MHz	L1 32 KB L2 512 KB	32 bits
PENTIUM ITANIUM	750 MHz a 1'7 GHz	L1 32 KB L2 96 KB L3 4 KB	64 bits
PENTIUM 4	1'3 a 3'8 GHz	L1 8 KB L2 512 MB	64 bits
INTEL CORE DUO	1'6 A 2'33 GHz	L1 64 KB L2 2024 KB	64 bits
INTEL CORE 2 DUO	1'8 a 3'33 GHz	L1 64 KB L2 2 MB	64 bits
INTEL CELERON DualCore	1'6 GHz	L1 64 KB L2 512 KB L3 3 MB	64 bits
INTEL CORE QUAD	2'4 a 3'0 GHz	L1 64 Y 128 KB L2 4 a 12 MB	64 bits
CORE 2 EXTREME	2'66 a 3'2 GHz	L1 128 Y 256 KB L2 8 Y 12 MB	64 bits
INTEL CORE I3	1'2 a 2'9 GHz	L1 2*32 KB L2 2*256 KB L3 4 MB	64 bits
INTEL CORE I5	2'8 GHz	L1 2*64 KB L2 2*256 KB L3 6 MB	64 bits
INTEL CORE I7	2'66 Y 2'93 GHz	Smart cache 8 MB	64 bits
CORE I7 EXTREME	2 a 3'2 GHz	Smart cache 8 MB	64 bits

Memoria caché nivel 1 (Caché L1)

TAMBIÉN LLAMADA **MEMORIA INTERNA**, SE ENCUENTRA EN EL NÚCLEO DEL MICROPROCESADOR. ES UTILIZADA PARA ALMACENAR Y ACCEDER A DATOS E INSTRUCCIONES IMPORTANTES Y DE USO FRECUENTE, AGILIZANDO LOS PROCESOS AL SER EL NIVEL QUE OFRECE UN TIEMPO DE RESPUESTA MENOR. SE DIVIDE EN DOS SUBNIVELES:

•**NIVEL 1 DATA CACHE:** SE ENCARGA DE ALMACENAR DATOS USADOS FRECUENTEMENTE.

•**NIVEL 1 INSTRUCTION CACHE:** SE ENCARGA DE ALMACENAR INSTRUCCIONES USADAS FRECUENTEMENTE.

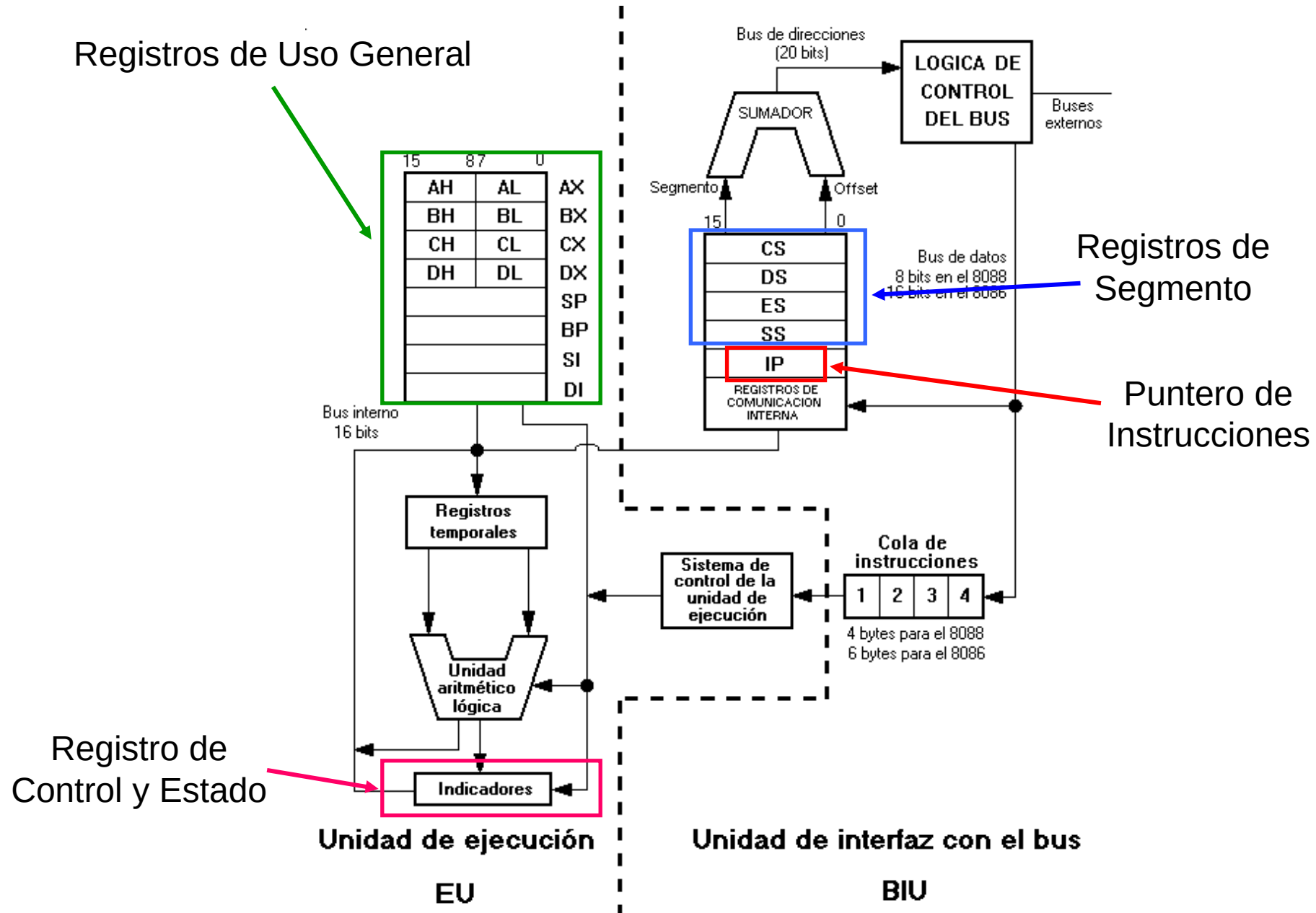
Memoria caché nivel 2 (Caché L2)

SE ENCARGA DE ALMACENAR DATOS DE USO FRECUENTE, ES MAYOR QUE LA CACHÉ L1, PERO A COSTA DE SER MÁS LENTA, AUN ASÍ ES MÁS RÁPIDA QUE LA MEMORIA PRINCIPAL (RAM). PUEDE SER INCLUSIVA Y CONTENER UNA COPIA DEL NIVEL 1 ADEMÁS DE INFORMACIÓN EXTRA, O EXCLUSIVA Y QUE SU CONTENIDO SEA TOTALMENTE DIFERENTE DE LA CACHE L1, PROPORCIONANDO ASÍ MAYOR CAPACIDAD TOTAL.

Memoria caché nivel 3 (Caché L3)

ES MÁS RÁPIDA QUE LA MEMORIA PRINCIPAL (RAM), PERO MÁS LENTA Y MAYOR QUE L2, AYUDA A QUE EL SISTEMA GUARDE GRAN CANTIDAD DE INFORMACIÓN AGILIZANDO LAS TAREAS DEL PROCESADOR. EN ESTA MEMORIA SE AGILIZA EL ACCESO A DATOS E INSTRUCCIONES QUE NO FUERON LOCALIZADAS EN L1 O L2. AL IGUAL QUE LA L2, PUEDE SER INCLUSIVA Y CONTENER UNA COPIA DE L2 ADEMÁS DE INFORMACIÓN EXTRA O, POR EL CONTRARIO, SER EXCLUSIVA Y CONTENER INFORMACIÓN TOTALMENTE DIFERENTE A LA DE LOS NIVELES ANTERIORES, CONSIGUIENDO ASÍ UNA MAYOR CAPACIDAD TOTAL.

Registros Internos del Procesador 8086/8088

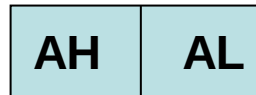


REGISTROS DE USO GENERAL

Los registros de propósito general son el **AX**, **BX**, **CX**, y **DX**, de 16 bits

Cada uno de ellos se divide en dos registros de 8 bits c/u, llamados **AH** y **AL**, **BH** y **BL**, **CH** y **CL**, y, **DH** y **DL**, “**H**” por **High (Alto)** y “**L**” por **Low (Bajo)**, indicando la parte alta o la parte baja del registro correspondiente de 16 bits.

Ej.: **AX=**



Un programa podía usar tanto los registros de 16 bits como los registros de 8 bits.

Aparte del uso general de los registros para hacer cálculos aritméticos y lógicos, existen instrucciones que usan estos registros con un uso particular especializado.

REGISTROS DE USO GENERAL

AX = REGISTRO ACUMULADOR, DIVIDIDO EN **AH** Y **AL** (8 BITS CADA UNO).
ES UTILIZADO PARA OPERACIONES QUE IMPLICAN ENTRADA/SALIDA, MULTIPLICACIÓN Y DIVISIÓN (ESTAS DOS ÚLTIMAS EN CONJUNTO CON EL REGISTRO DX), TAMBIÉN COMO REGISTRO DE PROPÓSITO GENERAL A DISPOSICIÓN DEL PROGRAMADOR.

BX = REGISTRO BASE, DIVIDIDO EN **BH** Y **BL**.
SE USA PRINCIPALMENTE PARA INDICAR POSICIONES DE MEMORIA (OFFSET).

CX = REGISTRO CONTADOR, DIVIDIDO EN **CH** Y **CL**.
SE UTILIZA COMO CONTADOR EN BUCLES (INSTRUCCIÓN LOOP), EN OPERACIONES CON CADENAS Y EN DESPLAZAMIENTOS Y ROTACIONES (USANDO EL REGISTRO **CL** EN LOS DOS ÚLTIMOS CASOS).

DX = REGISTRO DE DATOS, DIVIDIDO EN **DH** Y **DL**.
EL REGISTRO DX ES EL REGISTRO DE DATOS. EN ALGUNAS OPERACIONES SE INDICA MEDIANTE ESTE REGISTRO EL NÚMERO DE PUERTO DE ENTRADA/SALIDA, Y EN LAS OPERACIONES DE MULTIPLICACIÓN Y DIVISIÓN DE 16 BITS SE UTILIZA JUNTO CON EL ACUMULADOR AX.

REGISTROS APUNTADORES

SP = PUNTERO DE PILA (NO SE PUEDE SUBDIVIDIR).

EL PUNTERO DE PILA DE 16 BITS ESTÁ ASOCIADO CON EL SEGMENTO **SS** Y PROPORCIONA UN VALOR DE DESPLAZAMIENTO QUE SE REFIERE A LA PALABRA ACTUAL QUE ESTÁ SIENDO PROCESADA EN LA PILA. EL SISTEMA MANEJA DE MANERA AUTOMÁTICA ESTE REGISTRO, AUNQUE EL PROGRAMA PUEDE HACER CIERTAS MANIPULACIONES CON ÉL.

BP = PUNTERO BASE (NO SE PUEDE SUBDIVIDIR).

EL PUNTERO BASE DE 16 BITS SE UTILIZA COMO REGISTRO AUXILIAR Y EN DIRECCIONAMIENTOS A MEMORIA.

REGISTROS ÍNDICE

LOS REGISTROS **SI** Y **DI** ESTÁN DISPONIBLES PARA DIRECCIONAMIENTO INDEXADO Y PARA OPERACIONES CON CADENAS DE CARACTERES.

SI = **FUENTE ÍNDICE** (NO SE PUEDE SUBDIVIDIR).

EL REGISTRO ÍNDICE FUENTE DE 16 BITS ES REQUERIDO POR ALGUNAS OPERACIONES CON CADENAS DE CARACTERES.
ESTÁ ASOCIADO CON EL SEGMENTO **DS**.

DI = **DESTINO ÍNDICE** (NO SE PUEDE SUBDIVIDIR)

EL REGISTRO ÍNDICE DESTINO TAMBIÉN ES REQUERIDO POR ALGUNAS OPERACIONES CON CADENAS DE CARACTERES.
ESTÁ ASOCIADO CON EL SEGMENTO **ES**.

CUALQUIERA DE ESTOS REGISTROS PUEDE UTILIZARSE COMO FUENTE O DESTINO EN OPERACIONES ARITMÉTICAS Y LÓGICAS.

REGISTROS DE CONTROL Y ESTADO (INDICADORES)

Es un registro de 16 bits de los cuales 9 sirven para indicar el estado actual de la máquina y el resultado del procesamiento.

Muchas instrucciones aritméticas y de comparación cambian el estado de las banderas y apoyándose en ellas se pueden tomar decisiones para determinar la acción subsecuente.

Registro de indicadores (16 bits)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flag	--	--	--	--	OF	DF	IF	TF	SF	ZF	0	AF	0	PF	1	CF

Los bits de las banderas son las siguientes:

CF (acarreo): Contiene el acarreo del bit de mayor orden después de una operación aritmética; también almacena el contenido del último bit en una operación de desplazamiento o de rotación.

PF (paridad): Cuando está a 1, indica que hay un número par de bits activos.

AF (acarreo auxiliar): Se usa como bandera auxiliar. Se pone a 1 cuando hay necesidad de realizar ajustes tras una operación de tipo BCD.

ZF (cero): Indica el resultado de una operación aritmética o de comparación (0=resultado diferente de cero; 1=resultado igual a cero).

Registros de Control y Estado (Indicadores)

Registro de indicadores (16 bits)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flag	--	--	--	--	OF	DF	IF	TF	SF	ZF	0	AF	0	PF	1	CF

SF (signo): Contiene el signo resultante de una operación aritmética.

(0=positivo; 1=negativo)

TF (trampa): Permite la operación del procesador en modo de depuración (paso a paso).

IF (interrupción): Controla el disparo de las interrupciones.

(1 = habilita las interrupciones; 0 = deshabilita las interrupciones)

DF (dirección): Controla la selección de incremento o decremento de los registros **SI** y **DI** en las operaciones con cadenas de caracteres.

(1 = decremento automático; 0 = incremento automático)

Registros de Control y Estado (Indicadores)

Registro de indicadores (16 bits)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flag	--	--	--	--	OF	DF	IF	TF	SF	ZF	0	AF	0	PF	1	CF

OF (overflow, desbordamiento): Indica desbordamiento del bit de mayor orden después de una operación aritmética de números con signo

(1 = Existe overflow; 0 = No existe overflow).

Para operaciones sin signo, no se toma en cuenta esta bandera.

Es el principal indicador de error producido durante las operaciones con signo.

Vale 1 cuando:

- La suma de dos números con igual signo o la resta de dos números con signo opuesto producen un resultado que no se puede guardar (más de 16 bits).
- El bit más significativo (el signo) del operando ha cambiado durante una operación de desplazamiento aritmético.
- El resultado de una operación de división produce un cociente que no cabe en el registro de resultado.

Registros de Segmento

Definen áreas de 64 Kb dentro del espacio de direcciones de 1 Mb del 8086. No es posible acceder a una posición de memoria no definida por algún segmento: si es preciso, habrá de moverse alguno.

CS (Code Segment) = Registro de Segmento de Código.

Este registro es usado por el procesador, junto con el registro **IP**, para conocer dónde está la próxima instrucción a ejecutar.

DS (Data Segment) = Registro de Segmento de Datos.

Se usa para indicar dónde están todos los datos del programa en ejecución.

ES (Extra Segment) = Registro de Segmento Extra.

Es el registro que referencia al segmento extra, y se usa como apuntador de memoria auxiliar en operaciones complejas donde se necesitan dos punteros de datos simultáneos.

SS (Stack Segment) = Registro de Segmento de Pila.

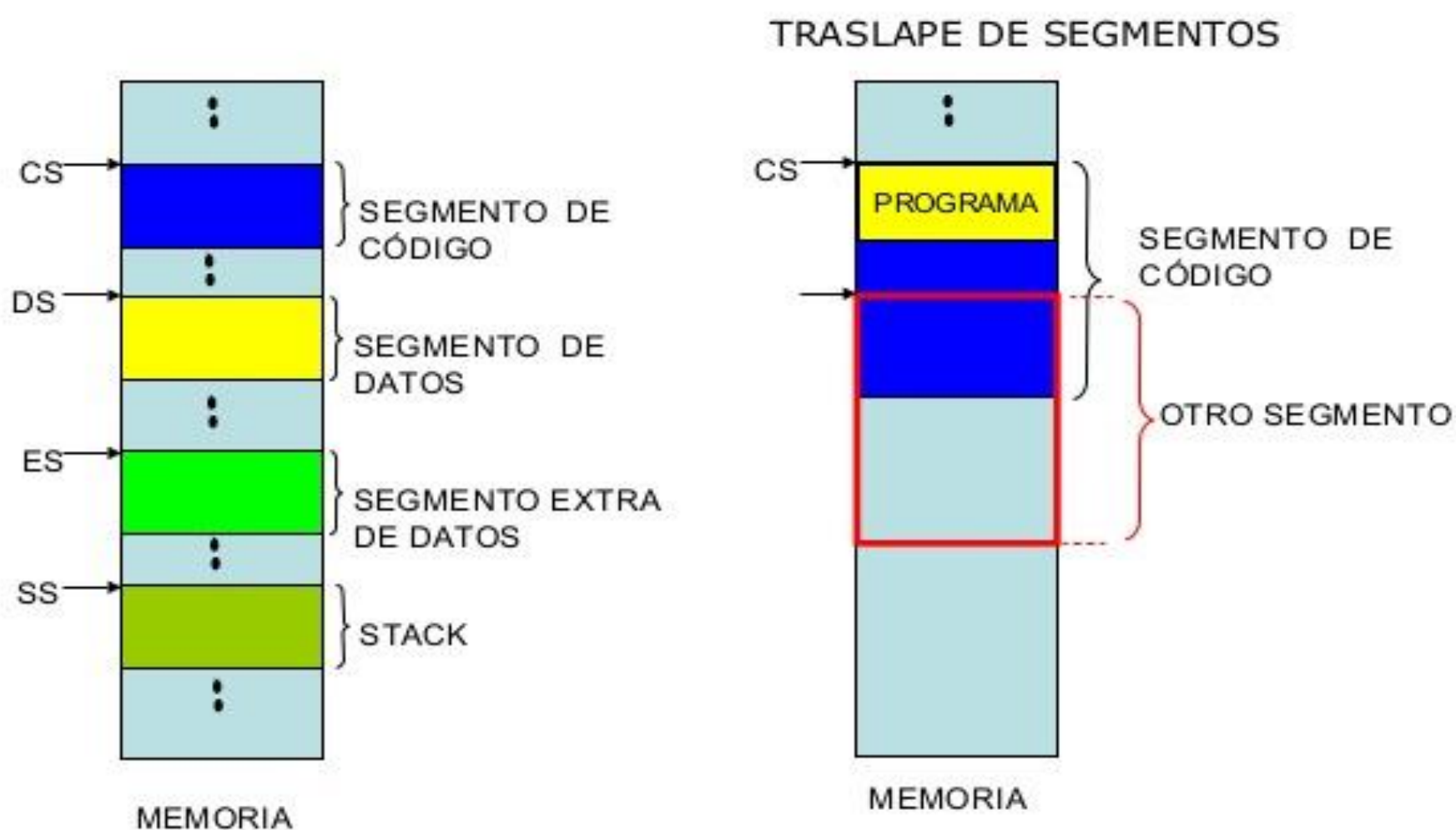
En este registro se indica al procesador dónde está la zona de memoria que se usa como segmento de pila.

Puntero de Instrucciones

Registro Apuntador de Instrucciones (**IP: Instruction Pointer**):

Este registro contiene el desplazamiento de dirección de la siguiente instrucción dentro del **Segmento de Código** (Registro **CS**) a ser ejecutada.

SEGMENTACIÓN



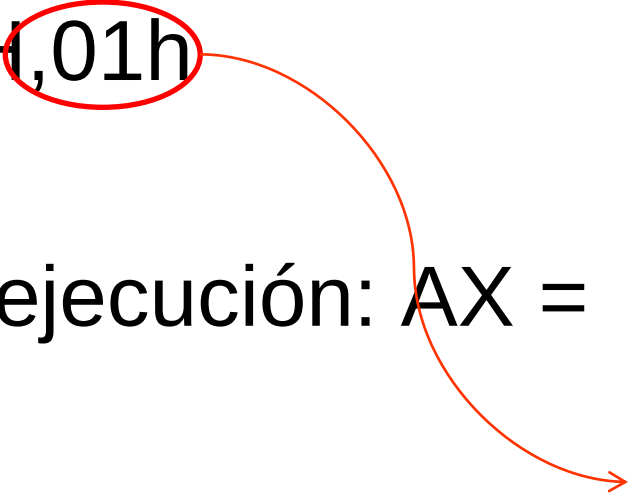
MODOS DE DIRECCIONAMIENTO

- 1. INMEDIATO**
- 2. DE REGISTRO DIRECTO**
- 3. DIRECTO**
- 4. DE REGISTRO INDIRECTO**
- 5. RELATIVO A LA BASE**
- 6. RELATIVO A UN REGISTRO INDICE**
- 7. INDEXADO A LA BASE**

1. MODO INMEDIATO

- Ej.: MOV AH,01h
 - Antes de la ejecución: AX =

23	88
----	----
 - Luego de la ejecución: AX =

01	88
----	----
- 

2. MODO DE REGISTRO DIRECTO

- Ej.: MOV AH,BL

• Antes de la ejecución: AX =

01	88
----	----

BX =

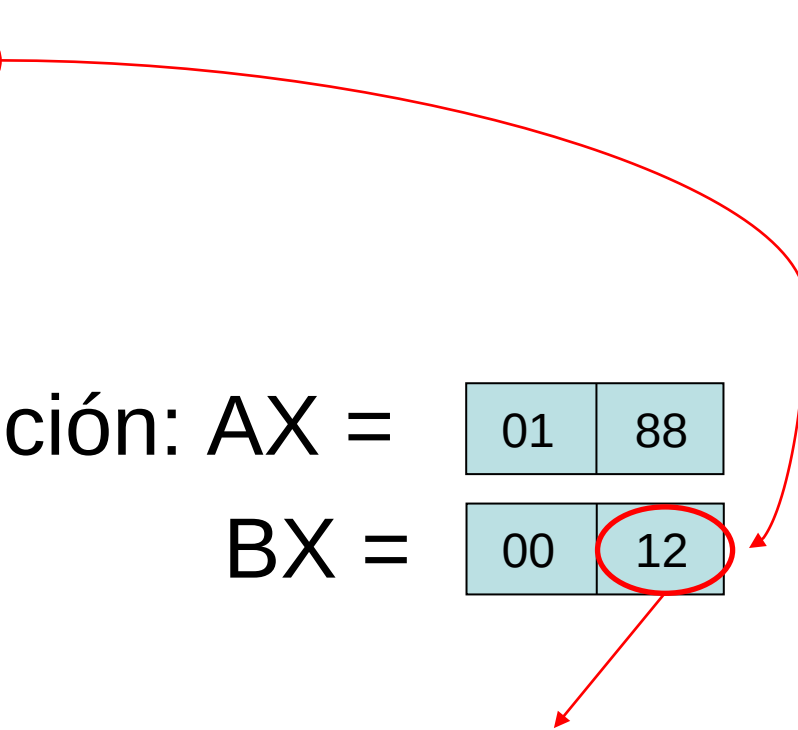
00	12
----	----

• Luego de la ejecución: AX =

12	88
----	----

BX =

00	12
----	----



3. MODO DIRECTO

- Ej.: MOV AX,DS:[0020h]

- Antes de la ejecución: AX =

12	88
----	----

Memoria

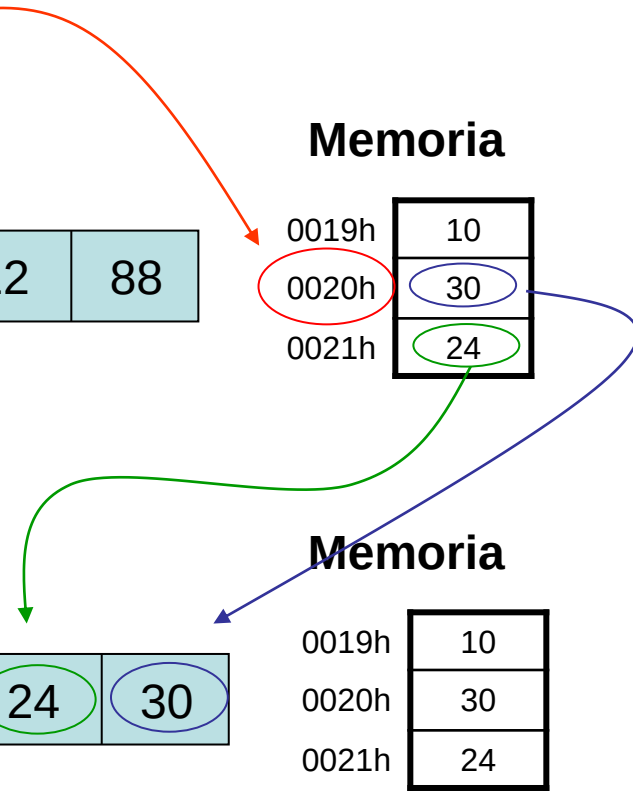
0019h	10
0020h	30
0021h	24

- Luego de la ejecución: AX =

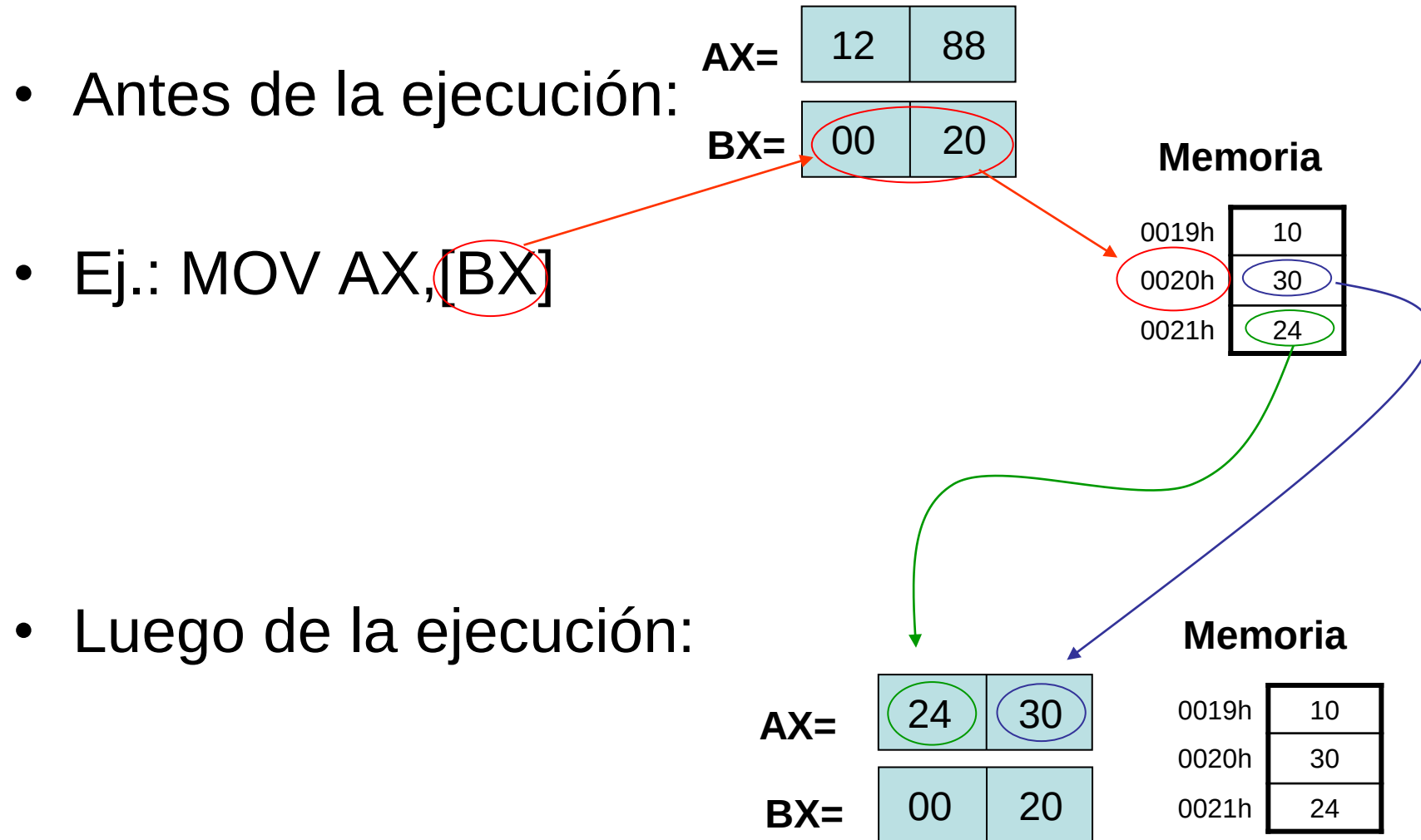
24	30
----	----

Memoria

0019h	10
0020h	30
0021h	24



4. MODO DE REGISTRO INDIRECTO



5. MODO DE RELATIVO A LA BASE

- Antes de la ejecución:

AX=	12	88
BX=	00	20

Ej.: MOV AX, [BX + 0030h]

Dir. Final = 0020h + 0030h = 0050h

Memoria

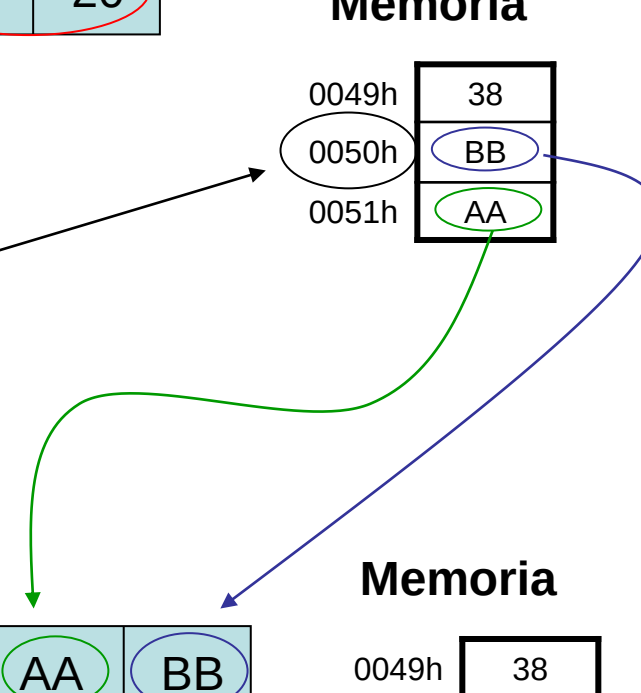
0049h	38
0050h	BB
0051h	AA

- Luego de la ejecución:

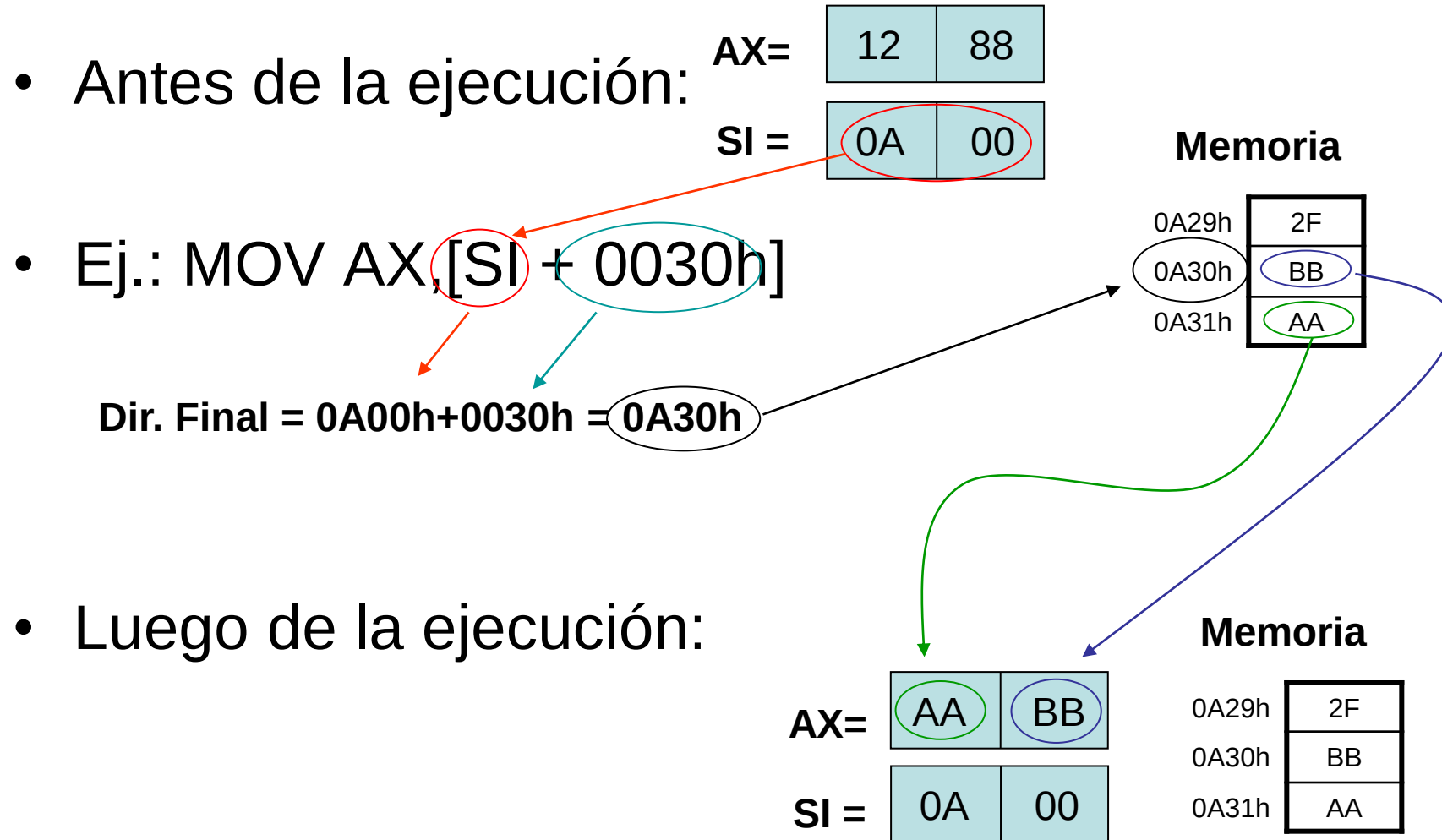
AX=	AA	BB
BX=	00	20

Memoria

0049h	38
0050h	BB
0051h	AA



6. MODO RELATIVO A UN REGISTRO INDICE



7. MODO INDEXADO A LA BASE

- Ej.: `MOV AX, [BX+SI+0002h]`

Dir. Final = $0028h + 0020h + 0002h = 0050h$

- Antes de la ejecución: $AX = \begin{bmatrix} 12 & 88 \end{bmatrix}$

$BX = \begin{bmatrix} 00 & 28 \end{bmatrix}$ $SI = \begin{bmatrix} 00 & 20 \end{bmatrix}$

- Luego de la ejecución: $AX = \begin{bmatrix} AA & BB \end{bmatrix}$

$BX = \begin{bmatrix} 00 & 28 \end{bmatrix}$ $SI = \begin{bmatrix} 00 & 20 \end{bmatrix}$

Memoria

0049h	2F
0050h	BB
0051h	AA

Memoria

0049h	2F
0050h	BB
0051h	AA

<u>MODO</u>	<u>INSTRUCCIÓN</u>	<u>REGISTRO</u>	<u>MEMORIA</u>
INMEDIATO	OPERANDO		
DE REGISTRO DIRECTO	REGISTRO → OPERANDO	OPERANDO	
DIRECTO	DIRECCION →		OPERANDO
DE REGISTRO INDIRECTO	REGISTRO → DIRECCION →	DIRECCION	OPERANDO
RELATIVO A LA BASE	DESPLAZAMIENTO REGISTRO BASE → DIRECCION	DESPLAZAMIENTO + DIRECCION →	OPERANDO
RELATIVO A UN REGISTRO INDICE	DESPLAZAMIENTO REGISTRO INDICE → DIRECCION	DESPLAZAMIENTO + DIRECCION →	OPERANDO
INDEXADO A LA BASE	DESPLAZAMIENTO (OPCIONAL) REGISTRO BASE → DIRECCION REGISTRO INDICE → DIRECCION	DESPLAZAMIENTO + DIRECCION + DIRECCION →	OPERANDO

Que son las direcciones y los contenidos de Memoria?

La Memoria Principal almacena bits (unos y ceros) en celdas independientes, aisladas entre si, que contienen **UN BYTE (8 bits)** de información.

Cada celda se localiza en el conjunto mediante un número binario identificador (“**DIRECCIÓN DE MEMORIA**”) el cual indica su “posición” en ese conjunto.

En cada dirección de memoria (**celda**) solo pueden leerse o escribirse **8 bits x vez**.

En una operación de lectura/escritura de Memoria se puede acceder a varias posiciones de memoria consecutivas.

Un cierto número estas (2, 4 u 8 bytes) suele denominarse PALABRA (**WORD**) DE MEMORIA.

Cómo se direcciona, se lee y se escribe en la Memoria Principal?

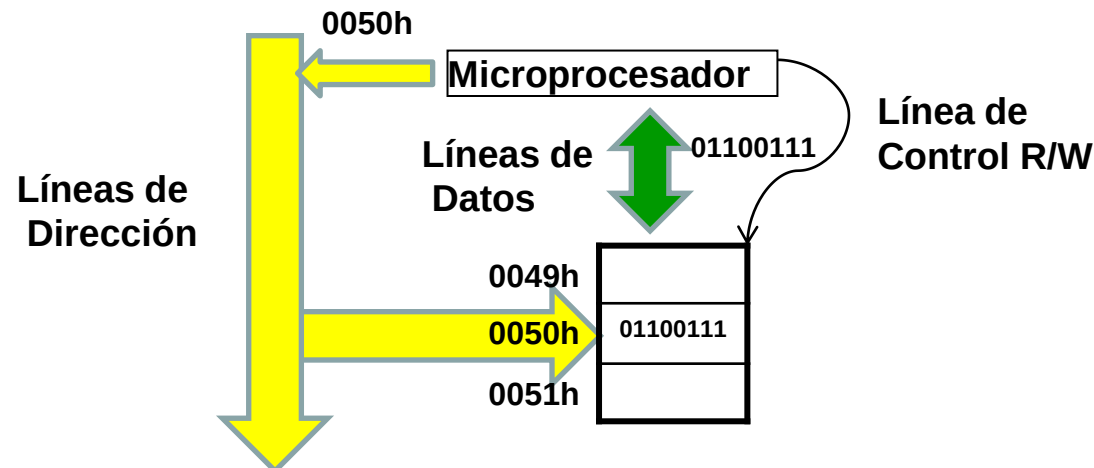
Sólo son posibles dos operaciones con la Memoria Principal, la Lectura y la Escritura. Para ello la U.C.P. primero debe direccionar la Memoria.

Direccionar:

Consiste en colocar en las líneas de Direcciones que llegan a la M.P. la dirección de la celda a la que se requiere acceder para completar la operación.

Tiempo de Acceso a Memoria:

Es el tiempo que transcurre entre que se direcciona una memoria, hasta que aparece en sus salidas (conectadas a las líneas de datos del BUS) el contenido de la celda seleccionada. Este tiempo se mide en Nanosegundos (10^{-9} segundos).



Unidades de Medidas de Almacenamiento

Medida	Simbologia	Equivalencia	Equivalente en Bytes
byte	b	8 bits	1 byte
kilobyte	Kb	1024 bytes	1 024 bytes
megabyte	MB	1024 KB	1 048 576 bytes
gigabyte	GB	1024 MB	1 073 741 824 bytes
terabyte	TB	1024 GB	1 099 511 627 776 bytes
Petabyte	PB	1024 TB	1 125 899 906 842 624 bytes
Exabyte	EB	1024 PB	1 152 921 504 606 846 976 bytes
Zetabyte	ZB	1024 EB	1 180 591 620 717 411 303 424 bytes
Yottabyte	YB	1024 ZB	1 208 925 819 614 629 174 706 176 bytes
Brontobyte	BB	1024 YB	1 237 940 039 285 380 274 899 124 224 bytes
Geopbyte	GB	1024 BB	1 267 650 600 228 229 401 496 703 205 376 bytes

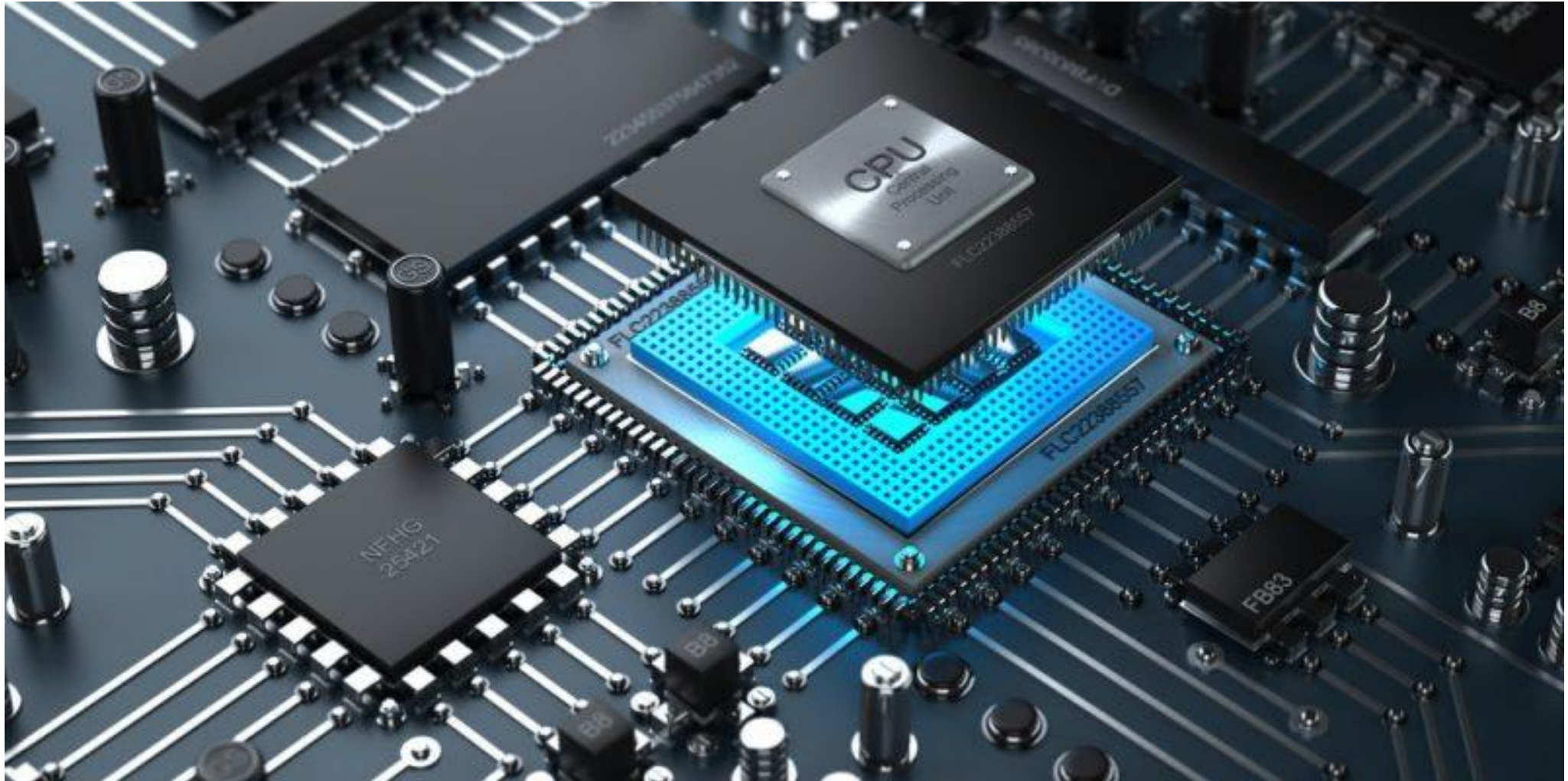
Unidades de Velocidad de Transmisión

No debe confundirse **Byte**, que es la unidad de medida utilizada en los sistemas de almacenamiento con **bits**, que se utiliza en minúscula y nunca de forma aislada.

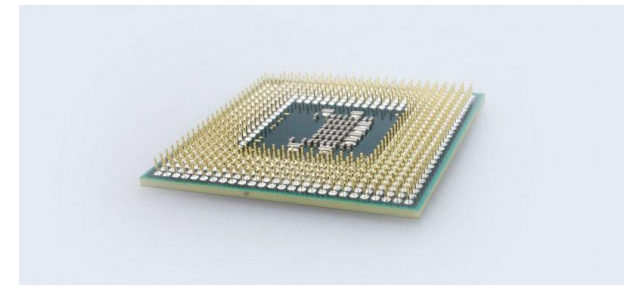
bits x segundo: (comienza en minúscula) y sus múltiplos se emplean para medir la velocidad de transferencia de datos.

Medida	Simbología	Equivalencia	Equivalencia en bits/seg
Bits x seg.	bps	1 bps	1 bps
Kilobits x seg.	Kbps	1.000 bps	1.000 bps
Megabits x seg.	1 Mbps	1.000.000 bps	1.000 Kbps
Gigabits x seg.	1 Gbps	10^9 bps	1.000 Mbps
Terabits x seg.	1 Tbps	10^{12} bps	1.000 Gbps

QUE ES UN MICROPROCESADOR ?



QUÉ ES UN MICROPROCESADOR?



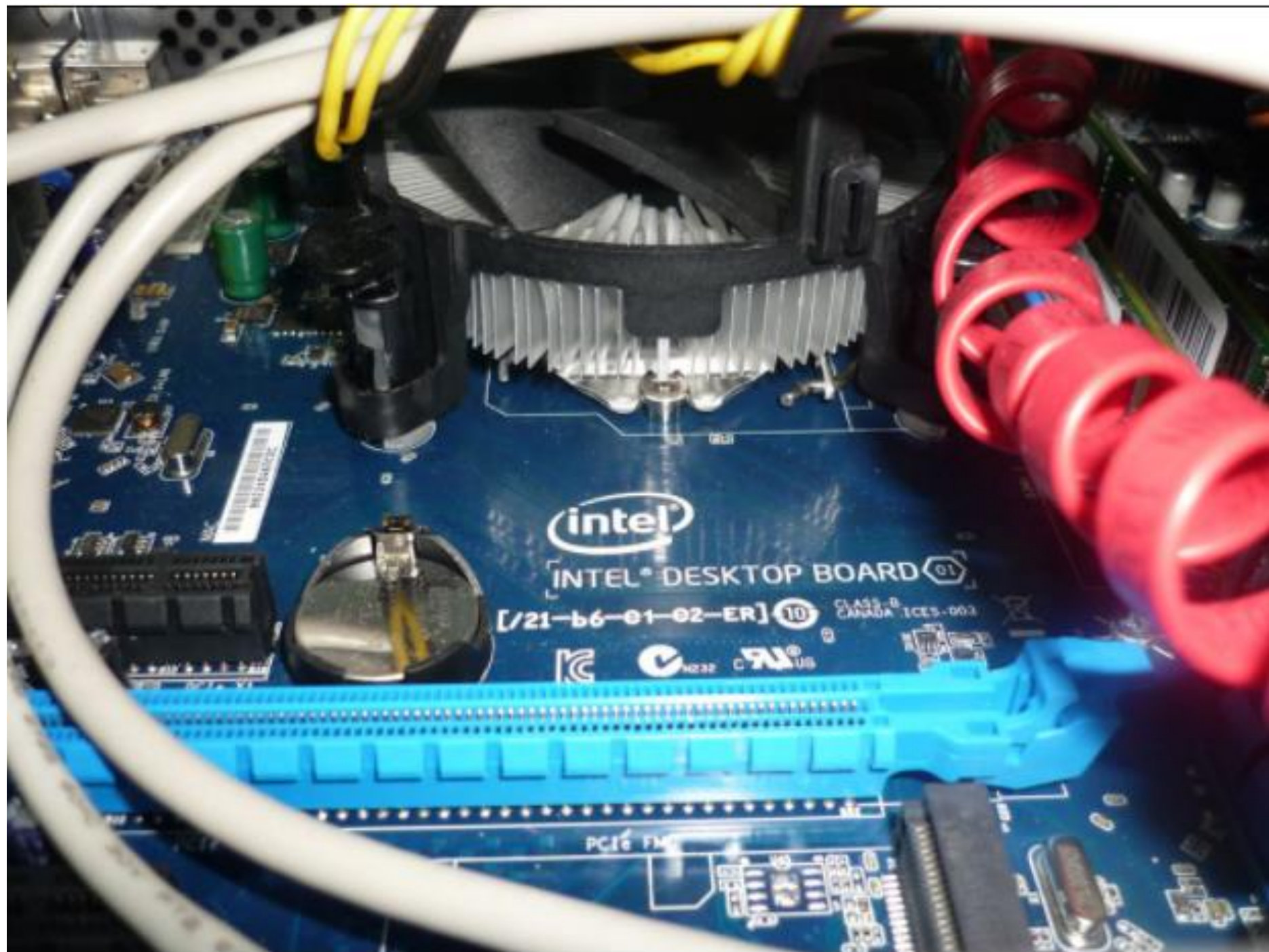
- SE LLAMA MICROPROCESADOR O SIMPLEMENTE PROCESADOR AL CIRCUITO INTEGRADO CENTRAL DE UN SISTEMA INFORMÁTICO, EN DONDE SE LLEVAN A CABO LAS OPERACIONES LÓGICAS Y ARITMÉTICAS (CÁLCULOS) PARA PERMITIR LA EJECUCIÓN DE LOS PROGRAMAS, DESDE EL SISTEMA OPERATIVO HASTA EL SOFTWARE DE APLICACIÓN.
- UN MICROPROCESADOR PUEDE OPERAR CON UNA O MÁS CPU (UNIDADES CENTRALES DE PROCESAMIENTO), CONSTITUIDAS CADA UNA POR REGISTROS, UNA UNIDAD DE CONTROL, UNA UNIDAD ARITMÉTICO-LÓGICA Y UNA UNIDAD DE CÁLCULO EN COMA FLOTANTE (O COPROCESADOR MATEMÁTICO).
- ASIMISMO, SE HALLA GENERALMENTE CONECTADO MEDIANTE UN ZÓCALO A LA PLACA BASE O TARJETA MADRE, JUNTO CON UN SISTEMA DISIPADOR DE CALOR QUE CONFORMAN CIERTOS MATERIALES DE DISIPACIÓN TÉRMICA Y UN FAN COOLER (VENTILADOR INTERNO).
- MIENTRAS QUE UN MISMO MICROPROCESADOR PUEDE CONTAR CON UNO O MÁS NÚCLEOS FÍSICOS O LÓGICOS, EN LOS QUE SE LLEVA A CABO TODA LA LABOR DE CÁLCULO, UN MISMO SISTEMA INFORMÁTICO PUEDE DISPONER DE VARIOS PROCESADORES TRABAJANDO EN PARALELO.
- EL RENDIMIENTO DE DICHOS PROCESADORES NO ES NADA FÁCIL DE MEDIR, PERO SE SUELE USAR LA FRECUENCIA DE RELOJ (MEDIDA EN HERZIOS) PARA DISTINGUIR ENTRE LA POTENCIA DE UNOS Y DE OTROS.







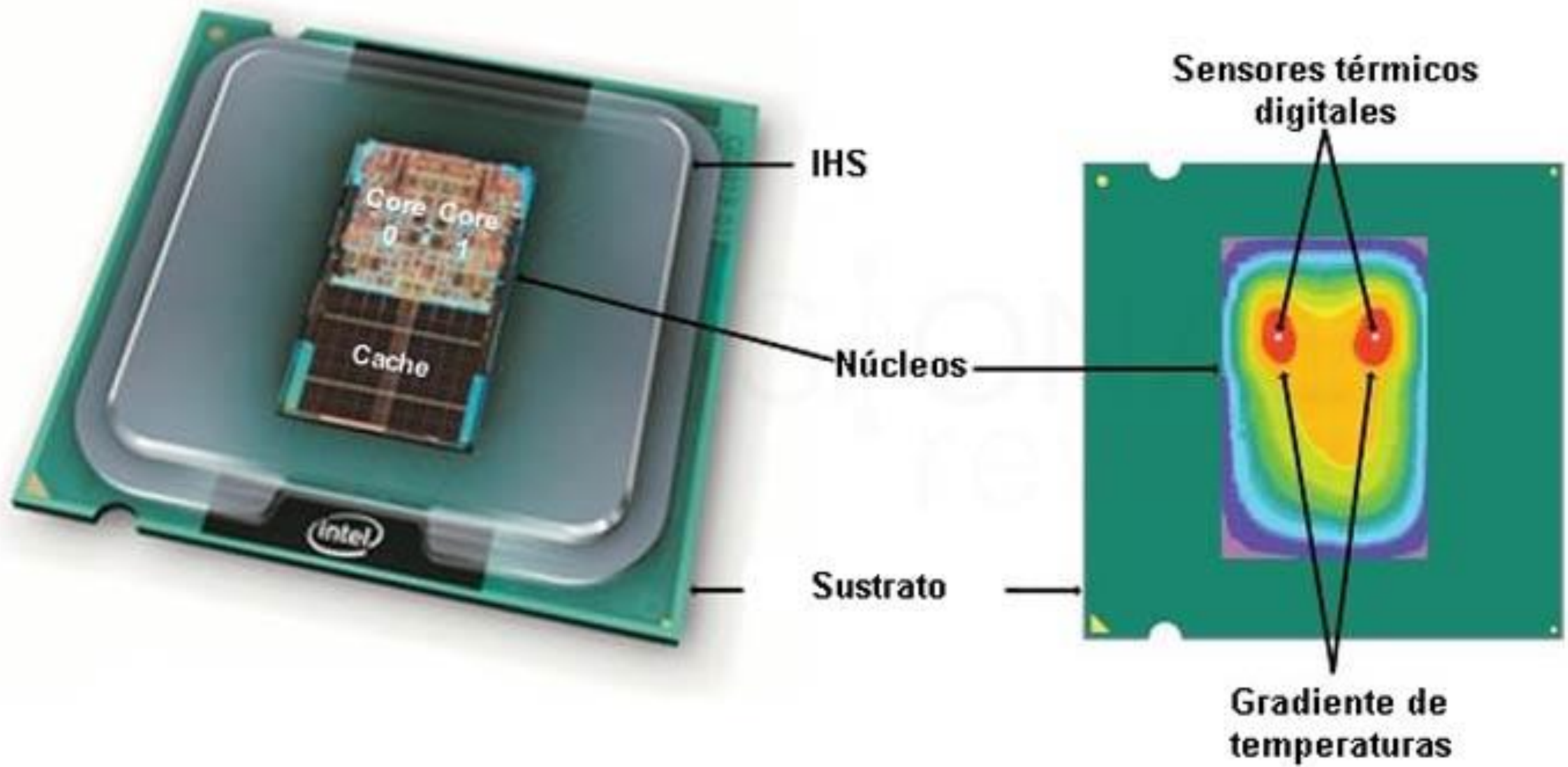
DISIPADOR DE CALOR

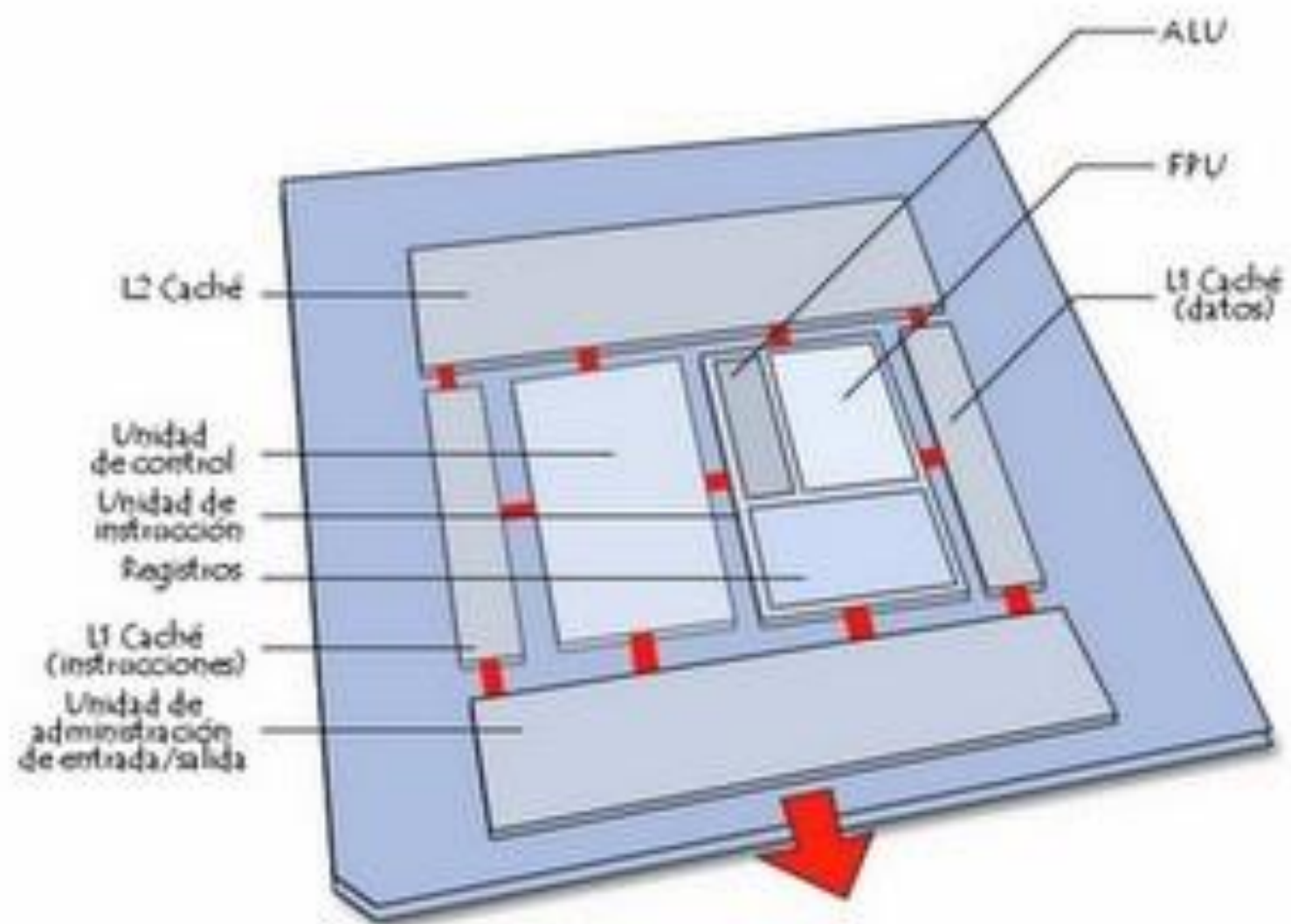




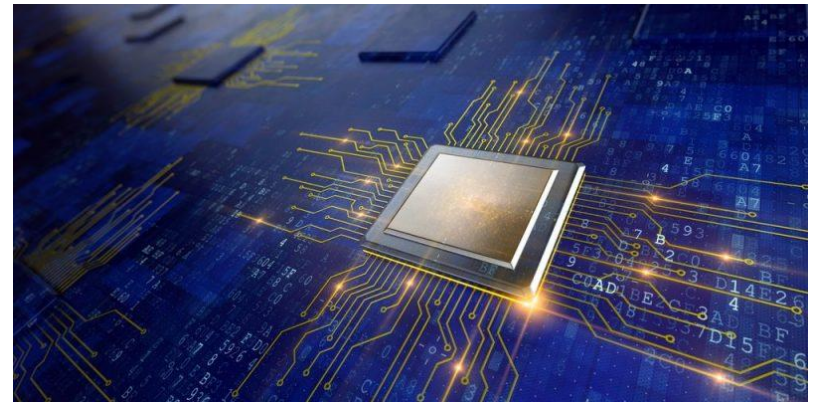
CARACTERÍSTICAS DEL MICROPROCESADOR

- **LOS MICROPROCESADORES ASEMEJAN UNA PEQUEÑA COMPUTADORA DIGITAL EN MINIATURA, POR LO QUE PRESENTA SU PROPIA ARQUITECTURA Y REALIZA OPERACIONES BAJO UN PROGRAMA DE CONTROL. DICHA ARQUITECTURA SE COMPONE DE:**
 - **ENCAPSULADO.** UNA CUBIERTA CERÁMICA QUE RECUBRE EL SILICIO Y LO PROTEGE DE LOS ELEMENTOS (COMO EL OXÍGENO DEL AIRE).
 - **CACHÉ.** UN TIPO DE MEMORIA ULTRARRÁPIDA DISPONIBLE PARA EL PROCESADOR, DE MODO QUE NO EMPLEE MEMORIA RAM SINO CUANDO SEA NECESARIO, YA QUE EN LOS VARIOS NIVELES DE LA MEMORIA CACHÉ SE GUARDAN DATOS EN USO PARA SU RECUPERACIÓN INMEDIATA.
 - **COPROCESADOR MATEMÁTICO.** LLAMADA UNIDAD DE COMA FLOTANTE, ES LA PORCIÓN DEL PROCESADOR QUE SE ENCARGA DE LAS OPERACIONES LÓGICAS Y FORMALES.
 - **REGISTROS.** UNA MEMORIA BREVE DE TRABAJO EN EL PROCESADOR, DISEÑADA PARA LLEVAR EL CONTROL DE SU PROPIO FUNCIONAMIENTO Y CONDICIONES.
 - **PUERTOS.** LOS CONDUCTOS QUE PERMITEN AL PROCESADOR COMUNICAR LA INFORMACIÓN CON EL RESTO DE LOS COMPONENTES DEL SISTEMA.





FUNCIÓN DEL MICROPROCESADOR



UN MICROPROCESADOR OPERA EN BASE A UNA SERIE DE INSTRUCCIONES ELEMENTALES QUE SON PREPROGRAMADAS Y ALMACENADAS BAJO LA FORMA DE CÓDIGO BINARIO. ESTAS INSTRUCCIONES VAN A ORGANIZARSE A LA MEMORIA PRINCIPAL, Y SE DAN DE ACUERDO A VARIAS FASES, QUE SON:

- **PREFETCH.** O PRELECTURA DE LA INSTRUCCIÓN DESDE LA MEMORIA PRINCIPAL DEL SISTEMA.
- **FETCH.** ENVÍO DE LA INSTRUCCIÓN ESPECÍFICA AL DECODIFICADOR.
- **DECODIFICACIÓN.** TRADUCCIÓN DE LA INSTRUCCIÓN EN UNA SERIE DE OPERACIONES A REALIZAR, Y LECTURA DE LOS OPERANDOS NECESARIOS PARA HACERLO.
- **EJECUCIÓN.** REALIZACIÓN DE LA INSTRUCCIÓN POR LOS COMPONENTES DEL SISTEMA.
- **ESCRITURA.** GRABADO DE LOS RESULTADOS DE VUELTA EN LA MEMORIA PRINCIPAL, O EN LOS REGISTROS.

ESTAS FASES SE LLEVAN A CABO EN VARIOS CICLOS DEL CPU, Y SU DURACIÓN DEPENDE DE LA FRECUENCIA A LA QUE TRABAJE EL MICROPROCESADOR.

LENGUAJE DE LOS CPU



- **LOS PROCESADORES NO INTERPRETAN CUALQUIER TIPO DE INFORMACIÓN, SINO QUE SÓLO PUEDEN INTERPRETAR INFORMACIÓN ESCRITA DE FORMA BINARIA.**
- **ESTE IDIOMA INFORMÁTICO SE BASA EN DOS CARACTERES, DE CEROS Y UNOS. ADEMÁS SIEMPRE ACUDE A LA INFORMACIÓN QUE OTORGA LA SEÑAL DEL RELOJ, A TRAVÉS DE LA CUAL SE ENCUENTRA EN SINCRONÍA Y CONECTANDO LAS DISTINTAS TAREAS Y OPERACIONES DEL CPU.**

QUÉ ES UN “NÚCLEO” DE UN PROCESADOR O UNA CPU?

- LOS PROCESADORES MÁS ANTIGUOS ERAN **SINGLE CORE**; ES DECIR, DE **UN ÚNICO NÚCLEO**.
- LOS ACTUALES VAN MUCHO MÁS ALLÁ. ESTOS NÚCLEOS SON, CADA UNO DE ELLOS, Y EXPLICADO DE LA FORMA MÁS SENCILLA POSIBLE, UNA UNIDAD DE PROCESAMIENTO EN SÍ MISMA.
- LOS NÚCLEOS SE DEDICAN A LEER INSTRUCCIONES Y A EJECUTAR ACCIONES ESPECÍFICAS. SI LLEVAMOS A CABO CUALQUIER TIPO DE ACCIÓN EN NUESTRO TELÉFONO INTELIGENTE, TABLETA, TELEVISOR Y ORDENADOR –ENTRE OTROS DISPOSITIVOS-, TODO ESTO PASA POR EL PROCESADOR. LOS NÚCLEOS DEL PROCESADOR, COMO DECÍAMOS, SON CAPACES DE HACER CICLOS DE PROCESAMIENTO DE FORMA INDEPENDIENTE, EN CUATRO PARTES:

- **LECTURA**
- **DECODIFICACIÓN**
- **EJECUCIÓN**
- **ESCRITURA**

¿DEPENDE EL RENDIMIENTO DE LOS NÚCLEOS?

SÍ, Y NO.

- ✓ UN NÚCLEO PUEDE LLEVAR A CABO UN CICLO DE INSTRUCCIÓN POR SÍ SOLO.
- ✓ ES DECIR, QUE SI TENEMOS DOS NÚCLEOS, PODRÍAMOS **PARALELIZAR** Y LLEVAR A CABO DOS CICLOS DE INSTRUCCIÓN INDEPENDIENTES, COMO SI TUVIÉRAMOS DOS PROCESADORES.
- ✓ SI TENEMOS CUATRO, ENTONCES PODEMOS LLEVAR A CABO CUATRO CICLOS DE INSTRUCCIÓN DE FORMA SIMULTÁNEA, A PRIORI. BÁSICAMENTE, TENEMOS MAYOR CAPACIDAD DE INSTRUCCIÓN SIMULTÁNEA, PERO LA VELOCIDAD CON LA QUE SE LLEVAN A CABO ESTOS CICLOS DE INSTRUCCIONES NO DEPENDE DE LOS NÚCLEOS, LUEGO EL RENDIMIENTO NO NECESARIAMENTE ESTÁ VINCULADO DE FORMA DIRECTA A LOS NÚCLEOS POR SU CANTIDAD.

Más núcleos no tiene por qué significar mayor rendimiento en un procesador, pero ¿por qué?

- EL DISEÑO DE UNA CPU ES REALMENTE COMPLEJO, Y SERÍA CONVENIENTE AHONDAR EN MUCHOS OTROS DETALLES SOBRE SUS CARACTERÍSTICAS TÉCNICAS. NO OBSTANTE, A NIVEL DE USUARIO Y PARA TENER UN CONCEPTO AMPLIO SOBRE EL RENDIMIENTO DE UN PROCESADOR, PODEMOS LIMITARNOS A DOS DATOS CLAVE: NÚCLEOS Y FRECUENCIA DE RELOJ.
- COMO DECÍAMOS, UN PROCESADOR, O UNA CPU, SE COMPONE DE VARIOS NÚCLEOS. A MÁS NÚCLEOS, MÁS UNIDADES DE PROCESAMIENTO INDEPENDIENTES Y, POR TANTO, MAYOR CANTIDAD DE INSTRUCCIONES SE PODRÁN PROCESAR DE FORMA SIMULTÁNEA.
- LA PARALELIZACIÓN EN EL PROCESAMIENTO DE INSTRUCCIONES PERMITE UN MAYOR RENDIMIENTO, PERO LA FRECUENCIA DE RELOJ ES LO QUE DEFINE LA VELOCIDAD CON QUE SE LLEVAN A CABO ESTOS CICLOS DE INSTRUCCIONES. ESTA VELOCIDAD SE MIDE EN GHZ; CUANTO MAYOR SEA, MÁS RÁPIDO SERÁ CAPAZ CADA UNO DE ESTOS NÚCLEOS EN COMPLETAR ESAS CUATRO PARTES QUE COMPONEN UN CICLO DE INSTRUCCIÓN COMPLETO.

- **ESTO ES PRECISAMENTE LO QUE EXPLICA QUE UN PROCESADOR DUALCORE PUEDA SER MÁS RÁPIDO QUE UN PROCESADOR QUADCORE.**
- **ADEMÁS DE POR OTROS FACTORES COMO LA ARQUITECTURA. EVIDENTEMENTE: LA MEMORIA CACHÉ DE LA CPU, EL PROCESO DE FABRICACIÓN, ETCÉTERA. PERO ES IMPORTANTE CONOCER QUÉ SON LOS NÚCLEOS DE UNA CPU Y SU FUNCIONAMIENTO, AL NIVEL MÁS BÁSICO, Y EN RELACIÓN A LA FRECUENCIA DE RELOJ, PARA DESMITIFICAR QUE LOS PROCESADORES DE OCHO NÚCLEOS EN TELÉFONOS INTELIGENTES, POR EJEMPLO, SEAN SIEMPRE LA MEJOR OPCIÓN.**
- **POR OTRO LADO, TAMBIÉN HAY QUE TENER EN CUENTA QUE LA OPTIMIZACIÓN DE SOFTWARE JUEGA UN PAPEL CRUCIAL PARA APROVECHAR, O NO, EL DISEÑO DE HARDWARE DE MUCHOS COMPONENTES Y, ENTRE OTROS, LA CPU. NO TODOS LOS PROGRAMAS, Y NO TODAS LAS APLICACIONES, ESTÁN PREPARADAS PARA APROVECHAR DETERMINADOS DISEÑOS DE CPU.**
- **AUNQUE CON ALGUNOS PROGRAMAS Y APPS SE PUEDAN APROVECHAR LAS PRESTACIONES DE UNA CPU OCTACORE, POR EJEMPLO, HABRÁ CON OTROS PROGRAMAS CON LOS QUE ESTA CARACTERÍSTICA NO IMPORTE LO MÁS MÍNIMO QUE CONTEMOS CON OCHO NÚCLEOS.**

ARQUITECTURA DEL PROCESADOR INTEL CORE i7



- **INTEL CORE i7** ES UNA FAMILIA DE PROCESADORES DE CUATRO NÚCLEOS DE LA ARQUITECTURA **INTEL X86-64**. LOS **CORE i7** SON LOS PRIMEROS PROCESADORES QUE USAN LA **MICROARQUITECTURA NEHALEM** DE INTEL Y ES EL SUCESOR DE LA FAMILIA **INTEL CORE 2**.
- EL IDENTIFICADOR **CORE i7** SE APLICA A LA FAMILIA INICIAL DE PROCESADORES CON EL NOMBRE CLAVE **BLOOMFIELD**. ESTA NUEVA GENERACIÓN DE PROCESADORES REFLEJA LOS RECIENTES E IMPORTANTES AVANCES EN TECNOLOGÍA DE INTEL.
- LOS PROCESADORES **CORE i7** ESTÁN BASADOS EN LA **MICRO-ARQUITECTURA INTEL NEHALEM**. CADA UNO DE ESTOS CHIPS OFRECE CUATRO NÚCLEOS Y VELOCIDADES QUE VAN DE LOS **2,66 GHZ** A LOS **3,20 GHZ**. ALGUNOS DE ESTOS PROCESADORES FORMAN LA SERIE **EXTREME EDITION**, COMO EL **CORE i7 965 EXTREME EDITION** TIENE UNA VELOCIDAD DE **3,20 GHZ** Y **TECNOLOGÍA QPI (QUICKPATH INTERFACE)** DE 6,4 GT/S QUE ES SU PRINCIPAL DIFERENCIADOR.

MICROARQUITECTURA NEHALEM

NEHALEM ES EL NOMBRE EN CLAVE PARA UN INTEL PROCESADOR MICROARQUITECTURA PUBLICADO EN NOVIEMBRE DE 2008. **NEHALEM** SE UTILIZÓ EN LA PRIMERA GENERACIÓN DE LOS PROCESADORES INTEL CORE (CORE i7 Y i5, CON i3 CORE BASÁNDOSE LA POSTERIOR WESTMERE Y SANDY BRIDGE MODELOS). NEHALEM ES EL SUCESOR DEL VIEJO MICROARQUITECTURA CORE (INTEL CORE 2).

- EL CAMBIO MÁS IMPORTANTE DE INTEL EN LOS ÚLTIMOS AÑOS

CON **NEHALEM** NO SÓLO SE INTRODUCE UNA NUEVA FAMILIA DE PROCESADORES, TAMBIÉN **CAMBIA LA ESTRUCTURA TRADICIONAL DE LAS PLATAFORMAS DE INTEL** DONDE LOS DISTINTOS ELEMENTOS DE UN SISTEMA SE COMUNICABAN MEDIANTE EL TRADICIONAL **FSB (FRONT SIDE BUS)** PARALELO. ASÍ, SI EL PROCESADOR TENÍA QUE COMUNICARSE CON LA MEMORIA DEL SISTEMA O CON OTROS COMPONENTES, INCLUYENDO OTROS PROCESADORES Y NÚCLEOS, DEBÍA HACERLO A TRAVÉS DEL BUS DEL SISTEMA Y DEL CHIPSET.

EL CAMBIO EN LA DISPOSICIÓN DE LOS SISTEMAS PC SE MODIFICA EN INTEL DEL MISMO MODO QUE OCURRÍA CON AMD CON SU ARQUITECTURA K8 Y EL ATHLON 64. EL BUS DEL SISTEMA DESAPARECE, PARA DEJAR PASO A UN CANAL DE COMUNICACIÓN SERIE DENOMINADO **QUICK PATH INTERCONNECT O QPI**. SE TRATA DE UN CANAL BIDIRECCIONAL CON UN ANCHO DE BANDA DE **MÁS DE 25 GB/S** QUE PERMITE LA COMUNICACIÓN DIRECTA ENTRE EL PROCESADOR Y EL CONTROLADOR I/O DE ENTRADA/SALIDA O ENTRE VARIOS PROCESADORES. GUARDA SIMILITUDES CON **HYPERTRANSPORT** DE AMD, PERO MEJORANDO SUS CARACTERÍSTICAS Y REDUCIENDO LA LATENCIA EN TODAS LAS OPERACIONES QUE TENGAN QUE VER CON ACCEDER A LA MEMORIA O A DATOS ALMACENADOS EN REGISTROS.

Generaciones de procesadores Intel Core: <https://www.profesionalreview.com/2020/07/05/cuantas-generaciones-de-procesadores-intel-core-hay-identifica-tu-cpu/>

QUICKPATH INTERCONNECT ("QUICKPATH", "QPI"):

ES UNA TECNOLOGÍA DE CONEXIÓN PUNTO A PUNTO CON EL MICROPROCESADOR, DESARROLLADO POR INTEL PARA REMPLAZAR EL **FRONT-SIDE BUS** EN COMPUTADORAS DE ESCRITORIO Y PLATAFORMAS DESKTOP, XEON E ITANIUM Y ASÍ COMPETIR CON EL HYPERTRANSPORT DE **AMD**. LA ARQUITECTURA **QUICKPATH** ASUME QUE EL PROCESADOR TIENE UN CONTROLADOR DE MEMORIA INTEGRADO, OBLIGANDO ASÍ A LOS MULTIPROCESADORES A USAR UNA ARQUITECTURA **NUMA***.

***NUMA (NON UNIFORM MEMORY ARCHITECTURE) ES UNA ARQUITECTURA DE MEMORIA COMPARTIDA.** LAS ARQUITECTURAS DE MEMORIA COMPARTIDA PERMITEN QUE TODOS LOS PROCESADORES DE UN SISTEMA ACCEDAN DIRECTAMENTE A LA MEMORIA GLOBAL DE ESE SISTEMA. NO SE REQUIERE EL "**PASE DE MENSAJES**" ENTRE DISTINTAS CPU'S PARA CONSEGUIR ESE ACCESO.

El **QPI** es una arquitectura de sistema que Intel la llama “Arquitectura QuickPath” e implementa lo que Intel llama “tecnología QuickPath”.

En una placa base con un solo procesador, **QPI** se usa para conectar el procesador con el Eje IO (por ejemplo, para conectar un procesador **Intel Core i7** a un chipset X58).

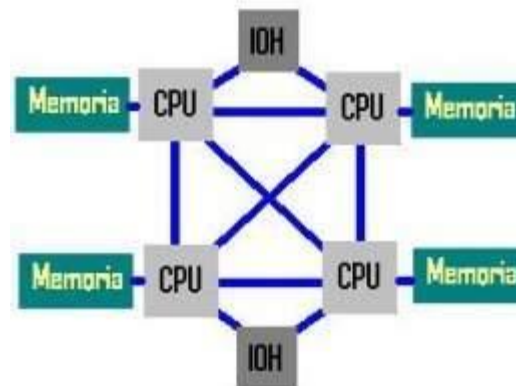
Tal como el **HyperTransport** de **AMD**, la arquitectura **QuickPath** asume que el procesador tiene un controlador de memoria integrado, obligando así a los multiprocesadores a usar una arquitectura **NUMA**.

Cada **QPI** comprime 2 conexiones punto a punto de 20-bit, una para cada dirección, para un total de 42 señales. Cada señal es un par diferencial, formando así un número de 84.

El **QuickPath** reporta velocidades de 4,8 a 6,4 GT/s por segundo por dirección. El ancho de banda va de 12,0 a 16,0 GB/s por dirección, o 24,0 a 32,0 GB/s por conexión.

La implementación inicial en el **Nehalem** usa una conexión de 25,6 GB/s a 20-bit. Esta conexión provee exactamente el doble del ancho de banda teórico de un **FSB** de Intel a 1600 MHz (usados en el **chipset Intel X48**).

Los procesadores actuales operan con conexiones de 16 bit. La velocidad de reloj a la cual opera viene determinada por el procesador; el **Core i7 920 y 940** funcionan con conexiones de 9,6 GB/s unidireccionales y 19,2 GB/s bidireccionales mientras el **Core i7 965XE** usa 12,8 GB/s y 25,6 GB/s respectivamente.



INTEL QUICKPATH INTERCONNECT

ESPECIFICACIONES DE FRECUENCIA:

QPI funciona a una frecuencia de reloj de **2,4 GHZ, 2,93 GHZ o 3,2 GHZ**.

La velocidad del reloj de un enlace en particular depende de las capacidades de los componentes en cada extremo del enlace, las características de la señal y de la trayectoria de la señal en el tablero de circuito impresos.

Los procesadores **Core i7 “non-extreme” 9xx** se limitan a una frecuencia de reloj de **2,4 GHZ** . las transferencias de bits se producen tanto en la subida como en los flancos de bajada del reloj, por lo que la tasa de transferencia es el doble de la velocidad del reloj.

ESPECIFICACIONES DE FRECUENCIA:

INTEL describe el procesamiento de datos (en GB/s), contando sólo los 64 bits de carga de datos en cada uno de 80 bits “FLIT”. (FLIT: Flow Control Digit)

Sin embargo, INTEL duplica el resultado debido a que el par de enlace unidireccional para enviar y recibir pueden estar activos simultáneamente. de este modo, INTEL describe un par de 20 carriles **QPI Link** (envío y recepción) con un reloj de **3,2 GHZ** que tiene una velocidad de hasta 25,6 GB/s. una frecuencia de reloj de **2,4 GHZ** proporciona rendimientos de una tasa de datos de 19.2 GB/s. en general, con esta definición, un enlace de dos carriles de 20 **QPI** transfiere ocho bytes por ciclo de reloj, cuatro en cada dirección.

La tasa se calcula como sigue:

- **3,2 GHz**
- **X 2 bits / Hz (doble velocidad de datos)**
- **X 20 (ancho QPI link)**
- **X (64/80) (bits de datos / bits flit)**
- **X 2 (unidireccional enviar y recibir al mismo tiempo de funcionamiento)**
- **÷ 8 (bits / byte)**
- **= 25,6 GB / s**

Características de Intel Core i7

NEHALEM representa el cambio de arquitectura más grande en la familia de procesadores **Intel x86** desde el **Pentium Pro en 1995**. La arquitectura **NEHALEM** tiene muchas nuevas características. la primera representa un cambio significativo desde el **CORE 2**:

- **FSB** es reemplazado por la interfaz **QUICKPATH** EN **i7 E i5 (SOCKET 1366)**, y sustituido a su vez en **i7, i5 e i3 (SOCKET 1156)** por el **DMI** eliminado el **NORTHBRIDGE** e implementando puertos **PCI EXPRESS** (16 LINEAS EN TOTAL) directamente, debido a que es más complejo y caro. Las placas base deben utilizar un chipset que soporte **QUICKPATH**. de momento sólo está disponible para placas base de **ASROCK, ASUS, DFI , EVGA , GIGABYTE , INTEL , MSI Y XFX**.

PCI Express (Peripheral Component Interconnect Express) es un bus seriado de datos de alta velocidad

- El controlador de memoria se encuentra integrado en el mismo procesador.
- Memoria de tres canales (ancho de datos de 192 bits): cada canal puede soportar una o dos memorias **DIMM DDR3**. Las placa base compatibles con **Core i7** tienen cuatro (3+1) o seis ranuras **DIMM** en lugar de dos o cuatro, y las **DIMM** deben ser instaladas en grupos de tres, no dos.
- DIMM según sus Siglas en Inglés: Dual In-Line Memory Module, y en Español: Módulo de Memoria con Contactos Duales

- **Northbridge:**

Es el chip que controla las funciones de acceso desde y hasta microprocesador, AGP o PCI-Express, memoria RAM, vídeo integrado (dependiendo de la placa) y Southbridge. Su función principal es la de controlar el funcionamiento del bus del procesador, la memoria y el puerto AGP o PCI-Express. De esa forma, sirve de conexión (de ahí su denominación de "puente") entre la placa madre y los principales componentes de la PC.

- **Southbridge:**

Es un circuito integrado que se encarga de coordinar los diferentes dispositivos de entrada y salida y algunas otras funcionalidades de baja velocidad dentro de la placa base. Suele integrar más bien la funcionalidad de E/S de una placa, por ello abarca las controladoras IDE, SATA, USB, de sonido, de red...

- Ambos chipsets van intercomunicados entre sí.

- **SOPORTE PARA DDR3 ÚNICAMENTE.**

- - **TURBO BOOST:** la misma permite a los distintos núcleos acelerarse "inteligentemente" por sí mismos cada 133 MHz por encima de su velocidad oficial, mientras que los requerimientos térmicos y eléctricos de la CPU no sobrepasen los predeterminados.
- - **DISPOSITIVO SINGLE-DIE:** los cuatro núcleos, el controlador de memoria, y la caché se encuentran dentro del mismo encapsulado.
- - **HYPERTHREADING REIMPLEMENTADO:** cada uno de los cuatro núcleos puede procesar dos tareas simultáneamente, por tanto el procesador aparece como ocho cpu's desde el sistema operativo. esta característica estaba presente en la antigua microarquitectura **NETBURST** introducida en los **Pentium 4 HT**.

- Sólo una **INTERFAZ QUICKPATH**: No concebida para placas base multiprocesador.
- - **TECNOLOGÍA DE PROCESO DE 45 NM O 32 NM.**
- - 731 millones de transistores (1.170 millones en el **CORE i7 980X**, con 6 núcleos y 12 MB de memoria caché).
- - Sofisticada administración de energía, puede colocar un núcleo no utilizado en modo sin energía.
- - Capacidad de OVERCLOCKING muy elevada (se puede acelerar sin problemas hasta los 4-4,1 GHz).

Mayor información

- **Capítulo 7 y 8**
 - ✓ Quiroga patricia 1^{ra} Ed
- **Capítulo 11 y 12**
 - ✓ Stallings, 7^{ma}. Ed.
- **Capítulo 5 y 6**
 - ✓ Osio, Morales, 1^{ma}. Ed.

CATEDRA DE ARQUITECTURA DE COMPUTADORAS

INSTRUCCIONES

Formato de una Instrucción

- En cualquier computador, cada instrucción está representada mediante una secuencia de bits.
- Esta secuencia se encuentra dividida en campos de acuerdo a los elementos que la componen.
- A este esquema se lo conoce como “Formato de la Instrucción”.
- Los datos (operandos), tanto fuente como destino pueden encontrarse en diferentes lugares :
 - * Registros de la CPU
 - * Memoria
 - * Dispositivo de E/S

CÓDIGO DE OPERACIÓN	DATO (OPERANDOS Fuente y Destino)
---------------------	--------------------------------------

Movimientos de operandos en una instrucción

La ejecución de una instrucción implica una secuencia de movimientos en **MP** y **RX** de la CPU establecidos por la UC en un orden determinado según el **Código de la Operación**

TRANSFERENCIA	EJEMPLO	SUMA	EJEMPLO
$RX \leftarrow RX$	MOV AX,BX	$RX \leftarrow RX + RX$	ADD AX,BX
$RX \leftarrow MPp$	MOV AX,0020)h	$RX \leftarrow RX + MPp$	ADD AX,0020)h
$RX \leftarrow MPd$	MOV AX,[0100]	$RX \leftarrow RX + MPd$	ADD AX,[0100]
$MPd \leftarrow RX$	MOV [0100],AX	$MPd \leftarrow MPd + RX$	ADD [0100],AX
$MPp \leftarrow RX$	No se utiliza, modificaría un programa	$MPp \leftarrow MPp + RX$	No se utiliza, modificaría un programa

Ciclos de una Instrucción



Estados del Ciclo

1- Cálculo dirección instrucción (IAC – Instruction Address Calculation)

Determina la dirección de la siguiente instrucción a ejecutarse. Normalmente implica añadir un nro. fijo a la dirección de la instrucción previa.

2- Captación de la instrucción (IF – Instruction Fetch)

La CPU la lee desde su posición en memoria.

3- Decodificación de la operación (IOD – Instruction Operation Decoding)

Analiza la instrucción para determinar el tipo de operación a realizar y el/los operandos a utilizar.

4- Cálculo dirección operando (OAC – Operand Address Calculation)

Si la instrucción implica la referencia a un operando en la memoria ó E/S, entonces se determina la dirección.

Estados del Ciclo

5- Captación del Operando (OF – Operand Fetch)

Capta el operando desde memoria o desde su ubicación.

6- Operación sobre los datos (DO – Data Operation)

Realiza la operación indicada en la instrucción

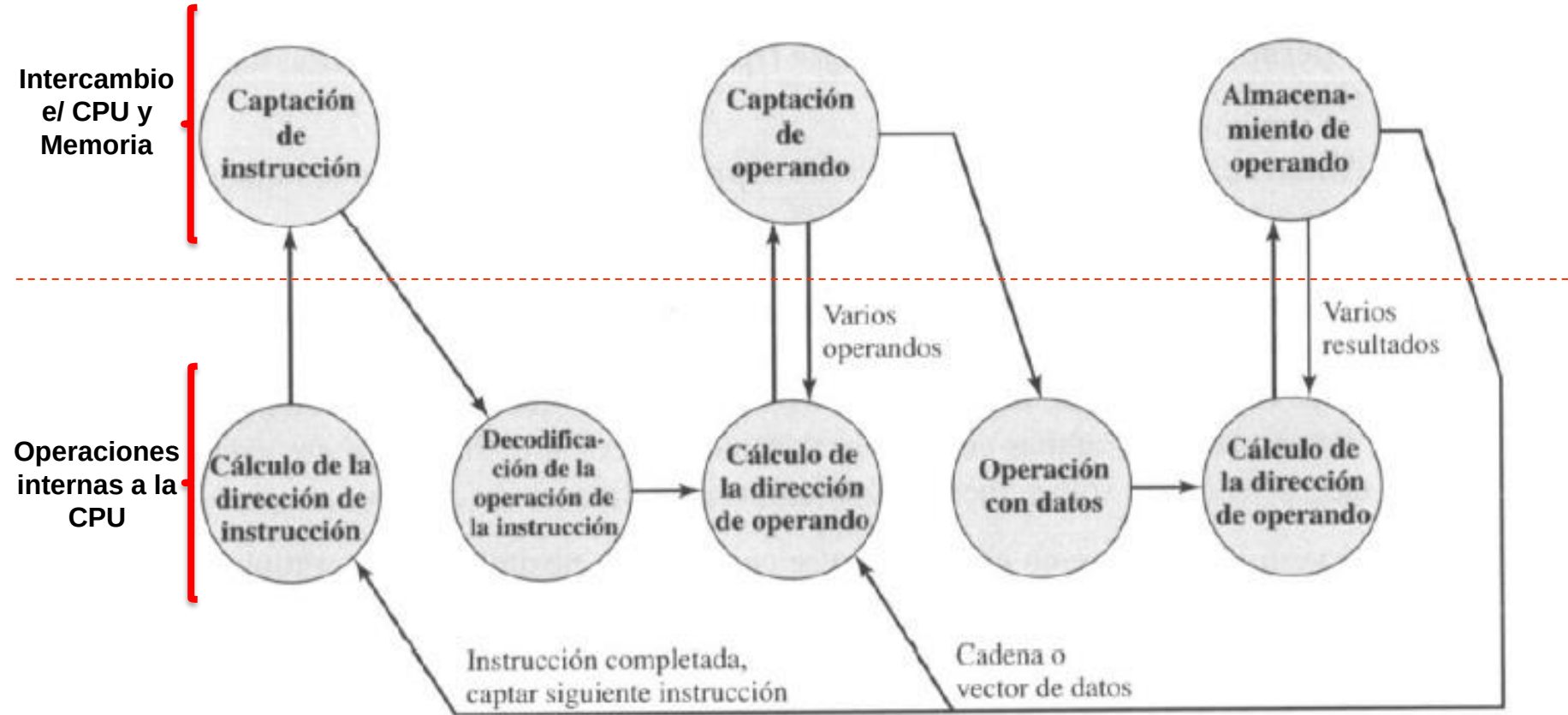
7- Cálculo dirección resultado (OAC – Operand Address Calculation)

(Idem 4 - **Cálculo dirección operando**)

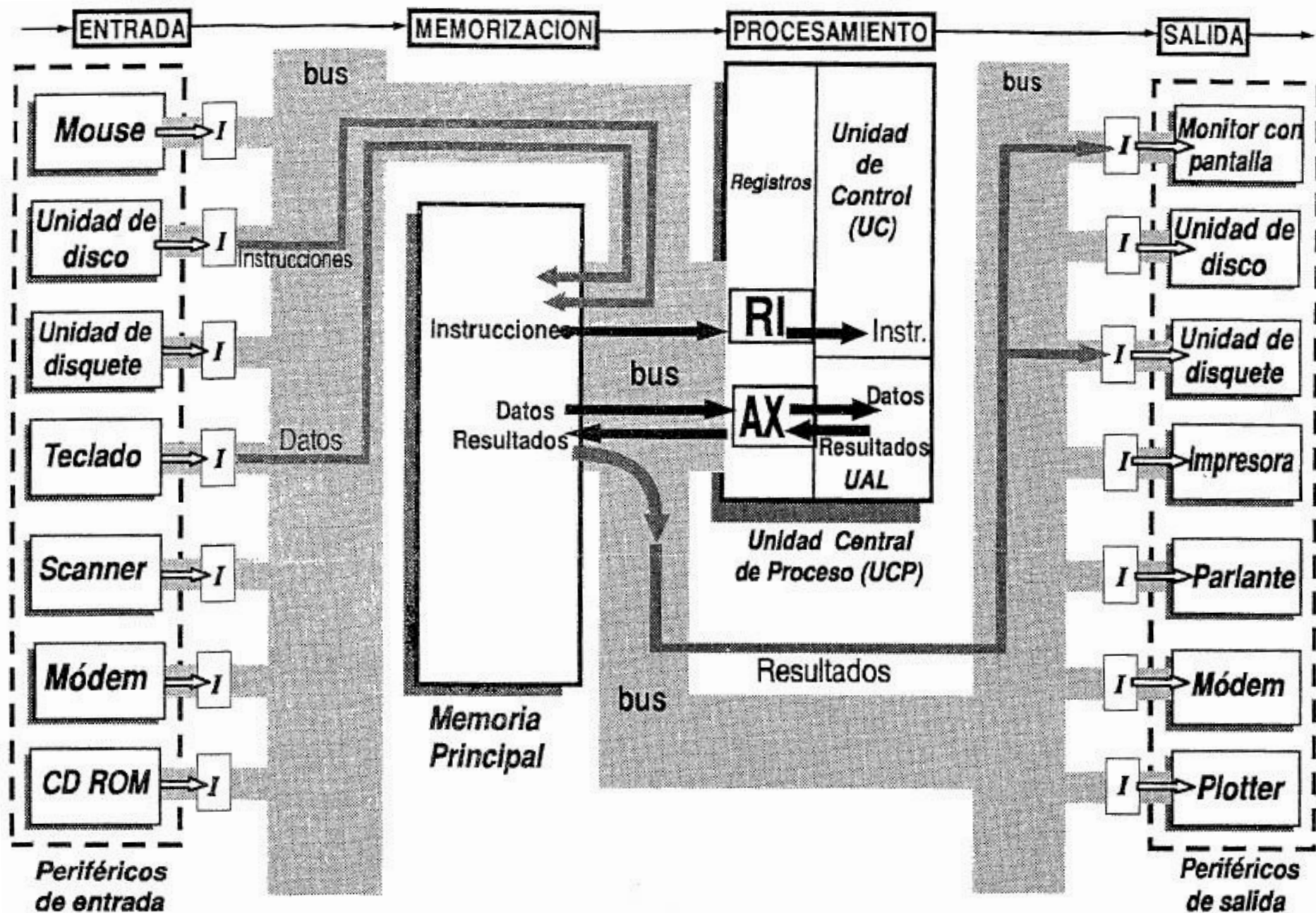
8- Almacenamiento del operando (Operand Store)

Escribe el resultado en memoria o lo envía a través de un dispositivo de E/S.

DIAGRAMA DE ESTADOS



Transferencia de Datos



Mayor información

➤ Capítulo 7 y 10

✓ Quiroga patricia 1^{ra} Ed

➤ Capítulo 10

✓ Stallings, 7^{ma.} Ed.

CATEDRA DE ARQUITECTURA DE COMPUTADORAS

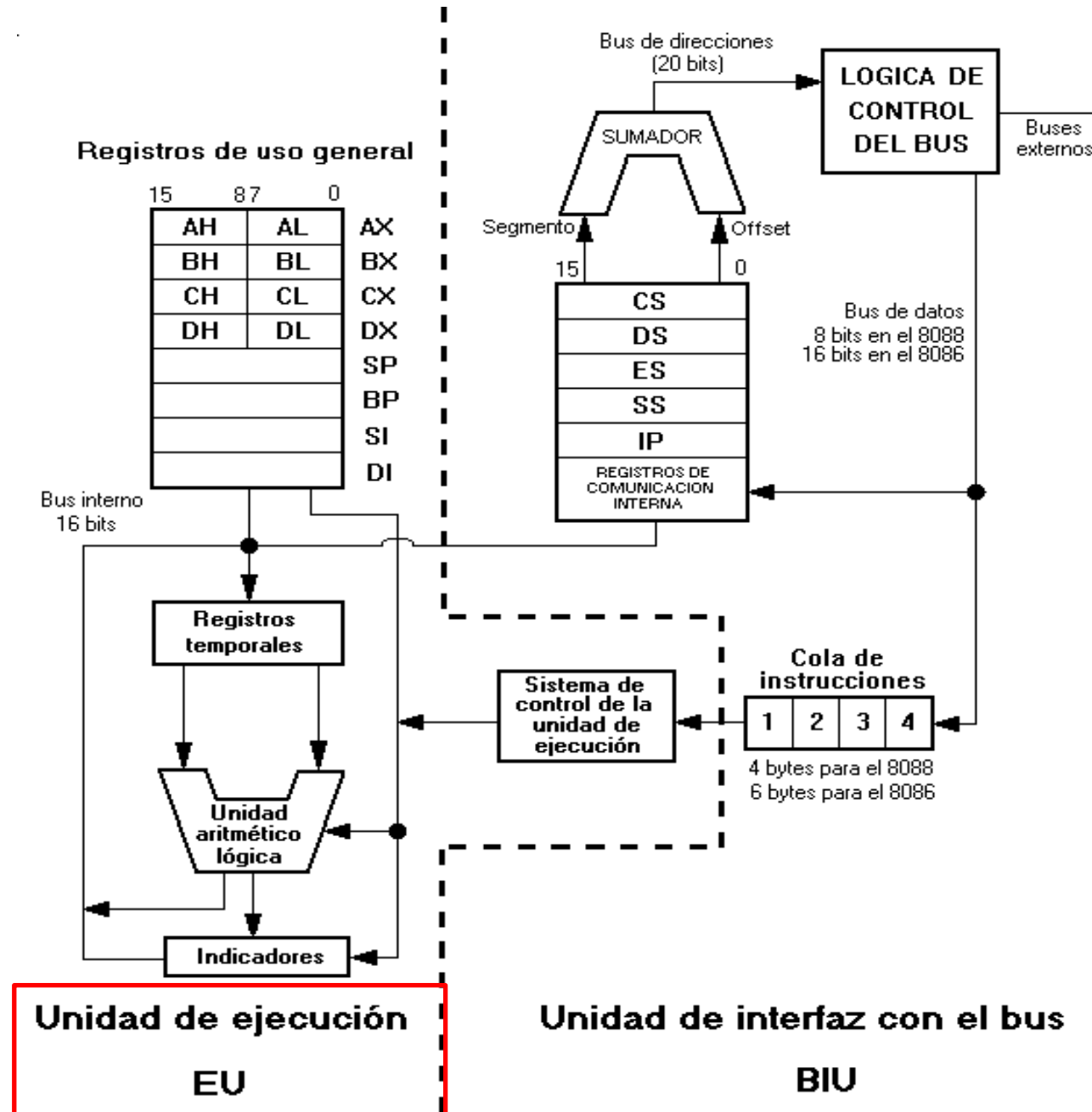
Estructura del Procesador

BUS DE SISTEMA						
MICROPROCESADOR			MEMORIA PRINCIPAL			BUS DE E/S
CU	Unidades de cálculo ALU	Registros internos				Interfaces E/S y Dispositivos de E/S. Por ejemplo, en interfaces SATA, en dispositivos de E/S discos, teclado, memoria, tarjeta de memoria
Otras unidades funcionales		Niveles de caché	Memoria programas	Memoria datos	Memoria E/S	

Esquema de una computadora basada en un microprocesador.

Estructura del Procesador

**ENCARGADA
DE REALIZAR
TODAS LAS
OPERACIONES**



UNIDAD DE EJECUCION

La unidad de ejecución (**EU**) se encarga de realizar las operaciones aritméticas y lógicas, además de proporcionar las direcciones lógicas al **BIU**.

Funciones Básicas:

- ✓ Leer e incrementar el **IP**
- ✓ Decodificar la instrucción leída
- ✓ Hacer que se ejecute la instrucción
- ✓ Resolver situaciones de conflicto y controlar la comunicación con los periféricos

Información utilizada para realizar sus tareas:

- ✓ La instrucción en ejecución
- ✓ El Registro de Estados
- ✓ El Contador de Períodos, accionado por el **Clock**
- ✓ Las señales de Control y Estado externas a la **CPU**

Operaciones Elementales

- ✓ Son los pequeños pasos que se realizan para ejecutar instrucciones (Por ej. Suma de Base + Desplazamiento, Lectura de un operando, incremento del IP, etc.)
- ✓ La ejecución de cada una de estas operaciones elementales requiere la activación de las correspondientes señales de control.
- ✓ El Objetivo de la **UNIDAD DE CONTROL** es la generación de las secuencias de señales de control que permitan realizar las distintas operaciones elementales.

Las Operaciones Elementales que puede realizar todo sistema digital síncrono pueden clasificarse en dos grupos:

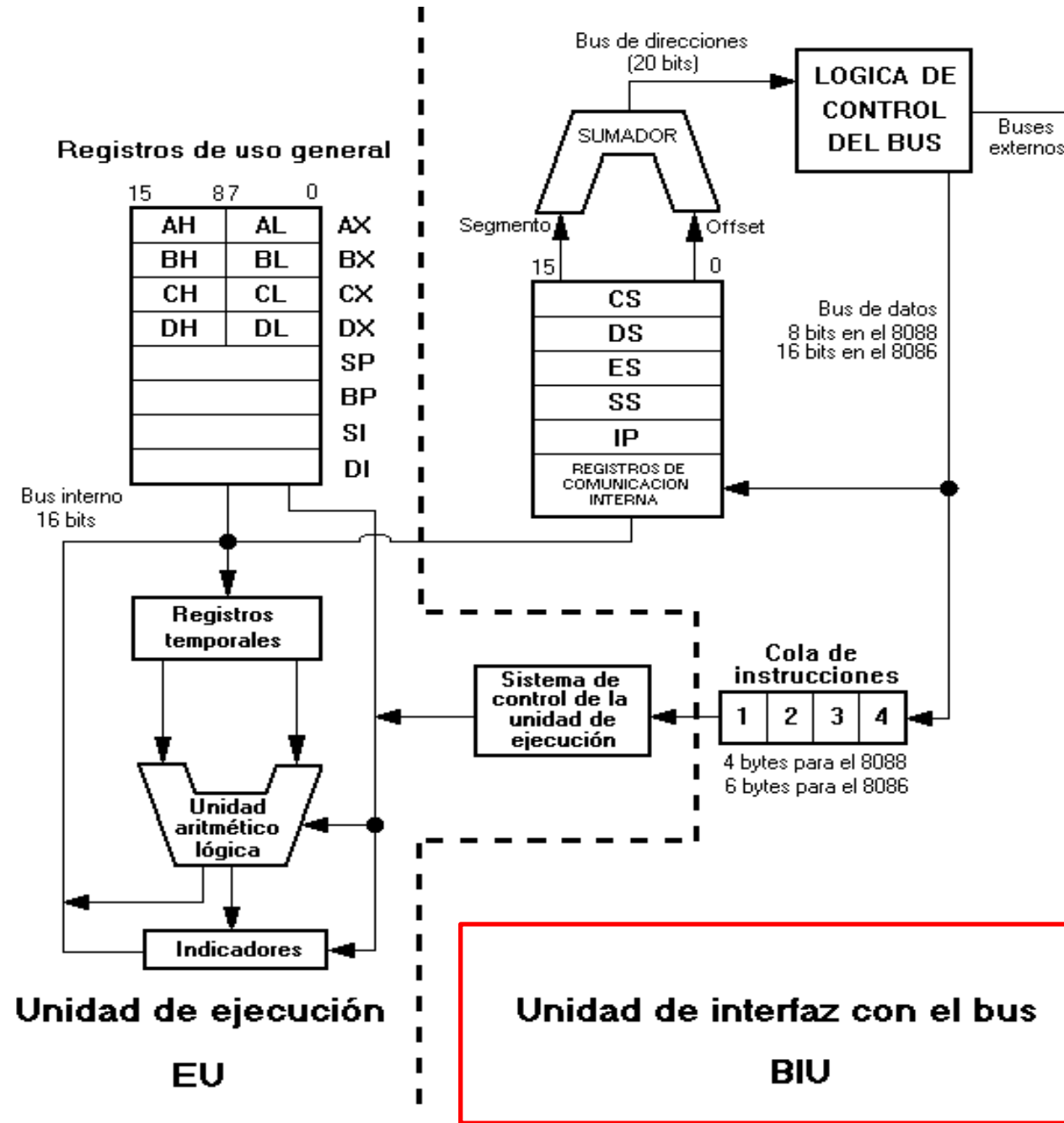
✓ **OPERACIONES DE TRANSFERENCIA**

Requieren dos elementos de almacenamiento (Registros por ej.), uno de origen y otro de destino. Para ello primero debe establecer un camino físico e/ la salida del origen y la entrada del destino. Una vez establecido se envía una señal al destino para que cargue lo que tiene en su entrada.

✓ **OPERACIONES DE PROCESO**

Tienen un planteamiento básico similar a las Operaciones de Transferencia, la diferencia fundamental radica en que la información sufre una **transformación** al pasar a través de un operador combinacional en su camino hacia el destino.

Estructura del Procesador



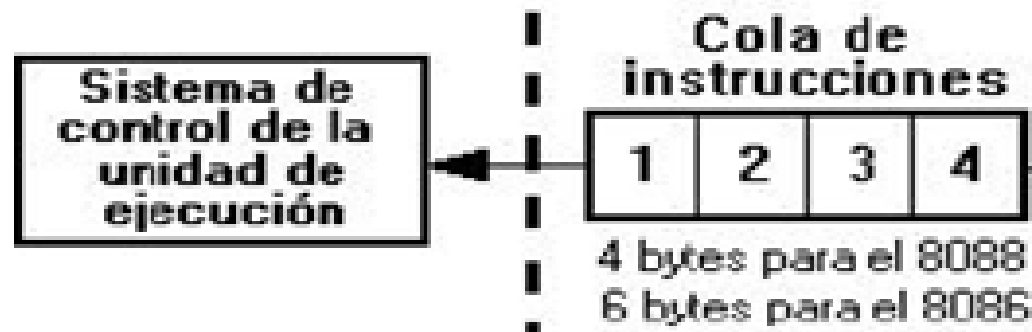
**ENCARGADA DE
ACCEDER A LOS
DATOS E
INSTRUCCIONES
DEL MUNDO
EXTERIOR**

UNIDAD DE INTERFAZ DEL BUS (B.I.U)

- ✓ Es la encargada de acceder a memoria y a otros dispositivos de E/S.
- ✓ Es la parte del procesador que se une al resto del hardware
- ✓ Se denomina así porque es la que realiza los movimientos de datos hacia el Bus de Datos del procesador (es el primer conducto de información hacia y desde el **CPU**)
- ✓ Es la responsable de responder a todas las señales que van al procesador y de generar todas las señales que van del procesador a las demás partes del sistema.
- ✓ Se encarga no sólo de obtener y almacenar datos en memoria principal, sino también de obtener las instrucciones del programa.

UNIDAD DE INTERFAZ DEL BUS (B.I.U)

- ✓ Sirve además de paso a las instrucciones y a los datos para que estos puedan alcanzar los registros de la **U.C.** y la **A.L.U.**



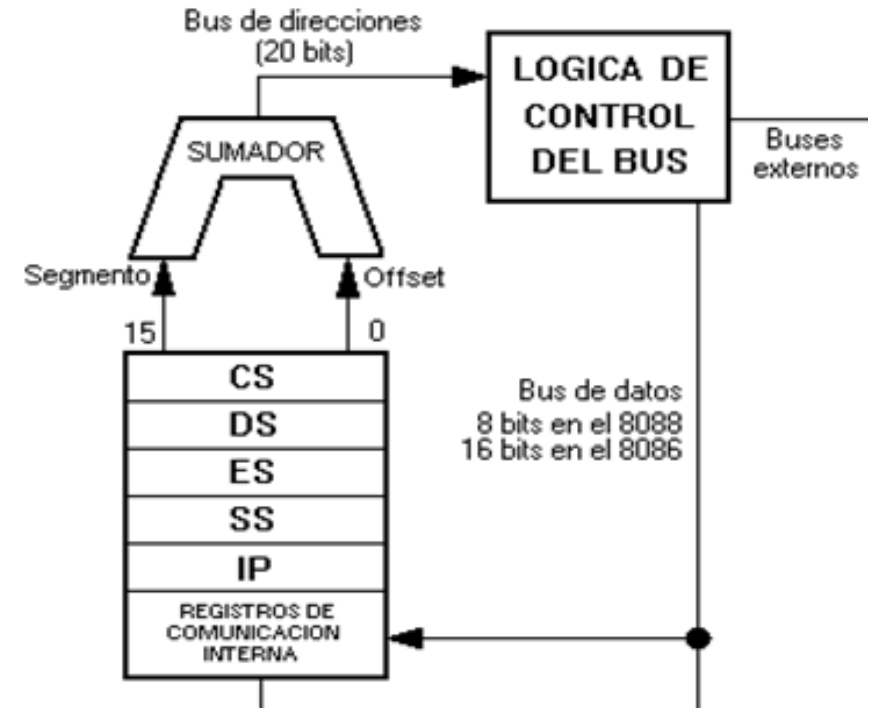
Las instrucciones son leídas previamente por la BIU y almacenadas en una cola de instrucciones con estructura **FIFO** (**First In, First Out**), o sea, la primera instrucción que ingresa en la cola es la primera que sale y se ejecuta.

UNIDAD DE INTERFAZ DEL BUS (B.I.U)

- ✓ La **BIU** se relaciona con los registros de segmento y con los registros punteros (p.ej., el **IP** o **Instruction Pointer**)
- ✓ Sincroniza los niveles de señales de los circuitos internos del Microprocesador con los de los otros componentes del sistema.

Posee dos conexiones:

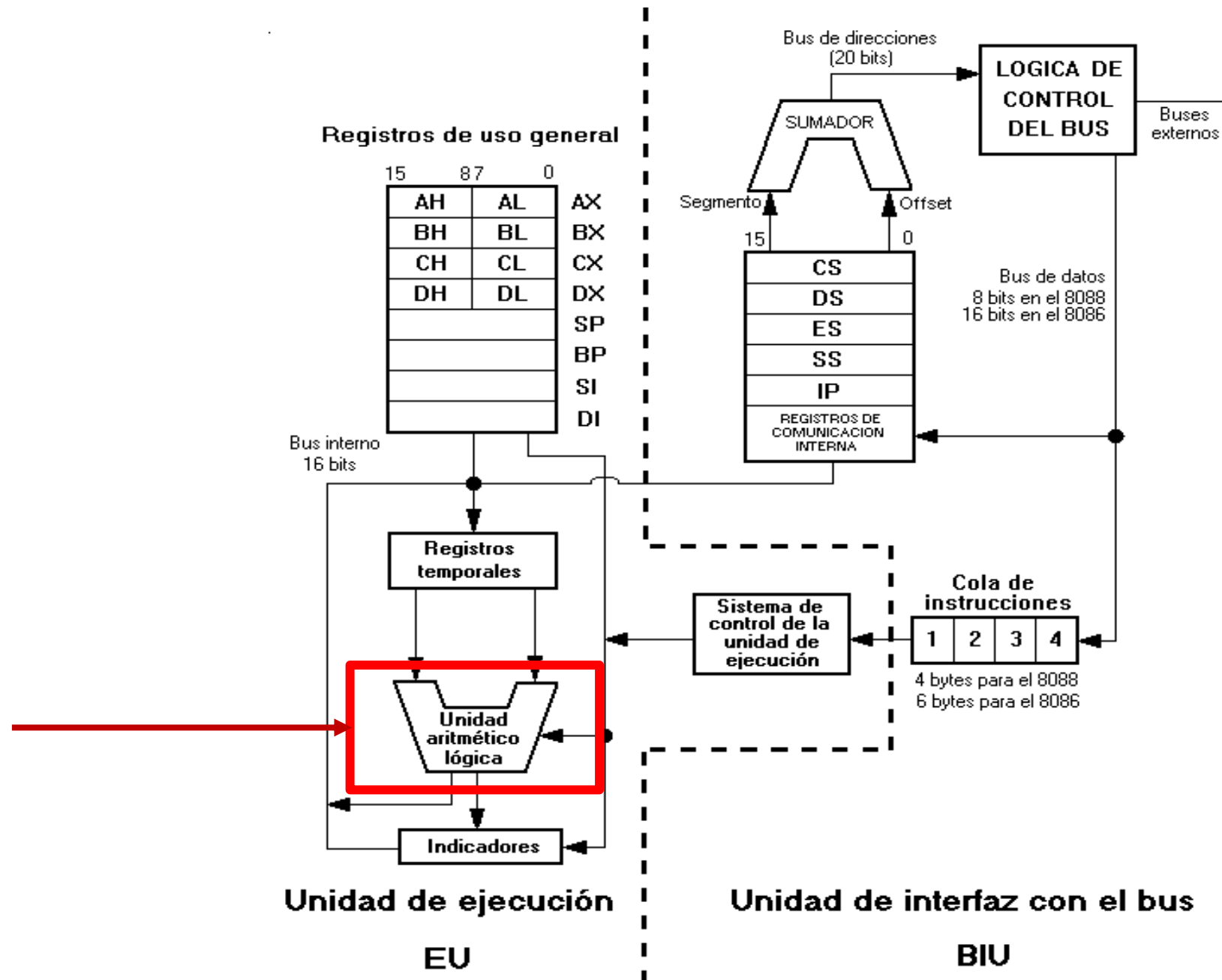
- ✓ **BUS DE DIRECCIONES:** Son aquellas conexiones que indican las localidades de memoria “hacia o desde” las cuales el microprocesador enviará o recibirá datos.
- ✓ **BUS DE DATOS:** Son las que transportan el significado de los datos de las instrucciones.



UNIDAD DE INTERFAZ DEL BUS (B.I.U)

- ✓ Posee además un **BUS DE CONTROL**, el cual se encuentra inmerso en la **U.C.** y sirve para enviar las señales de control como por ej.: Señales de carga o lectura de registros, corrimientos, etc.....
- ✓ El tamaño del **BUS DE DATOS** determina el tamaño del conjunto de bits que se transferirán en simultáneo
- ✓ El tamaño del **BUS DE DIRECCIONES** define la cantidad de memoria **RAM** que puede direccionar un Microprocesador.

Estructura del Procesador



UNIDAD ARITMÉTICO-LÓGICA (A.L.U)

- ✓ La **ALU** es básicamente la calculadora del sistema.
- ✓ Es la encargada de ejecutar todas las operaciones aritméticas y lógicas necesarias entre los datos que llegan a la **CPU**.
- ✓ Los elementos principales que componen de dicha unidad son los siguientes:
 - **REGISTROS**
 - Acumulador
 - Registros temporales
 - Registro de estado
 - **UNIDAD DE CALCULO (O ALU PROPIAMENTE DICHA)**

UNIDAD ARITMÉTICO-LÓGICA (A.L.U)

✓ Realiza dos tipos de funciones:

- Realización de las operaciones aritméticas como: suma, resta, multiplicación y división.
- Ejecución de operaciones de tipo lógico: comparación de dos datos.

INTERCONEXIÓN CON EL RESTO DEL PROCESADOR



UNIDAD ARITMÉTICO-LÓGICA (A.L.U)

- ✓ La **ALU** efectúa las **operaciones aritméticas** por medio de unos circuitos electrónicos que componen los siguientes dispositivos:
 - **Dispositivo de Adición**
 - **Registros**
 - **Dispositivo de control de cálculo.**
- ✓ **El dispositivo de Adición** sirve para calcular las operaciones de suma, resta, multiplicación y división. Esto es así ya que estas tres últimas operaciones se pueden efectuar mediante sumas sucesivas o cambiando el signo en la operación de sumar.
- ✓ **Los registros** se utilizan para contener los operandos, los resultados parciales que se van obteniendo en las distintas operaciones y los resultados finales.

UNIDAD ARITMÉTICO-LÓGICA (A.L.U)

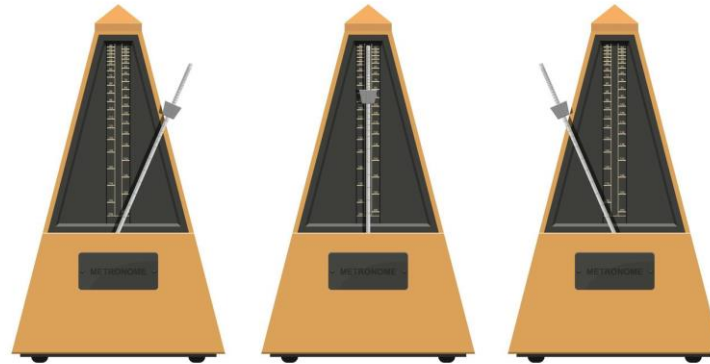
- ✓ **El dispositivo de Control de Cálculo** dirige y controla las operaciones de cálculo que se realizan en la **ALU**.
- ✓ Para realizar las **comparaciones** la **ALU** dispone de un circuito llamado **comparador**, que es capaz de detectar si dos datos son iguales, o cuál de los dos es el mayor o el menor.
- ✓ **La Unidad de Control** también se encarga de controlar otras operaciones como son el transporte de los resultados, proporcionar operandos, comenzar la operación siguiente, etc.

RELOJ DEL SISTEMA

- ✓ Toda computadora contiene un reloj del sistema, este reloj es accionado por un cristal de cuarzo que al momento de aplicarle electricidad las moléculas en el cristal vibran muchas veces cada segundo. Las vibraciones son usadas por la computadora para medir sus operaciones de procesamiento.
- ✓ Al paso de los años, las velocidades de los relojes se ha incrementado constantemente. La primera PC operaba a 4.77 Mhz. (millones de ciclos por segundo), actualmente un procesador i9 posee una frecuencia de reloj que va desde los 2.6 Ghz a los 5.0 Ghz. Un ciclo de reloj es el tiempo que le lleva un transistor en apagarse y encenderse.
- ✓ Los sistemas utilizan un reloj digital para sincronizar los impulsos.
- ✓ La separación entre impulsos es del orden de los nanosegundos (10^{-9} segundos), o sea, en un segundo podemos mover del orden de los mil millones de impulsos.

RELOJ DEL SISTEMA

- ✓ La velocidad de funcionamiento del Microprocesador está determinada por el ritmo de los impulsos de su reloj.
- ✓ Su funcionamiento es comparable con un metrónomo con su péndulo que oscila de derecha a izquierda. El intervalo de tiempo que el péndulo tarda en recorrer esa distancia y regresar a su punto inicial se denomina “CICLO”



- ✓ La velocidad del procesador está expresada en “CICLOS X SEGUNDO ó HZ.”

CATEDRA DE ARQUITECTURA DE COMPUTADORAS

Lenguaje Ensamblador

Lenguaje Ensamblador

- **Definición:** Es un lenguaje traductor (**recibe como entrada código escrito en un cierto lenguaje y produce como salida código en otro lenguaje**) que permite tener un control total sobre cada instrucción generada por una máquina en particular mediante el programa traductor, llamado *Ensamblador*.
- Es un código nemotécnico utilizado para escribir código de máquina mediante expresiones abreviadas.
- Se encuentra muy ligado al Hardware, al contrario que un lenguaje de alto nivel.
- Su ejecución es más rápida.
- Permite trabajar directamente con los registros del microprocesador, la memoria, dispositivos de E/S, etc.

Estructura de un Programa

✓ Básicamente los programas pueden estar englobados en cuatro bloques fundamentales:

- **DECLARACIONES Y DEFINICIONES**
- **SEGMENTO DE DATOS**
- **SEGMENTO DE PILA**
- **SEGMENTO DE CÓDIGO**

DECLARACIONES Y DEFINICIONES

- Donde se realiza la definición de constantes, importación de código o información para el compilador

Ejemplo:

```
CR EQU 13          ; Constante CR (Retorno de carro) en Decimal o bien,  
LF EQU 0Ah         ; Constante LF (Salto de línea) en Hexadecimal
```

SEGMENTO DE DATOS

- Reserva de espacio para las variables y constantes que utiliza el programa.
- Para la creación de un segmento, se utiliza la directiva SEGMENT, la cual indica el comienzo del mismo.
- El final del segmento viene dado por la directiva ENDS (END Segment).

Ejemplo:

```
DATOS SEGMENT      ; Inicio del segmento de datos  
    ;-----  
    MENSAJE DB CR,LF,'Hola Mundo !',CR,LF,'$'  
    ;-----  
DATOS ENDS         ; Fin del segmento de datos
```

SEGMENTO DE PILA

- ✓ Todos los programas han de llevar un pila, con el fin de depositar la información necesaria para las llamadas a funciones, o bien almacenar datos temporalmente. En este bloque se define un tamaño para la pila.
- ✓ El segmento de pila se define como cualquier otro segmento, pero especificando la palabra reservada STACK.
- ✓ En este segmento reservamos únicamente espacio para la pila, es decir, definimos su tamaño.

Ejemplo:

```
PILA SEGMENT STACK                ; Inicio del segmento de pila
    ;-----
    DB 64 DUP('PILA')              ; Inicialización de la pila
    ;-----
PILA ENDS                          ; Fin del segmento de pila
```

Definimos el tamaño de la pila, en este ejemplo añadimos 64 veces los bytes ASCII de la palabra PILA, con lo que otorgamos a la pila un tamaño de 256 bytes (64 x 4 bytes de la palabra PILA = 256 bytes).

SEGMENTO DE CÓDIGO

Definición de todos los procedimientos e instrucciones del programa.

Un procedimiento es un conjunto de sentencias, las cuales se engloban entre la directiva PROC (PROCedure) y la directiva ENDP (END Procedure).

Ejemplo:

```
CODIGO SEGMENT ; Inicio del segmento de código
proc1 PROC FAR ; Inicio procedimiento principal proc1
    ASSUME CS:CODIGO,DS:DATOS,SS:PILA ; Asignar segmentos
    MOV AX,DATOS ; AX=Dirección del segmento de datos
    MOV DS,AX ; DS=AX. Indicar el segmento de datos
    LEA DX,MENSAJE ; DS:DX = dirección de MENSAJE
    MOV AH,9 ; Función DOS para escribir texto en pantalla
    INT 21H ; Llamar a la interrupción del DOS
    MOV AX,4C00H ; Función para terminar el programa
    INT 21H ; Volver al DOS
proc1 ENDP ; Fin del procedimiento proc1
CODIGO ENDS ; Fin del segmento código
END proc1 ; Empezar a ejecutar el procedimiento proc1
```

Los elementos del código pueden ser visualizados mediante cuatro columnas:

ETIQUETA
INICIO

OPERADOR
ADD

OPERANDOS
al,02h

COMENTARIOS
; Suma 02h al contenido de al

ETIQUETAS :

- Permiten caracteres alfanuméricos
- Máximo: 31 caracteres

OPERADOR :

- En esta columna se situarán las instrucciones. **El campo del código de operación es el único que nunca puede estar vacío**, éste siempre contiene una instrucción o una directiva del ensamblador.

OPERANDOS :

- El campo de operandos o de dirección puede contener una dirección o un dato, o puede estar en blanco.

COMENTARIOS (opcional):

- Este campo es opcional. Aquí se situará cualquier comentario personalizado. Estos son útiles para comprender qué hace un programa sin tener que descifrar el código entero. El compilador (ensamblador) ignorará todo texto más allá del carácter punto y coma ";" y hasta el fin de línea.

Juegos de Instrucciones

Las instrucciones del 8086/88 se pueden dividir en varios grupos:

- ✓ Instrucciones de transferencia de datos
- ✓ Instrucciones aritméticas
- ✓ Instrucciones lógicas
- ✓ Instrucciones de desplazamiento y rotaciones
- ✓ Instrucciones de E/S
- ✓ Instrucciones de control del flujo del programa
- ✓ Instrucciones de cadena de caracteres

Instrucciones de transferencia de datos

- ✓ Las instrucciones de transferencia de datos copian datos de un sitio a otro.
- ✓ La instrucción “**MOV**” realiza la transferencia de datos del operando de origen al destino.
- ✓ “**MOV**” admite todos los tipos de direccionamiento.
- ✓ Ambos operandos deben ser del mismo tamaño y **NO** pueden estar ambos en memoria.

Instrucciones aritméticas

Este tipo de instrucciones realizan operaciones aritméticas con los operandos.

Ejemplos:

“**ADD**” y “**ADC**” realizan la suma y la suma con acarreo (bit **CF** del registro de estado) de dos operandos, respectivamente, y guardan el resultado en el primero de ellos. Admiten todos los Modos de Direccionamiento.

“**MUL**” realiza la multiplicación del contenido de **AX** y del operando indicado, guardando el resultado en **AX**, para operaciones de **8 bits** y en **DX:AX** para operaciones de **16 bits**.

Instrucciones lógicas

Realizan las operaciones lógicas: **OR, XOR, AND, NOT, TEST, CMP**.

OR, XOR y AND realizan las operaciones lógicas “O”, “O exclusiva” e “Y”, respectivamente, de dos operandos, guardando el resultado en el primero de ellos. Estas operaciones se efectúan **bit a bit**. Admiten todos los Modos de Direccionamiento.

NOT realiza la operación de negado lógico de los bits del Operando (**CA₁**), guardando el resultado en el mismo operando.

Admite todos los Modos Direccionamiento, excepto el Modo Inmediato.

Instrucciones de Comparación

Estas instrucciones realizan funciones de comparación no guardando el resultado, pero si afecta al registro de estado (**no alteran los operandos**). Son muy útiles en las instrucciones de salto que se verán más adelante.

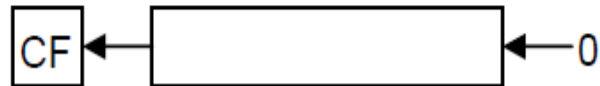
TEST realiza la operación lógica “Y” de dos operandos, pero **NO** afecta a ninguno de ellos, **SÓLO** afecta al registro de estado.

CMP realiza la resta de los dos operandos (como la instrucción **SUB**) pero **NO** afecta a ninguno de ellos, **SÓLO** afecta al registro de estado.

Instrucciones de Desplazamiento y Rotaciones

Realizan operaciones de desplazamiento y rotaciones de bits, y son: **SAL/SHL**, **SAR**, **SHR**, **ROL**, **ROR**, **RCL**, **RCR**.

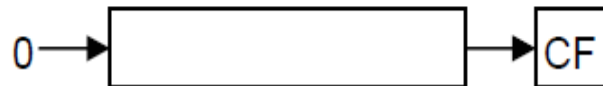
SAL/SHL realiza desplazamiento a la izquierda del primer operando tantos bits como indique el segundo operando, introduciendo un 0 y guardando el bit que sale en el bit CF del registro de estado.



SHL < D > , < c >

Corrimiento lógico de " c " bits a la izquierda

SHR realiza el desplazamiento a la derecha del operando, introduciendo un 0 y guardando el resultado en el bit CF del registro de estado.

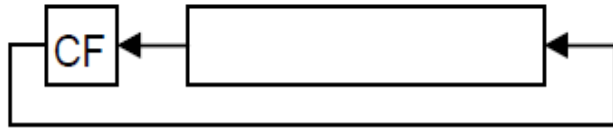


SHR < D > , < c >

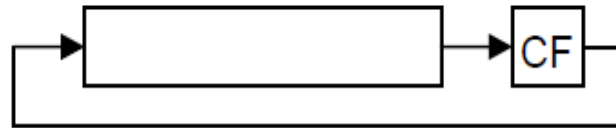
Corrimiento lógico de " c " bits a la derecha

Instrucciones de desplazamiento y rotaciones

RCL realiza la rotación a la izquierda de los bits del operando a través del bit CF (acarreo) del registro de estado.



RCR realiza la rotación a la derecha de los bits de operando a través del bit CF del registro de estado.



ROL realiza la rotación a la izquierda de los bits del operando, ignorando el bit CF del registro de estado, aunque en CF se almacena el bit que se rota.



Instrucciones de E/S

Se usan para la comunicación con los dispositivos periféricos. Y son **IN** y **OUT**.

IN lee de un puerto (sólo si la dirección del puerto es menor que 255).

OUT escribe en un puerto (sólo si la dirección del puerto es menor que 255).

Instrucciones de control del programa

Se utilizan para el control del programa, son instrucciones de salto, bucles y llamadas a procedimientos.

Las instrucciones de un programa se ejecutan unas tras otras, siguen una lógica secuencial, salvo que haya una instrucción de este tipo.

Instrucciones de salto

Estas instrucciones permiten saltar a otras partes del código.

Todas cambian el registro **IP** (contador de programa) y el registro **CS** (segmento de código) si es un salto lejano.

Un salto es lejano cuando la dirección a la que se salta no está en el mismo segmento de código.

Saltos Condicionales e Incondicionales

La mayoría de los programas constan de varios ciclos en los que una serie de pasos se repite hasta alcanzar un requisito específico y varias pruebas para determinar qué acción se realiza de entre varias posibles.

Requisitos como este implican la transferencia de control a la dirección de una instrucción que no sigue de inmediato de la que se está ejecutando actualmente. Una transferencia de control puede ser hacia adelante, para ejecutar una serie de pasos nuevos, o hacia atrás, para volver a ejecutar los mismos pasos.

Ciertas instrucciones pueden transferir el control fuera del flujo secuencial normal añadiendo un valor de desplazamiento al **IP**.

Dirección: Corta, Cercana y Lejana

Una operación de salto alcanza una **dirección corta** por medio de un **desplazamiento de un byte**, limitado a una distancia de **-128 a 127 bytes**.

Una operación de salto alcanza una **dirección cercana** por medio de un **desplazamiento de una palabra**, limitado a una distancia de **-32.768 a 32.767 bytes** **dentro del mismo segmento**.

Una **dirección lejana** puede estar en otro segmento y es alcanzada por medio de una **dirección de segmento y un desplazamiento**.

Etiquetas de Instrucciones

Las instrucciones **JMP**, **Jnnn** (salto condicional) y **LOOP** requieren un operando que se refiere a la etiqueta de una instrucción. El ejemplo siguiente salta a **A90**, que es una etiqueta dada a una instrucción **MOV**:

JMP A90

```
...  
A90: MOV AH, 00  
...
```

La etiqueta de una instrucción, tal como **A90:**, terminada con dos puntos (:) para darle atributo de cercana, esto es, la etiqueta esta dentro de un procedimiento en el mismo segmento de código.

Cuidado: Un error común es la omisión de los dos puntos. Note que una etiqueta de dirección en un operando de instrucción (como **JMP A90**) no posee el carácter de dos puntos.

La instrucción **JMP** (Salto incondicional)

Una instrucción usada comúnmente para la transferencia de control es la instrucción **JMP** (**J**ump, salto, bifurcación).

El salto es **incondicional**, ya que la operación transfiere el control bajo cualquier circunstancia. También **JMP** vacía el resultado de la instrucción previamente procesada; por lo que, un programa con muchas operaciones de salto puede perder velocidad de procesamiento.

El formato general para **JMP** es:

| [etiqueta] | **JMP** | dirección corta, cercana o lejana |

INSTRUCCIONES DE SALTO CONDICIONAL

El ensamblador permite usar una variedad de instrucciones de salto condicional que transfieren el control dependiendo de las configuraciones en el registro de banderas. Por ejemplo, puede comparar dos campos y después saltar de acuerdo con los valores de las banderas que la comparación establece.

El formato general para el salto condicional es:

| [etiqueta:] | Jnnn | dirección corta |

La instrucción LOOP

La instrucción **LOOP**, requiere un valor inicial en el registro CX.

En cada iteración, LOOP de forma automática disminuye 1 de CX. Si el valor en el CX es cero, el control pasa a la instrucción que sigue; si el valor en el CX no es cero, el control pasa a la dirección del operando. La distancia debe ser un **salto corto**, desde -128 hasta +127 bytes. Para una operación que exceda este límite, el ensamblador envía un mensaje como "salto relativo fuera de rango". El formato general de la instrucción LOOP es:

| [etiqueta:] | LOOP | dirección corta |

Ejemplo: “Si... entonces... sino”

Pseudocódigo:

```
SI  AH = BL
    AL = 01h
SINO
    AL = 00h
FIN SI
```

Assembler:

```
MOV AH,[0204h] ; Cargamos AH con el contenido de la dirección 0204h
MOV BL, [0A20h] ; Cargamos BL con el contenido de la dirección 0A20h
CMP AH,BL      ; Compara los contenidos de AH y BL
JNE ET01       ; Salta a “ET01” si los valores comparados son distintos
MOV AL,01h     ; Cargamos AL con el valor 01h si son iguales
JMP ET02       ; Salta a “ET02” para evitar la instrucción siguiente
ET01: MOV AL,00h ; Cargamos AL con el valor 00h si son distintos
ET02: .....    ; Continúa la ejecución del programa
```

Tipos de instrucciones del i8086)

Tipo de instrucción	Efecto
Instrucciones de transferencia	Mueven información entre registros, registros y posiciones de memoria, o entre registros y puertos de entrada/salida
Instrucciones aritméticas	Realizan operaciones aritméticas: sumas, restas, etc.
Instrucciones de manejo de bits	Realizan operaciones de desplazamiento, rotación y lógicas sobre registros o posiciones de memoria
Instrucciones de transferencia de control	Sirven para controlar la ejecución de las instrucciones del programa
Instrucciones de entrada salida	Mueven información entre registros y puertos de entrada/salida
Instrucciones de manejo de cadenas	Realizan operaciones sobre cadena de bytes o palabras
Instrucciones de interrupción	Provocan que el microprocesador realice un servicio que se le solicita diferente a las instrucciones que está ejecutando

Instrucciones de transferencia de datos (I)

- **Nombre:** MOV
- **Formato:** MOV destino, origen
- **Descripción:**
Transfiere un byte o una palabra desde el operando origen al operando destino
- **Ejemplos:**
 - MOV CX, 112h ; CX = 112h
 - MOV BL, 255 ; BL = 255 = FFh
 - MOV ES, AX ; ES = AX
 - MOV AL, 12h ; AL = 12h
 - MOV PAL_MEM, BX ; PAL_MEM = BX

Instrucciones de transferencia de datos (II)

- **Nombre:** PUSH
- **Formato:** PUSH origen
- **Descripción:**
Decrementa el puntero de pila (SP) en 2 y luego transfiere la palabra que se ha especificado en el operando origen a lo alto de la pila
- **Ejemplo:**
 - PUSH BX ; Pone BX en la pila

Instrucciones de transferencia de datos (y III)

- **Nombre:** POP
- **Formato:** POP destino
- **Descripción:**
Transfiere un byte o una palabra desde la cima de la pila al operando destino y luego incrementa la pila en 2
- **Ejemplo:**
 - POP BX ; Pone en BX el contenido de la cima de la pila

CATEDRA DE ARQUITECTURA DE COMPUTADORAS

INTERRUPCIONES

INTERRUPCIONES

Definición:

- ✓ Una interrupción es un evento no programado que altera el flujo normal de ejecución de una tarea.
- ✓ La rutina de interrupción se ejecuta y generalmente regresa el control al procedimiento que fue interrumpido, el cual entonces puede continuar con su ejecución de manera normal.
- ✓ Son un método del que disponen los dispositivos e incluso los procesos para hacer notar a la CPU la aparición de alguna circunstancia que requiera su intervención

Funcionamiento del Mecanismo de Interrupciones

El mecanismo de la interrupción es simple:

- ✓ **En un momento determinado se activa la entrada de la interrupción al procesador.**
- ✓ **El procesador suspende entonces la ejecución del programa en curso, pasando el control al programa de interrupción.**
- ✓ **Este programa tratará la causa de la interrupción, desactivará el dispositivo que la ha producido y devolverá el control al programa principal.**
- ✓ **El programa principal es transparente a este proceso y no tiene ninguna indicación de que se ha producido, salvo, por supuesto, por el tiempo de proceso transcurrido.**

- ✓ Una función que ejecutan todas las interrupciones al activarse el programa de interrupción es “**salvar**” los registros del procesador para evitar que sean modificados y alteren el funcionamiento del programa suspendido.
- ✓ Estos registros se restauran justo antes de volver al programa principal. Éste tiene, sin embargo, la posibilidad de bloquear las interrupciones mediante instrucciones que activan o desactivan una bandera de interrupción (**bandera IF**).
- ✓ Tal posibilidad es necesaria cuando el programa principal accede a datos o periféricos que son también utilizados por las interrupciones, pues se pueden producir situaciones erróneas cuando los dos (proceso e interrupción) intenten utilizar los mismos recursos simultáneamente.

La mayoría de los procesadores disponen de una entrada especial de interrupción que no puede ser impedida por software:

- ✓ Se trata de la **NMI (Non-Maskable Interrupt)**.
- ✓ Esta entrada es necesaria para aplicaciones de alta prioridad y urgencia (como es el caso de un fallo en la alimentación) en los que el procesador tiene escaso tiempo para detener los procesos y salvar los datos vitales de la máquina

Vectores de Interrupción

- ✓ **La familia IBM PC, como todos los ordenadores basados en la familia de microprocesadores Intel 8086/8088, se controla en gran medida a través de la utilización de interrupciones, las cuales pueden ser generadas por Hardware y Software.**
- ✓ **Las rutinas de servicios del BIOS no son una excepción y cada una de ellas tiene asignada un número de interrupción al que se puede invocar cada vez que se quiera usar ese servicio.**

Vectores de Interrupción

- ✓ **Cuando se produce una interrupción, el control del ordenador se transfiere a una subrutina de tratamiento de la interrupción que a menudo está almacenada en la ROM del sistema (una rutina de servicio del BIOS no es más que una rutina de tratamiento de interrupción).**
- ✓ **A esta rutina de tratamiento de la interrupción se le llama cargando su dirección de segmento y desplazamiento en los registros CS e IP, respectivamente.**
- ✓ **Las direcciones de segmentos que ubican las rutinas de tratamiento de la interrupción se llaman vectores de interrupción.**

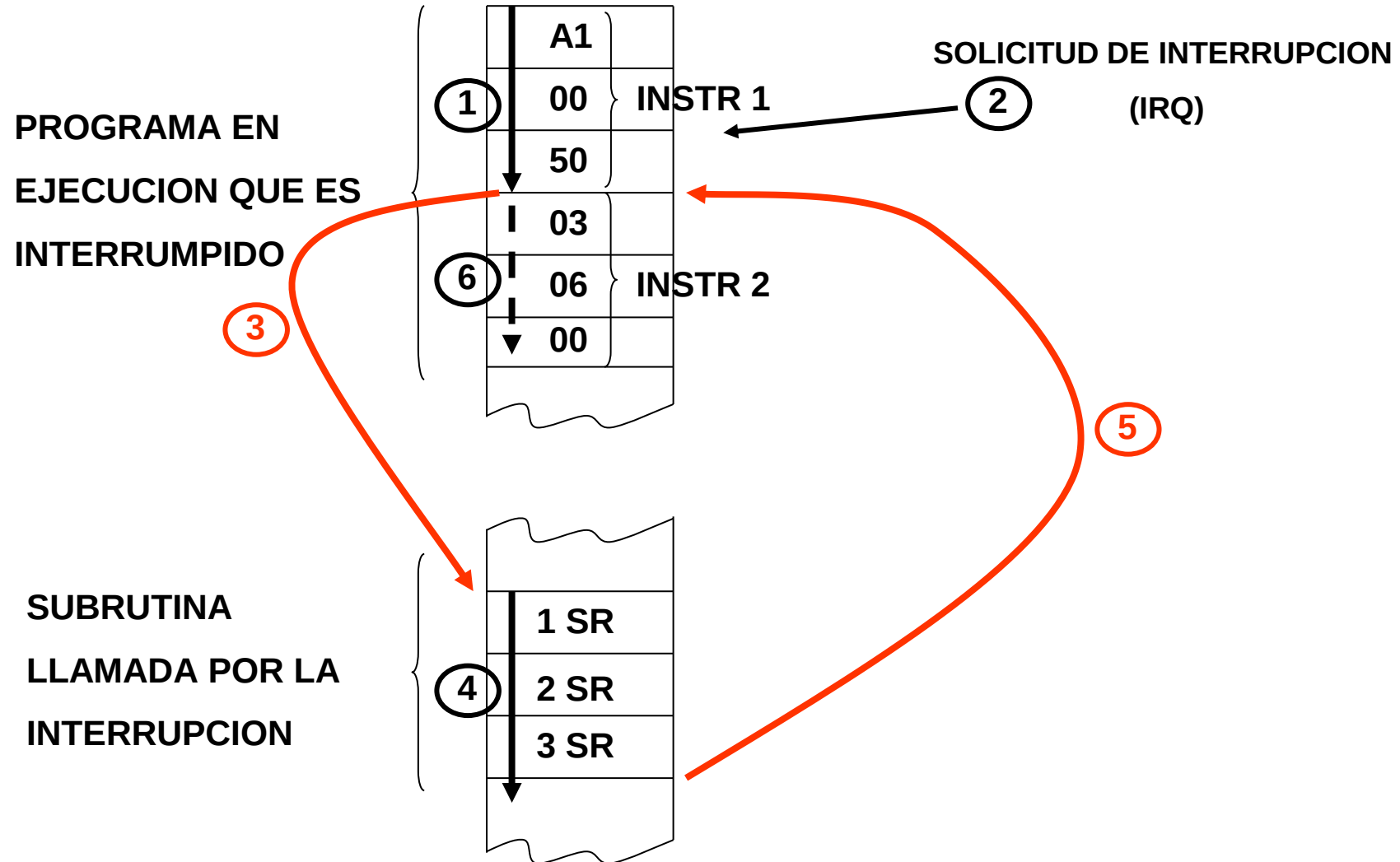
Funcionamiento del Mecanismo de Interrupciones

- ✓ Un procesador que no tenga un **Controlador de Interrupciones** integrado, suele tener una única línea de interrupción llamada habitualmente **INT**.
- ✓ Esta línea es activada por el **Controlador de Interrupciones** cuando tiene una interrupción que servir. Al activarse esta línea, el procesador consulta los registros del **Controlador de Interrupciones** para averiguar que **IRQ** debe atender.
- ✓ A partir del **Número del IRQ** busca en la tabla de vectores de interrupción la dirección de la rutina a la que debe llamar para atender la petición del dispositivo asociado a dicha **IRQ**.

Procesamiento de una Interrupción

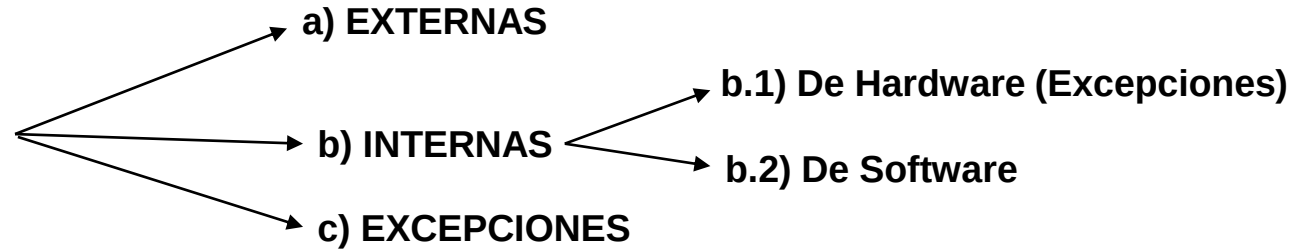
1. **Terminar** la ejecución de la instrucción máquina en curso.
2. **Salvar** el valor del contador de programa (**IP**) y de los **registros internos** en la pila, de manera que en la CPU, al terminar el proceso, pueda seguir ejecutando el programa a partir de la última instrucción con el mismo estado que poseía previo a la interrupción.
3. La CPU **salta** a la dirección donde está almacenada la rutina de servicio de interrupción (**Interrupt Service Routine**, o abreviado **ISR**) y ejecuta esa rutina que tiene como objetivo atender al dispositivo que generó la interrupción.
4. Una vez que la rutina de la interrupción termina, el procesador **restaura** el estado que había guardado en la pila en el paso 2 y retorna al programa que se estaba usando anteriormente.

Cómo se Ejecuta una Interrupción



Tipos de Interrupciones

TIPOS DE INTERRUPCIONES:



a) INTERRUPCIONES DE HARDWARE EXTERNAS: (a la UCP)

Las generan los dispositivos de E/S o sensores que son periféricos de la CPU. Estas interrupciones son asíncronas, pues nunca se sabe con exactitud cuándo van a producirse.

b) INTERRUPCIONES INTERNAS: (Síncronas, no depende en absoluto de circunstancias externas al programa)

b.1) De Hardware (Excepciones)

Ocurren en el interior de la U.C. si existe algún problema mientras se ejecuta una instrucción (código de operación desconocido, división por cero, overflow, etc.) Cada una genera una interrupción distinta y el momento exacto en que se produce depende de cada instrucción.

b.2) Simuladas ó de Software

Son generadas por instrucciones del lenguaje incluidas en el código de un programa para simular una interrupción. (instrucción “**INT**”)

Tabla 8-6. Diferencias entre distintos tipos de interrupciones.

Externas o hardware: son convocadas en forma asincrónica, no dependen del programa en ejecución.	No enmascarables (NMI) La CPU es avisada por una señal de control llamada comúnmente NMI (<i>Non Mascarable Interrupt</i>).	Siempre son atendidas. Se consideran de máxima importancia, pues las provoca el hardware ante acontecimientos que no pueden ser por lo menos "avisados" (p. ej., la caída de tensión de la alimentación).
	Enmascarables La CPU es avisada por otra señal que la diferencia de la NMI.	Se consulta la señal de interrupción por cada ciclo de ejecución de instrucción. Como no siempre debe ser atendida, se consulta también una bandera de estado de habilitación de interrupciones. Si la bandera lo autoriza, entonces la CPU suspende de manera momentánea la ejecución de programa activo y ejecuta el servicio de interrupción. Una función hardware externa, determina la prioridad que tiene esta interrupción respecto de otras peticiones pendientes y le da curso (p. ej., recepción de un byte en un puerto). Si la bandera de interrupción está desactivada, la CPU no la tiene en cuenta y continúa la ejecución del programa activo; de ahí deriva el nombre de "mascable" o "enmascarable".
Internas o software: son convocadas por el programa.		Se pueden producir por la ejecución de una instrucción específica dentro de un programa, para solicitar una interrupción que cumpla con alguna función determinada. La forma de convocarla es INT #, donde # corresponde al número que identifica la interrupción.
Excepciones: son provocadas como consecuencia de anomalías que se producen y detectan durante la ejecución del programa y a causa de ella.	Faltas o errores	Son las que se pueden detectar y corregir antes de que se produzca la ejecución de una instrucción determinada (p. ej., se produce una falla al invocar una página que no está en memoria principal). La atención de esta interrupción provoca que el sistema operativo localice la página faltante y la cargue.
	Trampas	Son las que se detectan una vez ejecutada la instrucción que las provoca (p. ej., las que de alguna manera incorpora el usuario en el programa, como sobreflujo u <i>overflow</i>).
	Abortos	Son las que se detectan sin localizar la instrucción que las provoca, abortando la ejecución del programa (p. ej., un valor no válido en un registro de sistema).

Servicios de Interrupciones

- ✓ La **BIOS** (chip que contiene el **Sistema Básico de Entrada-Salida**) posee las rutinas necesarias para que, un programa, se comuniquen con los dispositivos sin tener que conocer la electrónica.
- ✓ La principal operación (para que un programa pueda acceder a la BIOS) es la **INT** (que lleva un argumento) que se relaciona con la operación a efectuar. Las más importantes son:
 - **10h**: manejador de la Terminal de Video.
 - **13h**: manejador de Discos.
 - **16h**: manejador de Teclado.
 - **17h**: manejador de Impresora.
 - **21h**: funciones del DOS (**Sistema Operativo de Discos**).

EJEMPLOS

- **POSICIONAMIENTO DEL CURSOR DE PANTALLA:**

AH: 02h

BH: página de Video donde se realiza la operación.

DH: fila de la pantalla donde se ubicará el cursor.

DL: columna.

Ejemplo: **MOV AH , 02h**

MOV BH , 0 ; página 0

MOV DH , 5 ; fila 5

MOV DL , 3 ; columna 3

INT 10h ; Posiciona, el cursor (de página 0), en fila 5 y columna 3.

- **ESCRIBIR UN CARACTER EN PANTALLA:**

AH: 0Ah

AL: código ASCII del carácter a mostrar.

BH: página de Video donde se realiza la operación.

CX: cantidad de veces a mostrar el carácter.

Ejemplo: **MOV AH , 0Ah**

MOV AL , 164 ; código ASCII correspondiente al carácter “ñ”

MOV BH , 0 ; página 0

MOV CX , 5 ; lo va a escribir 5 veces seguidas

INT 10h ; Escribe cinco " ñ " a partir de donde está el cursor.

Máquinas Virtuales

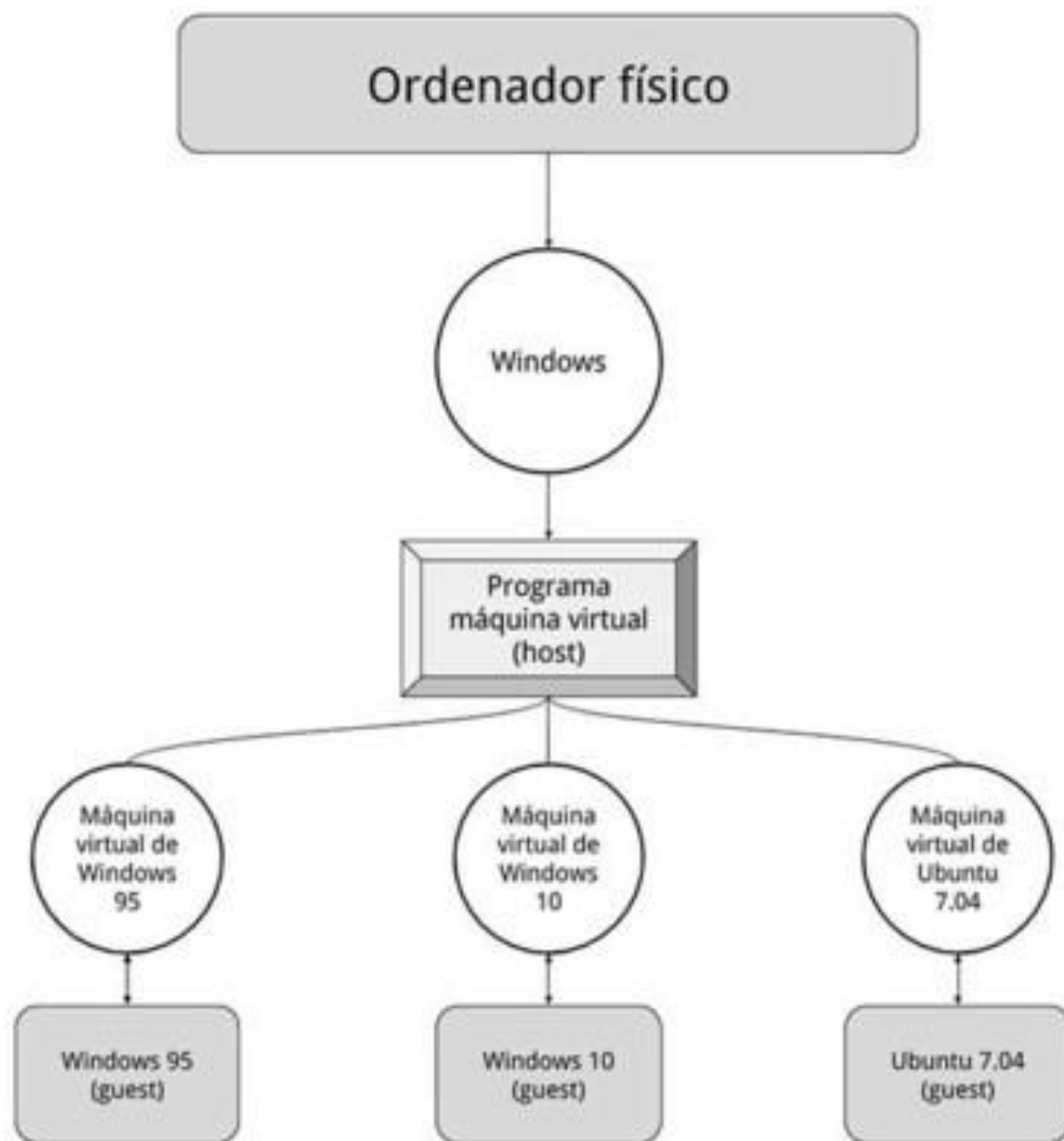
- Una máquina virtual es un archivo de PC, que suele denominarse **“imagen”**.
- Se comporta igual que un equipo real.
- Equivale a crear un equipo dentro de un equipo.
- Se ejecuta en una ventana, igual que cualquier otro programa, y permite que el usuario final tenga la misma experiencia en una máquina virtual que tendría en el propio sistema operativo host.

- ✓ La máquina virtual se sitúa en un espacio aislado del resto del sistema.
- ✓ Esto crea un entorno ideal para probar otros sistemas operativos, incluidas versiones beta, acceder a datos infectados por virus, crear copias de seguridad de sistemas operativos y ejecutar software o aplicaciones en sistemas operativos para los que no se habían creado inicialmente.

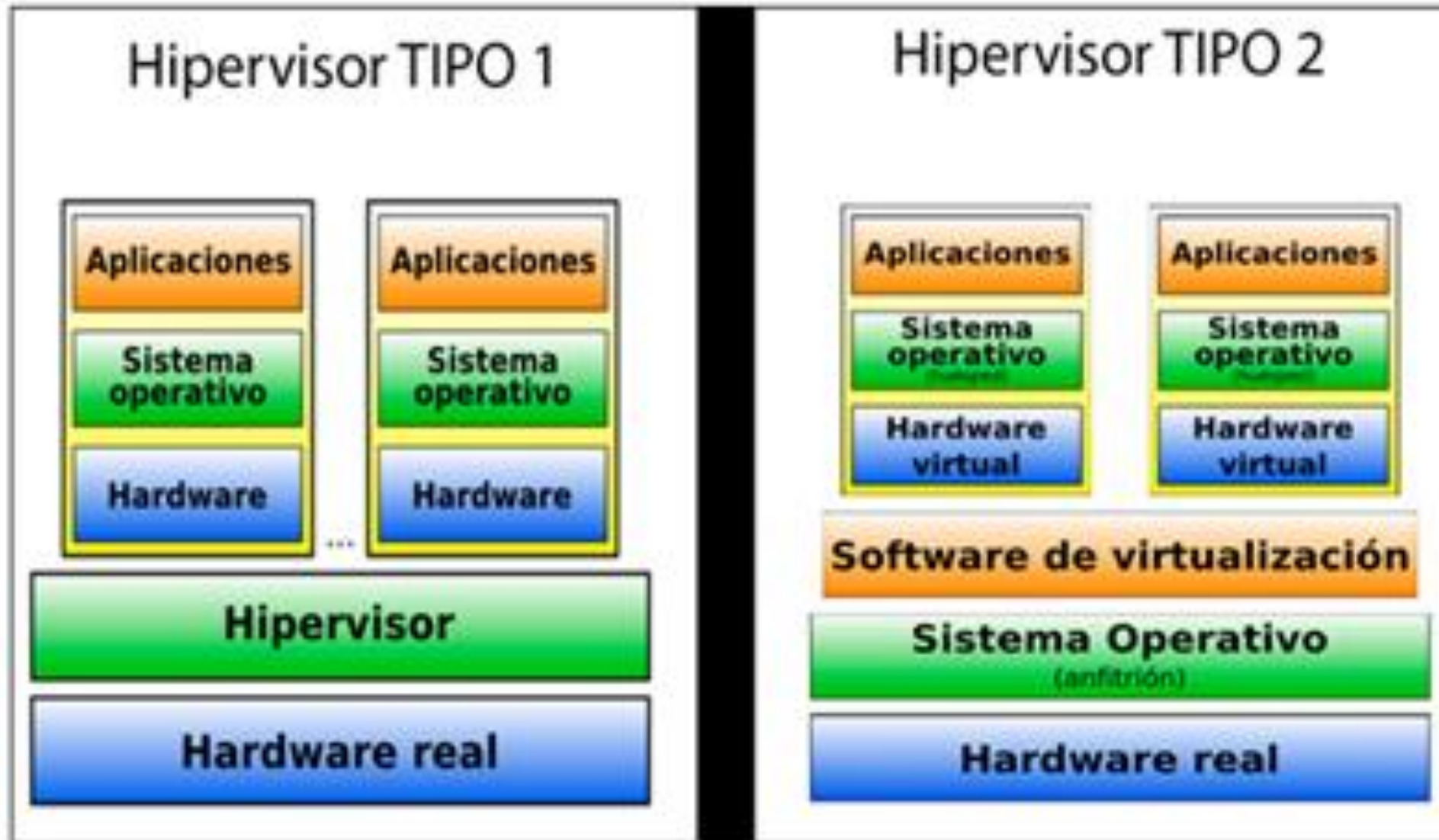
- Se pueden ejecutar varias máquinas virtuales a la vez en un mismo equipo físico.
- Para servidores, los diversos sistemas operativos se ejecutan en paralelo con un fragmento de software denominado hipervisor para administrarlos, mientras que los equipos de escritorio suelen implementar un solo sistema operativo para ejecutar los demás sistemas operativos en ventanas de programa.

- Cada máquina virtual proporciona su propio hardware virtual, incluidas las CPU, memoria, unidades de disco duro, interfaces de red y otros dispositivos.
- El hardware virtual se asigna después al hardware real de la máquina física, lo que permite ahorrar costos, porque reduce la necesidad de tener sistemas de hardware físico, con los costos de mantenimiento que conllevan, y también reduce la demanda de alimentación y refrigeración.

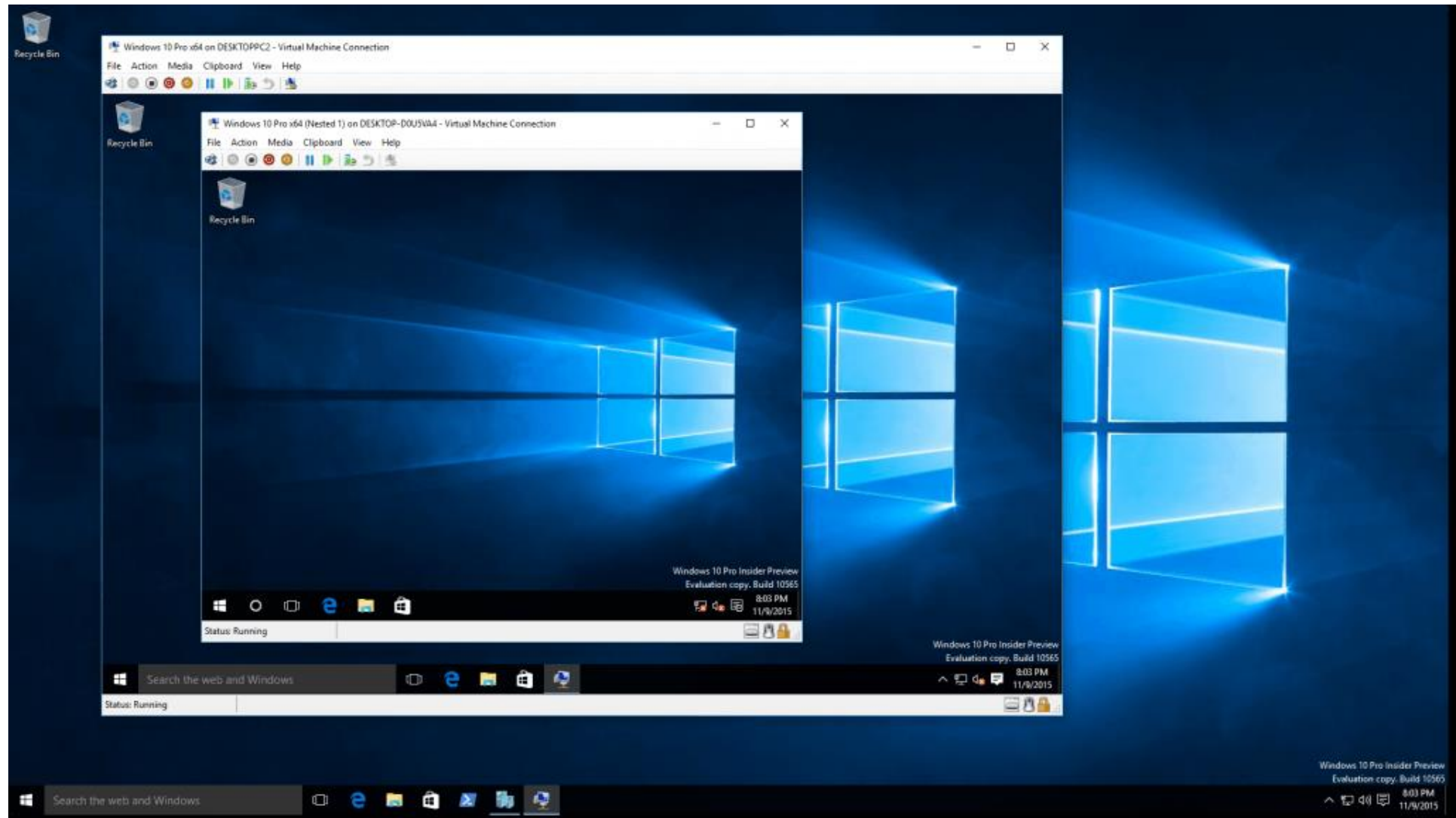
- Para el sistema operativo que se ejecuta dentro de la máquina virtual toda esta emulación es transparente e invisible.
- Todo funciona igual a si se estuviera ejecutando en un PC normal, sin que sepa que en verdad está metido dentro de una burbuja dentro de otro sistema operativo.



Hipervisor vs Virtualización



Windows 10... dentro de Windows 10... dentro de Windows 10



- En su burbuja, la máquina virtual no puede acceder al resto de datos de la máquina anfitrión a pesar de estar físicamente funcionando en la misma, están aisladas.
- No obstante, las principales aplicaciones de máquinas virtuales como **VirtualBox** o **VMWare** disponen de atajos y herramientas para facilitar la tarea de pasar archivos de una máquina a otra.

CATEDRA DE ARQUITECTURA DE COMPUTADORAS

MICROCODIGO

INTRODUCCIÓN

- ✓ **Microcódigo o Microprograma es un lenguaje de menor nivel que el código de máquina y permite la representación de las microoperaciones organizadas de manera secuencial para la ejecución de las instrucciones.**
- ✓ **La eficiencia del microcódigo establece la cantidad de operaciones invertidas en la ejecución de una instrucción, puede ser que una CPU realice una multiplicación en muchas menos operaciones que otra y esto mejora la velocidad de ejecución, independientemente del ritmo que impone el reloj.**

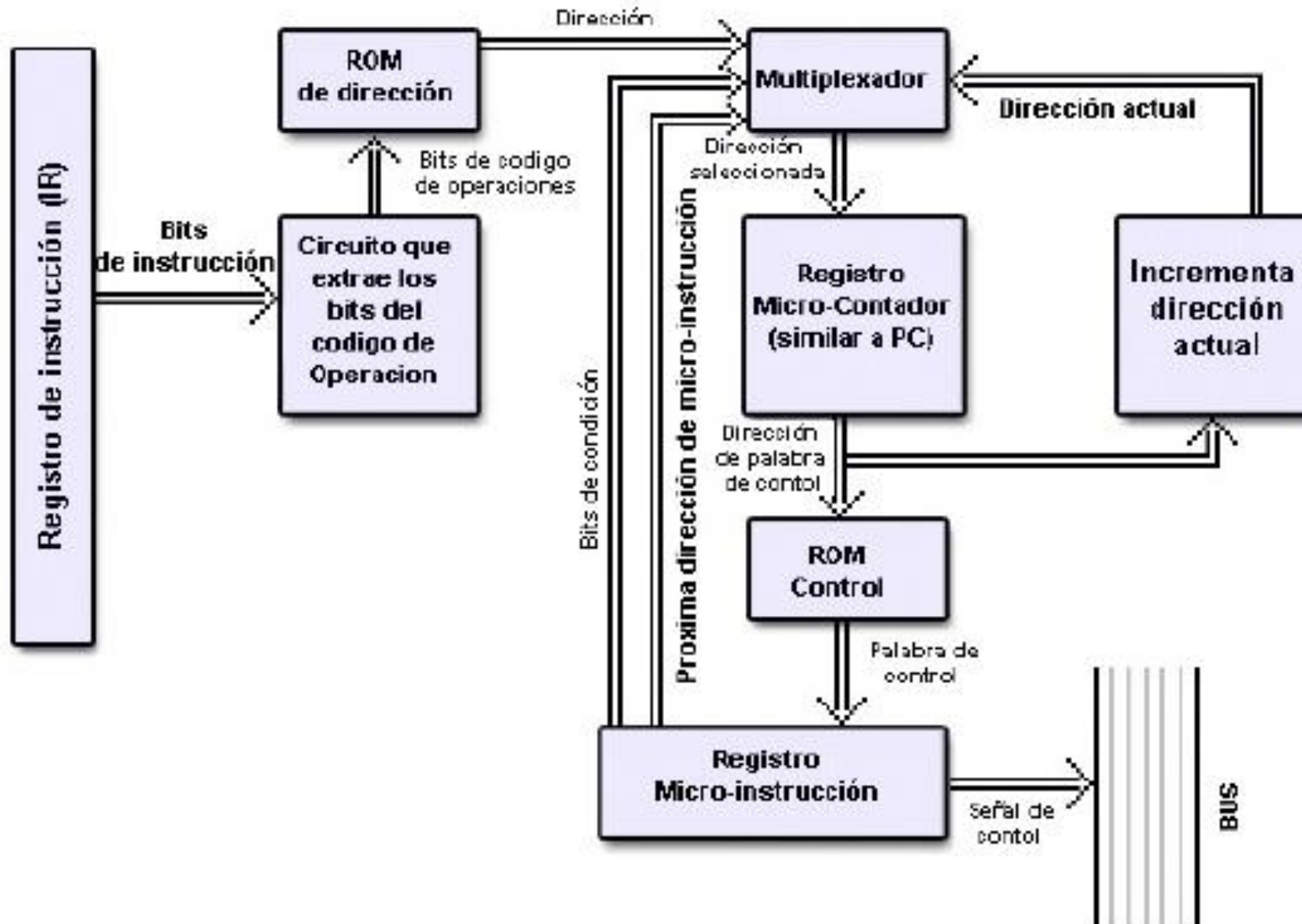
MICROINSTRUCCIONES

- ✓ La Unidad de Control parece un dispositivo en principio sencillo.
- ✓ Implementar una U.C. como interconexión de elementos lógicos básicos no es una tarea fácil.
- ✓ Todo el hardware que permita control y secuenciamiento de microoperaciones es difícil de diseñar y verificar, además de ser relativamente inflexible. (Por ej. Si uno deseara modificar el diseño para agregar una nueva instrucción de máquina)
- ✓ Como alternativa surgió la idea en los procesadores CISC de realizar U.C. microprogramadas.

- ✓ **Una secuencia de microinstrucciones comprende un Microprograma o Firmware (que se encuentra a mitad de camino entre hardware y software).**
- ✓ **Es más fácil diseñar en firmware que en hardware.**
- ✓ **La ventaja que aporta el uso de la microprogramación para implementar una U.C. es que simplifica su diseño. Lo que permite una implementación más económica y menos propensa a errores.**

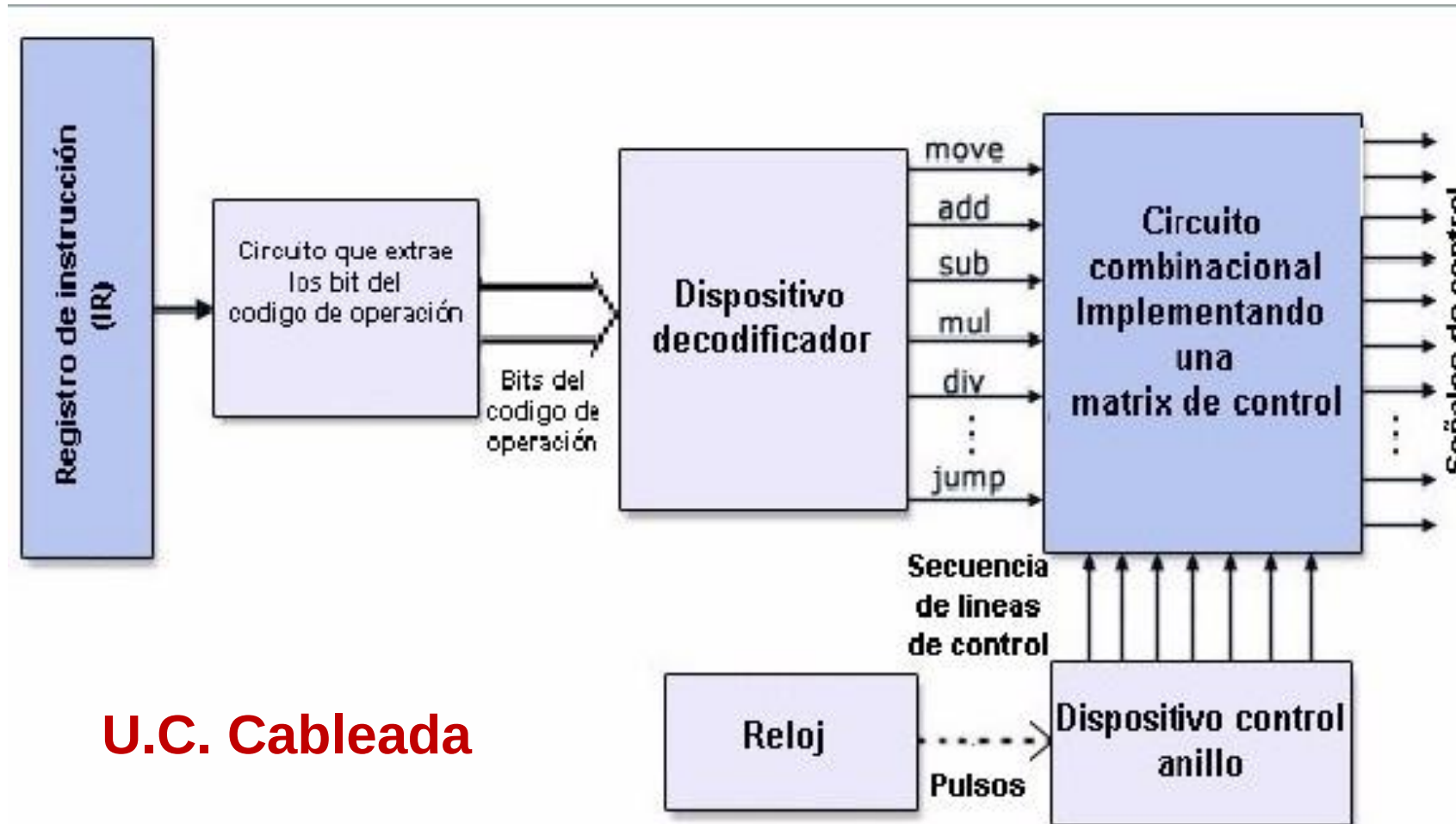
- ✓ **El principal inconveniente de una U.C. microprogramada es que será algo más lenta que una unidad cableada de tecnología comparable.**
- ✓ **A pesar de ello la microprogramación es la técnica dominante para implementar U.C. en las arquitecturas CISC puras, debido a su facilidad de implementación.**
- ✓ **En cambio los procesadores RISC, dado que tienen un formato de instrucción más sencillo emplean normalmente U.C. cableadas.**

U.C. MICROPROGRAMADA



El control cableado consiste de una máquina de estados y un circuito combinacional.

Es un autómata que se mueve de un estado a otro dependiendo de la entrada recibida.



U.C. Cableada

- ✓ **Los Microprogramas consisten en una serie de microinstrucciones. Estas microinstrucciones controlan la CPU a un nivel fundamental de circuitos de hardware.**
- ✓ **Por ejemplo, una sola microinstrucción típica podría especificar las siguientes operaciones:**
 - ✓ **Conecte Registro 1 de la parte "A" de la ALU**
 - ✓ **Conecte Registro 2 de la parte "B" de la ALU**
 - ✓ **Establezca la ALU para realizar sumas en complemento a dos**
 - ✓ **Establecer entrada de acarreo de la ALU a cero**
 - ✓ **Guarde el valor del resultado de Registro 8**

CODIFICACIÓN HEXADECIMAL DE INSTRUCCIONES

✓ Convenciones utilizadas:

“D”: Bit de Dirección de Registro

- D=1 : REG = OPERANDO DESTINO (REG -> REG)
- D=0 : REG = OPERANDO FUENTE

“W”: Bit de Tamaño del Dato

(Especifica si la operación será realizada sobre media palabra o palabra completa)

- W=0 : DATOS DE 8 BITS
- W=1 : DATOS DE 16 BITS

“MOD”: Indica el Modo de Direccionamiento

- 00 : MODO MEMORIA SIN DESPLAZAMIENTO
- 01 : MODO MEMORIA CON DESPLAZAMIENTO 8 BITS
- 10 : MODO MEMORIA CON DESPLAZAMIENTO 16 BITS
- 11 : MODO REGISTRO

CODIFICACIÓN HEXADECIMAL DE INSTRUCCIONES

✓ Convenciones utilizadas:

“REG”: Utilizado para identificar los Registros (operando " destino ",
excepto cuando " [..] " está del lado destino :

REG	W=0 (8 Bits)	W=1 (16 Bits)
000	AL	AX
001	CL	CX
010	DL	DX
011	BL	BX
100	AH	SP
101	CH	BP
110	DH	SI
111	BH	DI

EJEMPLO

MOV BX , 018DH (MODO INMEDIATO A REGISTRO 16 BITS)

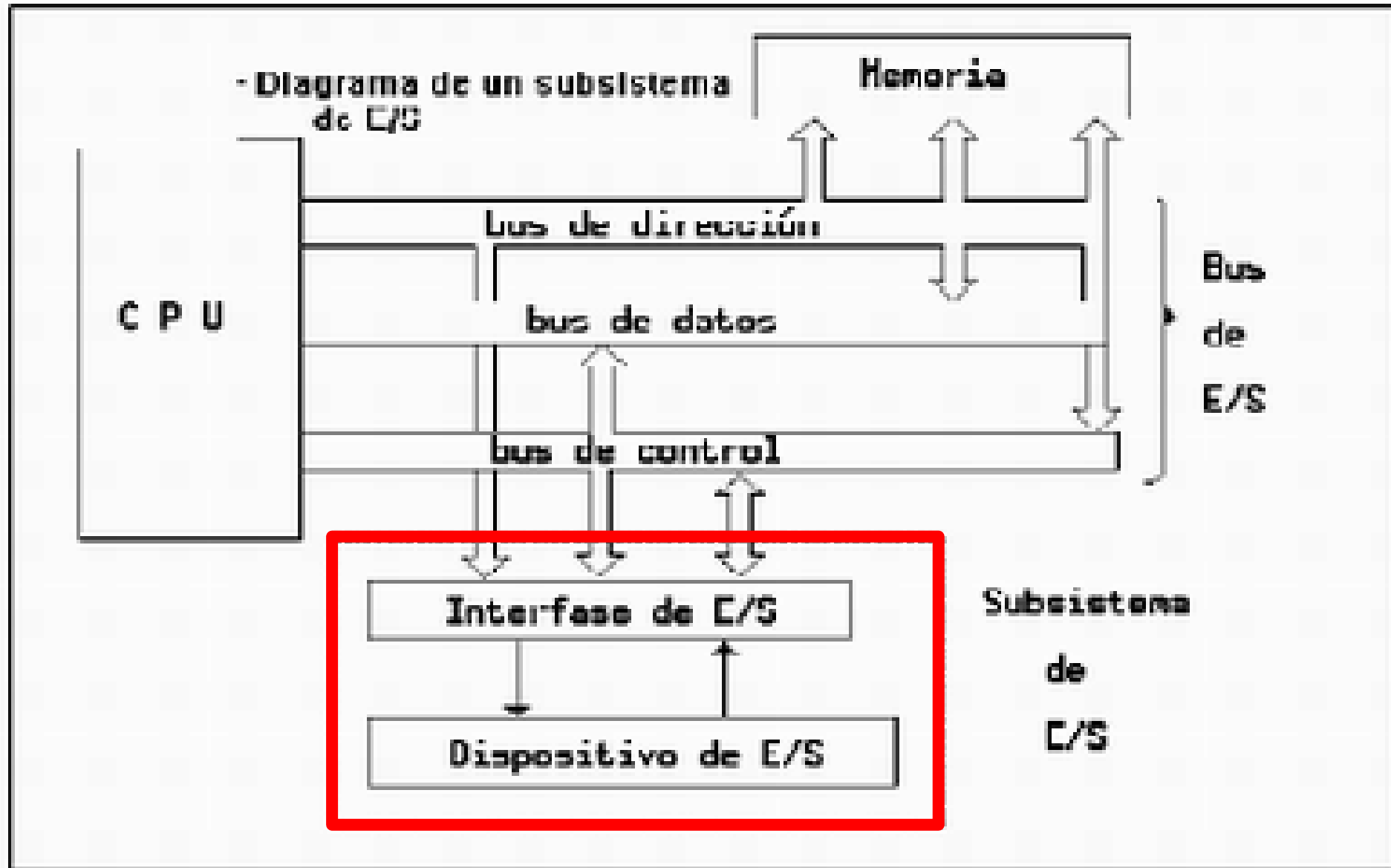
En la cartilla:

1011	W	REG	DATO	DATO (si W=1)	=> CODIF. HEXADECIMAL= BB018D
1011	1	011	00000001	10001101	
B		B	0 1	8 D	

CATEDRA DE ARQUITECTURA DE COMPUTADORAS

INTERFAZ DE ENTRADA / SALIDA

INTERFAZ DE E/S



INTRODUCCIÓN

- Un subsistema de E/S consiste en interfases de E/S y dispositivos periféricos.
- La interfaz de E/S controla la operatoria de los dispositivos conectados a ella. Las operaciones de control (por ejemplo rebobinado, posicionamiento, etc.) se arrancan mediante comandos emitidos por la CPU.
- **Periférico:** Es cualquier elemento capaz de realizar un intercambio de datos con la CPU o la Memoria.

FUNCIONES DE LA INTERFAZ DE E/S

- ✓ **Almacenar los datos y realizar las conversiones que se le requieran.**
- ✓ **Detectar errores en la transmisión y ser capaz de reiniciar la transacción en estos casos.**
- ✓ **Testear, arrancar y detener el dispositivo según las directivas impartidas por la CPU.**
- ✓ **Realizar el Control y Temporización del Flujo de Datos.**
- ✓ **Establecer la Comunicación con la CPU**
 - **Decodificación de comandos**
 - **Intercambio de los datos de E/S con la CPU**
 - **Informar el estado del dispositivo**
 - **Reconocimiento de la dirección del dispositivo**

PROBLEMÁTICA

La cantidad y variedad de dispositivos es muy grande, y el **Sistema de E/S** debe proveer soluciones para los siguientes problemas:

- ✓ **Direccionamiento:**

Coexisten varios periféricos, y las operaciones de lectura, escritura y control se deben realizar en forma independiente.

- ✓ **Velocidades diferentes:**

Los periféricos pueden tener un rango de velocidades muy amplio.

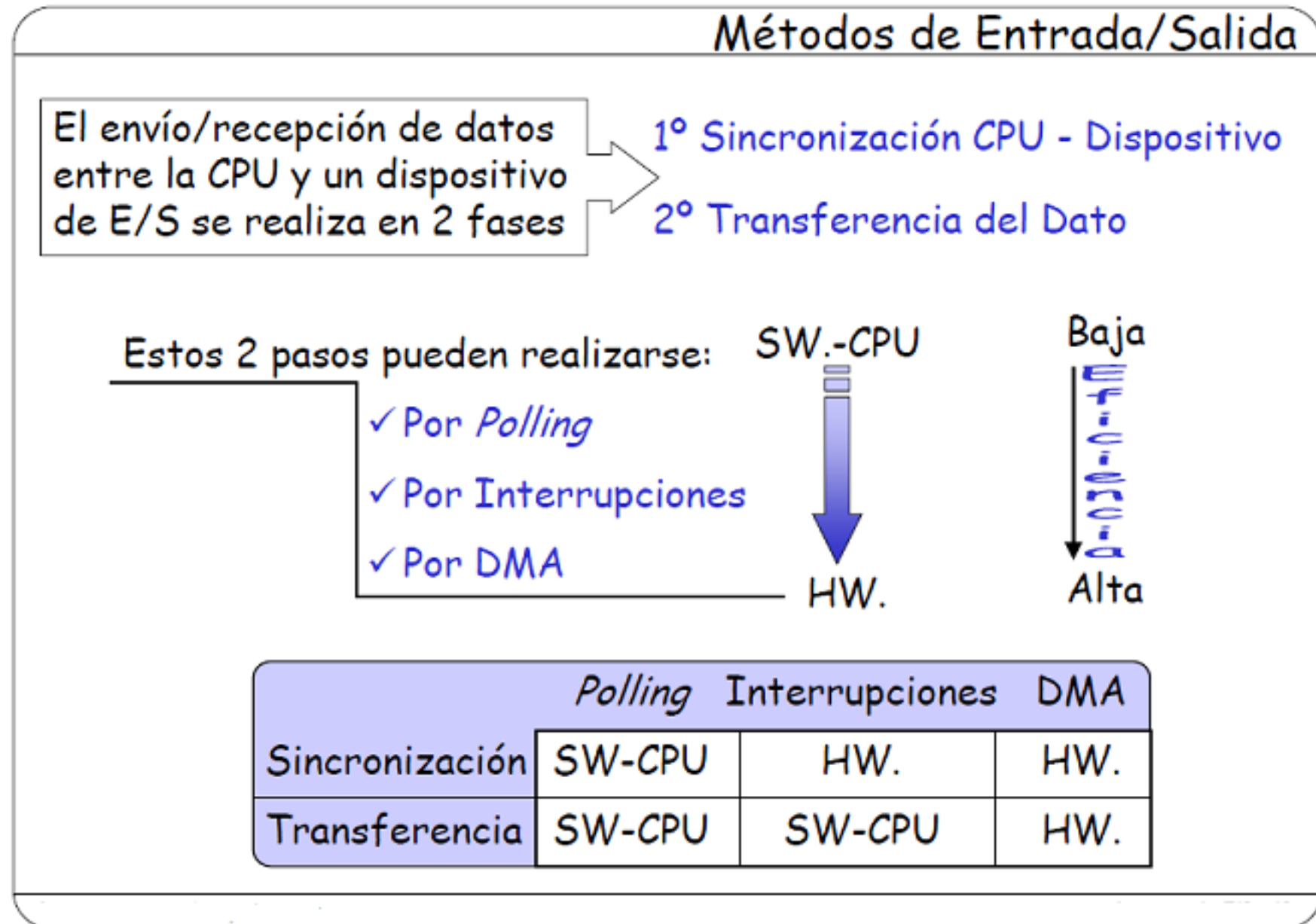
- ✓ **Acceso lectura/escritura:**

Existen periféricos de sólo lectura, de escritura y de lectura/escritura.

TRANSFERENCIA ELEMENTAL Y OPERACIONES DE E/S

- ✓ Los modos de operación con la E/S son diferentes en función de que se trate de:
- ✓ **Transferencia Elemental:** Intercambio de una única unidad de información con el periférico.
 - ✓ Es necesario establecer una comunicación física entre el periférico y el resto del sistema y a continuación llevar a cabo el control del periférico mediante la consulta y modificación de su estado.
- ✓ **Operación de E/S:** Transferencia de un conjunto de datos.
 - ✓ Aparte de contar el número de datos hay que sincronizar las velocidades de los periféricos con el resto del sistema, donde difiere del caso anterior y detectar posibles errores mediante códigos de paridad o polinomiales, y si es preciso repetir la transmisión.

FORMAS DE REALIZAR LA TRANSFERENCIA DE E/S



Formas de Realizar la Transferencia de E/S

El envío o recepción de un dato hacia / desde un dispositivo de E/S se debe realizar en dos pasos:

1 - SINCRONIZACION CPU-DISPOSITIVO

Ambos realizan tareas distintas y a velocidades distintas

2 - TRANSFERENCIA DEL DATO

Existe tres métodos principales de E/S:

➤ **Pooling (Encuesta):** (Es la forma mas sencilla)

El sincronismo entre CPU y dispositivos se consigue sondeando a cada uno de ellos.

CPU

1. Leer Registro de Estado del controlador
3. Comprobar estado.
Si no preparado, ir a Paso 1
4. Escribir dato en el reg. de datos del controlador
8. Si hay más datos para enviar, ir a Paso 1

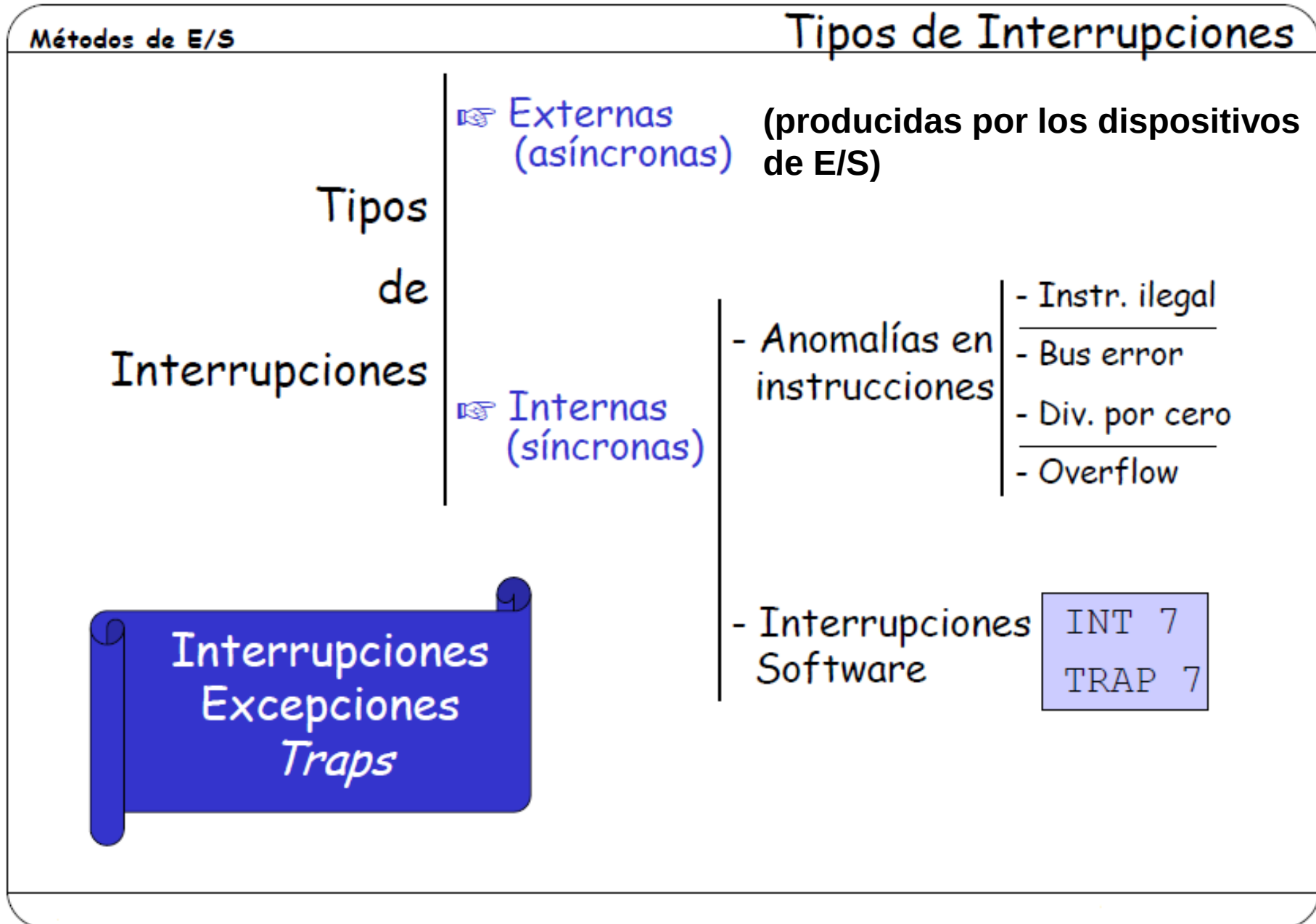
Controlador

2. Enviar estado
5. Aceptar dato y poner estado a "No preparado"
6. Esperar a que el dato se escriba en el dispositivo
7. Poner estado a "Preparado"

PROBLEMAS

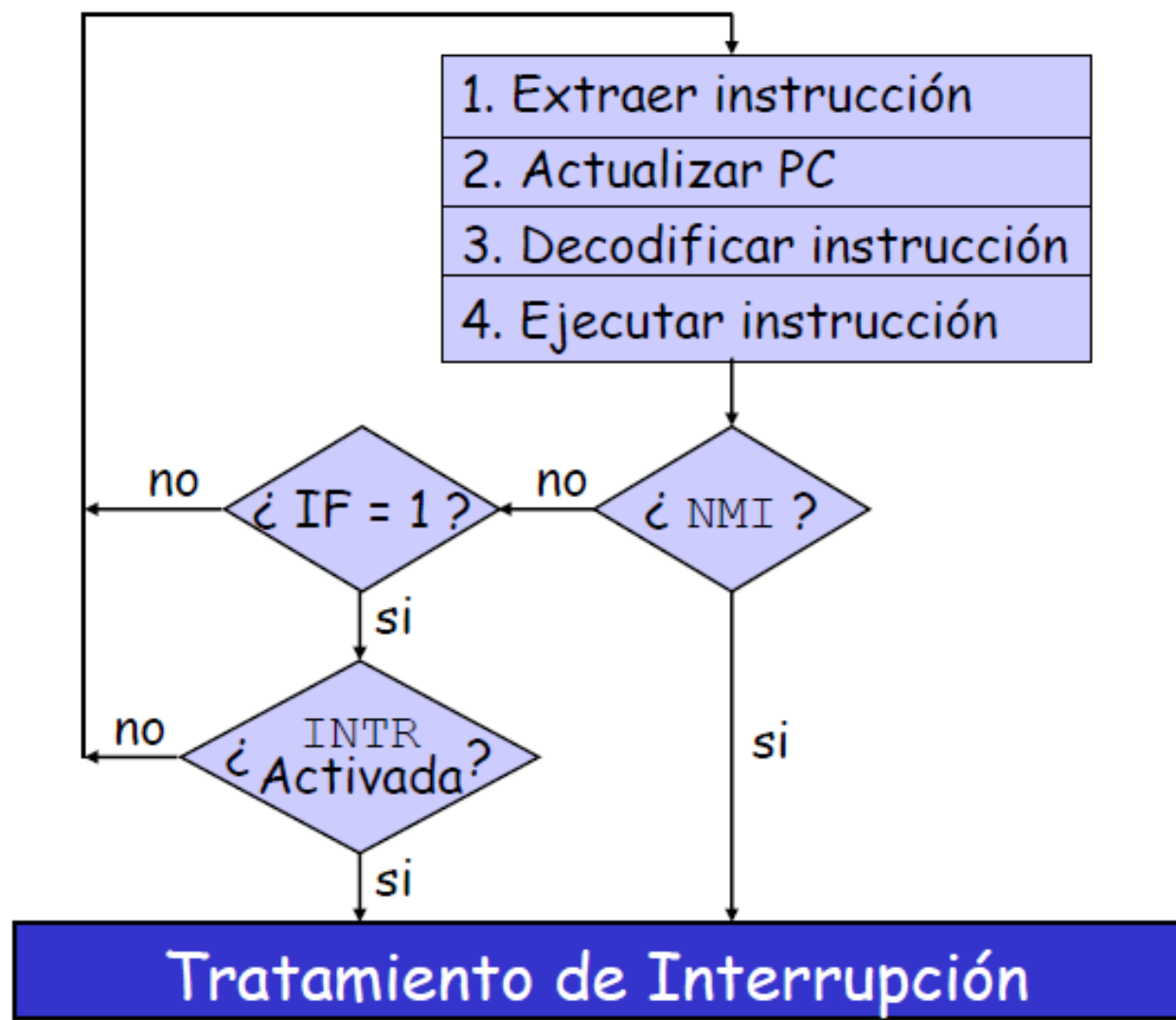
- ✓ Pérdida de Tiempo (Espera activa)
- ✓ Lentitud (si no se atiende pronto)

➤ Por Interrupciones:



¿Siempre hay que atender a las interrupciones?

Interrupciones	<u>No</u> <u>Enmascarables</u>	- <i>Reset</i> (RESET) - Error de Bus (BERR) - NMI (NMI)
	<u>Enmascarables</u>	ENABLE DISABLE ↓ Flag en Reg. Estado

Aceptación

➤ **Por DMA (Direct Memory Access):**

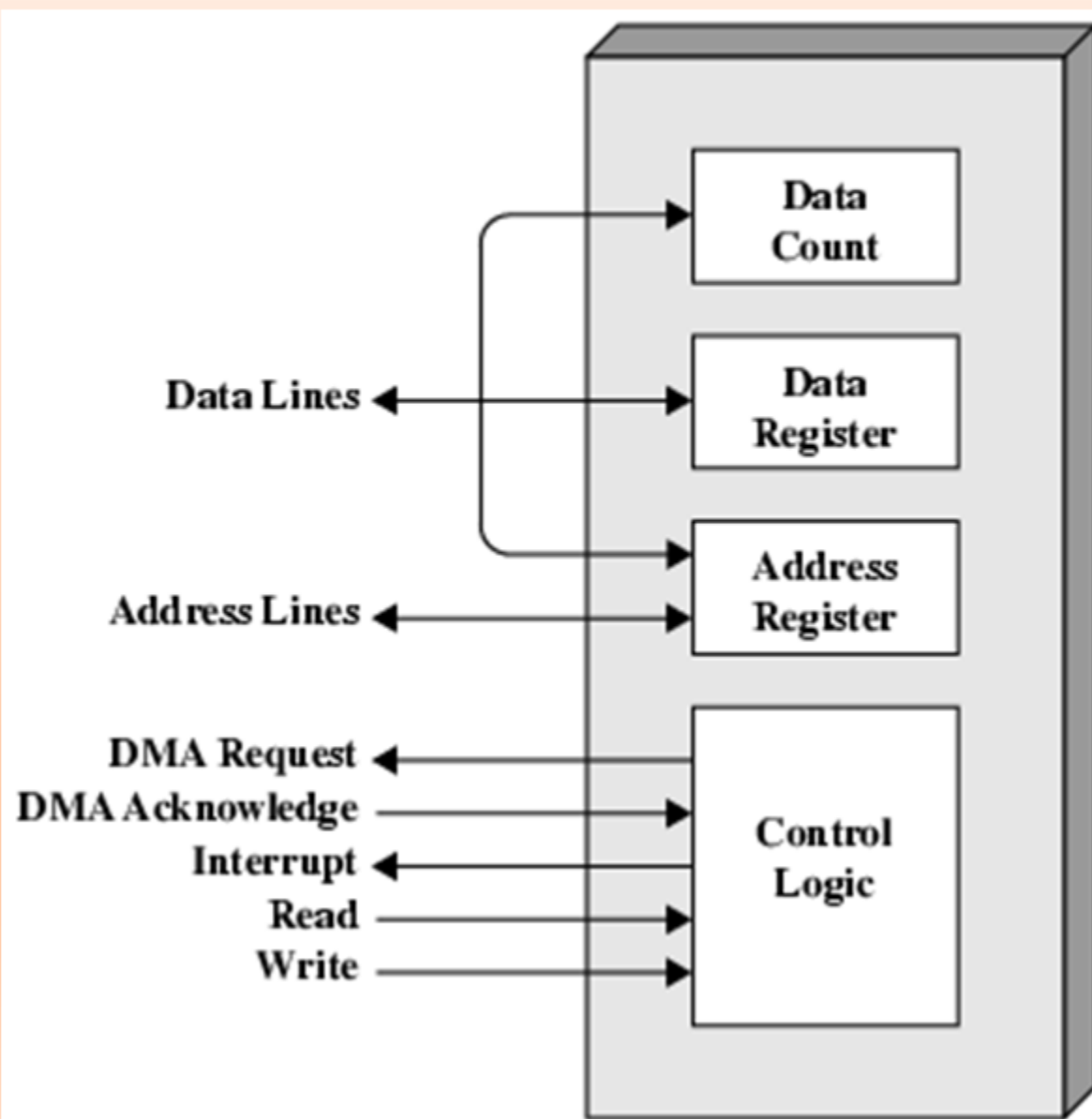
El periférico se comunica directamente con la MP sin intervención de la CPU, esto acelera el proceso.

Requiere de un CONTROLADOR DMA (especialmente adecuado para transferencias de Bloques de Datos)

Para poder realizar operaciones de E/S por DMA el controlador debe conocer:

- ✓ **Dirección del periférico**
- ✓ **Rango de memoria donde se realiza la operación**
- ✓ **Tipo de operación a Realizar (Lectura o Escritura)**
- ✓ **Número de operaciones elementales que van a integrar esa operación de E/S**

Diagrama de controlador DMA



MODO BLOQUE (BURST)

Cuando el **DMA** toma posesión del bus, se realiza la transferencia de un bloque de un gran número de palabras de datos, y la **CPU** queda en estado de **HALT** o **STOP** al no poder utilizar los buses.

Al finalizar la transferencia, el controlador de **DMA** envía una interrupción a la **CPU**, con lo que esta sale del estado **STOP**, atiende la interrupción y continúa la ejecución del programa que se interrumpió al ceder el bus al **DMA**.

Se utiliza con dispositivos de memoria secundaria, como los discos, donde las transferencias de datos no se pueden detener o ralentizar (pérdida de información).

Transferencias de datos más rápidas a costa de tener la **CPU detenida** (sin ejecutar instrucciones) mientras se realiza la transferencia.

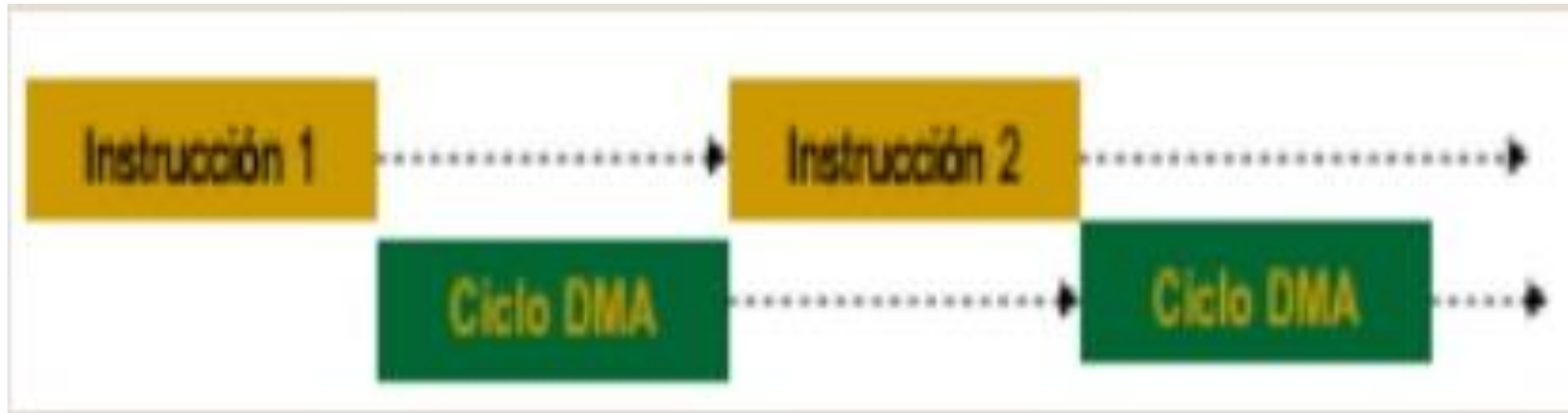
ROBO DE CICLO

Alternativa al modo Bloque, Cuando se le **cede el control del bus al DMA**, se le permite que lo utilice para **transferir una única palabra de datos**, una vez finalizada la transferencia debe devolver el control del bus al **CPU**.

Con este sistema, la transferencia de bloques de datos tiene que realizarse mediante **una serie de ciclos de bus de DMA** intercalados con **ciclos de bus de la CPU**.

Este modo reduce la velocidad de transferencia respecto al **Modo Bloque**, pero también reduce la interferencia que se le causa a la **CPU**.

ROBO DE CICLO

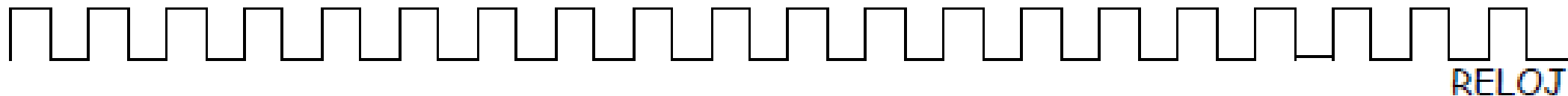


Ambas actividades, la del procesador y la del controlador, **se realizan en paralelo**, aunque es evidente que ambas resultan ralentizadas al tener que **repartirse** el tiempo de posesión del bus.

BUS TRANSPARENTE

La interferencia ocasionada por los métodos anteriores puede eliminarse completamente si se diseña una interfaz de **DMA** que **solamente utilice ciclos de bus cuando la CPU no lo necesita**, ya que esta no requiere la posesión del bus el 100% del tiempo de ejecución de una instrucción.

El bus puede quedar **libre** cuando la CPU está **decodificando una instrucción**, cuando está **calculando la dirección de los operandos o del resultado**, o cuando está **utilizando la ALU**.



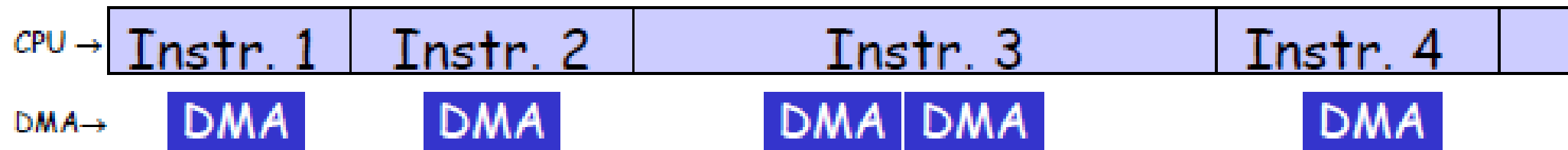
Transferencia de Bloque



Robo de Ciclo



Bus Transparente



La Evolución del Funcionamiento de la E/S

1. **La CPU controlaba directamente al periférico.** Esta situación se observa en los dispositivos controlados por microprocesadores.
2. **Se añade un controlador o módulo de E/S.** La CPU utiliza E/S programada sin interrupciones (**Polling**). De esta forma, la CPU se independiza de los detalles específicos de las interfaces de los dispositivos externos.
3. Se utiliza la misma configuración de la etapa 2, pero **ahora se emplean interrupciones**. La CPU no necesita esperar a que se realice la operación de E/S, incrementándose la eficiencia.
4. El módulo de E/S tiene acceso directo a memoria a través del **controlador DMA**. Permite transferir un bloque de datos hacia o desde memoria sin utilizar CPU.
5. El módulo de E/S se mejora haciendo que **se comporte como un procesador en si**, con un repertorio especializado de instrucciones orientados a las operaciones de E/S.
6. **El módulo de E/S tiene una memoria local propia y es, de hecho, un computador en si mismo.** Con esta arquitectura, permite controlar un conjunto grande de dispositivos con la mínima intervención de la CPU.

E/S - Conclusiones

En toda transferencia se utilizan señales de control, datos y dirección, esta vez sobre los buses de E/S.

Las señales de control y tiempo se utilizan para regular la transferencia elemental, indicando como y cuando debe ocurrir.

O sea que permiten contestar las siguientes preguntas:

- ✓ Puede producirse una transferencia ahora?
- ✓ Que tipo de transferencia se va a realizar?
- ✓ Cuando comienza?
- ✓ Cuando termina?
- ✓ Se realizó con éxito?

Estas preguntas nos permiten deducir que las funciones de la "lógica o inteligencia" de un bus de E/S son:

- ✓ Comunicarse con el periférico y el sistema CPU-memoria.**
- ✓ Controlar la temporización durante la transferencia.**
- ✓ Almacenar temporalmente bits para "paliar" la diferencia de velocidad entre emisor y receptor.**
- ✓ Detectar si se produjeron errores durante la transferencia.**

Cuando la actividad del bus de E/S es “síncronica” se utilizan señales de **clock** que regulan la transferencia, por lo tanto, la **frecuencia de clock** es un parámetro a tener en cuenta para la velocidad de transferencia.

Un bus síncronico requiere que los dispositivos conectados a él, que se encuentren “**sintonizados**” a esa frecuencia y que todas las actividades se produzcan en intervalos de tiempo fijos.

Cuando la actividad del bus es “asíncronica”, las señales de **clock** no regulan su operación.

En este caso, su velocidad depende de los tiempos de los dispositivos conectados a ellos.

Tipos de señales diferentes comunes a la mayoría de los buses de E/S para **"controlar"** la transferencia:

- ✓ **Señal de clock**, que permite sincronizar el ciclo del bus con la operación de la CPU (CLK).
- ✓ **Señal que puede habilitar un tiempo de espera.**
- ✓ **Señal de lectura o escritura** a un dispositivo de E/S o a la memoria (IOR o IOW).
- ✓ **Señales de interrupción a la CPU (IRQn).**
- ✓ **Señales de reconocimiento (aceptación o requerimiento)**, que permiten el dialogo de los dispositivos durante la transferencia (p. ej., la técnica de handshaking o "apretón de manos").
- ✓ **Señal de bus cedido u ocupado.**

Las Señales de Dirección permiten representar la dirección del emisor y el receptor (A_n = address “sub” n).

Cada dispositivo de E/S tiene **asociada una combinación binaria**.

Por ejemplo, cuando se envía una transferencia al receptor, este "**reconoce**" su dirección tomando los bits A_n del bus y decodificándolos.

O sea que las **señales de dirección** permiten contestar las preguntas:

¿De quien? y ¿A quien?

Las Señales de Datos representan los bits del mensaje que se debe transferir

$$(D_n = \text{data "sub" } n)$$

El “**ancho de bus**” o bien la cantidad de líneas afectadas a la transferencia elemental dan una medida de potencial de trabajo:

A mayor cantidad de bits transferidos en paralelo => Mayor capacidad de transferencia.

Las unidades de comunicación con el bus son unidades hardware, que actúan de intermediarias en la comunicación:

“CPU - memoria interna” => “controlador – periférico”

Resuelven, fundamentalmente, el problema de disparidad en los tiempos de operación entre las unidades que conectan.

Se pueden agrupar en:

- ✓ **Interfaces paralelo.**
- ✓ **Interfaces serie.**
- ✓ **Controlador DMA (o sólo DMA's).**
- ✓ **Canal o procesador de entrada-salida (Input Output Processor o IOP).**

- ✓ La elección del tipo de intermediario tiene que ver con las características de cada dispositivo que se ha de conectar.
- ✓ En un mismo sistema pueden existir todos o algunos de ellos.
- ✓ A su vez, cada intermediario puede estar asociado con más de un dispositivo.

❖ Controladores

El término controlador se utiliza en gran medida para definir cualquier unidad hardware que "**gobierne**" a otra.

Un controlador de periférico es un dispositivo asociado en forma directa al periférico, que puede estar físicamente integrado a él, o bien separado de éste, y está constituido por:

- ✓ Un **buffer interno (memoria RAM)**, que permite el almacenamiento de la información que "viaja" desde o hacia el soporte.
- ✓ Una **lógica de control**, que interpreta comandos del periférico, genera señales para su ejecución y gobierna así la unidad.

Ej: Controlador de Disco, Controlador de Impresora, etc.

❖ Controladores

Su función se puede resumir en, al menos, dos operaciones fundamentales:

- ✓ Aislar el software de servicio de entrada/salida, que se "ocupa" de la transferencia de los detalles específicos del hardware del periférico y los convierte en invisibles.
- ✓ Compatibilizar la velocidad del periférico respecto de la del resto del sistema.
- ❑ Otros controladores, cuya función permite gobernar actividades del sistema, liberan a la CPU de ciertas tareas que pueden llevarse a cabo de manera independiente.
- ❑ Cuando un controlador se involucra con una transferencia, la CPU recibe parámetros de un programa que ejecuta a través de una interfaz, este programa forma parte de la administración de entrada/salida y se denomina **manejador de dispositivo (device driver)**.

❖ Interfaces

Una interfaz es un hardware que actúa de nexo entre un periférico o un adaptador y el bus. Sirve principalmente para adecuar las señales y preparar la transferencia elemental basada en un protocolo (p.ej., un byte).

No tiene capacidad suficiente para tomar la responsabilidad de la transferencia completa (**bloque**), la CPU o el canal son los que asumen el control de una transferencia completa.

Tipos:

✓ *Interfaz Paralela*

✓ *Interfaz Serie*

Interfaz Paralela



Puerto paralelo de impresora

- **Interfaz Paralela**

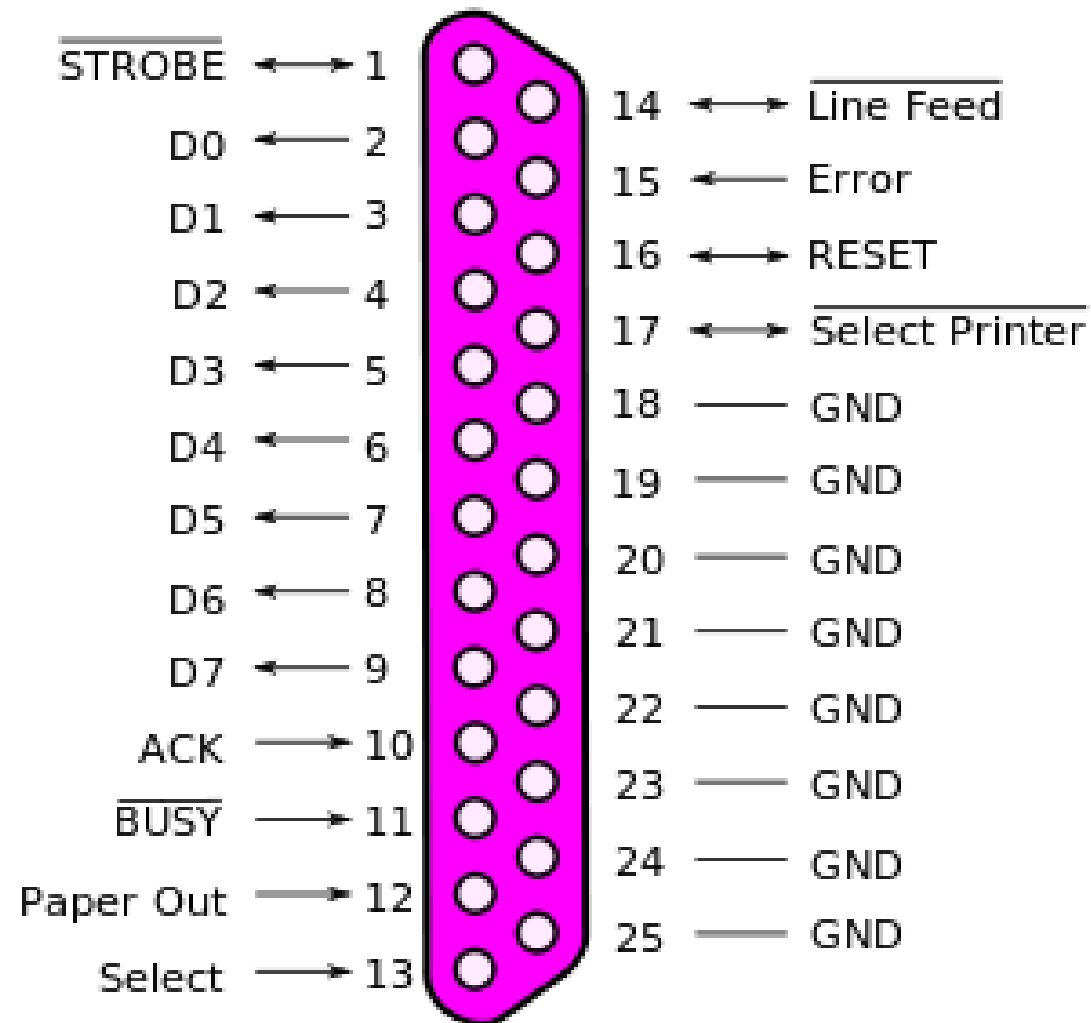
Es un dispositivo hardware que permite el control de la transferencia “en paralelo” entre el bus de sistema y un periférico.

Está asociada a una lógica de direccionamiento que permite establecer que esa interfaz ha sido seleccionada.

La interfaz cuenta con registros denominados **ports**. Un **port** está dividido en partes, por ejemplo, registro de dato (o port de dato) y registro de control (o port de control).

El registro de control tiene una función doble:

- ✓ En primer lugar, recibe un comando que puede enviar al periférico.
- ✓ En segundo lugar, recibe señales de control de estado; o sea que cada uno de sus bits se puede interpretar como indicadores de estado o flags que informen acerca de la transferencia.



Distribución de entradas y salidas en el conector DB25 para el puerto paralelo.

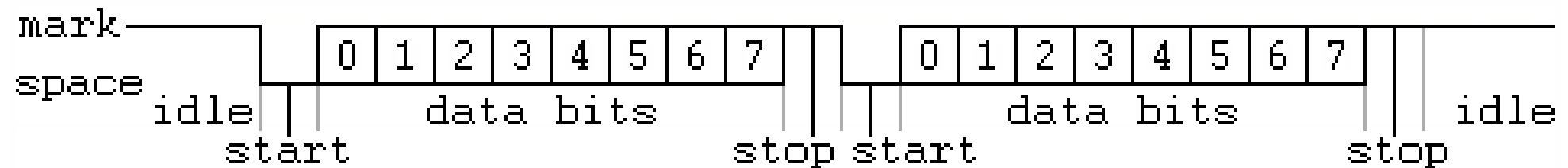
Los pines del puerto paralelo con conector DB25 son:

Pin No (DB25)	Pin No (36 pin)	Nombre de la señal	Dirección	Registro - bit	Invertidas
1	1	Strobe	E/S	Control-0	Si
2	2	Data0	Salida	Data-0	No
3	3	Data1	Salida	Data-1	No
4	4	Data2	Salida	Data-2	No
5	5	Data3	Salida	Data-3	No
6	6	Data4	Salida	Data-4	No
7	7	Data5	Salida	Data-5	No
8	8	Data6	Salida	Data-6	No
9	9	Data7	Salida	Data-7	No
10	10	Ack	Entrada	Status-6	No
11	11	Busy	Entrada	Status-7	Si
12	12	Paper-Out	Entrada	Status-5	No
13	13	Select	Entrada	Status-4	No
14	14	Linefeed	E/S	Control-1	Si
15	15	Error	Entrada	Status-3	No
16	16	Reset	E/S	Control-2	No
17	17	Select-Printer	E/S	Control-3	Si
18-25	18-25	Tierra	-	-	-

Interfaz Serie



Puerto Serie



- **Interfaz Serie**

Es un dispositivo hardware que permite el control de la transferencia de bits “en serie” entre el bus y un dispositivo de E/S.

Está asociado con una lógica de direccionamiento que permite establecer que esa interfaz fue seleccionada por el procesador para la transferencia.

Los registros de la interfaz constituyen el denominado puerto serie.

La interfaz se inicializa colocando un byte en su registro de control, que establece el tipo de transferencia.

En el **modo salida** la interfaz recibe un dato enviado por la CPU a través del bus de datos; este dato se almacena en un registro de recepción relacionado con un registro de desplazamiento, que provoca la serialización de la unidad de información para transferir hacia el dispositivo.

- **Interfaz Serie**

En el **modo entrada** otro registro de desplazamiento recibe la información bit tras bit desde el dispositivo y la coloca a disposición de la CPU cuando la unidad de información está completa en el registro.

La interfaz serie acepta las modalidades sincrónica y asincrónica; sin embargo, cuando los datos se transmiten en una línea desde un punto al otro, se produce un retardo de la señal en el medio de transmisión, que provoca incertidumbre acerca de dónde termina un bit y dónde empieza el otro.

Además, otra desventaja con respecto a la transmisión paralela es que la transmisión serie crea la dificultad de la delimitación de los caracteres; esto es, al enviarse los bits uno tras otro, no se sabe dónde empieza y dónde termina un carácter.

La técnica utilizada para resolver el problema depende de que la transmisión sea sincrónica o asincrónica.

CATEDRA DE ARQUITECTURA DE COMPUTADORAS

TRADUCTORES DE LENGUAJE

Traductores de Lenguaje

- Son programas cuya función es convertir los programas escritos en **lenguaje simbólico** a **lenguaje de máquina**.
- Programa escrito en **Lenguaje simbólico = Código Fuente**
- Programa en **Lenguaje de Máquina = Código Ejecutable**



•Traducción:

- Comprende el análisis del léxico y la sintaxis de cada instrucción o sentencia, así como las referencias lógicas a las que apunta cada instrucción.
- Si del análisis surgen errores => se generará un informe indicando:
 - * Lugar donde se produjo
 - * Tipo de error cometido

•Traducción:

- Para realizar el análisis Sintáctico se controla cada sentencia del **Programa Fuente:**
 - Nombres de datos
 - Signos de puntuación
 - Palabras propias del lenguaje que tengan una construcción aceptada por el traductor

Se destacan tres tipos de traductores de lenguaje:

- **ENSAMBLADORES**
- **INTÉRPRETES**
- **COMPILADORES.**

ENSAMBLADORES

Se encarga de traducir un **archivo fuente** escrito en un lenguaje **Assembler** a un archivo cuyas instrucciones estén en **Código Máquina**, que es ejecutable directamente por el procesador para el que se creó.

La razón por la que se creó este tipo de traductor es la de “**humanizar**”, por medio de un **lenguaje simbólico**, la escritura de programas a nivel **código de máquina**, que es el código binario “**decodificable**” por el procesador pero imposible de escribir en la práctica.

Ejemplo:

El código **8E26** (**1000111000100110**) se representa en Assembler x86 como **MOV BH**, donde:

MOV => Mover

BH => Registro de CPU.

Por error se cree que el Assembler es un lenguaje "antiguo" porque la mayoría de los programadores trabaja con lenguajes más evolucionados (Por ej. lenguajes visuales).

Sin embargo, a la hora de programar a nivel hardware, la única posibilidad la ofrece el set de instrucciones Assembler del procesador que se esté utilizando.

Todo procesador, grande o pequeño, desde el de una computadora personal hasta el de una supercomputadora, posee su lenguaje máquina y, por lo tanto, su simbólico de máquina.

Los traductores se dividen en dos grupos en función de *la relación entre lenguaje fuente y lenguaje de máquina*.

El primer grupo traduce una instrucción de un **lenguaje fuente** y genera una única **instrucción de máquina**; ese lenguaje fuente es su Assembler.

El segundo grupo lo constituyen los lenguajes de alto nivel en los que una sentencia se traduce a varias instrucciones en código de máquina.

INTÉRPRETES

Es un traductor de lenguaje que **convierte** una instrucción en **lenguaje de alto nivel** a **lenguaje de máquina** y, **de ser correcta, la ejecuta inmediatamente**.

Si encuentra un error de sintaxis => lo señala e interrumpe la ejecución.

Ventaja de la traducción con intérpretes:

- El programa se va probando a medida que se confecciona, o sea que permite una programación "interactiva".

Desventaja:

- Debe traducirse cada vez que se quiere ejecutar, aun cuando no haya sufrido modificaciones.

Ejemplos:

PHP, JavaScript, ASP, Perl, Python, Ruby, etc.

COMPILADORES

Es un traductor de lenguaje que **convierte** un programa escrito en **lenguaje de alto nivel** a **lenguaje de máquina**, pero tiene algunas diferencias significativas respecto del intérprete.

Fundamentalmente, separa la traducción de la ejecución del programa y agiliza tanto una como otra

La **ejecución** del programa **sólo se realiza cuando la compilación terminó de manera satisfactoria**

Permite obtener el código de máquina del programa compilado y **hacer un resguardo del mismo en una memoria externa**

Luego se puede **invocar al programa ejecutable tantas veces como sea necesario**, con la única condición de que se lo transfiera desde la memoria externa hacia la memoria principal.

El proceso de compilación de un programa puede interpretarse en tres etapas:

- **Análisis del Léxico:**

Se separa la cadena de caracteres en símbolos elementales como se especificó anteriormente: nombres de datos, signos de puntuación y palabras especiales, también llamadas palabras reservadas.

- **Análisis Sintáctico:**

Se determina la estructura sintáctica de acuerdo con un patrón de reglas gramaticales correspondiente al lenguaje que se utilice, y se analiza la secuencia de señales generadas en el proceso de Análisis del léxico.

- **Generación de Código:**

Se genera el código de las instrucciones para cada elemento sintáctico del programa.

Una cuarta etapa podría ser la de **optimización**, cuyo objetivo es reducir el programa o hacerlo más veloz, utilizando técnicas como detección y eliminación de instrucciones redundantes.

Ejemplos:

Delphi, C, C++, Pascal, etc.

CATEDRA DE ARQUITECTURA DE COMPUTADORAS

ARQUITECTURA CISC vs RISC

ARQUITECTURA CISC

Complex Instruction Set Computer (Computadora con un Conjunto de Instrucciones Complejo)

Los microprocesadores **CISC** poseen un conjunto de instrucciones que se caracteriza por ser muy amplio y permitir operaciones complejas entre operandos situados en la memoria o en los registros internos.

“**Los pasos**” de una ejecución están representados por una línea de código binario, que se presenta como salida de la unidad de control por cada pulso de reloj.

ARQUITECTURA CISC

Una instrucción compleja utiliza varias microinstrucciones y, por lo tanto, una Unidad de Control para un set de instrucciones **CISC utiliza una ROM de microcódigo.**

Un set de instrucciones **CISC admite múltiples modos de obtención del dato y, por lo tanto, un mismo verbo tendrá en el set tantas instrucciones distintas como modos de direccionamiento acepte el verbo.**

A continuación, se muestran varias “versiones” de la instrucción “**ADD**”:

Dirección	Código de instrucción		Mnemónico
<i>1531:0103</i>	<i>03060300</i>		ADD AX,[0003]
<i>1531:0107</i>	<i>01C0</i>		ADD AX,AX
<i>1531:0109</i>	<i>0304</i>		ADD AX,[SI]
<i>1531:010B</i>	<i>0302</i>		ADD AX,[BP+SI]
<i>1531:010D</i>	<i>034203</i>		ADD AX,[BP+SI+03]

Cuando un set de instrucciones admite tanta variedad de modos de direccionamiento, aumenta el número de instrucciones del set, además de incrementarse su complejidad en la medida en que aumenta la del procedimiento para acceder al dato.

La instrucción no se obtendrá de la memoria en la fase **FETCH en el mismo número de Ciclos para cada una de las variantes.**

Esto agregado a los diversos tiempos en el acceso al dato, debido a los distintos modos de direccionamiento mencionados, atenta contra el rendimiento "ideal" del procesamiento.

“Todas las instrucciones deberían ocupar prácticamente la misma cantidad de ciclos de reloj para lograr mayor eficiencia.”

Este tipo de arquitectura dificulta el paralelismo entre instrucciones, por lo que, en la actualidad, la mayoría de los sistemas **CISC de alto rendimiento implementan un sistema que convierte dichas instrucciones complejas en varias instrucciones simples del tipo **RISC**.**

La microprogramación es una característica importante y esencial de casi todas las arquitecturas **CISC. Como por ejemplo:**

Intel 8086, 8088, 80286, 80386, 80486.

La microprogramación significa que cada instrucción de máquina es interpretada por un microprograma localizado en una memoria en el circuito integrado del procesador.

En la década del '60 la microprogramación, por sus características, era la técnica más apropiada para las tecnologías de memorias existentes en esa época y permitía desarrollar también procesadores con compatibilidad ascendente.

En consecuencia, los procesadores se dotaron de poderosos conjuntos de instrucciones.

Las instrucciones compuestas son decodificadas internamente y ejecutadas con una serie de microinstrucciones almacenadas en una ROM interna.

Para esto se requieren de varios ciclos de reloj (al menos uno por microinstrucción).

CARACTERISTICAS DE LA ARQUITECTURA CISC

- Propósito de la Arquitectura CISC:

Proporcionar una única instrucción de máquina para cada instrucción que esté escrita en un lenguaje de alto nivel.

- Posee formatos de instrucciones de tamaño variable.
- Las instrucciones que necesitan operandos de registros pueden tener sólo dos bytes de longitud, pero las instrucciones que necesitan direcciones de memoria pueden necesitar cinco bytes para incluir todo el código de la instrucción.
- Las instrucciones **CISC** proporcionan manipulación directa de los operandos que residen en la memoria.

Por ejemplo, una instrucción **ADD** puede especificar un operando en la memoria mediante un direccionamiento de índice o por medio de un direccionamiento directo.

ARQUITECTURA RISC

Reduced Instruction Set Computer (Computadora con Conjunto de Instrucciones Reducido)

En la arquitectura **RISC**, las instrucciones de transferencia de datos están limitadas a las instrucciones cargar y almacenar.

Estas instrucciones utilizan un direccionamiento directo por registro.

Para evitar conflictos entre un acceso a memoria con el fin de recuperar una instrucción y cargar y almacenar un operando, utilizan dos canales separados con dos memorias: Una para almacenar las instrucciones y otra para almacenar los datos.

En algún momento, las dos memorias pueden operar a la misma velocidad que el reloj del CPU y se denomina memoria cache.

ARQUITECTURA RISC

Una de las de las principales ventajas es su capacidad para ejecutar instrucciones a una velocidad de una por ciclo de reloj.

En realidad, lo que se hace es empezar cada instrucción con cada ciclo de reloj y mediante su arquitectura paralela el procesador logra el propósito de una ejecución de instrucción en un solo ciclo.

La ventaja de la **RISC** sobre la **CISC** es que la **RISC** puede tener segmentos paralelos, al requerir un solo ciclo de reloj , en tanto la **CISC** utiliza muchos segmentos en su arquitectura paralela y el segmento mas grande requiere dos o mas ciclo de reloj).

ARQUITECTURA RISC

En la arquitectura **RISC** existen solo dos instrucciones para acceder a memoria, **Load** y **Store**.

Se utilizan para cargar la gran cantidad de registros disponibles en esta arquitectura para almacenar operandos.

Las instrucciones que realizan operaciones aritméticas son de referencia a registro y, en general, tienen especificados dos para los operandos y uno para el resultado.

Esta característica permite una independencia entre las instrucciones de acceso a memoria y las de cálculo, (facilita la concurrencia de ambas actividades y favorece el procesamiento segmentado).

Al tener instrucciones de tres operandos, el resultado no afecta a ninguno de éstos; en consecuencia, de ser necesaria la reutilización del operando en la ejecución de próximas instrucciones (y si el registro no fue actualizado) ya está disponible en el registro sin necesidad de acceder a memoria de nuevo.

Formato típico de una instrucción aritmética RISC:



Las instrucciones con formatos fijos permiten que los campos de códigos de operación y de los registros que contienen los operandos estén siempre codificados en las mismas posiciones, permitiendo el acceso a los registros de instrucciones en etapa de ejecución, al mismo tiempo que se decodifica el código de operación actual.

CARACTERÍSTICAS DE LA ARQUITECTURA RISC

- Una instrucción de máquina por cada ciclo de máquina.
- Operaciones registro a registro.
- Codificación uniforme de instrucciones (ejemplo: el código de operación se encuentra siempre en la misma posición de bit en cada instrucción, la cual es siempre una palabra), lo que permite una decodificación más rápida.
- Un conjunto de registros homogéneo, permitiendo que cualquier registro sea utilizado en cualquier contexto y así simplificar el diseño del compilador (aunque existen muchas formas de separar los ficheros de registro de entero y coma flotante).

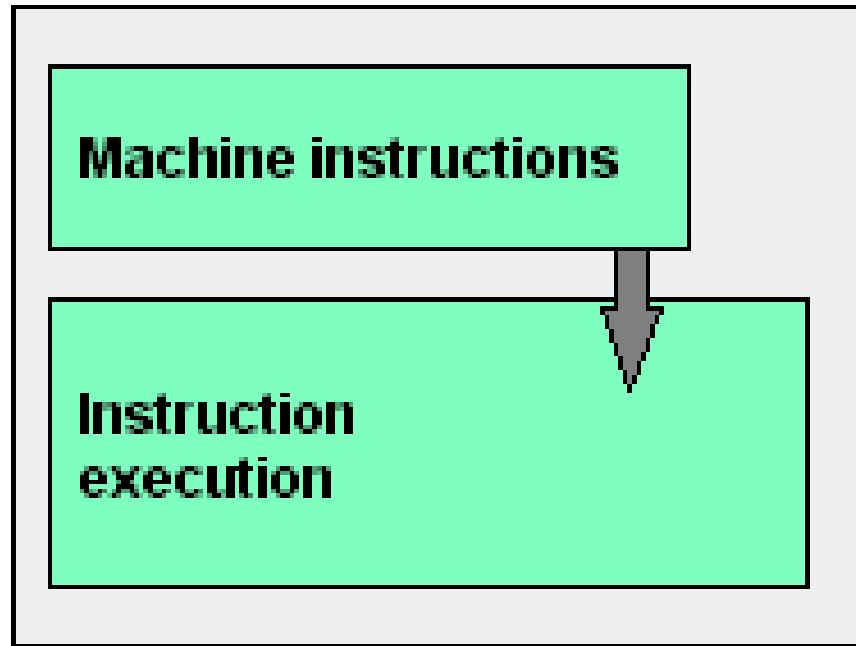
CARACTERÍSTICAS DE LA ARQUITECTURA RISC

- Modos de direccionamiento simples con modos más complejos reemplazados por secuencias de instrucciones aritméticas simples.
- Algunos tipos de datos soportados en el hardware (por ejemplo, algunas máquinas **CISC** tiene instrucciones para tratar con tipos byte, cadena; tales instrucciones no se encuentran en una máquina **RISC**).

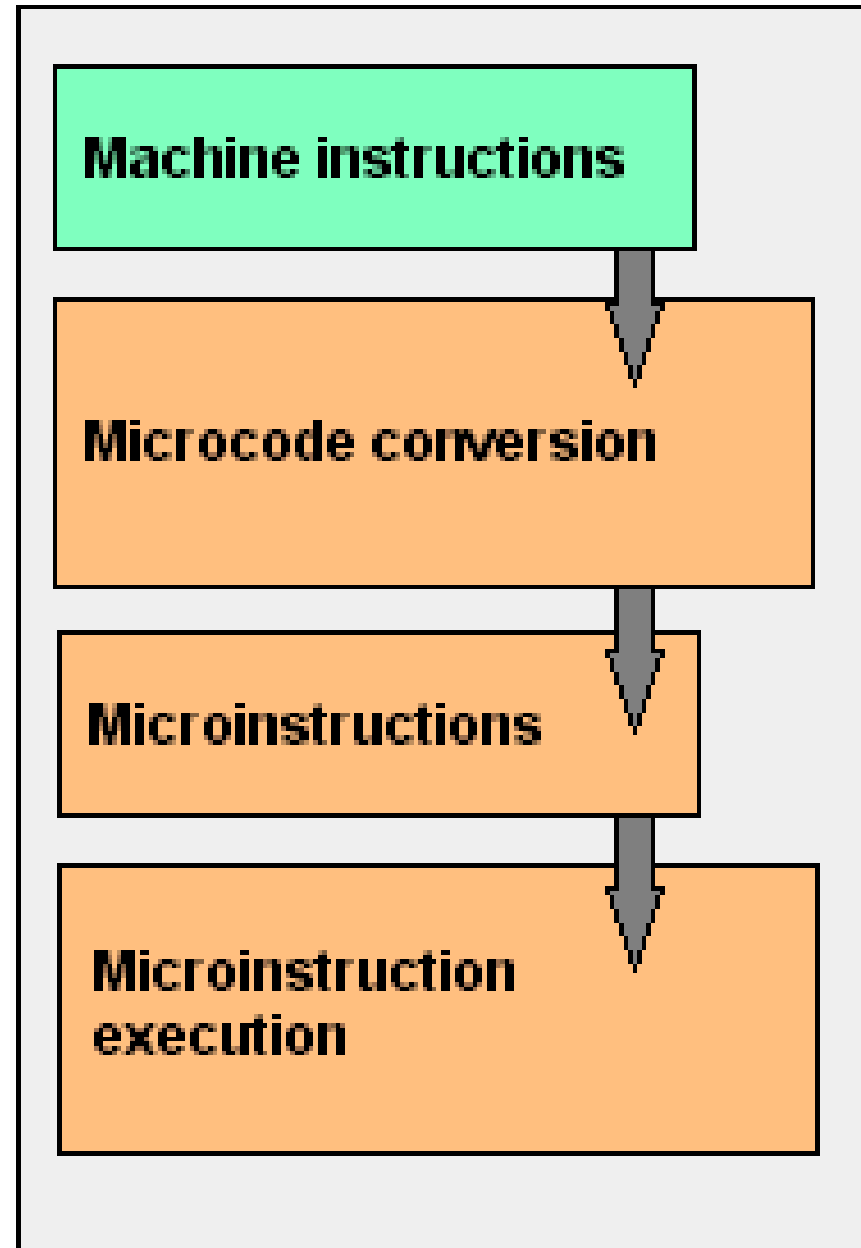
TABLA COMPARATIVA

	RISC	CISC
1	Instrucciones sencillas en un ciclo	Instrucciones complejas en varios ciclos
2	Sólo LOAD/STORE hacen referencia a memoria	Cualquier instrucción puede referencia memoria
3	Procesamiento serie de varias etapas	Poco procesamiento en serie
4	Instrucciones ejecutadas por hardware	Instrucciones interpretadas por el microprograma
5	Instrucciones de formato fijo	Instrucciones de formato variable
6	Pocas instrucciones y modo	Muchas instrucciones y modo
7	La complejidad está en el compilador	La complejidad está en el microprograma
8	Varios conjuntos de registros	Un sólo conjunto de registros

RISC



CISC



Ejemplos de Arquitectura CISC:

- Intel 8086, 8088, 80286, 80386, 80486.
- Motorola 68000, 68010, 68020, 68030, 6840.

Ejemplos de Arquitectura RISC:

- POWERPC
- DEC ALPHA
- MIPS
- ARM
- APPLE

FIN